

Aplicación del paradigma orientado a objetos

Breve descripción:

Durante el transcurso de este componente formativo, se abordan contenidos relacionados con la implementación de clases, objetos, atributos, constructores, métodos, herencia y relaciones en el lenguaje de programación Java.

Junio 2024

Tabla de contenido

Introducción	4
1. Características del lenguaje de programación orientada a objetos.....	6
2. Implementación de clases y objetos	10
Estructura de una clase en Java	10
2.1. Atributos y métodos de la clase	13
2.2. Constructores y destructores	17
2.3. Métodos accesorios y modificadores	19
Métodos accesorios (get, set)	21
2.4. Sobrecarga de métodos (overload).....	23
3. Comunicación entre clases	26
Implementación de asociaciones	26
Implementación de composición/agregación	30
Implementación de herencia.....	31
Síntesis	34
Material complementario	35
Glosario	36
Referencias bibliográficas	38
Créditos	39



Introducción

La Programación Orientada a Objetos (POO) se caracteriza por aplicar principios de abstracción, encapsulación, herencia y polimorfismo. En POO, los datos (atributos) y los métodos (comportamientos) se encapsulan dentro de objetos relacionados entre sí, permitiendo la ocultación de detalles de implementación. Esta ocultación es fundamental, ya que, similar a cómo se puede manejar una moto sin conocer el funcionamiento interno de sus sistemas, en la ingeniería de software permite la comunicación entre objetos a través de interfaces definidas sin revelar cómo están implementados otros objetos.

Este componente formativo ofrece una inmersión en los principios y aplicaciones de la Programación Orientada a Objetos, guiando a los estudiantes a través de los siguientes aspectos clave:

Introducción a las características fundamentales de la Programación Orientada a Objetos (POO).

Profundización en la implementación de clases y objetos, abarcando:

- Atributos y métodos de las clases.
- Constructores y destructores para la gestión de objetos.
- Métodos accesorios y modificadores, esenciales para la manipulación de datos.
- Sobrecarga de métodos (overload), facilitando la flexibilidad y reutilización de código.

Exploración de la comunicación entre clases mediante:

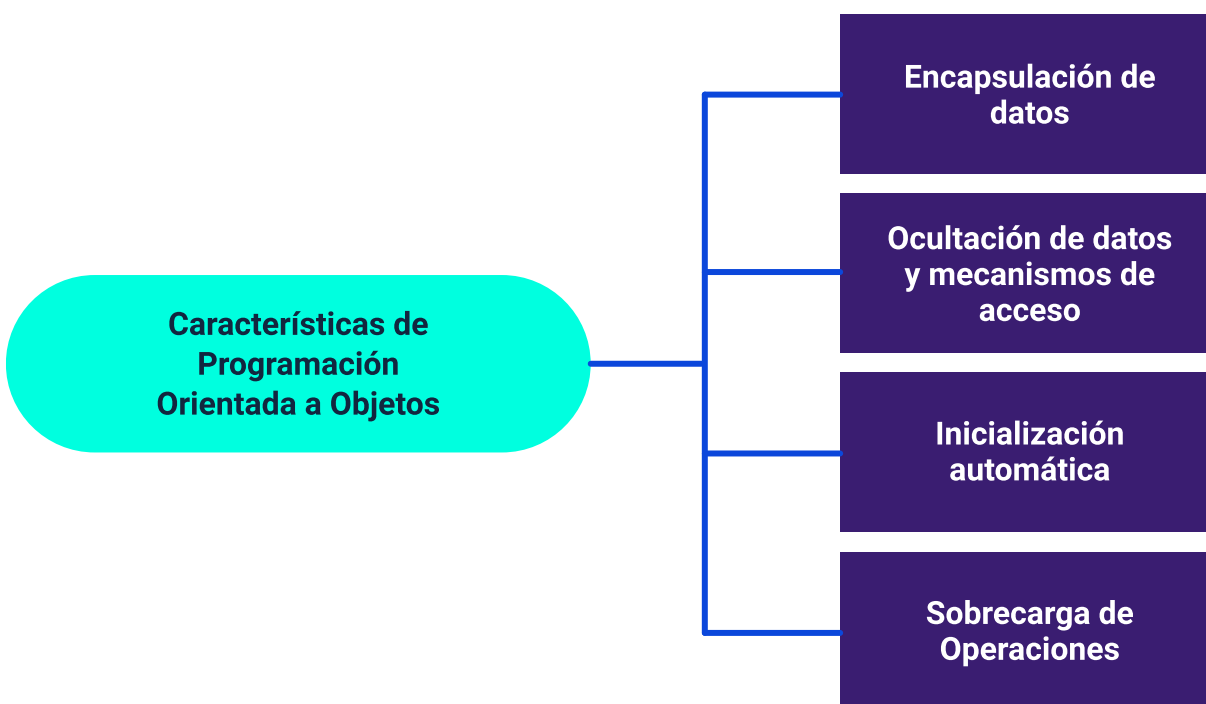
- Implementación de asociaciones, para relacionar objetos de manera lógica.

- Implementación de composición/agregación, definiendo relaciones de pertenencia.
- Implementación de herencia, permitiendo la extensión y reutilización de funcionalidades.

1. Características del lenguaje de programación orientada a objetos

Aunque es posible implementar Programación Orientada a Objetos (POO) en lenguajes como C y PASCAL, esta se torna compleja y confusa en programas de gran tamaño. Resulta más apropiado utilizar un lenguaje específicamente diseñado para soportar los principios de la POO para su implementación. Esto se debe a que, dependiendo de las características que ofrecen, los lenguajes de programación se pueden clasificar en dos categorías: lenguajes basados en objetos y lenguajes orientados a objetos. La programación basada en objetos es un estilo que soporta, principalmente, la encapsulación e identidad de los objetos, centrando sus características principales en:

Figura 1. Características de POO



Se dice que los lenguajes de programación basados en objetos son aquellos que soportan el uso de objetos, pero no incorporan todas las características de la

programación orientada a objetos, como la herencia o el polimorfismo. Un ejemplo de lenguaje de programación que no es completamente orientado a objetos es Ada, que aunque utiliza objetos, no incluye la herencia ni la ligadura dinámica, y es además un lenguaje multipropósito.

La programación orientada a objetos (POO), por otro lado, incluye todas las características de los lenguajes basados en objetos y añade funcionalidades clave como la herencia y la ligadura dinámica (también conocida como polimorfismo), que permiten una mayor flexibilidad y reutilización del código.

Las características de programación orientada a objetos son:

- Herencia
- Ligadura dinámica
- Características basadas en objetos

Entre los lenguajes que incluyen soporte para la programación orientada a objetos se cuentan C++, Smalltalk, Object Pascal y Java. Existe una amplia variedad de lenguajes de programación que ofrecen funcionalidades tanto de programación basada en objetos como de orientación a objetos. En particular, Java es un lenguaje de programación orientada a objetos que fue desarrollado por Sun Microsystems y lanzado en 1995. Java compila el código fuente en un bytecode que es ejecutado por la Máquina Virtual de Java (JVM), lo que lo diferencia de lenguajes como C y C++, que son principalmente compilados a código máquina y no interpretados. Java combina características de lenguajes interpretados a través de su JVM, garantizando portabilidad y eficiencia en la ejecución de sus programas.

Sus características principales son:

- **Sencillo**

Es fácil de entender, elimina la complejidad de otros lenguajes.

- **Orientado a objetos**

Al ser un paradigma orientado a objetos la filosofía de programación orientada a objetos facilita la creación y mantenimiento de programas.

- **Independiente de la arquitectura y portable**

Al compilar un programa en Java, se obtiene un tipo de código binario conocido como Java Bytecode. Este código es interpretable por diferentes computadoras de igual manera. Como el código compilado de Java es interpretado, un programa compilado de Java puede ser utilizado por cualquier computadora que tenga implementado el intérprete de Java.

- **Robusto**

Java simplifica la gestión de la memoria.

- **Multitarea**

Java puede ejecutar diferentes hilos de código al mismo tiempo.

- **Dinámico**

En Java no es necesario cargar completamente el programa en memoria, sino que las clases compiladas pueden ser cargadas en bajo demanda en tiempo de ejecución.

Además de las características mencionadas, hay diversos entornos de desarrollo integrados (IDE, por sus siglas en inglés "Integrated Development Environment") disponibles para la programación en Java. Estos entornos proporcionan a los

programadores un espacio de trabajo unificado que facilita todo el proceso de desarrollo de aplicaciones. Entre los IDE más reconocidos se encuentran:

Tabla 1. Entornos de desarrollo Java

IDE	Enlace para descarga
NetBeans	https://netbeans.apache.org/download/index.html
Eclipse	http://www.eclipse.org/downloads/
JBlue	https://bluej.org/versions.html

2. Implementación de clases y objetos

Los elementos centrales de un lenguaje de programación son las clases, ya que se utilizan para realizar todas las operaciones lógicas durante el desarrollo de software. Las clases actúan como estructuras o plantillas que modelan objetos del mundo real y pueden definirse como las abstracciones de estos objetos en el código. Los objetos derivados de estas clases pueden representar entidades físicas o conceptuales, como cosas, lugares o personas. Por lo general, las clases definen propiedades, comportamientos y relaciones con otras clases.

En Java, las convenciones de codificación estándar incluyen reglas específicas para nombrar clases, métodos y atributos. Las reglas para nombrar clases son las siguientes:

- a) La primera letra de cada nombre de clase debe ser mayúscula.
- b) Se debe utilizar la nomenclatura camelCase para nombres compuestos, donde la primera letra de cada palabra interna es mayúscula; por ejemplo, "CuentaAhorros".
- c) Los nombres de las clases deben ser sustantivos, indicativos de lo que representan.

Estructura de una clase en Java

Una clase en Java se compone por las siguientes declaraciones:

- Paquete.
- Comentarios.
- Definición de clases.
- Atributos.

- Constructores.
- Métodos.

Ejemplo de estructura:

```
public class Nombre de la Clase {}
```

- Donde la palabra public indica que el alcance de la clase será público.
- La palabra class indica que es una clase.
- Y las llaves {} se usan para agrupar los bloques de código.

Basado en lo anterior, a continuación, se relaciona un ejemplo sobre el diseño de la clase de producto con sus atributos y métodos:

Producto

- -código: int
- -nombre: String
- -precio: double
- +obtenerPrecio()
- +calcularTotal()

En Java este diseño se representaría de la siguiente manera:

Tabla 2. Implementación de la clase Producto en Java

Línea de código	Programación	Explicación
12	public class Producto { // Clase	Donde la línea 12 del código representa la estructura de la clase que es de acceso público y su nombre es Producto.
13		

Línea de código	Programación	Explicación
14 15 16 17	<pre>private int codigo; // atributos private String nombre; // atributos private double precio; // atributos</pre>	De la línea 14 a la 16 representa los atributos de la clase.
18 19 20 21 22 23 24 25 26	<pre>public void obtenerPrecio() { // método System.out.println("El precio es" + precio); } public void calcularTotal() { // método double total = precio*5000; } }</pre>	A partir de la línea 18 se encuentran los métodos.

La creación de un objeto en Java se realiza mediante la instanciación de una clase. Al instanciar una clase, se crea un objeto que hereda los atributos y métodos de dicha clase. Es posible crear múltiples instancias de una misma clase.

Por ejemplo, al instanciar un objeto de la clase Producto, asignamos valores a sus atributos, como código, nombre y precio, y podemos utilizar sus métodos, tales como obtenerPrecio() y calcularTotal(). En Java, la instanciación se lleva a cabo con la palabra reservada new, seguida del constructor de la clase. La sintaxis para crear un objeto de la clase Producto sería:

```
Producto p = new Producto();
```


Una vez instanciado, el objeto `p` tiene acceso a los métodos y atributos de la clase `Producto`. La forma correcta de invocar los métodos sería a través del objeto creado:

```
p.obtenerPrecio();
```

```
p.calcularTotal();
```

2.1. Atributos y métodos de la clase

Los atributos de una clase son definidos según esta sintaxis: `[modifVisibilidad] tipo nombre Variable [= valor Inicial];`

- **Public**

Indica que los atributos son accesibles desde cualquier lugar del programa. No hay restricciones en el alcance de los miembros de datos públicos.

- **Private**

Indica que los atributos solo son accesibles dentro de la clase en la que se declaran.

- **Protected**

Indica que los atributos son accesibles dentro del mismo paquete o sub-clases en paquetes diferentes. Al heredar si se puede usar desde la clase derivada.

- **Tipo**

Es el tipo de la variable, pudiendo ser un tipo básico o un objeto.

A continuación, se relacionan los tipos de datos que existen en Java:

Tabla 3. Tipos de datos en Java

Tipo	Formato	Descripción
Byte	8 bit	Entero de un byte

Tipo	Formato	Descripción
Short	16 bit	Entero corto
Int	32 bit	Entero
Long	64 bit	Entero largo
Float	32 bit	Flotante simple
Double	64 bit	Flotante doble
Char	16 bit	Un solo carácter

El nombre que se asigna a una variable, conocido como nombreVariable, debe adherirse a ciertas convenciones en Java:

- Los nombres de las variables deben comenzar con una letra minúscula. En el caso de nombres compuestos, se emplea la nomenclatura camelCase, donde la primera letra de cada palabra subsiguiente es mayúscula.
- Los nombres de variables no pueden contener espacios, iniciar con números ni incluir caracteres especiales.
- No se permite que una variable tenga el mismo nombre que una palabra clave reservada por el lenguaje.
- Dentro de un mismo ámbito, cada variable debe tener un nombre único y no puede repetirse el nombre de otras variables previamente declaradas.

A continuación, se proporcionan ejemplos que siguen estas convenciones:

Ejemplo variable datos Java

primerNombre

apellidoCliente

edad

La explicación indica que se puede inicializar una variable con un valor utilizando el formato = valorInicial;

Ejemplo variable datos Java:

Se permite definir más de una variable, separándolas por coma, por ejemplo:

```
public int a = 5, b, c = 4;
```

Ejemplos de declaración de atributos o variables:

```
public int edad=5;
```

```
private char genero='M';
```

```
private double estatura;
```

Nota: todos los atributos terminan en punto y coma.

Los métodos se definen la siguiente manera:

```
[modifVisibilidad] tipo nombreMetodo (listaParámetros) {}
```

En los atributos y métodos para las clases también existen:

- **Modificador de visibilidad**

Aplica las mismas normas que para los atributos. Indica el nivel de acceso: público, privado o protegido.

- **Tipo**

Corresponde al tipo de valor que el método retorna, como int, double, char, String, etc.

- **Nombre del método**

Debe ser un verbo en minúscula o un compuesto iniciando con mayúscula en sustantivos, ej. `revisarCuenta()`.

- **Lista de parámetros**

Incluye los parámetros entre paréntesis; son los datos que recibe el método, y puede ser vacía.

- **Cuerpo del método {}**

Contiene el código a ejecutar encerrado en llaves; no termina en punto y coma como los atributos.

A continuación, se presentan algunos ejemplos de métodos:

Ejemplo método atributos y métodos:

```
public int obtenerEdad ()  
  
{  
  
    return edad;  
  
}
```

Este es un método público que retorna un valor entero, cuyo nombre es `obtenerEdad`, no recibe parámetros y devuelve la edad.

Ejemplo listado de parámetros:

Ejemplo método:

```
public void cambiarEdad (int laEdad)  
  
{  
  
    edad = laEdad;
```

```
}
```

Este es un método público que no retorna ningún valor porque tiene la palabra reservada `void`, su nombre es `cambiarEdad` y recibe un parámetro de tipo entero, en el cuerpo del método asigna a la variable `edad` el valor del parámetro que viene almacenado en la variable `laEdad`.

2.2. Constructores y destructores

El constructor de una clase es un método diseñado específicamente para inicializar los objetos de dicha clase. Por otro lado, un destructor es un método que se invoca automáticamente cuando el objeto es destruido. Se utiliza un constructor al instanciar un objeto para inicializar sus variables con valores adecuados. Un constructor, que es un método especial de la clase, tiene las siguientes características:

El nombre del constructor coincide con el nombre de la clase.

- No retorna ningún valor.
- Es posible definir múltiples constructores, siempre que tengan diferentes listas de parámetros.
- El código dentro de un constructor generalmente incluye inicializaciones de variables y objetos para que el objeto sea creado con los valores iniciales deseados.

La sintaxis para definir un constructor es la siguiente:

```
[modificadorDeVisibilidad]
```

```
NombreDelConstructor(listaDeParámetros) {
```

```
    // Código de inicialización
```

}

Donde:

- `modificadorDeVisibilidad` indica si el constructor es `public`, `private` o `protected`, siguiendo las mismas normas que para los atributos y métodos.
- `NombreDelConstructor` debe coincidir con el nombre de la clase.
- `listaDeParámetros` es la enumeración de parámetros que el constructor acepta, separados por comas y definidos por su tipo y nombre, por ejemplo, `(int a, int b)`.

En el ejemplo de la clase `Producto`, se crea un constructor público llamado `Producto` con una lista de parámetros. Este constructor asigna los valores de los parámetros a las variables de instancia de la clase. La palabra reservada `this` se utiliza para referirse a los miembros de la propia clase en el objeto actual.

- **Un constructor**

Sin parámetros se denomina constructor vacío. Si no se define explícitamente un constructor, el compilador proporciona uno por defecto, permitiendo la creación de objetos sin argumentos mediante `new Clase{}`.

- **Los destructores**

Son métodos que liberan la memoria ocupada por un objeto. En Java, no existen los destructores como tal debido al recolector de basura de la máquina virtual de Java, que administra la memoria de forma automática, liberándola cuando detecta objetos que ya no se utilizan.

Aunque el manejo de memoria en Java es automático gracias al recolector de basura, los usuarios pueden sugerir la ejecución del recolector de basura mediante una

instrucción específica, aunque el momento exacto de la recolección de basura no es garantizado.

2.3. Métodos accesores y modificadores

En la programación orientada a objetos, existen dos tipos fundamentales de modificadores de acceso para los miembros de una clase: público (`public`) y privado (`private`).

- **Público (`public`)**

Un miembro público es accesible desde cualquier parte del código, incluso fuera de su clase de definición. Esto permite que cualquier otra clase o componente del programa interactúe con el miembro público sin restricciones. El acceso es universal, permitiendo que cualquier otro código, independientemente de su paquete o clase, utilice el miembro público.

- **Privado (`private`)**

Un miembro privado solo puede ser accedido por otros métodos dentro de la misma clase. Esto se utiliza para controlar la exposición y manipulación de los datos internos del objeto, evitando el acceso o modificación directa por parte de código externo a la clase. El acceso está estrictamente limitado a la clase donde el miembro privado es declarado. Otros componentes del programa no pueden acceder directamente a estos miembros privados.

Hay cuatro tipos de modificadores de acceso disponibles en Java:

- **Predeterminado (`default`)**

Cuando no se especifica un modificador de acceso para una clase, método o miembro de datos, se aplica el modificador de acceso predeterminado por defecto. Los miembros de datos, clases o métodos con acceso predeterminado son accesibles solo dentro del mismo paquete.

Por ejemplo, la clase Producto y el método mostrar, al tener modificador de acceso predeterminado, no pueden ser accedidos desde otro paquete.

- **Privado (private)**

Se indica con la palabra clave “private”. Los métodos o miembros de datos declarados como privados son accesibles únicamente dentro de la clase donde se declaran.

Ninguna otra clase del mismo paquete puede acceder a los miembros privados.

Las clases e interfaces no pueden declararse como privadas.

- **Protegido (protected)**

Se especifica con la palabra clave “protected”. Los métodos o miembros de datos declarados como “protected” son accesibles dentro del mismo paquete o en subclases situadas en paquetes diferentes.

Accesible dentro del mismo paquete y por subclases en paquetes diferentes, promoviendo así una forma de herencia protegida.

- **Público (public)**

Se indica con la palabra clave “public”. Este modificador de acceso tiene el alcance más amplio de todos.

Las clases, métodos o miembros de datos declarados como públicos son accesibles desde cualquier lugar del programa, sin restricciones en el alcance. Por ejemplo, declarar el método mostrar de la clase Producto

como público permite que se acceda a este método desde cualquier parte del programa.

Métodos accesorios (get, set)

Los métodos accesorios permiten obtener o modificar los atributos de un objeto y se clasifican en dos tipos: “get” y “set”.

- Los métodos “get” permiten acceder al valor de un atributo.
- Los métodos “set” permiten modificar el valor de un atributo.

El nombre de estos métodos comienza con “get” o “set”, seguido del nombre del atributo con la primera letra en mayúscula. Su modificador de acceso es siempre público para facilitar el acceso o modificación de los atributos desde fuera de la clase. Por ejemplo, “getNombre” o “setNombre”.

Ejemplo de sintaxis del método “get”:

```
public tipoDatoAtributo getAtributo() {  
  
    return atributo;  
  
}
```

Ejemplo de sintaxis del método “set”:

```
public void setAtributo(tipoDatoAtributo variable) {  
  
    this.atributo = variable;  
  
}
```

Ejemplo implementación de métodos get y set en Java:

```
public class Producto { // Clase
```

```
private int codigo; // atributos

private String nombre; // atributos


public int getCodigo() { // método get del atributo codigo

    return codigo; // obtiene el valor del atributo codigo

}


public void setCodigo(int codigo) { // método set del atributo codigo

    this.codigo = codigo; // cambia el valor del atributo codigo

}


public String getNombre() { // método get del atributo nombre

    return nombre; // obtiene el valor del atributo nombre

}


public void setNombre(String nombre) { // método set del atributo nombre

    this.nombre = nombre; // cambia el valor del atributo nombre

}

}
```

En el ejemplo se presenta la clase Producto con sus atributos código y nombre, por cada uno de estos atributos encontramos sus métodos get y set para obtener y cambiar el valor de cada uno de ellos.

2.4. Sobrecarga de métodos (overload)

La sobrecarga de métodos permite definir múltiples constructores o métodos con el mismo nombre dentro de una clase, bajo la condición de que estos no compartan la misma cantidad y tipo de parámetros. En esencia, la sobrecarga facilita la declaración de métodos homónimos que difieren en los parámetros que aceptan. Por lo tanto, no es posible tener dos métodos con idéntico nombre y parámetros coincidentes. En este contexto, los argumentos proporcionados al llamar al método determinan cuál de ellos se ejecuta.

Ejemplo implementación de sobrecarga de constructores y métodos en Java:

```
public class Producto { // Clase

    private int codigo; // atributos

    private String nombre; // atributos

    private double precio; // atributos

    public Producto() {

    }

    public Producto(int codigo, String nombre, double precio) {
```

```
this.codigo = codigo;

this.nombre = nombre;

this.precio = precio;
}

public void descontarProducto(double precio, String nombre) {

    double descuento = precio * 0.10;

    System.out.println("El producto " + nombre + " tiene descuento");
}

public void descontarProducto(int precio) {

    double descuento = precio * 0.10;
}
}
```

El ejemplo presenta la sobrecarga tanto de constructores como de métodos. Se evidencia sobrecarga en los constructores de la clase Producto, donde se han creado dos constructores con el mismo nombre pero con distintos parámetros. Una situación similar ocurre con el método descontarProducto, que se define dos veces, cada uno con diferentes parámetros.



3. Comunicación entre clases

Las clases, así como los objetos, no operan de manera aislada. La Programación Orientada a Objetos (POO) busca simular las aplicaciones del mundo real de la manera más precisa posible, lo cual implica reflejar las relaciones entre clases y objetos de forma adecuada. De este modo, la comunicación entre clases se facilita exclusivamente a través de sus interfaces públicas.

Existen tres tipos fundamentales de relaciones entre objetos:

- **Asociación**

Describe cómo los objetos se relacionan y se utilizan entre sí.

- **Agregación/Composición**

Define una relación en la que un objeto está compuesto por uno o más objetos, mostrando una relación de tipo 'parte-todo'.

- **Generalización/Especialización (Herencia)**

Expresa una jerarquía entre clases donde una clase derivada (subclase) hereda comportamientos y atributos de una clase base (superclase), permitiendo la reutilización y extensión de código.

Implementación de asociaciones

En Java, la implementación de la asociación se lleva a cabo mediante campos de instancia. Esta relación puede ser bidireccional, donde cada clase posee una referencia hacia la otra. Tanto la agregación como la composición son variantes específicas de relaciones de asociación. A continuación, se detallan estos tipos de relaciones:

- **Asociación unidireccional**

Una asociación unidireccional ocurre cuando una clase está vinculada a otra, pero solo una de ellas conoce la existencia de la otra. Esto significa que un objeto de la primera clase puede acceder a los miembros de la segunda clase, pero no a la inversa.

Para implementar una asociación unidireccional en Java, se debe incluir una referencia en la clase que conoce a la otra. Esto se logra declarando un atributo del tipo de la clase conocida dentro de la clase que posee el conocimiento.

Por ejemplo, en una relación entre una clase Pedido y una clase Cliente, la clase Pedido puede tener una referencia a la clase Cliente para saber a quién pertenece, pero la clase Cliente no necesita tener una referencia a la clase Pedido si la lógica del negocio no requiere que un cliente conozca sus pedidos.

Aquí está cómo se podría ver esto en el código:

```
public class Cliente {  
  
    private String nombre;  
  
    // Otros atributos y métodos del Cliente  
  
}  
  
public class Pedido {  
  
    private Cliente cliente; // Referencia unidireccional  
  
    private double total;
```

```
public Pedido(Cliente cliente) {  
  
    this.cliente = cliente;  
  
}  
  
// Otros atributos y métodos del Pedido  
  
}
```

En este caso, Pedido conoce a #Cliente, pero Cliente no tiene conocimiento de Pedido, lo que hace que la relación sea unidireccional desde Pedido hacia Cliente.

- **Asociación bidireccional**

En una asociación bidireccional, dos clases están vinculadas entre sí de tal manera que ambas pueden conocer la existencia y utilizar los miembros de la otra. Esto significa que un objeto de una clase mantiene una referencia a uno o varios objetos de la otra clase y viceversa; ambos tienen la capacidad de interactuar entre sí.

Para implementar una asociación bidireccional en Java, se deben crear instancias mutuas en ambas clases. Cada clase tendrá un atributo que referencia a la otra, y típicamente se proporcionarán métodos para establecer y modificar estas referencias de manera controlada.

Tomando como ejemplo una relación entre Cliente y Cuenta, no solo el Cliente tendría una Cuenta, sino que la Cuenta también tendría una referencia a su Cliente para formar una relación bidireccional completa.

Aquí se muestra cómo se podría representar en el código:

```
public class Cliente {  
  
    private String nombre;  
  
    private Cuenta cuenta; // Referencia bidireccional a Cuenta  
  
    // Constructor, getters y setters  
  
    public void vincularCuenta(Cuenta cuenta) {  
  
        this.cuenta = cuenta;  
  
        cuenta.vincularCliente(this); // Establece la referencia bidireccional  
    }  
}  
  
public class Cuenta {  
  
    private double saldo;  
  
    private Cliente cliente; // Referencia bidireccional a Cliente
```

```
// Constructor, getters y setters

public void vincularCliente(Cliente cliente) {

    this.cliente = cliente;

    // Si es necesario, también establecer la referencia desde Cliente a Cuenta

    if (cliente.getCuenta() != this) {

        cliente.vincularCuenta(this);

    }

}

}
```

En este código, tanto la clase Cliente como la clase Cuenta tienen métodos que permiten vincular una a la otra. Es importante manejar estas referencias cuidadosamente para evitar bucles infinitos o inconsistencias en las relaciones. Cuando se establece la vinculación desde una instancia de Cliente a Cuenta, también se llama a un método en Cuenta para establecer la vinculación inversa, asegurando así una asociación bidireccional coherente.

Implementación de composición/agregación

La agregación es una relación entre clases que modela una jerarquía de tipo "todo-parte", donde las partes pueden existir independientemente del todo. En cambio, la composición es una forma específica de agregación en la que las partes están exclusivamente asociadas a un único todo y su existencia depende de ese todo.

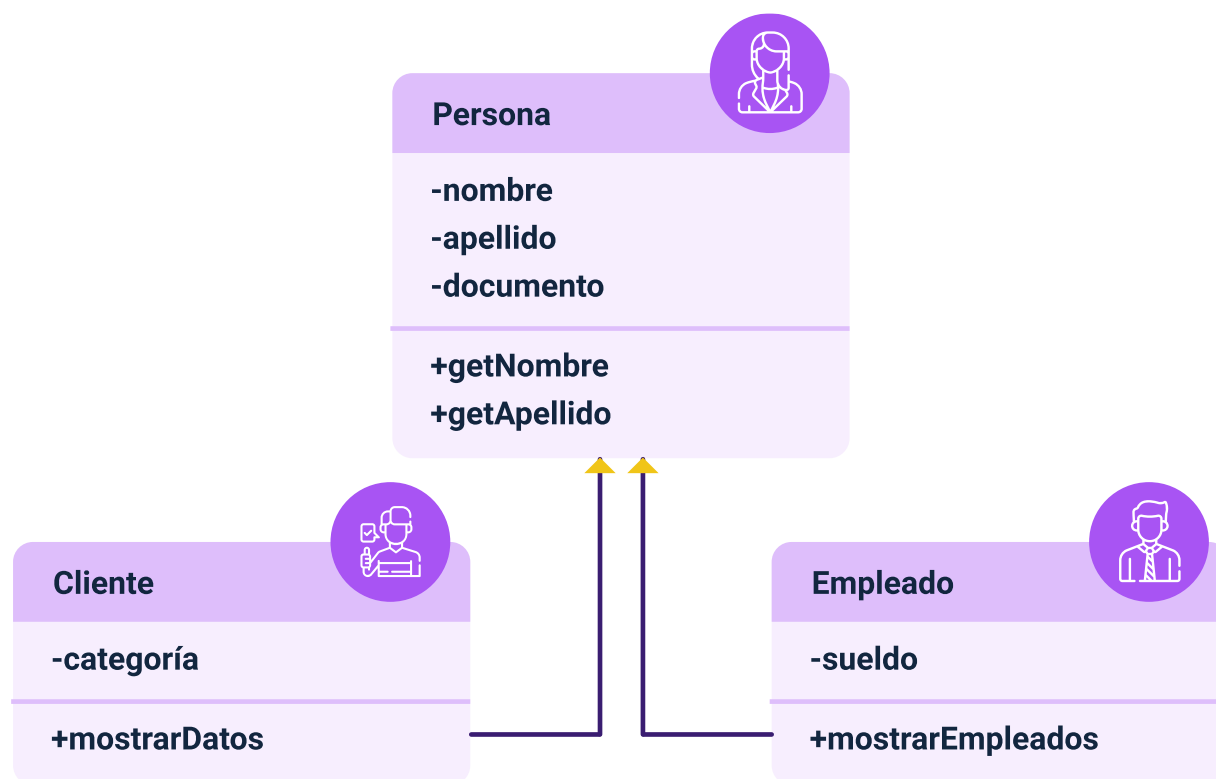
En Java, la composición se implementa creando una clase que contiene objetos de otras clases como atributos privados, inicializándolos generalmente dentro del constructor de la clase contenedora para asegurar el acoplamiento de sus ciclos de vida.

Implementación de herencia

La herencia es el mecanismo en la programación orientada a objetos que permite que una clase, conocida como subclase, herede propiedades y comportamientos de otra clase, llamada superclase.

Por ejemplo, a partir de la clase Persona, es posible definir una subclase Empleado, que hereda las características de Persona. La herencia permite la reutilización de código y la extensión de funcionalidades. En Java, la herencia se implementa mediante la palabra clave `extends`, lo que indica que la subclase Empleado adquiere los métodos y atributos accesibles de la clase Persona.

Figura 2. Ejemplo de herencia en UML



La superclase **Persona** posee los atributos nombre, apellido y documento, que son igualmente necesarios para las clases **Cliente** y **Empleado**. La herencia permite la reutilización de código, de modo que las clases derivadas no necesitan declarar nuevamente estos atributos, ya que los heredan directamente de la clase **Persona**.

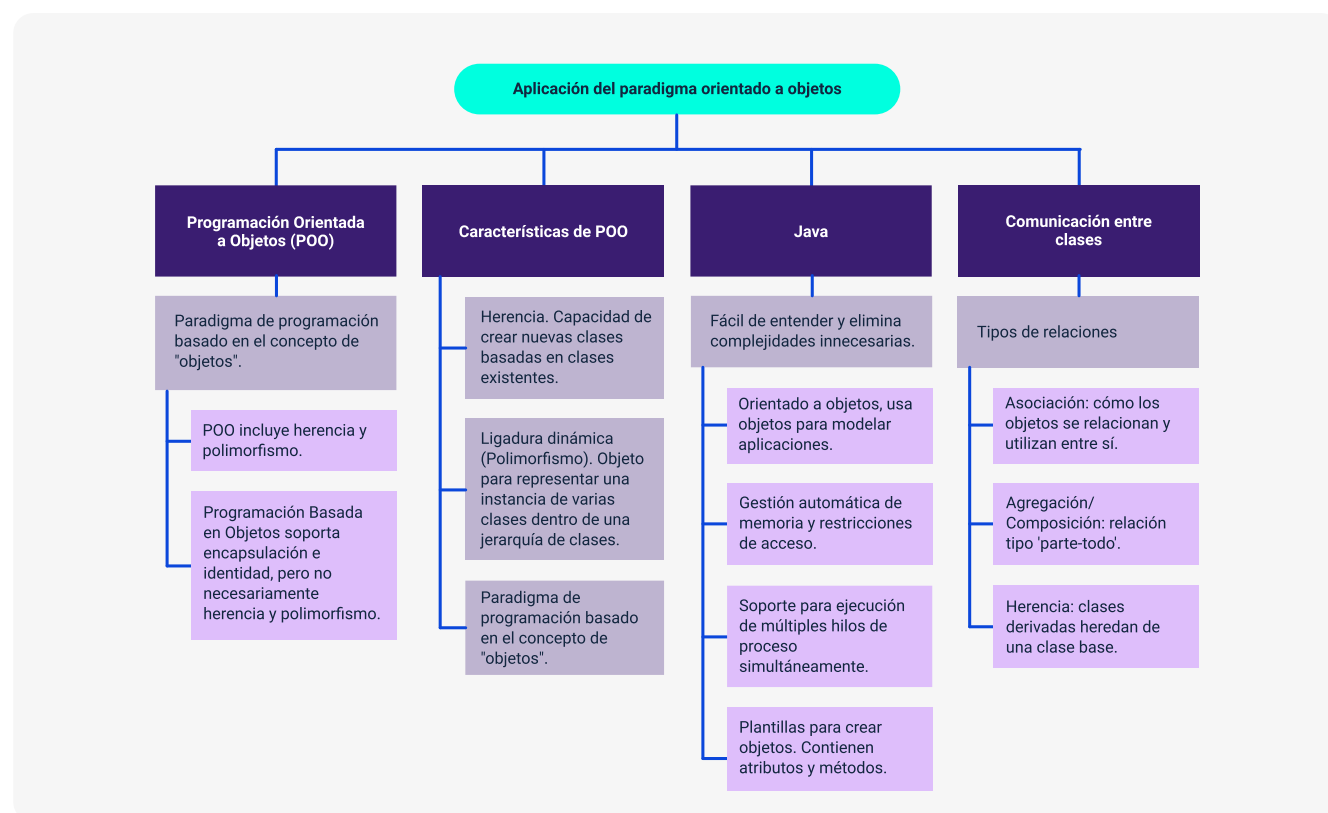
Para implementar la relación de herencia del ejemplo que incluye a las clases **Persona**, **Cliente** y **Empleado** en Java, se procedería de la siguiente manera:

Se crea la clase **Persona**, la superclase, con sus atributos, constructores y métodos “get” y “set”.

Se crea la clase Cliente con su propio atributo categoría y su método mostrarDatos. Para heredar los atributos de la superclase Persona, se utiliza la palabra reservada “extends”.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Implementación de clases y objetos java	Códigos de Programación - MR. (2017). Curso Java - 7: Clases, Métodos y Objetos.	Video	https://youtu.be/Z6ispQvMy8s?feature=shared
Sobrecarga de métodos (overload)	Códigos de Programación - MR. (2017). Curso Java - 8: Constructores y Sobrecarga de Métodos.	Video	https://www.youtube.com/watch?v=DUSaU58GDk&ab_channel=C%C3%B3digosdeProgramaci%C3%B3n-MR
Comunicación entre clases	TodoCode. (2022). RELACIONES entre CLASES en JAVA POO.	Video	https://www.youtube.com/watch?v=XKk5k9OrAUg&ab_channel=TodoCode

Glosario

Agregación: tipo especial de asociación que representa una relación "parte de" donde las partes pueden existir independientemente del todo.

Asociación: relación entre dos objetos en la que uno utiliza las capacidades o servicios del otro.

Bytecode: código intermedio más abstracto que el código máquina, que es ejecutado por la JVM.

Clases: estructuras fundamentales de la POO que definen las propiedades y comportamientos de los objetos.

Composición: forma de agregación con una relación más fuerte donde las partes no pueden existir independientemente del todo.

Herencia: característica de la POO que permite que una clase derive de otra, heredando sus métodos y atributos.

Java: lenguaje de programación orientado a objetos, independiente de la plataforma, que compila a bytecode, el cual se ejecuta en la Máquina Virtual de Java (JVM).

JVM (Máquina Virtual de Java): entorno de ejecución para programas Java que permite la portabilidad del código entre diferentes plataformas.

Objetos: instancias de clases que contienen datos y comportamientos definidos por su clase.

Polimorfismo (Ligadura Dinámica): capacidad de una variable de tipo base para referirse a objetos de tipos derivados, permitiendo que se ejecuten diferentes métodos a través de una interfaz común.

POO (Programación Orientada a Objetos): paradigma de programación que utiliza objetos y sus interacciones para diseñar aplicaciones y programas informáticos.

Referencias bibliográficas

Archivo General de la Nación (2022). Guía para la formulación de un esquema de metadatos para la gestión de documentos.

https://www.archivogeneral.gov.co/sites/default/files/Estructura_Web/5_Conulte/Recursos/Publicacionees/GuiaDeMetadatos.pdf

Instituto Colombiano de Normas Técnicas y Certificación. (2022). Normas técnicas de la información y seguridad de la familia ISO 27000. Icontec.

<https://www.icontec.org/servicio-educacion/modulares-de-educacion/>

Ministerio de Educación. (2022). Guía para la clasificación de la información.

https://www.mineduacion.gov.co/1759/articles-407695_galeria_14.pdf

Ministerio de Tecnologías de la Información y Comunicaciones - MinTIC. (2022). Entidades del sector. MinTIC.

<https://www.mintic.gov.co/portal/inicio/Ministerio/Entidades-del-sector/>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Zulema León Escobar	Experta Temática	Grupo De Apoyo Administrativo - Regional Distrito Capital
Paola Alexandra Moya Peralta	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Juan Daniel Polanco Muñoz	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Jhon Jairo Urueta Álvarez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Integración continua

Breve descripción:

En este componente formativo se abordarán los conceptos relacionados con el funcionamiento de un sistema web, soportado por la arquitectura de internet. A continuación, se revisarán las tecnologías más importantes para el desarrollo web, tanto del lado del cliente como del servidor. Por último, se explorarán los conceptos básicos de una arquitectura orientada a servicios.

Julio 2024

Tabla de contenido

Introducción	4
1. Sistemas de control de versiones.....	5
Sistemas de control de versiones locales	6
Sistemas de control de versiones locales	7
Sistemas de control de versiones centralizados	8
Sistemas de control de versiones distribuidas.....	9
2. Sistema de control de versiones – Git.....	11
Instalación de Git en sistemas operativos	14
3. Comandos básicos de Git.....	15
4. Git en entornos remotos	18
5. Plataformas para implementar integración continua – Gitlab	20
Creación de cuenta.....	21
Aceptación de términos de servicios.....	22
Interfaz de inicio	22
Síntesis	33
Material complementario	34
Glosario	35
Referencias bibliográficas	36



Créditos	37
----------------	----

Introducción

La integración continua es una práctica esencial en DevOps que asegura un espacio compartido donde los equipos pueden gestionar su trabajo de manera unificada mediante sistemas de control de versiones. Estos espacios, conocidos como repositorios, no solo agrupan archivos, sino que también administran metadatos sobre los cambios y los usuarios que los realizan.

Los sistemas de control de versiones registran los cambios en archivos a lo largo del tiempo, permitiendo su recuperación precisa. Existen tres tipos principales: locales, centralizados y distribuidos. Los sistemas locales suelen ser manuales y propensos a errores, mientras que los centralizados dependen de un servidor único, lo que puede ser riesgoso.

Los sistemas distribuidos, como Git, ofrecen una solución más robusta al permitir que cada cliente tenga una copia completa del repositorio, lo que facilita la restauración en caso de fallos del servidor central. Git, ampliamente utilizado, nació para la comunidad del kernel de Linux y permite trabajar sin conexión, garantizando la integridad de los datos mediante algoritmos criptográficos.

Plataformas como GitHub, GitLab y BitBucket, basadas en Git, facilitan la integración continua proporcionando repositorios accesibles por internet, donde se pueden gestionar proyectos de software de manera colaborativa y segura, optimizando así el desarrollo y mantenimiento de los proyectos.

1. Sistemas de control de versiones

Un sistema de control de versiones se encarga del registro de los cambios realizados a lo largo del tiempo en uno o un conjunto de archivos (versiones), de forma tal que estos puedan ser recuperados con precisión según las necesidades de los usuarios. A este tipo de procesos se le puede denominar versionamiento y, aunque estas actividades son comunes en archivos o código fuente en la industria del desarrollo de software, el proceso es aplicable a cualquier tipo de archivo.

El proceso de hacer copias de seguridad y crear versiones de los archivos considerados importantes es una actividad muy común.

Así, mediante el uso de un sistema de control de versiones, se obtienen múltiples posibilidades, entre las cuales se encuentran:

Figura 1. Beneficios del control de versiones



Respecto a los sistemas de control de versiones, estos pueden ser categorizados según sus características en tres grupos: sistemas de control de versiones locales,

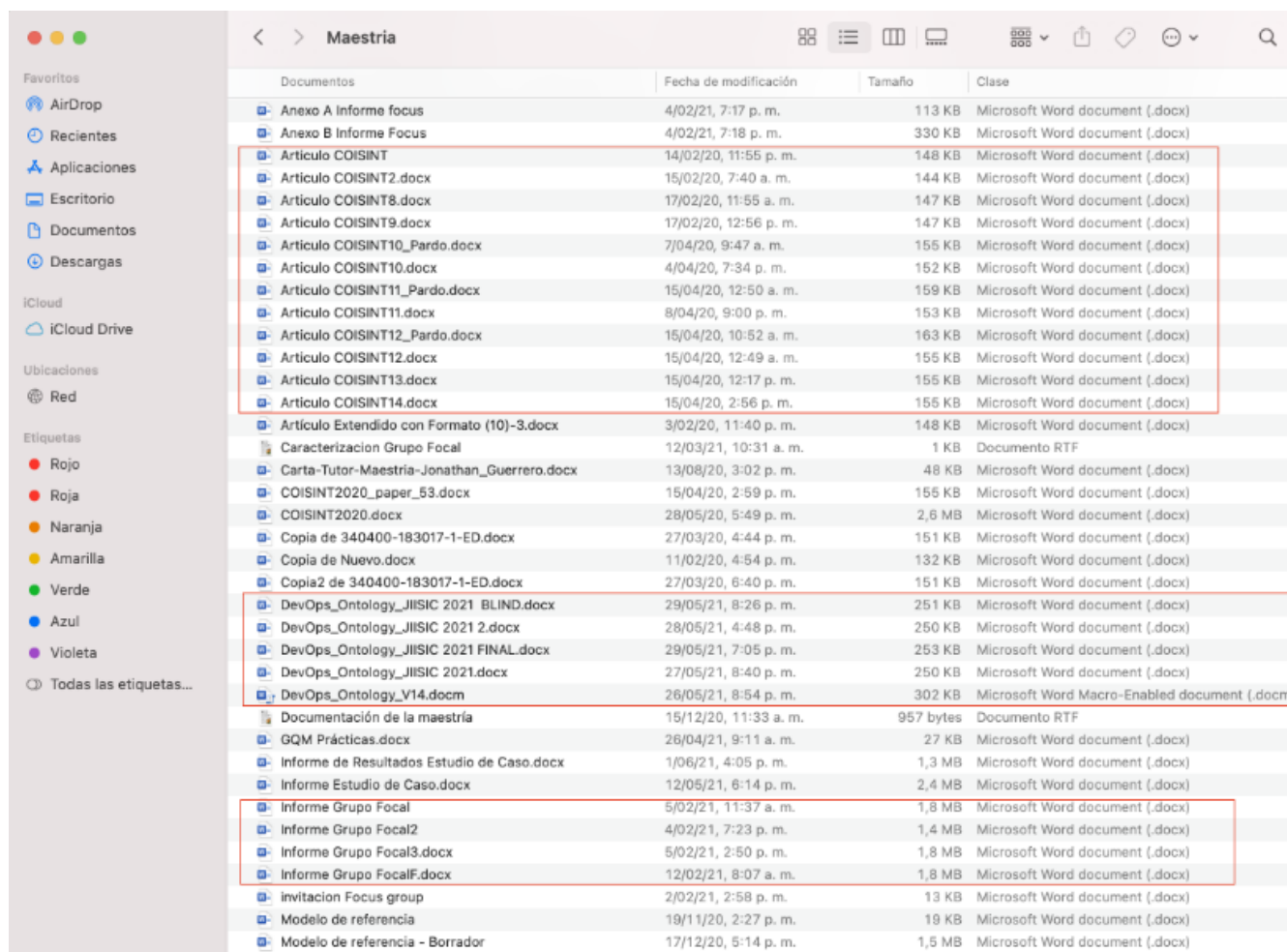
sistemas de control de versiones centralizados y sistemas de control de versiones distribuidos.

A continuación, se podrán conocer sobre los sistemas de control de versiones locales, centralizadas y distribuidas:

Sistemas de control de versiones locales

Una forma de llevar control de versiones locales es crear copias manuales en un directorio exclusivo o de fácil reconocimiento, donde se registre los cambios realizados sobre un documento particular, generalmente mediante una numeración o incluso llevando registro de la fecha de realización de la versión, como se aprecia en la siguiente figura.

Figura 2. Ejemplo de los sistemas de control de versiones locales



Documentos	Fecha de modificación	Tamaño	Clase
Anexo A Informe focus	4/02/21, 7:17 p. m.	113 KB	Microsoft Word document (.docx)
Anexo B Informe Focus	4/02/21, 7:18 p. m.	330 KB	Microsoft Word document (.docx)
Artículo COISINT	14/02/20, 11:55 p. m.	148 KB	Microsoft Word document (.docx)
Artículo COISINT2.docx	15/02/20, 7:40 a. m.	144 KB	Microsoft Word document (.docx)
Artículo COISINT8.docx	17/02/20, 11:55 a. m.	147 KB	Microsoft Word document (.docx)
Artículo COISINT9.docx	17/02/20, 12:56 p. m.	147 KB	Microsoft Word document (.docx)
Artículo COISINT10_Pardo.docx	7/04/20, 9:47 a. m.	155 KB	Microsoft Word document (.docx)
Artículo COISINT10.docx	4/04/20, 7:34 p. m.	152 KB	Microsoft Word document (.docx)
Artículo COISINT11_Pardo.docx	15/04/20, 12:50 a. m.	159 KB	Microsoft Word document (.docx)
Artículo COISINT11.docx	8/04/20, 9:00 p. m.	153 KB	Microsoft Word document (.docx)
Artículo COISINT12_Pardo.docx	15/04/20, 10:52 a. m.	163 KB	Microsoft Word document (.docx)
Artículo COISINT12.docx	15/04/20, 12:49 a. m.	155 KB	Microsoft Word document (.docx)
Artículo COISINT13.docx	15/04/20, 12:17 p. m.	155 KB	Microsoft Word document (.docx)
Artículo COISINT14.docx	15/04/20, 2:56 p. m.	155 KB	Microsoft Word document (.docx)
Artículo Extendido con Formato (10)-3.docx	3/02/20, 11:40 p. m.	148 KB	Microsoft Word document (.docx)
Caracterización Grupo Focal	12/03/21, 10:31 a. m.	1 KB	Documento RTF
Carta-Tutor-Maestría-Jonathan_Guerrero.docx	13/08/20, 3:02 p. m.	48 KB	Microsoft Word document (.docx)
COISINT2020_paper_53.docx	15/04/20, 2:59 p. m.	155 KB	Microsoft Word document (.docx)
COISINT2020.docx	28/05/20, 5:49 p. m.	2,6 MB	Microsoft Word document (.docx)
Copia de 340400-183017-1-ED.docx	27/03/20, 4:44 p. m.	151 KB	Microsoft Word document (.docx)
Copia de Nuevo.docx	11/02/20, 4:54 p. m.	132 KB	Microsoft Word document (.docx)
Copia2 de 340400-183017-1-ED.docx	27/03/20, 6:40 p. m.	151 KB	Microsoft Word document (.docx)
DevOps_Ontology_JIISIC 2021 BLIND.docx	29/05/21, 8:26 p. m.	251 KB	Microsoft Word document (.docx)
DevOps_Ontology_JIISIC 2021 2.docx	28/05/21, 4:48 p. m.	250 KB	Microsoft Word document (.docx)
DevOps_Ontology_JIISIC 2021 FINAL.docx	29/05/21, 7:05 p. m.	253 KB	Microsoft Word document (.docx)
DevOps_Ontology_JIISIC 2021.docx	27/05/21, 8:40 p. m.	250 KB	Microsoft Word document (.docx)
DevOps_Ontology_V14.docm	26/05/21, 8:54 p. m.	302 KB	Microsoft Word Macro-Enabled document (.docm)
Documentación de la maestría	15/12/20, 11:33 a. m.	957 bytes	Documento RTF
GQM Prácticas.docx	26/04/21, 9:11 a. m.	27 KB	Microsoft Word document (.docx)
Informe de Resultados Estudio de Caso.docx	1/06/21, 4:05 p. m.	1,3 MB	Microsoft Word document (.docx)
Informe Estudio de Caso.docx	12/05/21, 6:14 p. m.	2,4 MB	Microsoft Word document (.docx)
Informe Grupo Focal	5/02/21, 11:37 a. m.	1,8 MB	Microsoft Word document (.docx)
Informe Grupo Focal2	4/02/21, 7:23 p. m.	1,4 MB	Microsoft Word document (.docx)
Informe Grupo Focal3.docx	5/02/21, 2:50 p. m.	1,8 MB	Microsoft Word document (.docx)
Informe Grupo FocalF.docx	12/02/21, 8:07 a. m.	1,8 MB	Microsoft Word document (.docx)
Invitación Focus group	2/02/21, 2:58 p. m.	13 KB	Microsoft Word document (.docx)
Modelo de referencia	19/11/20, 2:27 p. m.	19 KB	Microsoft Word document (.docx)
Modelo de referencia - Borrador	17/12/20, 5:14 p. m.	1,5 MB	Microsoft Word document (.docx)

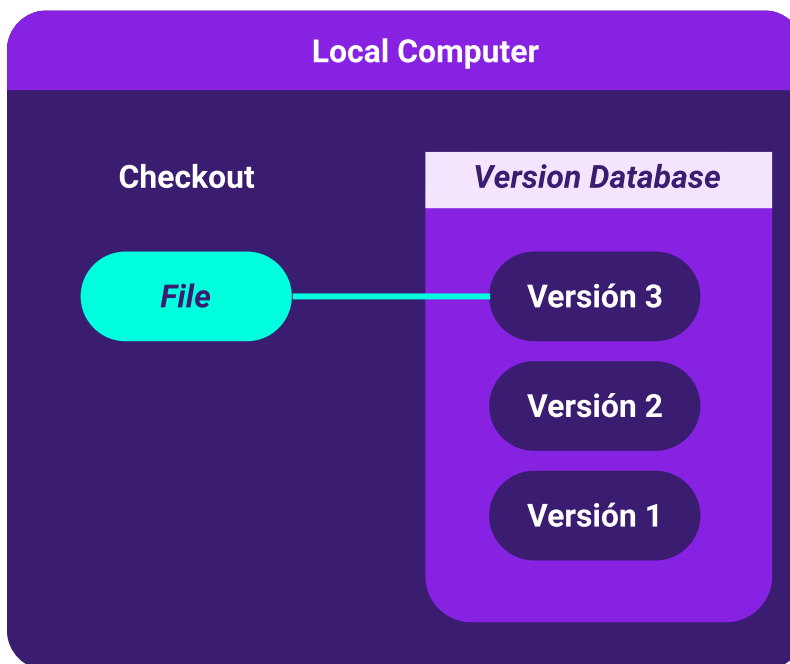
Sistemas de control de versiones locales

Lo anterior, aunque puede ser una práctica común y fácil de implementar, resulta muy riesgoso, ya que la posibilidad de introducir errores, olvidar la ubicación del directorio, sobrescribir por accidente un archivo o nombrar incorrectamente un archivo es muy alta.

Para solucionar este problema, surgieron hace un tiempo los sistemas de control de versiones locales, que contienen una base de datos simple donde se registra

información de todos los cambios realizados sobre los archivos, como se ilustra en la siguiente figura.

Figura 3. Esquema de control de versiones en un computador local

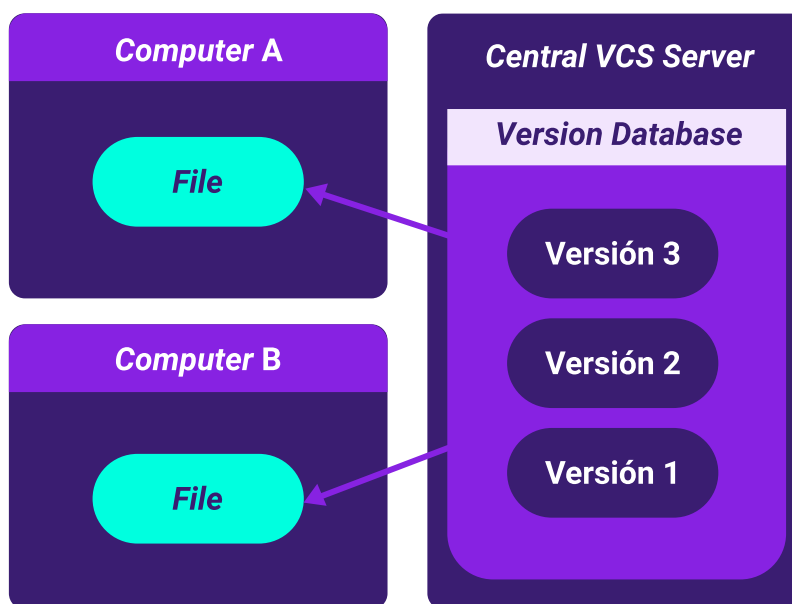


Sistemas de control de versiones centralizados

Este tipo de sistemas surge para solucionar los problemas asociados a la colaboración en grupo. Bajo el sistema de control de versiones locales, la única forma de colaborar en equipo era mediante un proceso de sincronización manual. Los miembros de un equipo debían sacar copias de los archivos procesados localmente y enviarlas a sus compañeros, quienes, manualmente, debían integrar sus propios cambios y luego repetir el proceso de regreso.

En los sistemas de control de versiones centralizados, se dispone de un servidor donde se almacenan todos los archivos y a partir del cual varios clientes pueden realizar descargas, como se ilustra en la siguiente figura.

Figura 4. Ejemplo sistemas de control de versiones centralizados



Este tipo de sistemas de control de versiones ha sido ampliamente utilizado en la industria del software, ya que facilitan el trabajo colaborativo, permiten un control detallado por parte de los administradores y, son mucho más fáciles de gestionar en comparación con los procesos de sincronización manual que debían realizarse en sistemas de control de versiones locales al trabajar en equipo. Sin embargo, al tratarse de sistemas centralizados, existe una dependencia muy peligrosa del equipo central. Si esta falla por alguna razón bloqueará automáticamente a todos los usuarios y, en casos más extremos, se puede perder todo si el equipo central sufre un fallo irreparable, como fallas en los discos.

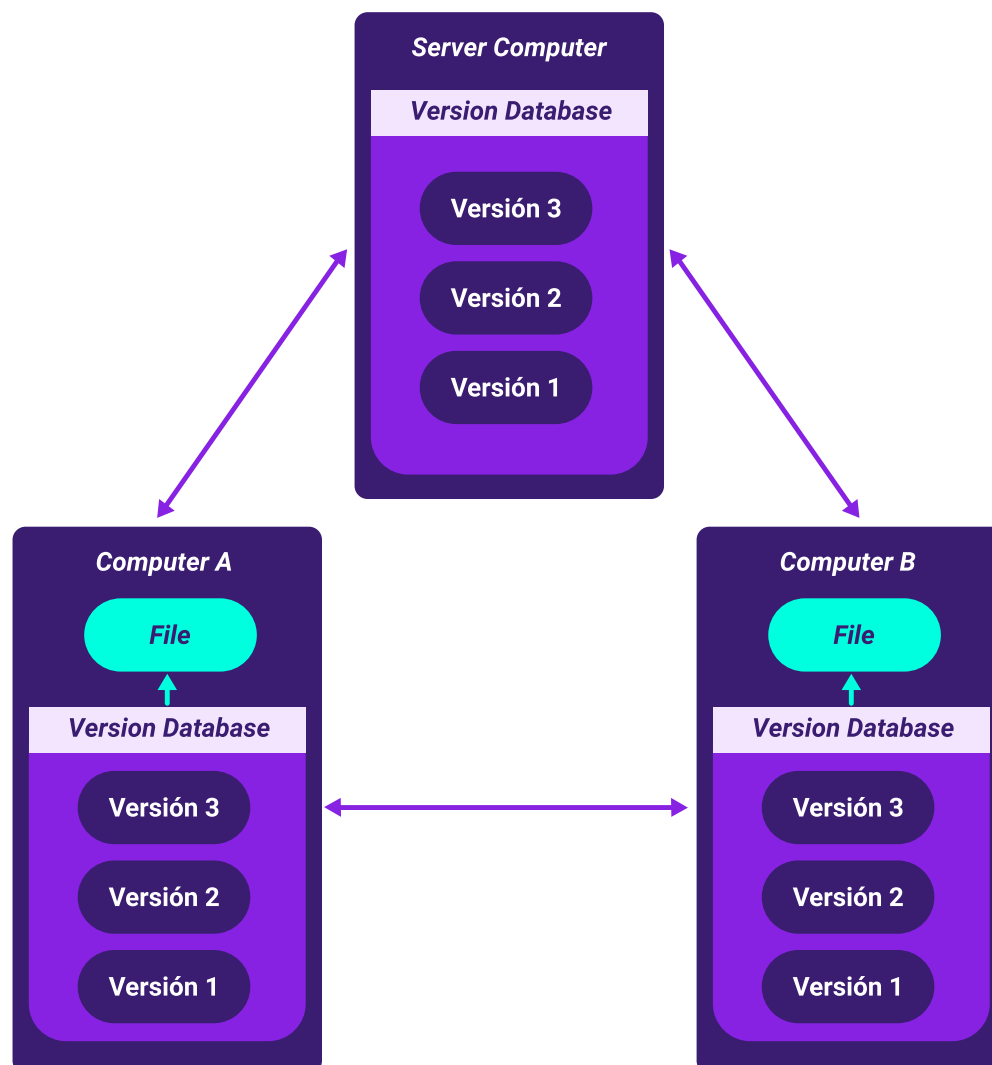
Sistemas de control de versiones distribuidas

En este tipo de sistemas, los clientes, además de descargar una copia actual de los archivos presentes en el repositorio, crean una réplica completa del mismo. De esta forma, si por alguna razón el servidor falla, los repositorios presentes en los clientes

sirven para realizar el proceso de restauración. Es decir, en estos sistemas, los clientes se convierten en clones del repositorio en el servidor.

Este tipo de sistemas permite administrar múltiples repositorios remotos con los que los usuarios pueden trabajar simultáneamente con diferentes grupos de personas en varios proyectos, e incluso dentro del mismo proyecto y de distintas formas. Esto posibilita el establecimiento de varios flujos de trabajo que no son posibles de implementar en sistemas centralizados y mucho menos en sistemas locales.

Figura 5. Ejemplo sistemas de control de versiones distribuidas



2. Sistema de control de versiones – Git

Un sistema de control de versiones se encarga del registro de los cambios realizados a lo largo del tiempo en uno o un conjunto de archivos (versiones), de forma tal que estos puedan ser recuperados con precisión según las necesidades de los usuarios. A este tipo de procesos se le puede denominar versionamiento y, aunque estas actividades son comunes en archivos o código fuente en la industria del desarrollo de software, el proceso es aplicable a cualquier tipo de archivo.

El proceso de hacer copias de seguridad y crear versiones de los archivos considerados importantes es una actividad muy común.

- Velocidad.
- Diseño sencillo.
- Soporte para múltiples desarrollos no lineales.
- Sistema totalmente distribuido.
- Capacidad de manejar eficientemente proyectos de gran envergadura.

Git, a diferencia de otros sistemas de control de versiones, posee un sistema de almacenamiento de información en el que cada versión tiene una copia de instantáneas en miniatura. Es decir, ofrece la posibilidad de trabajo incluso cuando no se tiene conectividad al servidor central, ya que localmente se posee toda la información requerida, incluso si se desea hacer procesos de recuperación a versiones anteriores.

Git realiza un proceso de comprobación antes de almacenar datos mediante algoritmos con códigos criptográficos SHA-1, que se calculan a partir del contenido y la estructura del directorio del proyecto. Esto hace imposible cambiar el contenido de algún archivo o la estructura de directorios sin que Git lo detecte.

Debido a que Git solo añade información a la base de datos con cualquier acción realizada, es difícil llegar a un estado en el que este sistema de control de versiones no pueda realizar un proceso de recuperación o que se elimine información de alguna manera una vez exista una confirmación en los cambios.

Según Git (2021), el sistema de control de versiones Git maneja tres estados principales para cada uno de los archivos de un proyecto, los cuales son:

- **Confirmado (committed)**

Indica que los datos se encuentran almacenados de forma segura en la base de datos local. Se considera una versión concreta.

- **Preparado (staged)**

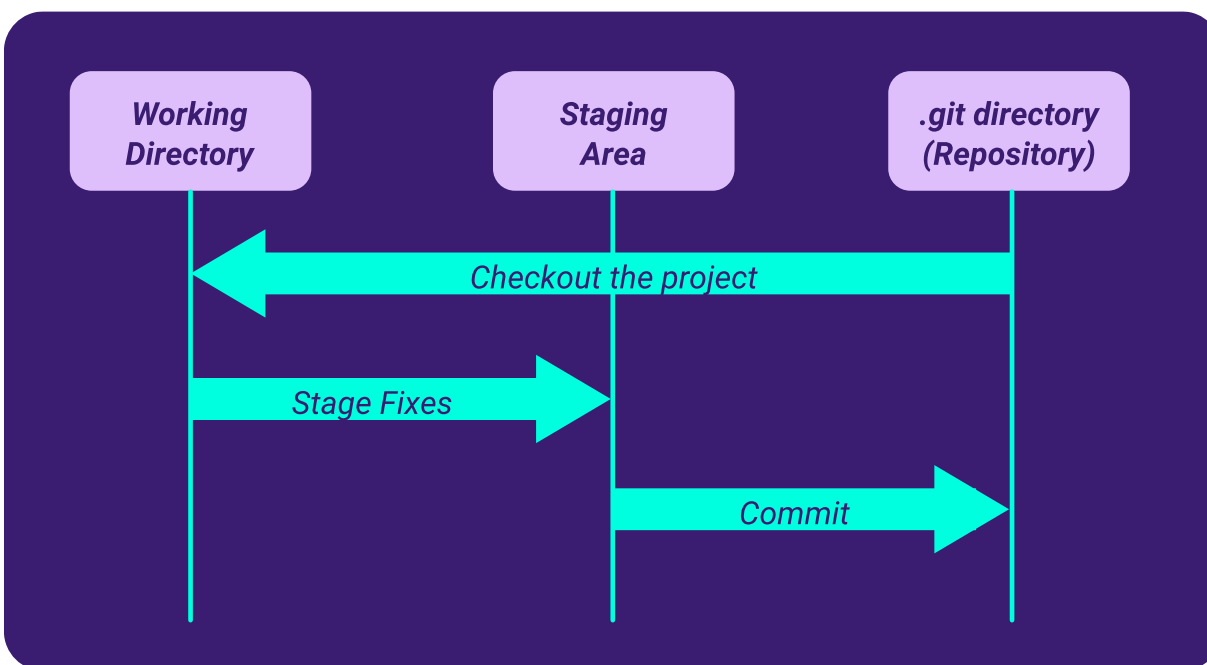
Indica archivo modificado que ha sido marcado en su versión actual para una próxima confirmación.

- **Modificado (modified)**

Indica que el archivo ha sufrido alguna modificación, pero esta aún no se ha preparado ni confirmado en la base de datos.

En la siguiente figura se puede identificar la interacción que ocurre entre el directorio de trabajo, el área de preparación y el directorio Git. El directorio de trabajo es donde se desarrollan las labores de construcción del proyecto y corresponde a una copia de la versión del proyecto. El área de preparación es un archivo que almacena información de lo que va en la próxima confirmación. El directorio Git es donde se almacenan los metadatos y la base de datos de objetos para el proyecto. Este directorio Git es lo que se copia en el proceso de clonación del repositorio desde otro equipo de trabajo.

Figura 6. Ejemplo sistemas de control de versiones centralizados



Teniendo en cuenta lo anterior, un flujo normal de acciones para trabajar con Git sería el siguiente:

- Realizar acciones de construcción del proyecto sobre el directorio de trabajo.
- Realizar la preparación de los archivos, los cuales se marcaron para ser añadidos al área de preparación.
- Confirmar los cambios, lo cual toma los archivos del área de preparación y los almacena como una copia instantánea permanente en el directorio Git.

Git se puede instalar en cualquier tipo y distribución de sistema operativo e incluso se puede integrar con diferentes IDE para la conexión con varios repositorios locales y en la nube. Para un mejor entendimiento de los comandos más comúnmente

utilizados, se optará por la instalación común y la manipulación de este por consola de comandos.

Instalación de Git en sistemas operativos

La forma más sencilla de instalar Git en sistemas operativos Windows y Linux es por medio del sitio oficial disponible.

Instalación de Git en sistemas operativos Windows

Ingrese al paso a paso del proceso de instalación para este sistema, lo invitamos a consultar el PDF "**Instalación de Git en sistemas operativos Windows**", el cual se encuentra en la carpeta Anexos.

Instalación de Git distribuciones de Linux

Ingrese al paso a paso del proceso de instalación para este sistema, lo invitamos a consultar el PDF "**Instalación de Git distribuciones de Linux**", el cual se encuentra en la carpeta Anexos.

3. Comandos básicos de Git

Para trabajar con Git es importante conocer los diferentes comandos que posibilitan la creación de repositorios, a continuación, se podrá acceder a ellos a través del siguiente recurso:

- **Inicializar un repositorio**

Para inicializar un repositorio desde un directorio existente, se debe usar el comando: `$ git init`.

- **Crear subdirectorio .git**

Este comando creará un nuevo subdirectorio llamado `.git`, el cual contendrá todos los archivos necesarios para el repositorio. Si se desea obtener una copia desde un repositorio existente, se debe utilizar el comando de clonación: `$ git clone https://url\_del\_repositorio`

- **Clonación del repositorio**

Este comando permite recibir una copia de casi todos los archivos existentes en el repositorio de origen, incluyendo cada versión de cada uno de los archivos. Así, si el servidor original llegara a dañarse, se podrá realizar una restauración a partir de todos los clones generados.

- **Estados de los archivos**

Una vez se dispone de un espacio de trabajo inicializado, todos los archivos van a tener dos posibles estados: Rastreados (tracked files): Son todos aquellos archivos que estaban en la última versión del proyecto, independientemente de si estos archivos están modificados, sin modificar o preparados. Cuando se clona un proyecto a partir de un repositorio existente, todos estos archivos pertenecen al estado rastreado y sin

modificar. Una vez un archivo es editado, pasa al estado modificado; luego se debe pasar al estado de preparación para finalmente confirmarlos. Sin rastrear: Corresponden a todos aquellos archivos que no formaban parte de la última versión del proyecto y no están en el área de preparación (staging area).

- **Visualizar el estado**

Para visualizar el estado actual de tus archivos indicando si están o no rastreados por Git, se puede ejecutar el siguiente comando: `$ git status`.

- **Rastrear archivos nuevos**

Para todos los archivos nuevos que se desean ser rastreados por Git, se debe indicar con el siguiente comando: `$ git add Nombre_archivo`.

- **Rastrear directorios**

También es posible indicar a Git que haga rastreo de un directorio, lo cual implica que recursivamente se hace rastreo de todos los archivos en el interior del directorio: `$ git add Directorio`.

- **Preparar archivos**

El comando `git add` además de servir para iniciar el rastreo de un archivo o directorio que no estaba en la última versión, también sirve para preparar archivos (enviarlos al staging area) y para marcar archivos como resueltos en los casos donde estos entran en conflictos por operaciones de combinación. Dicho de una manera más simplificada: el comando `add` indica que un archivo o directorio debe ser añadido en la próxima confirmación.

- **Confirmar cambios**

Para confirmar cambios, se ejecuta el comando: `$ git commit`.

- **Mensaje de confirmación**

Al ejecutar la confirmación, el sistema desplegará un mecanismo para recibir un mensaje de confirmación que será asociado a esta operación de commit. También es posible agregar el comentario explícitamente en la ejecución de la confirmación usando la opción `-m`: `$ git commit -m "En esta versión se arregló el archivo W"`.

- **Confirmación automática**

También es posible ejecutar una operación de confirmación que salte el paso de preparación que se logra con la ejecución del comando `add`. Es decir, la operación de confirmación se encarga de preparar todos los archivos rastreados y luego confirmar. Esto es posible agregando la opción `-a`: `$ git commit -a -m 'comentario de esta confirmación'`.

- **Visualizar el historial**

Para visualizar el histórico de las confirmaciones desde la más reciente hasta la más antigua realizadas sobre un repositorio, se ejecuta el comando: `$ git log`.

4. Git en entornos remotos

Para poder colaborar con un equipo de trabajo e implementar el concepto de integración continua, es necesario hospedar los proyectos en repositorios remotos, los cuales son accesibles por medio de internet o por un entorno de red.

En estos repositorios remotos se definen una serie de permisos que indican si se tiene la posibilidad de solo lectura o de lectura y escritura. Colaborar implica la gestión de estos repositorios, enviando y obteniendo información desde estos, teniendo en cuenta los estados que ya se han mencionado anteriormente.

A continuación, se relacionan los pasos a seguir para los entornos remotos.

- **Ver repositorios remotos**

Para ver los repositorios remotos configurados, se puede ejecutar el comando: `$ git remote`.

- **Definir un repositorio remoto**

Para definir un repositorio remoto y asociarlo a un nombre para su referenciación, se utiliza el siguiente comando: `git remote add [nombre-remoto] [url]`.

- **Ejemplo de definición**

Donde nombre-remoto corresponde al nombre con que se referencia el repositorio y URL es la ubicación lógica del mismo en un entorno de red o en una dirección de internet. En el siguiente ejemplo se define un repositorio ubicado en un servidor de Github y cuyo nombre de referencia es ref: `$ git remote add ref https://github.com/paulboone/ticgit`.

- **Extraer datos del remote**

Una vez definido el remote, se pueden extraer los datos utilizando el siguiente comando: `$ git fetch [nombre-remoto]`.

- **Conexión al repositorio remoto**

El comando anterior se conecta al repositorio remoto y trae todos los datos de los cuales aún no se tiene copia en tu repositorio local. Para enviar información desde el repositorio local hacia el servidor remoto, se utiliza el siguiente comando: `git push [nombre-remoto] [nombre-rama]`.

- **Enviar información**

Recordemos que, si se ha clonado un repositorio desde alguna ubicación, Git asigna el nombre origin al servidor del que se ha realizado la clonación. Así, si por ejemplo queremos enviar nuestra rama master al servidor origin, se debe ejecutar el comando de la siguiente forma: `$ git push origin master`.

5. Plataformas para implementar integración continua – Gitlab

En la actualidad, existe una gran cantidad de plataformas con servicios en la nube donde se pueden implementar procesos de integración continua. Estas plataformas generalmente ofrecen la posibilidad de crear repositorios públicos y privados, se integran con sistemas de control de versiones, permiten el registro de grupos de trabajo, y muchas otras funcionalidades. Entre las más utilizadas en la industria, basadas en sistemas de control de versiones en Git, se encuentran las descritas a continuación:

- **GitHub**

Conoce GitHub, un portal creado para alojar códigos de las aplicaciones.

Enlace web: <https://github.com/>

- **GitLab**

Conoce sobre GitLab, un servicio web que permite controlar las versiones y desarrollos de software colaborativo basado en Git.

Enlace web: <https://about.gitlab.com/>

- **BitBucket**

Conoce a Bitbucket, un servicio de alojamiento basado en web para los proyectos que utilizan sistema de control de versiones Mercurial y Git.

Enlace web: <https://bitbucket.org/>

Para complementar la información sobre los anteriores sistemas, a continuación, se presentarán las diferentes versiones gratuitas con sus funcionalidades básicas, las cuales son accesibles simplemente con la creación de una cuenta de usuario.

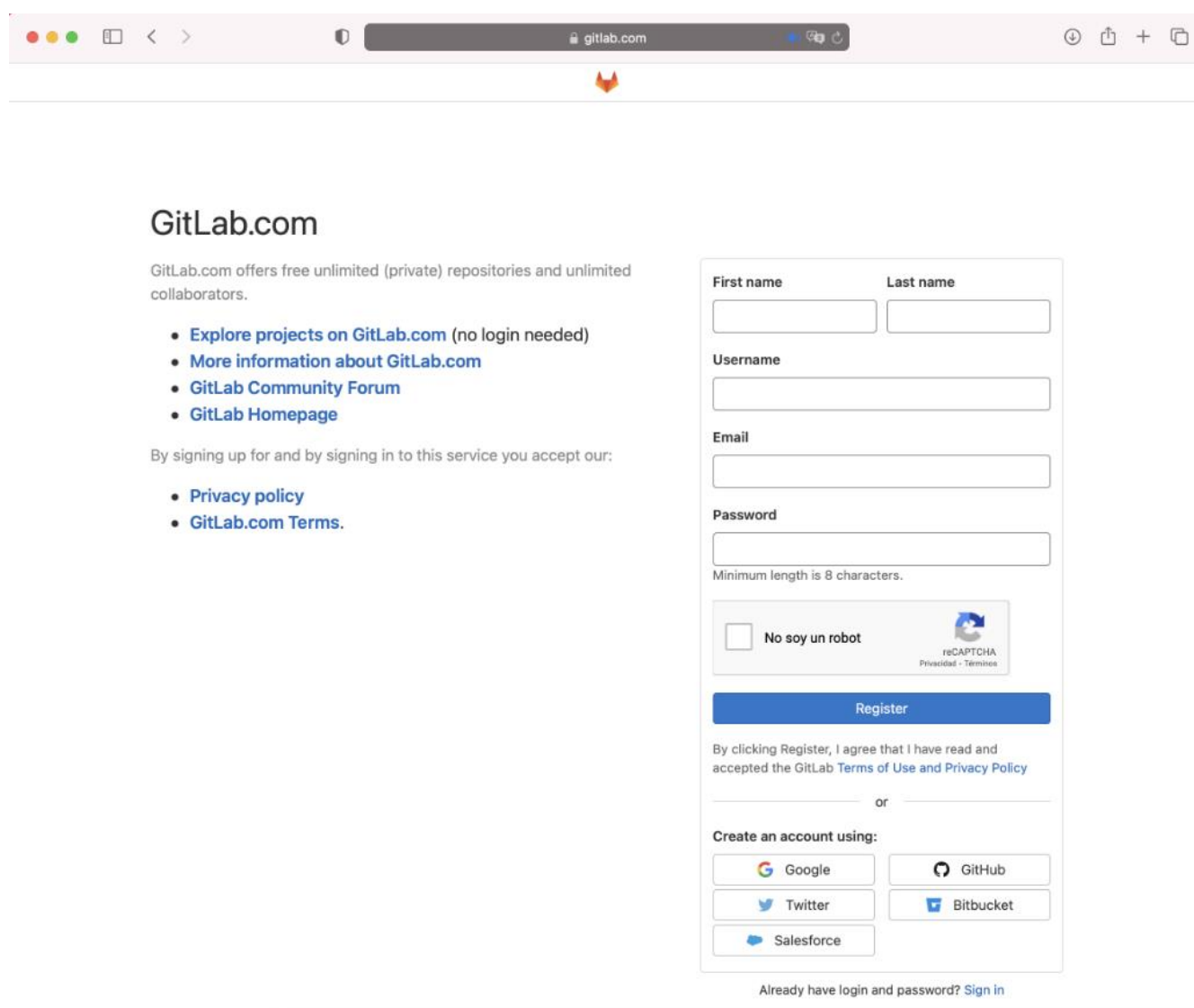
Creación de cuenta

Primero, se debe crear una cuenta en Gitlab directamente desde el enlace:

https://gitlab.com/users/sign_up

El proceso se puede realizar de forma tradicional o usando cuentas ya existentes de otras plataformas como Google, Github, Twitter, Bitbucket o Salesforce.

Figura 7. Página de registro de GitLab



The screenshot shows the GitLab.com registration page in a web browser. The browser's address bar displays 'gitlab.com'. The page features the GitLab logo (a red fox head) and the text 'GitLab.com'. Below this, it states: 'GitLab.com offers free unlimited (private) repositories and unlimited collaborators.' A list of links is provided: 'Explore projects on GitLab.com (no login needed)', 'More information about GitLab.com', 'GitLab Community Forum', and 'GitLab Homepage'. A note says: 'By signing up for and by signing in to this service you accept our:' followed by links to 'Privacy policy' and 'GitLab.com Terms.'.

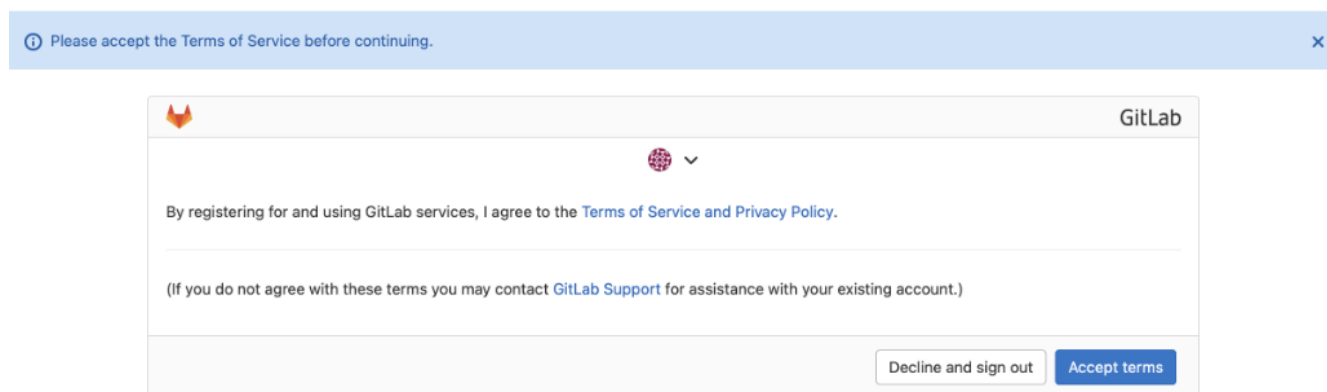
The registration form on the right includes the following fields and elements:

- First name** and **Last name** input fields.
- Username** input field.
- Email** input field.
- Password** input field with a note: 'Minimum length is 8 characters.'
- A checkbox labeled 'No soy un robot' next to a reCAPTCHA logo and the text 'reCAPTCHA Privacidad - Términos'.
- A blue **Register** button.
- A statement: 'By clicking Register, I agree that I have read and accepted the GitLab Terms of Use and Privacy Policy'.
- The word 'or'.
- A section titled 'Create an account using:' with buttons for Google, GitHub, Twitter, Bitbucket, and Salesforce.
- A link at the bottom: 'Already have login and password? Sign in'.

Aceptación de términos de servicios

Una vez se realiza el registro, se deben aceptar los términos del servicio y las políticas de privacidad.

Figura 8. Aceptación de términos y condiciones



Please accept the Terms of Service before continuing.

GitLab

By registering for and using GitLab services, I agree to the [Terms of Service and Privacy Policy](#).

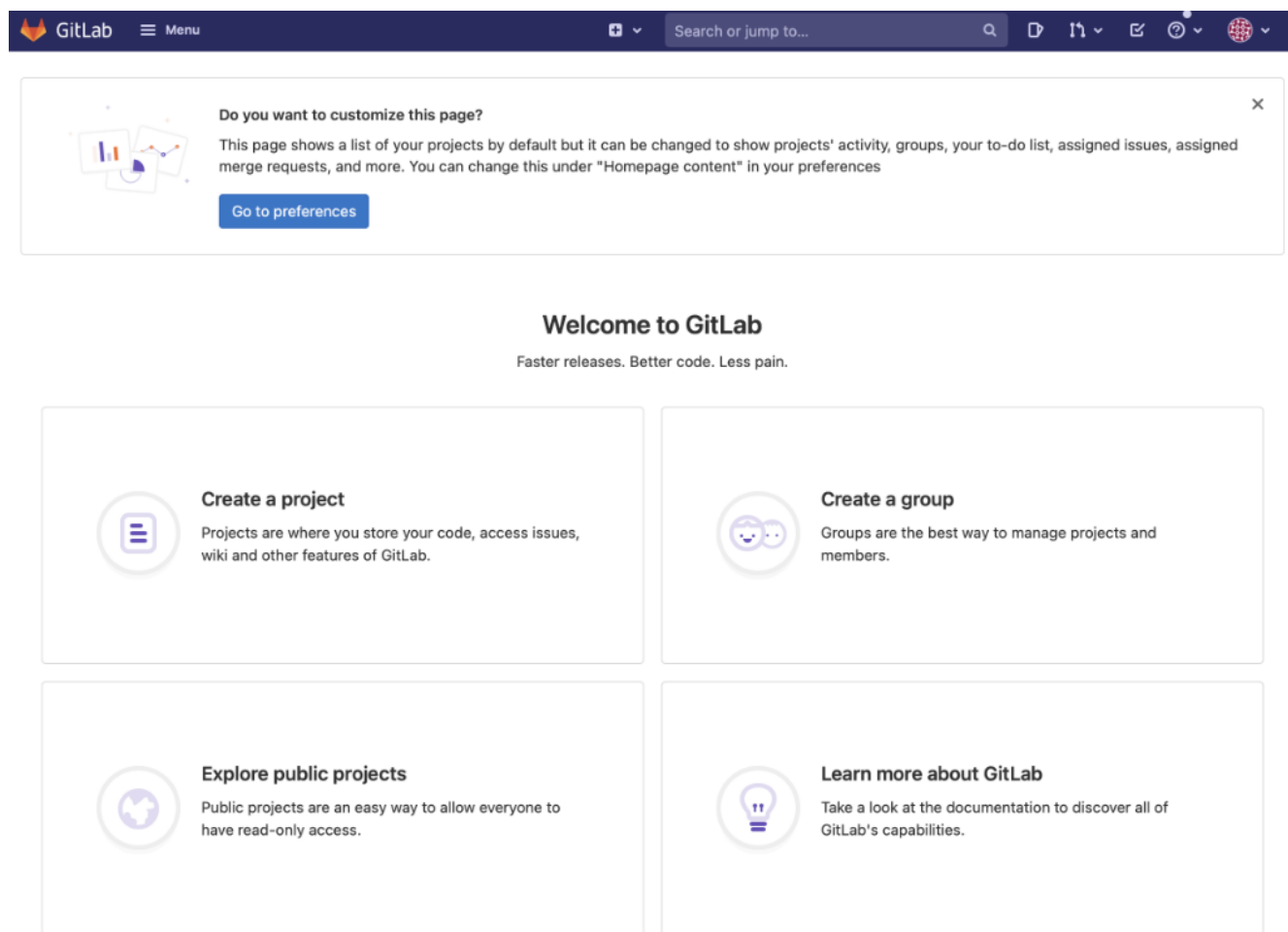
(If you do not agree with these terms you may contact [GitLab Support](#) for assistance with your existing account.)

Decline and sign out Accept terms

Interfaz de inicio

Después de esto, se muestra la interfaz de inicio donde se puede crear directamente el proyecto (repositorio).

Figura 9. Página de bienvenida de GitLab



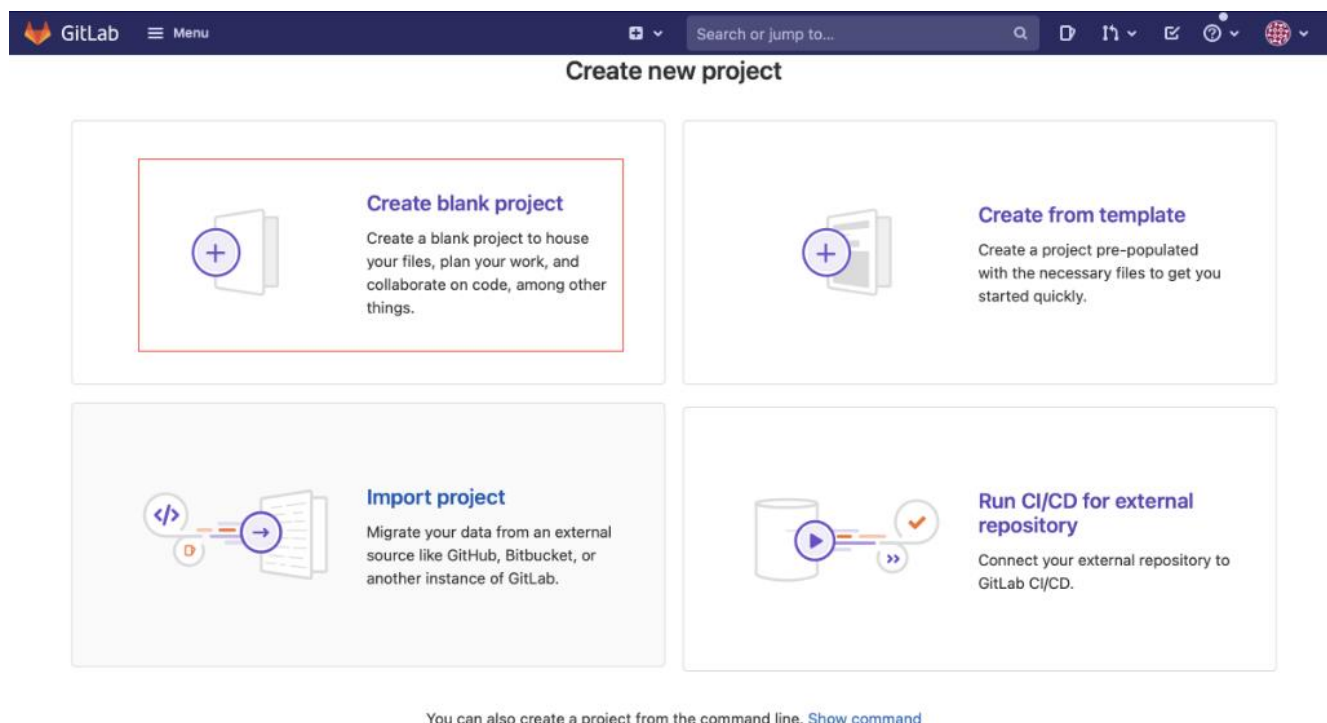
a) Creación de proyecto

En la parte de creación de proyectos, se ofrecen cuatro posibilidades:

- Creación de proyecto en blanco.
- Creación de proyecto desde plantilla.
- Importación de proyectos desde otros sistemas.
- Conexión de repositorios externos con GitLab CI/CD.

Para los grupos de desarrollo que apenas inician con prácticas de integración continua, el proyecto en blanco es la mejor opción.

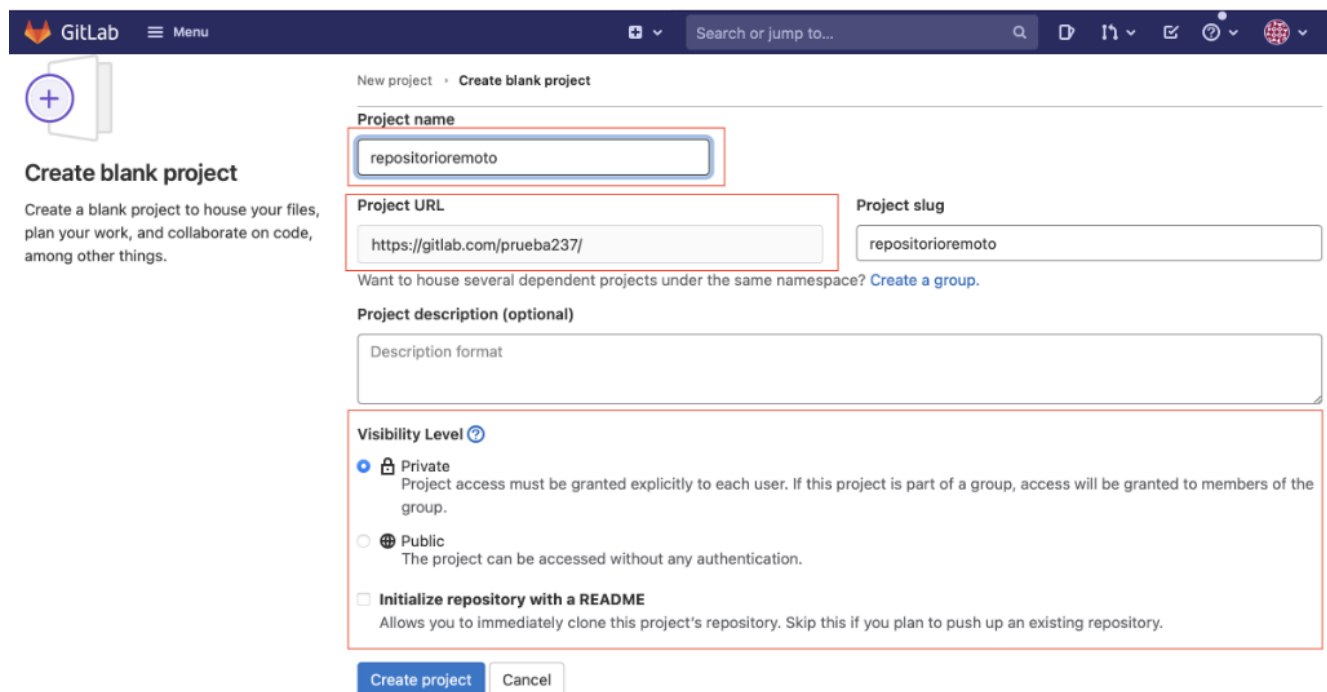
Figura 10. Creación de un nuevo proyecto



b) Nombre del proyecto

A continuación, se debe definir el nombre del proyecto, agregar una descripción y definir el tipo de visibilidad, que puede ser pública (cualquiera podría acceder sin autenticarse) o privada (se deben tener permisos asignados para ingresar). La URL asignada dependerá del nombre de usuario asignado a la cuenta de Gitlab creada y del nombre definido al repositorio.

Figura 11. Creación de un proyecto en blanco



Create blank project

Create a blank project to house your files, plan your work, and collaborate on code, among other things.

New project › Create blank project

Project name
repositorioremoto

Project URL
https://gitlab.com/prueba237/

Project slug
repositorioremoto

Want to house several dependent projects under the same namespace? [Create a group.](#)

Project description (optional)
Description format

Visibility Level ⓘ

☒ **Private**
Project access must be granted explicitly to each user. If this project is part of a group, access will be granted to members of the group.

☐ **Public**
The project can be accessed without any authentication.

☐ **Initialize repository with a README**
Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.

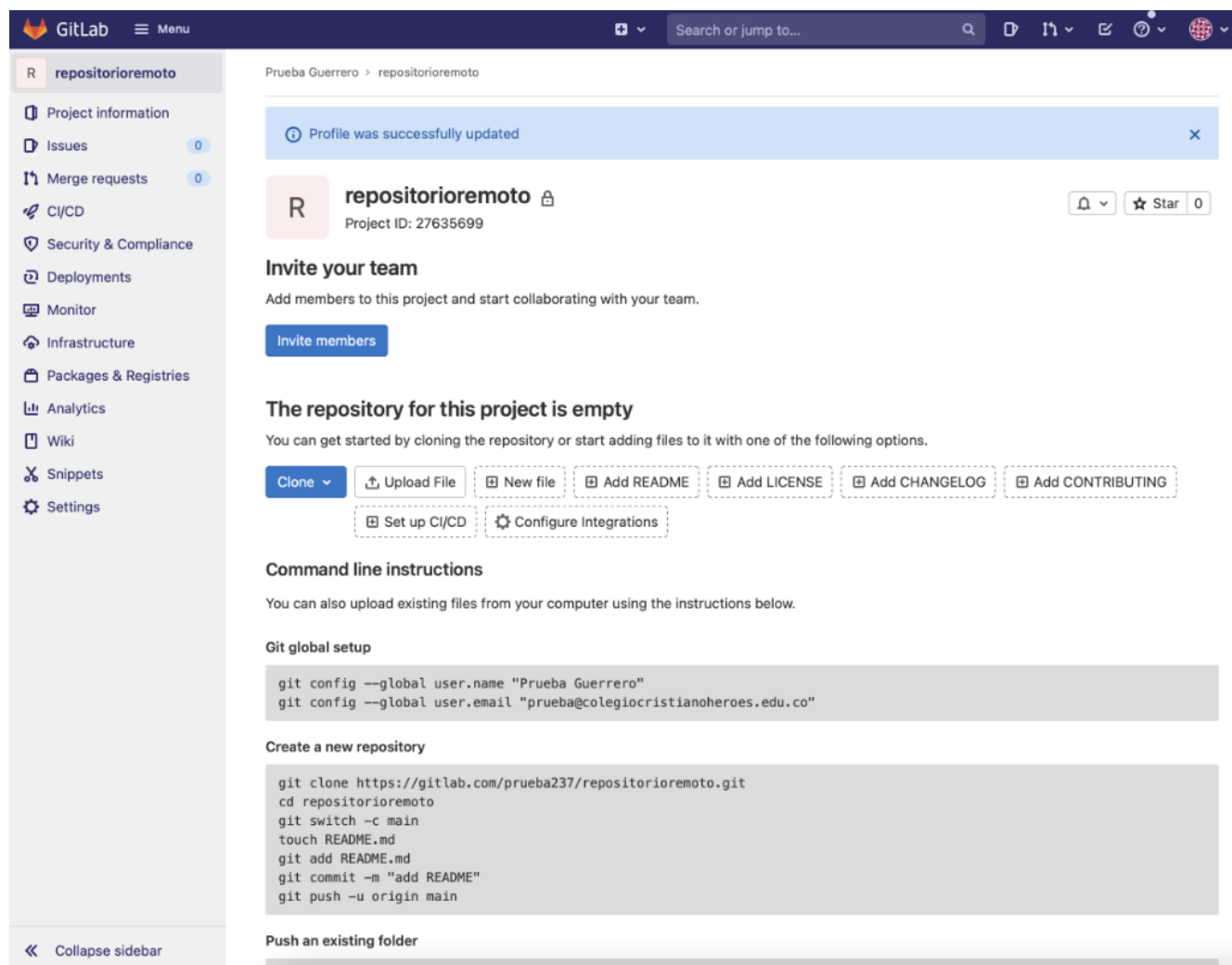
[Create project](#) [Cancel](#)

c) Instrucción de uso

Una vez finalizado el proceso, el sistema proporciona un conjunto de instrucciones para usar el repositorio, entre las cuales se encuentra la configuración del usuario y correo de acuerdo con las credenciales de Gitlab y los comandos Git para hacer su respectiva gestión.

A continuación, lo único que queda por hacer es conectar el equipo local con el repositorio remoto por medio de Git usando los comandos mencionados anteriormente. En este caso, se usará la ayuda ofrecida por Gitlab para crear un nuevo repositorio local, realizando la clonación respectiva desde el repositorio remoto.

Figura 12. Repositorio vacío



d) Modificación de usuario

En primer lugar, se debe modificar el usuario y la cuenta de correo en la máquina local para que coincida con las credenciales autorizadas en el repositorio remoto.

```
root@ubuntu-VirtualBox:/home/ubuntuuser# git config --global user.name "Prueba Guerrero"
```

```
root@ubuntu-VirtualBox:/home/ubuntuuser# git config --global user.email "prueba@colegiocristianoheroes.edu.co"
```

```
root@ubuntu-VirtualBox:/home/ubuntuser#
```

e) Clonación de repositorio

Luego, se clona el repositorio creado en Gitlab para copiarlo en la máquina local. En este punto, se debe tener claridad sobre el usuario y su contraseña en Gitlab, ya que esta información será requerida durante el proceso de clonación. Una vez se verifican las credenciales, se realiza el proceso de clonación. En este caso, es un repositorio vacío, pero se encontrará en el sistema local de archivos una carpeta con el mismo nombre definido en el repositorio remoto, con toda la configuración necesaria para la gestión de estados con Git. Este repositorio se convierte en el directorio de trabajo local.

```
root@ubuntu-VirtualBox:/home/ubuntuser# git clone
```

```
https://gitlab.com/prueba237/repositorioremoto.git
```

```
Clonando en 'repositorioremoto'...
```

```
Username for 'https://gitlab.com': prueba237
```

```
Password for 'https://prueba237@gitlab.com':
```

```
warning: Pareces haber clonado un repositorio sin contenido.
```

```
root@ubuntu-VirtualBox:/home/ubuntuser#
```

f) Acceso al directorio

Ahora, se accede al directorio de trabajo para empezar la construcción de archivos, ponerlos en rastreo, prepararlos, confirmarlos localmente y luego enviar la versión confirmada localmente al repositorio remoto. Se debe utilizar el comando `git switch -c nombre` para crear una nueva rama de trabajo local, donde "nombre" corresponde al nombre de la rama.

g) Creación de archivo local

Después de esto, se probará crear un archivo local vacío llamado README.md, el cual se preparará, confirmará y enviará al repositorio remoto.

```
root@ubuntu-VirtualBox:/home/ubuntuser/repositorioemoto# touch  
README.md
```

h) Archivo de rastreo

Luego, se agrega este archivo para que sea rastreado y agregado al área de preparación.

```
root@ubuntu-VirtualBox:/home/ubuntuser/repositorioemoto# git add  
README.md
```

i) Archivo de repositorio local

Se confirma el archivo en el repositorio local, lo cual genera una nueva versión en el repositorio local.

```
root@ubuntu-VirtualBox:/home/ubuntuser/repositorioemoto# git  
commit -m "Agregado readme"
```

En la rama main

Tu rama está actualizada con 'origin/main'.

j) Envío al repositorio remoto

Luego de tener una versión confirmada, se envía al repositorio remoto. Se deberán ingresar las credenciales de usuario en Gitlab.

```
root@ubuntu-VirtualBox:/home/ubuntuser/repositorioremoto# git push -u  
origin main
```

```
Username for 'https://gitlab.com': prueba237
```

```
Password for 'https://prueba237@gitlab.com':
```

```
Enumerando objetos: 3, listo.
```

```
Contando objetos: 100% (3/3), listo.
```

```
Escribiendo objetos: 100% (3/3), 230 bytes | 57.00 KiB/s, listo.
```

```
Total 3 (delta 0), reusado 0 (delta 0)
```

```
To https://gitlab.com/prueba237/repositorioremoto.git
```

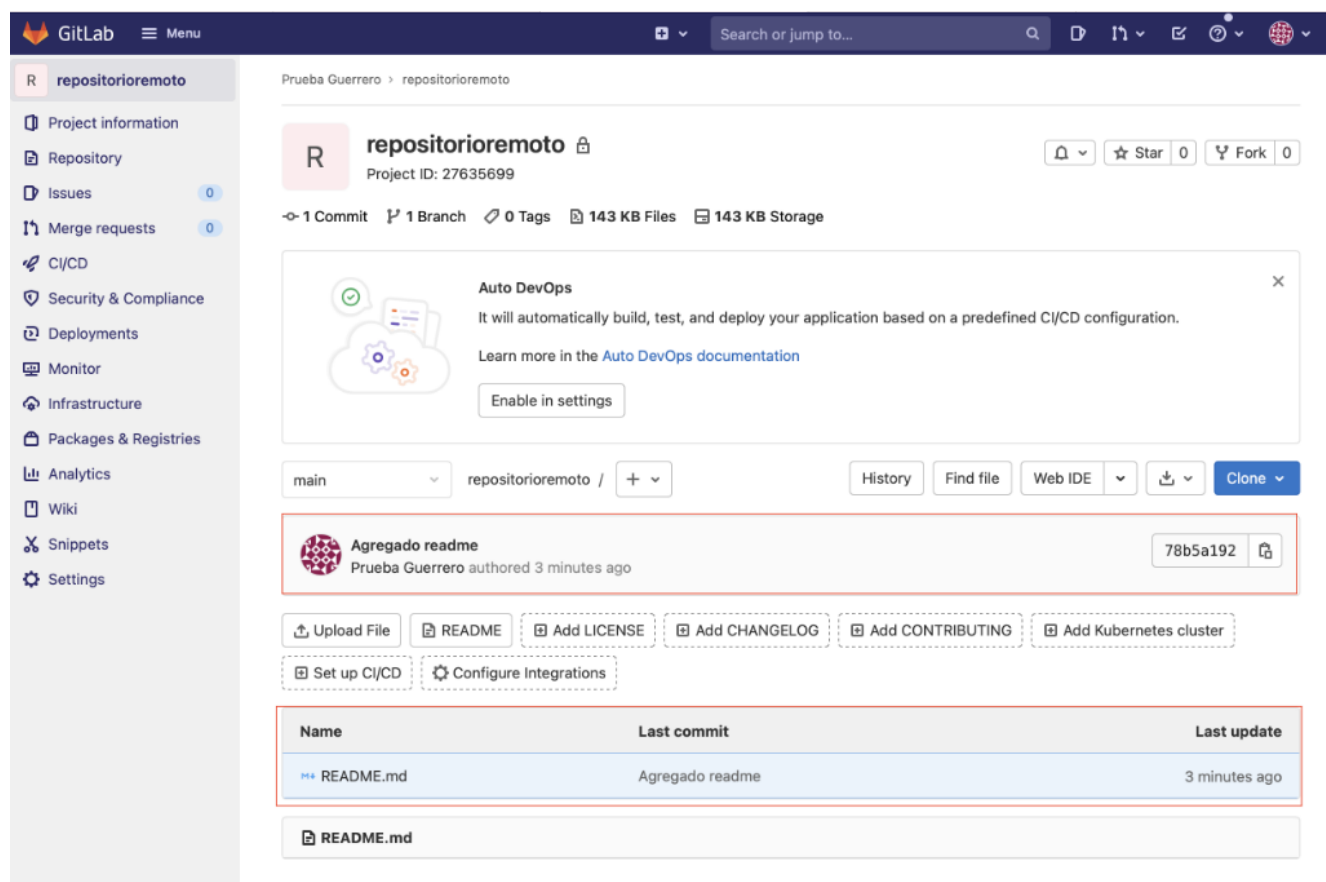
```
* [new branch]    main -> main
```

Rama 'main' configurada para hacer seguimiento a la rama remota 'main' de 'origin'.

k) Repositorio Gitlab

En este punto, los cambios aplicados ya se pueden visualizar directamente desde la plataforma Gitlab.

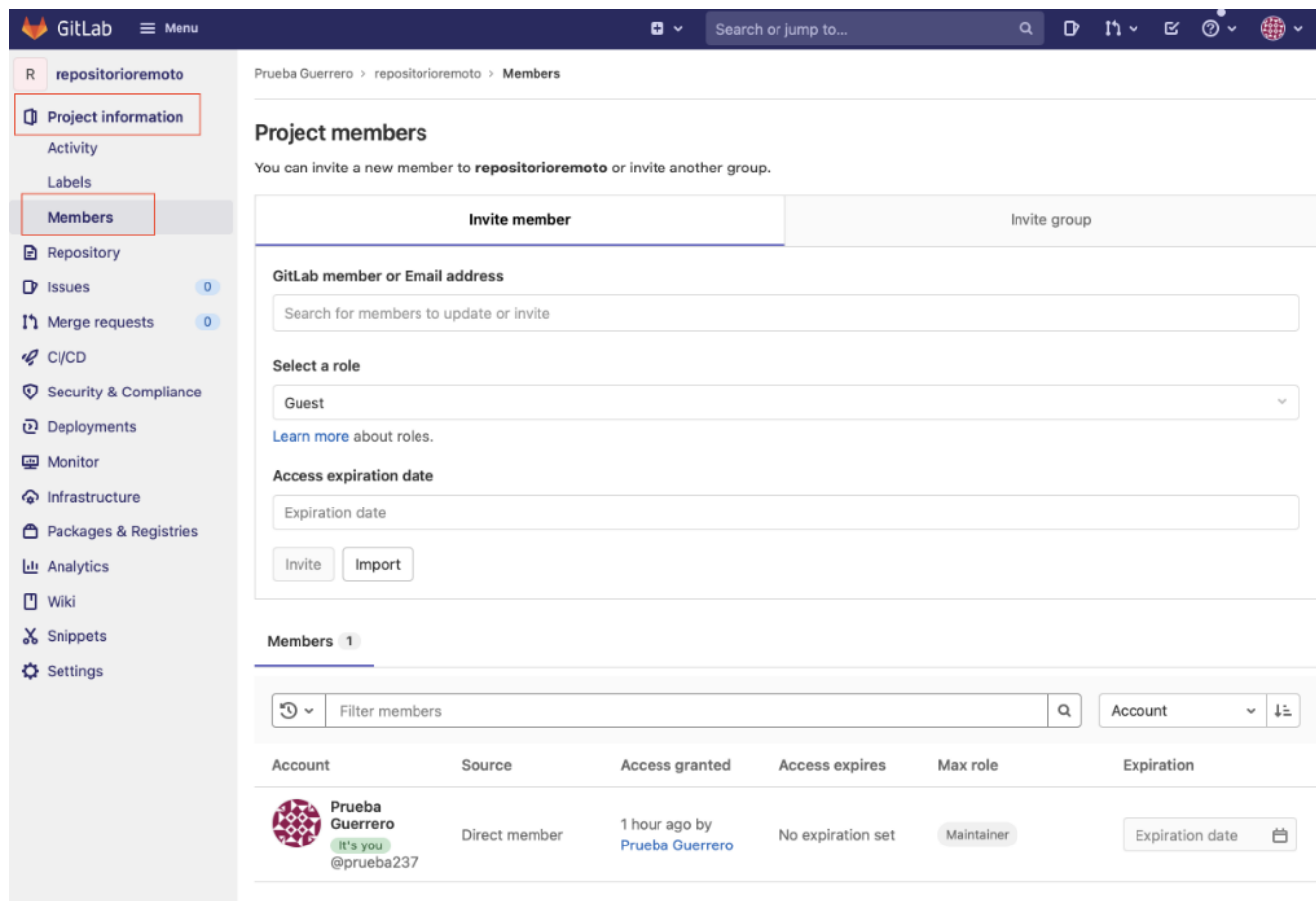
Figura 13. Vista del repositorio con archivo README



I) Agregar colaboradores

Para agregar colaboradores que también puedan clonar, preparar, confirmar y subir cambios al repositorio, es necesario agregarlos como colaboradores y asignarles los permisos requeridos, ya que el repositorio fue definido como privado. Para esto, se debe regresar a Gitlab y, en el menú del panel del lado izquierdo, seleccionar la opción de información del proyecto y luego miembros.

Figura 14. Gestión de miembros del proyecto



Project members

You can invite a new member to **repositorioremoto** or invite another group.

Invite member

Invite group

GitLab member or Email address

Search for members to update or invite

Select a role

Guest

[Learn more](#) about roles.

Access expiration date

Expiration date

Invite Import

Members 1

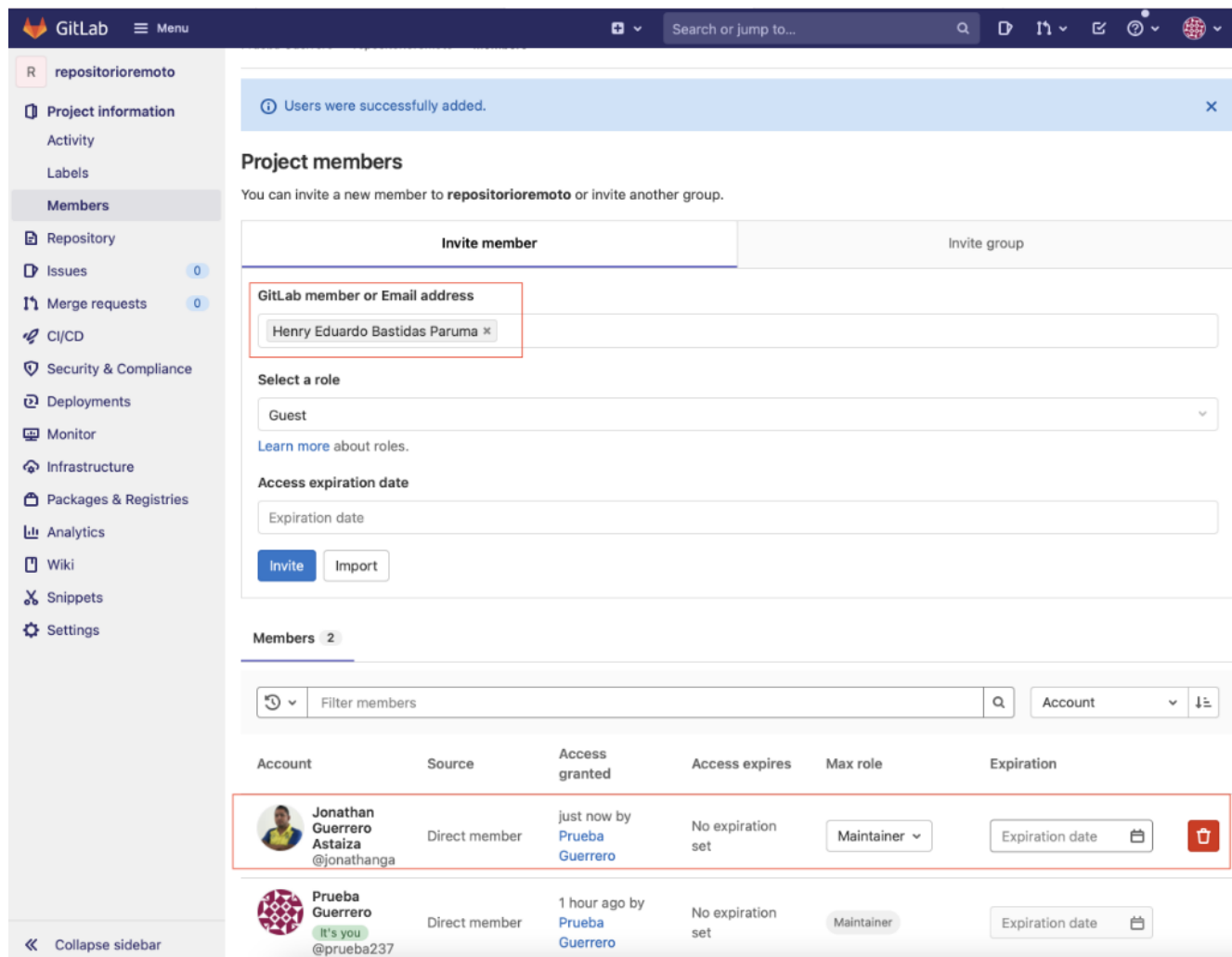
Account	Source	Access granted	Access expires	Max role	Expiration
 Prueba Guerrero It's you @prueba237	Direct member	1 hour ago by Prueba Guerrero	No expiration set	Maintainer	Expiration date 

m) Configuración de roles

Por defecto, el usuario creador del repositorio aparece como miembro de este. Mediante esta interfaz, se pueden invitar a nuevos miembros por correo electrónico o por nombre de usuario, si ya están registrados en Gitlab. Adicionalmente, se puede configurar el rol asociado a este usuario e incluso definir la fecha en la que expira la autorización asignada. Si se desea asignar permisos de lectura y escritura, se recomienda asignar el rol Maintainer. Usando este mismo espacio, se pueden revocar permisos y eliminar colaboradores del proyecto. Todos los usuarios autorizados

podrán realizar la clonación del repositorio e interactuar con él por medio de los comandos Git vistos a lo largo de este módulo.

Figura 15. Adición de un nuevo miembro al proyecto



Project members

You can invite a new member to **repositorioremoto** or invite another group.

Invite member

GitLab member or Email address

Henry Eduardo Bastidas Paruma ✕

Select a role

Guest

[Learn more](#) about roles.

Access expiration date

Expiration date

Invite Import

Invite group

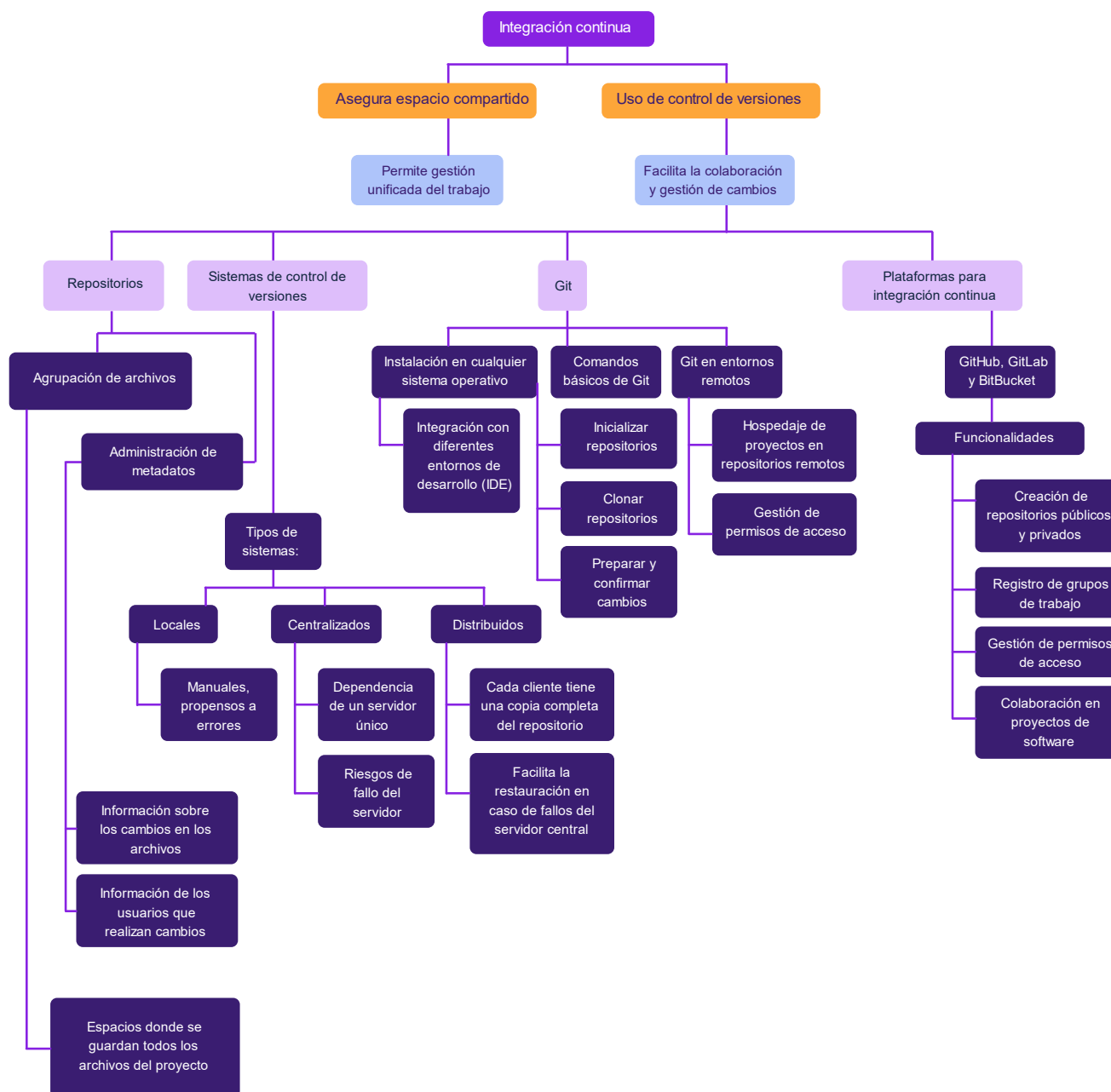
Members 2

Filter members

Account	Source	Access granted	Access expires	Max role	Expiration
 Jonathan Guerrero Astaiza @jonathanga	Direct member	just now by Prueba Guerrero	No expiration set	Maintainer	Expiration date  
 Prueba Guerrero It's you @prueba237	Direct member	1 hour ago by Prueba Guerrero	No expiration set	Maintainer	Expiration date 

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Comandos básicos de Git	Fazt. (2014). Git y Github – Curso práctico de Git y Github desde cero. [Video]. YouTube.	Video	https://youtu.be/HiXLkL42tMU
Comandos básicos de Git	GIT. (s.f.). Git - Download for Windows.	Software	http://git-scm.com/download/win

Glosario

BitBucket: servicio de alojamiento basado en web para proyectos que utilizan Mercurial y Git.

Comandos básicos de Git: comandos como git init, git clone, git add, git commit, git push para gestionar repositorios.

DevOps: metodología de desarrollo de software que integra el desarrollo (Dev) y las operaciones (Ops) para mejorar la colaboración y productividad mediante la automatización de procesos y la entrega continua de software.

Git: sistema de control de versiones distribuido que permite trabajar sin conexión y garantiza la integridad de datos.

GitHub: plataforma para alojar código de aplicaciones y colaborar en proyectos.

GitLab: plataforma web que permite el control de versiones y desarrollo colaborativo basado en Git.

Hospedaje de proyectos: proceso de alojar proyectos en repositorios remotos accesibles por internet.

Integración continua: práctica de DevOps que asegura la disponibilidad de un espacio compartido y gestión unificada del trabajo.

Repositorios: espacios donde se agrupan archivos y se administran metadatos relacionados con los cambios realizados.

Sistemas de control de versiones: sistemas que registran los cambios en archivos a lo largo del tiempo, permitiendo su recuperación.

Referencias bibliográficas

Atlassian. (s.f.). Qué es el control de versiones | Atlassian Git Tutorial.

<https://www.atlassian.com/es/git/tutorials/what-is-version-control>

Git. (2021) Git - Acerca del control de versiones. Git --Local-Branching-on-the-Cheap. <https://Git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Acerca-del-Control-de-Versiones>

Younes. (2021).Gitlab VS Github VS BitBucket. Which one deserve your time? DEV Community. <https://dev.to/yafkari/gitlab-vs-github-vs-bitbucket-which-one-deserve-your-time-2npm>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Jonathan Guerrero Astaiza	Experto Temático	Grupo De Apoyo Administrativo - Regional Distrito Capital
Paola Alexandra Moya Peralta	Evaluable Instruccionale	Centro de Servicios de Salud - Regional Antioquia
Andrés Felipe Herrera Roldán	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Luis Jesús Pérez Madariaga	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Jaime Hernán Tejada Llano	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Desarrollo de aplicaciones web con Java

Breve descripción:

El componente proporciona una guía sobre tecnologías y frameworks en el ecosistema Java. Cubre Java con JDBC para la conectividad de bases de datos, Servlets y JSP para el desarrollo web, React JS para interfaces de usuario, Hibernate para ORM, y Spring Framework para aplicaciones empresariales, destacando la inyección de dependencias y la configuración simplificada con Spring Boot.

Julio 2024

Tabla de contenido

Introducción	5
1. Java con JDBC.....	7
1.1. Clases e interfaces Java	8
1.2. Conexión a base de datos desde aplicación Java.....	10
2. Servlets y JSP	14
2.1. Introducción a servlets	17
Ciclo de vida de un Servlet	19
Ejemplo de Servlet	20
2.2. Formularios HTML con Servlets.....	23
2.3. HTTP Session	25
2.4. Elementos de JSP.....	28
2.5. MVC.....	31
3. Fundamentos de React JS	33
¿Qué es React JS?	34
¿Qué es JSX?	34
3.1. Instalando React JS.....	34
3.2. Crear un proyecto.....	43
3.3. Propiedades y estados de los componentes	48

Eventos en React	50
Listas y Keys	51
Formularios	53
Concepto de clases en React	54
Funciones Hooks.....	55
Ciclos de vida de los componentes de React	56
Ejecutar la aplicación.....	61
4. Framework Hibernate	64
4.1. Operaciones básicas	70
Método Save	72
Método Get	75
Método CreateQuery	78
Actualizaciones	82
Método Delete	86
4.2. Asociaciones.....	91
Relaciones OneToOne	91
Relaciones OneToMany - ManyToOne	93
Relaciones ManyToMany	94
4.3. Ciclo de vida	95

5. Framework Spring.....	98
5.1. Introducción	100
Módulos del Framework	103
Creación de la aplicación	106
5.2. Spring Data	112
Módulos de Spring Data	114
5.3. Spring Boot y MySQL.....	116
Contenedor de aplicaciones integrado.....	117
Starters	117
5.4. Validaciones.....	127
Síntesis	130
Material complementario	131
Glosario	132
Referencias bibliográficas	134
Créditos	135

Introducción

En el ámbito del desarrollo de software, Java se ha consolidado como uno de los lenguajes de programación más robustos y versátiles. Este documento proporciona una guía comprensiva sobre algunas de las tecnologías y frameworks más importantes utilizados en el ecosistema Java, con un enfoque particular en la conectividad de bases de datos, el desarrollo web y la gestión de la persistencia de datos.

El contenido de este documento se estructura en cinco secciones principales:

- **Java con JDBC**

Se examina el uso de Java Database Connectivity (JDBC), una API fundamental para conectar aplicaciones Java con bases de datos relacionales. Se describen las clases e interfaces necesarias, los pasos para establecer una conexión y se proporciona información sobre los drivers JDBC para distintos sistemas de gestión de bases de datos (SGBD).

- **Servlets y JSP**

Este tema introduce los Servlets y JavaServer Pages (JSP), tecnologías clave para el desarrollo web en Java. Se abordan su arquitectura, ciclo de vida, y la forma de manejar formularios HTML y sesiones HTTP, así como la implementación del patrón de diseño Modelo-Vista-Controlador (MVC).

- **Fundamentos de React JS**

Aquí se exploran los fundamentos de React JS, una biblioteca de JavaScript desarrollada por Facebook para construir interfaces de usuario. Se incluye la instalación, creación de proyectos, manejo de componentes y estado,

eventos, listas y keys, formularios, ciclo de vida de los componentes y el uso de hooks.

- **Hibernate**

Hibernate es un framework de mapeo objeto-relacional (ORM) que facilita la interacción con bases de datos en aplicaciones Java. Esta sección cubre la configuración de Hibernate, el uso de clases de entidad y anotaciones, y las operaciones básicas como guardar, obtener, actualizar y eliminar registros, así como las asociaciones entre entidades y el ciclo de vida de los objetos.

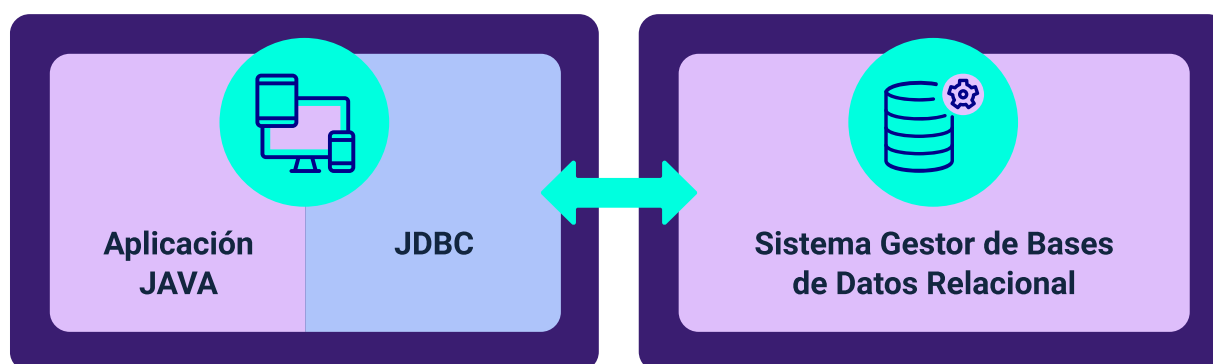
- **Spring**

Se presenta el Spring Framework, una herramienta poderosa para el desarrollo de aplicaciones empresariales en Java. Se destacan la inyección de dependencias (DI), el principio de inversión de control (IOC), y se examinan módulos importantes como Spring Data y Spring Boot. Además, se discuten las validaciones en Spring Boot y se proporciona un ejemplo de mapeo con JPA.

1. Java con JDBC

Cuando se habla de JDBC (Java Database Connectivity), se hace referencia a un driver que encapsula un conjunto de clases e interfaces escritas en Java, creadas para establecer conexiones, enviar sentencias SQL y procesar los datos resultantes sobre bases de datos relacionales.

Figura 1. Java con JDBC



En términos más generales, JDBC es uno de los puentes que puede ser utilizado en Java para comunicar una aplicación Java con un servidor de bases de datos relacionales. Trabajar con conexiones a bases de datos mediante JDBC implica realizar cuidadosamente las siguientes acciones:

- Manejar manualmente las conexiones y desconexiones a la base de datos.
- Ser cuidadosos en la escritura de sentencias SQL a ser ejecutadas en el servidor.
- Conocer las relaciones existentes en el modelo de base de datos a manipular.
- Conocer las características de la base de datos a manipular, ya que esto podría afectar las consultas.

- **Tenga presente**

Cada sistema gestor de base de datos (SGBD) provee su propio JDBC para la manipulación de sus sistemas con la aplicación Java. Dependiendo de la base de datos a la que se desee conectar, se debe descargar el driver JDBC correspondiente.

La siguiente tabla muestra las direcciones oficiales para la descarga del driver JDBC para las bases de datos relacionales más comúnmente utilizadas en la industria del software.

Tabla 1. Drivers JDBC para SGBD

Base de Datos	Página oficial para descarga de JDBC
Mysql	https://dev.mysql.com/downloads/connector/j/
SQL Server	https://docs.microsoft.com/en-us/sql/connect/jdbc/download-microsoft-jdbc-driver-for-sql-server?view=sql-server-ver15
Oracle	https://www.oracle.com/database/technologies/jdbc-upc-downloads.html
PostgreSQL	https://jdbc.postgresql.org/download/
DB2	https://www.ibm.com/support/pages/db2-jdbc-driver-versions-and-downloads

1.1. Clases e interfaces Java

Además de disponer del driver JDBC correspondiente para la base de datos que se desea conectar, se requiere un conjunto adicional de clases e interfaces Java para hacer uso del driver. Para ello, es necesario importar los paquetes java.sql o javax.sql. A continuación, se describen algunas de las clases más importantes dentro de estas

librerías, y otros elementos, para gestionar correctamente una base de datos por medio de JDBC.

- **DriverManager**

Clase estática que permite la gestión del driver JDBC disponible para la aplicación. Por medio del método `getConnection`, crea una instancia de tipo `Connection` mediante la cual se realiza el flujo de envío y recibo de información hacia y desde la base de datos destino.

- **Connection**

Interfaz Java que permite establecer una conexión con una base de datos específica por medio de la cual se puede extraer información de la descripción de tablas, procedimientos almacenados y ejecutar sentencias SQL. Se instancia un objeto de este tipo invocando el método `getConnection` del `Driver Manager`, al cual se le deben pasar como parámetros las credenciales (usuario y contraseña) del usuario de la base de datos destino y la cadena de conexión correspondiente.

- **Statement**

Interfaz Java usada para la ejecución de sentencias SQL estáticas y retornar los resultados que son producidos por estas sentencias. Una instancia de este tipo se obtiene de la ejecución del método `createStatement()` desde una instancia de un objeto de tipo `Connection`.

- **ResultSet**

Tipo especial de dato que representa un conjunto de datos de tipo tabular, ideal para referenciar resultados obtenidos por medio de una consulta que retorna un conjunto de datos estructurados, como, por ejemplo, los de una

consulta SELECT. Un ResultSet es un apuntador, tipo cursor, direccionado a la primera posición de toda la estructura tabular que está referenciando.

- **El método next()**

Además de desplazarse una posición de forma lineal dentro de la estructura, también devuelve como resultado un valor booleano para informar si el desplazamiento realizado encontró, o no, un conjunto de datos válidos. Así es posible determinar cuándo se alcanza el final de los elementos.

Clases e interfaces Java

Para profundizar y conocer los elementos más importantes de las clases e interfaces Java más usadas, lo invitamos a consultar el PDF "**Clases e interfaces Java**", el cual se encuentra en la carpeta Anexos.

1.2. Conexión a base de datos desde aplicación Java

Garantizar la conexión a una base de datos requiere la aplicación concreta de los pasos ya explicados.

Los pasos que deben realizarse para garantizar una conexión a base de datos desde una aplicación Java, utilizando JDBC, son:

- a) Descarga del driver JDBC (Si no se tiene).
- b) Importar el driver descargado en las librerías del proyecto JAVA.
- c) Importar el paquete `java.sql.*`.
- d) Inicializar el driver usando la sentencia `Class.forName`.
- e) Crear un Objeto de tipo `Connection` por medio del `DriverManager`.

- f) Crear un objeto Statement que permite configurar las sentencias SQL desde el Objeto de Connection creado en el punto anterior.
- g) Ejecutar la sentencia SQL usando cualquiera de los métodos del objeto Statement.
- h) Procesar los resultados de la sentencia ejecutada por medio de un ResultSet y sus métodos de apoyo.

Para afianzar los conocimientos logrados hasta este punto, se recomienda revisar el video propuesto a continuación. En este, se presenta un ejemplo sencillo y se describen las prácticas de cada paso descrito.

Video 1. Conexión JDBC



[Enlace de reproducción del video](#)

Síntesis del video: Conexión JDBC

En este video se explica cómo conectar una aplicación Java a un sistema gestor de bases de datos utilizando el driver JDBC. El ejemplo se desarrolla utilizando un servidor local de bases de datos MySQL, instalado a través del paquete XAMPP, y el entorno de desarrollo integrado (IDE) NetBeans.

Primero, se descarga el driver correspondiente para MySQL y se configura en el proyecto de NetBeans. Se menciona la importancia de contar con tres elementos clave: una cuenta de usuario y contraseña en el servidor, y la URL de conexión. La URL, en este caso, es localhost con el puerto por defecto 3306 y la base de datos llamada "prueba".

Se crea un proyecto en NetBeans y se configura para conectarse a la base de datos MySQL. La tabla utilizada en el ejemplo se llama "usuario" y tiene tres columnas: user_id, username y user_password. Para establecer la conexión, se agrega el driver descargado al proyecto y se instancian las librerías necesarias.

A continuación, se define la cadena de conexión con los parámetros de usuario, contraseña y URL del servicio. Se importan las librerías necesarias en Java, como java.sql, que permiten la creación de objetos de tipo Connection, Statement y ResultSet.

Luego, se instancia el driver con la sentencia Class.forName() y se maneja cualquier excepción con un bloque try-catch. Se establece la conexión con el DriverManager y se crea un objeto Statement para ejecutar consultas SQL.

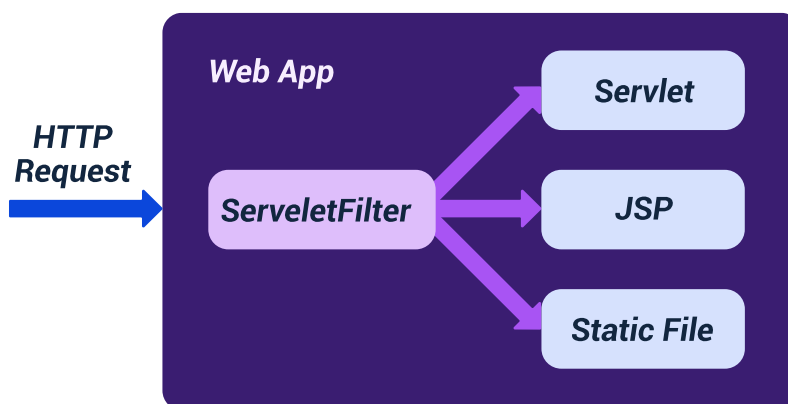
El video describe cómo insertar datos en la tabla "usuario" y cómo verificar la inserción mediante una consulta SELECT. Se utiliza un ciclo para procesar y mostrar los resultados de la consulta.

Finalmente, se realiza la conexión a la base de datos MySQL utilizando el driver JDBC y se destacan las funcionalidades básicas de inserción y consulta de datos.

2. Servlets y JSP

Los Servlets y los JSP (Java Server Pages), son tecnologías que se utilizan para realizar ejecuciones en servidores Java, permitiendo extender los servicios de un servidor web. (Berzal, 2007)

Figura 2. Java con JDBC



Un ejemplo sencillo de lo que se puede hacer con JSP y Servlet es que, por medio de una página JSP, se puede crear un formulario que contenga una serie de datos como: primer nombre, segundo nombre, edad y correo. Luego, a través de un servlet, se realiza el envío de estos registros para que sean almacenados en una base de datos y, finalmente, el usuario pueda acceder a esta información.

Conozca, a continuación, los procesos principales de estas tecnologías y algunos aspectos generales del uso de JSP y Servlet:

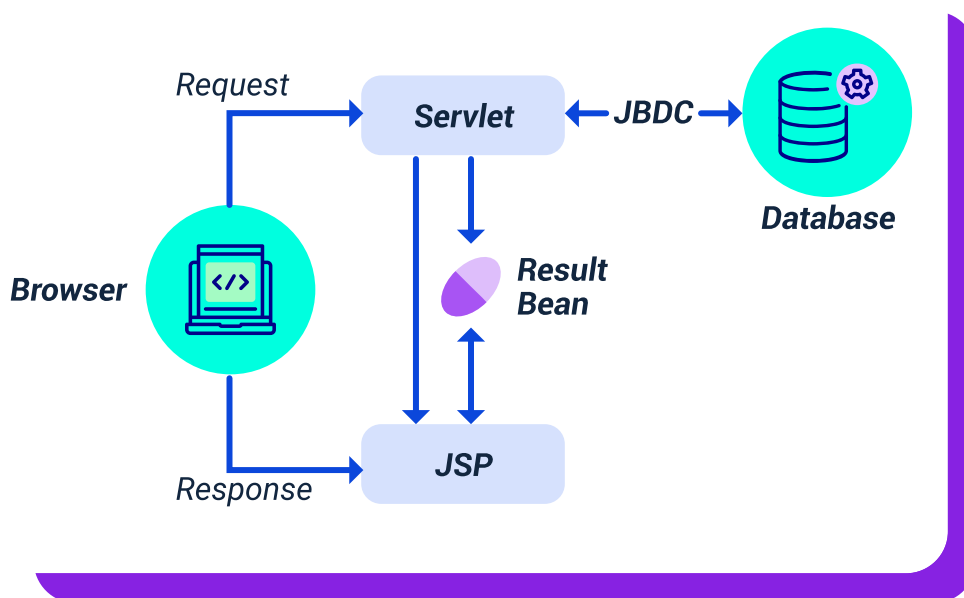
- **Los servlets**

Adecuan el proceso de las peticiones que entran al servidor, procesan los datos de formularios, generan contenidos de varios tipos y redireccionan las peticiones.

- **JSP (JavaServer Pages)**

Es un código Java que se ejecuta en un servidor remoto. Lee las acciones del usuario desde formularios HTML, donde el servidor realiza consultas, usualmente a bases de datos, para obtener la información solicitada y devolver una página HTML dinámica y en tiempo real.

Figura 3. Arquitectura web que involucra JSP y Servlets



- **Los archivos JSP**

Están formados por código Java dentro de etiquetas <html> con extensión .jsp. Estos archivos deben ser guardados en la carpeta del contenido web del servidor. El código JSP inicia con <% %>.

- **Ejemplo**

Este es un ejemplo sencillo de código JSP.

```
<%@ page language='java' contentType='text/html; charset=iso-8859-1'%>
```

```
<%@ page import='java.util.Date' %>
```

```
<html>
```

```
<head>
```

```
    <title>Hola Planeta</title>
```

```
</head>
```

```
<body>
```

```
    <p>Bienvenido a la página JSP. </p>
```

```
    <p>La hora del servidor es <%= new Date() %> </p>
```

```
    <% out.println("HOLA PLANETA"); %>
```

```
</body>
```

```
</html>
```

- **Software requerido**

El software necesario para trabajar con JSP y servlets es: Java JEE IDE (NetBeans, Eclipse) y un servidor web (Tomcat).

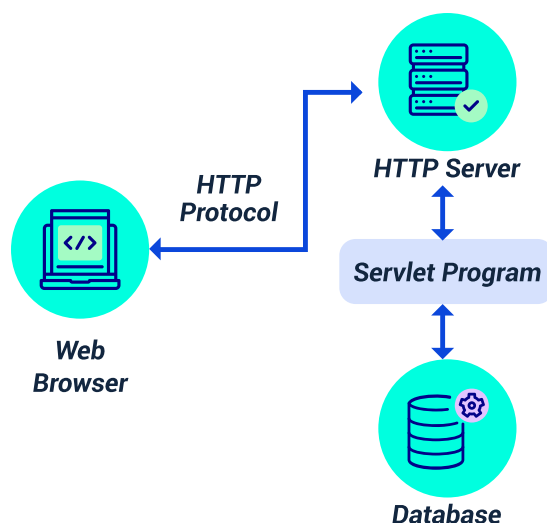
Figura 4. Entorno de desarrollo Eclipse IDE



2.1. Introducción a servlets

Servlet es un programa que se lleva a cabo en una aplicación o servidor web; se emplea como una capa que intercede entre la base de datos y el servidor de peticiones HTTP, por medio de un navegador web o cualquier otro cliente HTTP (Gómez, 2017).

Figura 5. Arquitectura de un servlet



A continuación, se presentan algunas ventajas de utilizar servlets:

- Rendimiento: a diferencia de CGI (Common Gateway Interface), el rendimiento de un servlet es mucho mejor.
- No necesita procesos independientes para operar las solicitudes del cliente.
- Dado que los servlets están escritos en Java, son independientes de la plataforma.
- Fiabilidad: resguardan los recursos del servidor gracias a la seguridad proporcionada por Java.

Las tareas principales de un servlet son:

- **Datos de transmisión**

Se encarga de leer claramente los datos de transmisión de un cliente (navegador), incluyendo un formulario HTML.

- **Remisión de peticiones HTTP**

Remite una petición HTTP de los datos contenidos en el navegador, incluyendo cookies.

- **Tratamiento de datos**

Trata los datos y produce resultados accediendo a la base de datos.

- **Envío de documentos**

Envía documentos al cliente (navegador), los cuales pueden ser de varios tipos como textos, binarios, etc.

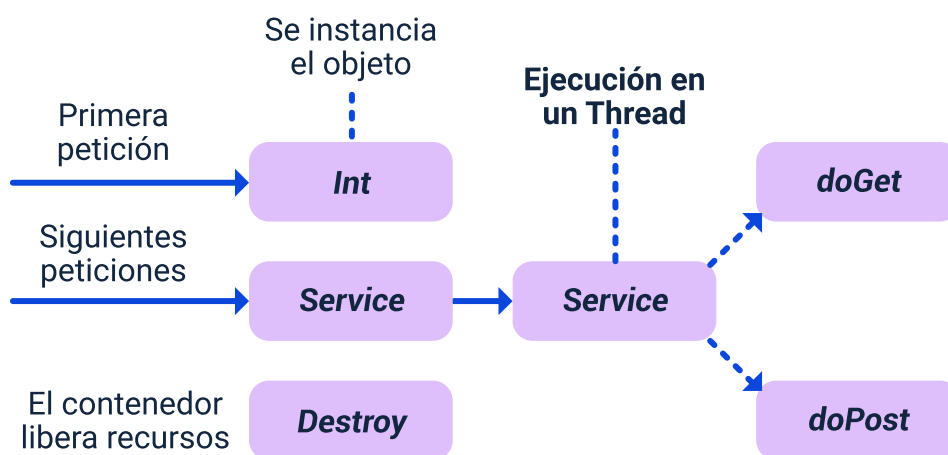
- **Envíos de respuesta**

Realiza envíos de respuesta HTTP implícita al cliente (navegador).

Ciclo de vida de un Servlet

A diferencia de las clases en Java, que inician con un método `main()`, en los servlets se invocan ciertos métodos al recibir peticiones. A esta metodología se le llama ciclo de vida de un servlet y se compone de tres métodos: `init`, `service` y `destroy`.

Figura 6. Ciclo de vida de un Servlets



Conocer el ciclo de vida de un Servlet; procurar llevar registro, en la libreta personal de apuntes, de los aspectos más importantes y destacados.

- **Init**

El servlet inicia por medio de una llamada al método `init()`. Este método está definido para ser llamado una sola vez. Cuando el servlet es invocado, se crea una instancia del servlet, generando un hilo nuevo para cada solicitud.

- **Service**

En segundo lugar, es llamado el método de servicio para gestionar la solicitud del cliente. La función del método `service` es atender las

peticiones GET y POST de los clientes; cuando se llama a este método, invoca a doGet() y doPost(), dependiendo del caso.

- **Destroy**

Se llama al método destroy para finalizar el servlet. Como su nombre lo indica, se encarga de destruir el servlet sobrescribiendo el método destroy().

Ejemplo de Servlet

El siguiente código presenta un "Hola Mundo" en un servlet, con sus métodos de ciclo de vida.

```
public class HolaMundo extends HttpServlet {  
  
    private String mensaje;  
  
    public void init () throws ServletException  
  
        { // inicia la asignación de mensaje hola mundo  
  
            mensaje = "Hola Mundo";  
  
        }  
  
    public void doGet (HttpServletRequest request,HttpServletResponse response)  
        throws ServletException, IOException  
  
        { // Se especifica el formato de la respuesta utilizando método getWriter  
  
            salida = response.getWriter ();  
  
            // Genera respuesta de la petición
```

```
        salida.println("<h1>" + mensaje + "</ h1>");  
    }  
  
    public void destroy ()  
  
    {.
```

En el código anterior, se hace lo siguiente:

- a) Se crea la clase HolaMundoServlet que hereda de HttpServlet.
- b) La clase HttpServlet contiene el método doGet que se encarga de recibir la petición del usuario desde el navegador.
- c) El método doGet recibe dos argumentos: request (para procesar la petición) y response (para dar respuesta a la petición).
- d) Se sobrescribe el método doGet para crear la lógica de respuesta al usuario, se establece el formato de la respuesta en HTML y se establece el contenido de la respuesta.

Explore el vídeo que se propone para ahondar en los aspectos más importantes de Servlet.

Video 2. Servlet



Enlace de reproducción del video

Síntesis del video: Servlet

El video explica cómo crear un primer Servlet utilizando NetBeans y un servidor web Tomcat. Comienza con la preparación del entorno, que incluye la instalación de NetBeans o Eclipse y la configuración de Tomcat. Luego, se procede a crear un nuevo proyecto de aplicación web en NetBeans, configurando el servidor y seleccionando la versión adecuada de Java EE. Dentro del proyecto, se genera una estructura que incluye un archivo index.html y se crea un paquete para contener el Servlet. Se configura el descriptor web (web.xml) para mapear el Servlet y se asignan un nombre y una URL.

La clase del Servlet extiende `HttpServlet` e implementa los métodos `doGet` y `doPost` para manejar las solicitudes HTTP. El método `doGet` se configura para responder con un mensaje HTML simple. Una vez que se verifica la configuración en `web.xml`, se ejecuta el proyecto y se asegura de que el servidor web esté funcionando. Al acceder a la URL configurada, el Servlet responde con un mensaje "Hola mundo". El video concluye con la explicación de este proceso, destacando la importancia de la configuración correcta y la verificación del servidor antes de la ejecución.

2.2. Formularios HTML con Servlets

Las páginas web contienen formularios HTML para la solicitud de información al usuario. El proceso de recolección de datos y respuesta de este formulario se debe realizar por medio de un programa. Esto lo hace un servlet que recoge las peticiones de un formulario y las procesa.

La declaración de un formulario HTML se hace por medio de la etiqueta `<form>`, donde es primordial que se especifique el método por el que se realizará el envío de los datos, los cuales pueden ser `get` o `post`.

A continuación, se presentan los aspectos más relevantes en lo relativo a formularios HTML con servlets:

Video 3. Formulario Servlet



[Enlace de reproducción del video](#)

Síntesis del video: Formulario Servlet

El video presenta un ejemplo práctico de cómo crear formularios HTML y procesarlos utilizando servlets en Java. Comienza con la creación de un proyecto en un entorno de desarrollo y la configuración de las opciones del servidor. Se explica la creación de un formulario HTML que solicita al usuario ingresar un nombre de usuario y una contraseña, destacando la importancia de la etiqueta `<form>` y la especificación del método (get o post) y la acción que apunta al servlet encargado de procesar los datos. También se muestra cómo crear un servlet que recibe y procesa

los datos enviados desde el formulario, sobrescribiendo el método doGet para manejar las peticiones y respuestas del usuario.

El video también cubre la construcción visual del formulario utilizando herramientas de arrastrar y soltar, especificando los nombres de los campos del formulario para su procesamiento en el servlet. Se detalla cómo el servlet recibe los datos del formulario a través de las variables request y response, almacenando los datos en variables y validándolos. Finalmente, se compara la información ingresada con valores predefinidos para verificar su validez y se envía una respuesta al usuario indicando si el registro fue exitoso o si hubo algún error. El video concluye mostrando la respuesta obtenida al enviar el formulario, validando así el funcionamiento del servlet tanto para datos correctos como incorrectos.

2.3. HTTP Session

Las sesiones se crean para hacer un seguimiento y memorizar las acciones de un usuario en un sitio web. Las sesiones en Java permiten guardar información entre las diversas peticiones HTTP, utilizando una clase que realiza el seguimiento de las sesiones; estos objetos también se almacenan del lado del servidor.

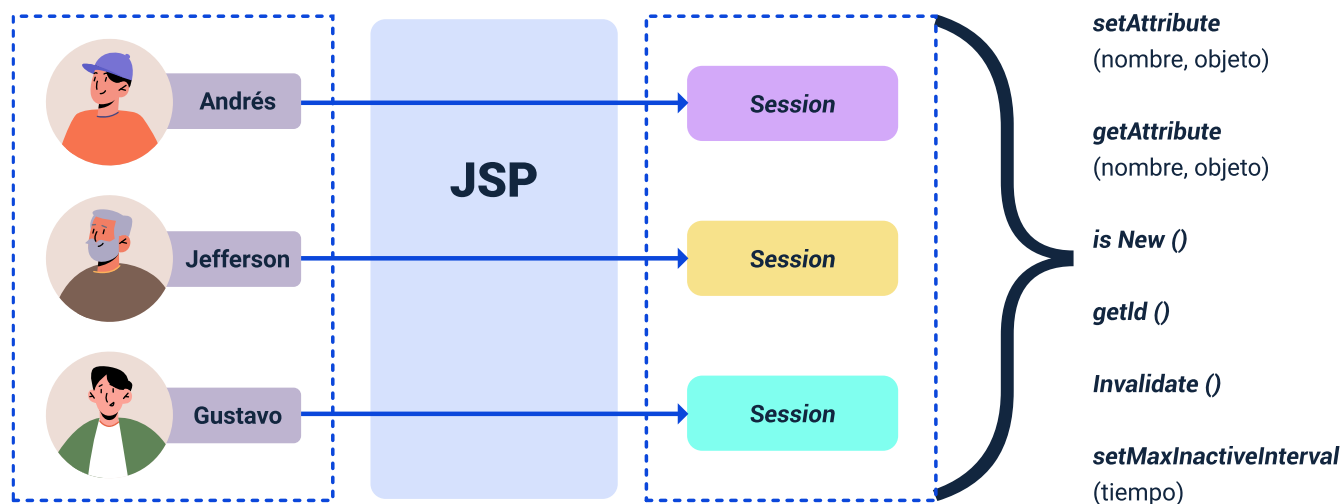
Para poder hacer uso de este objeto, es necesario obtener un objeto de la sesión para poder escribir en él, eliminarlo o leerlo, incluso después de que la sesión haya sido cerrada.

Los métodos más utilizados de HttpSession son:

- a) **getId()** Retorna el identificador que es asignado a la sesión. Se utiliza para la identificación de la sesión.
 - b) **isNew()** Retorna el valor booleano true en caso de que el cliente no tenga una sesión.
 - c) **setAttribute()** Establece un objeto en la sesión, empleando un nombre explícito.
 - d) **getAttribute()** Retorna el objeto que fue asignado a la sesión, identificado con el nombre que le fue asignado.
 - e) **setInactiveInterval()** Detalla el mayor tiempo que puede existir entre las peticiones continuas del cliente.
- El método utilizado para obtener la sesión de un usuario es getSession() este método regresa una interfaz de tipo HttpSession, Esta interfaz está dentro del paquete javax. servlet.http y es usada por el contenedor de páginas JSP que permite la creación de una sesión entre el cliente y el servidor.
<%HttpSession session = request.getSession(); %>

A continuación, se puede conocer el funcionamiento básico de las sesiones:

Figura 7. HTTP Session



- Analice detenidamente la gráfica e identifique cómo representa a tres usuarios que navegan por el mismo sitio web.
- Cada uno de estos tres usuarios accede al archivo JSP, el cual crea un objeto de sesión único para cada uno de ellos.
- La gráfica también presenta cómo estos objetos de sesión son guardados en la memoria del servidor. Estos permiten acceder a una serie de métodos que pertenecen a los objetos de tipo sesión.
- El método `setAttribute` tiene dos parámetros (nombre, objeto), donde el objeto es el que se almacenará en la sesión.
- Identifique cómo el método `getAttribute` devuelve el objeto que se ha almacenado en la sesión.
- Revise en la gráfica la representación de cómo el método `isNew` permite comprobar si la sesión es nueva o no.

2.4. Elementos de JSP

Las sesiones se crean para hacer un seguimiento y memorizar las acciones de un usuario en un sitio web. Las sesiones en Java permiten guardar información entre las diversas peticiones HTTP, utilizando una clase que realiza el seguimiento de las sesiones; estos objetos también se almacenan del lado del servidor. Para poder hacer uso de este objeto, es necesario obtener un objeto de la sesión para poder escribir en él, eliminarlo o leerlo, incluso después de que la sesión haya sido cerrada.

- **Códigos**

Se puede introducir código Java de varias clases como variables, métodos, sentencias, etc., y ser ejecutado en el contenedor JSP.

- **Directivas**

Brindan información general de la página y muestran lo que debe hacer el motor JSP con la página JSP. Para indicar una directiva se utiliza el símbolo arroba (@). Existen tres tipos: Page, Include y Taglib.

- **Acciones**

Facilitan al motor datos referentes a las acciones que debe realizar cuando se procesa la página (redirecciones). Hay 7 tipos: Param, Forward, Include, UseBean, getProperty, setProperty, Plugin.

- **Inserción de código Java**

Existen tres formas para insertar código Java en JSP: Expresiones `<%= expresión java %>`, Scriptlets `<% varias líneas de código java %>`, Declaraciones `<%! Declara variables o métodos %>`.

- **Expresiones**

Encerradas en la etiqueta `<%= expresión %>`, evalúan la expresión para convertir su resultado en un String, siendo este insertado en la salida. Solo tienen una línea de código. Ejemplos: `<%= new Date() %>`, `<%= 10+1 %>`, `<%= 10<4 %>`.

- **Scriptlets**

Inician con `<%` y terminan con `%>`. No tienen signo igual y pueden agregar varias líneas de Java, como bucles y condicionales. Ejemplo: `<% for(int i=0; i<10; i++) System.out.println("La cantidad es " + i); %>`.

- **Declaraciones**

Inician con `<%!` y terminan con `%>`. Se pueden escribir varias líneas de código y realizar declaraciones. Ejemplo: `<%! private int total; public int suma(int a, int b) { total = a + b; return total; } %>`.

Para afianzar sus conocimientos en elementos JSP, le invitamos a revisar el siguiente vídeo.

Video 4. Tipos de código JSP



[Enlace de reproducción del video](#)

Síntesis del video: Tipos de código JSP

El video Tipos Código JSP proporciona una guía sobre la inserción de código en páginas JSP. Comienza con la creación de un proyecto web en Eclipse y la configuración del servidor. Luego, detalla el trabajo con archivos JSP, que combinan HTML con Java. Se explican tres formas principales de inserción de código en JSP:

Expresiones: Utilizan `<%= %>`. Ejemplo: conversión de texto a mayúsculas.

Scriptlets: Bloques de código Java con `<% %>`. Ejemplo: bucles y sentencias de control.

Declaraciones: Definiciones de variables o métodos con `<%! %>`. Ejemplo: declaración y uso de métodos dentro de la página JSP.

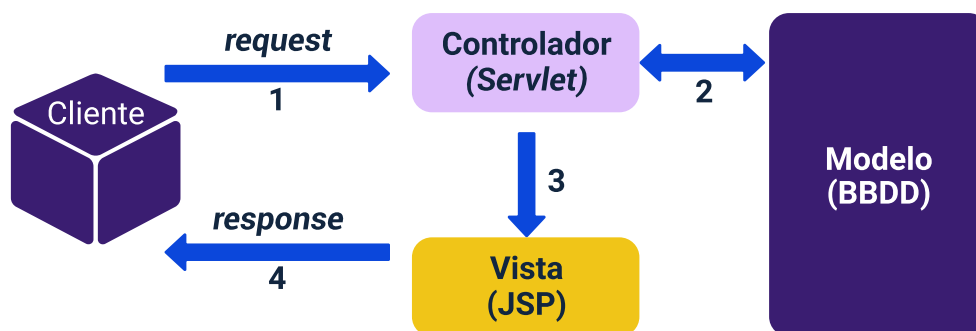
El video incluye ejemplos prácticos para ilustrar cada tipo de inserción de código, como operaciones matemáticas y manipulación de cadenas. Concluye con la combinación de estos métodos para la creación de aplicaciones web funcionales.

2.5. MVC

El MVC (Modelo-Vista-Controlador) es un patrón de diseño estándar que se utiliza para crear aplicaciones dinámicas con Java. Consiste en separar la parte lógica de los proyectos de la parte visual, integrando servlets y JSP. En este patrón, los servlets se encargan de la parte lógica y los JSP de la parte visual.

División de la aplicación realizada por el patrón:

Figura 8. Arquitectura MVC



- **El modelo**

Realiza el almacenamiento y la manipulación de los datos.

- **La vista**

Se encarga de la presentación con la que el usuario realizará la interacción, habitualmente en HTML.

- **El controlador**

Recoge los datos de entrada que envía el usuario, realiza la solicitud de acciones al modelo y modera la vista.

3. Fundamentos de React JS

React JS es una nueva tecnología importante para la construcción de software en su componente Front-end, que requiere tener en cuenta algunas consideraciones, como se pueden conocer a continuación:

- **¿Qué es?**

Es una nueva tecnología de reciente desarrollo, relacionada con una librería o biblioteca.

- **¿En qué se basa?**

Trabaja principalmente sobre componentes.

- **¿Cuál es su propósito?**

Inicialmente, fue pensada para solucionar el problema de interfaces que están actualizando información constantemente. La mayoría de las aplicaciones usadas en la actualidad actualizan información a cada momento mediante nuevos comentarios, notificaciones, publicaciones, imágenes, etc.

- **¿Quién la creó?**

Fue creada originalmente por Facebook, y hoy en día es una de las librerías más populares que existen para páginas web y front-end.

- **¿Cómo trabaja?**

Trabaja mediante una librería de JS. Una librería es una colección de procesos almacenados en objetos o módulos, es decir, códigos que aceleran el flujo de trabajo, pues consiste en reciclar algo que alguien ya ha creado antes.

¿Qué es React JS?

Es una biblioteca JavaScript de código abierto, desarrollada por la multinacional Facebook para crear interfaces de usuario. Es importante tener en cuenta que una biblioteca o librería es un conjunto de funcionalidades ya construidas y reutilizables siguiendo su sintaxis. Su principal objetivo, además de facilitar el desarrollo de aplicaciones de una sola página, es ser sencilla, declarativa y fácil de combinar.

- ¡Importante! Los datos deben ser atractivos, entendibles y accesibles para el usuario. Para esto, se deben tener en cuenta los objetivos de alistamiento y preparación de la información.

¿Qué es JSX?

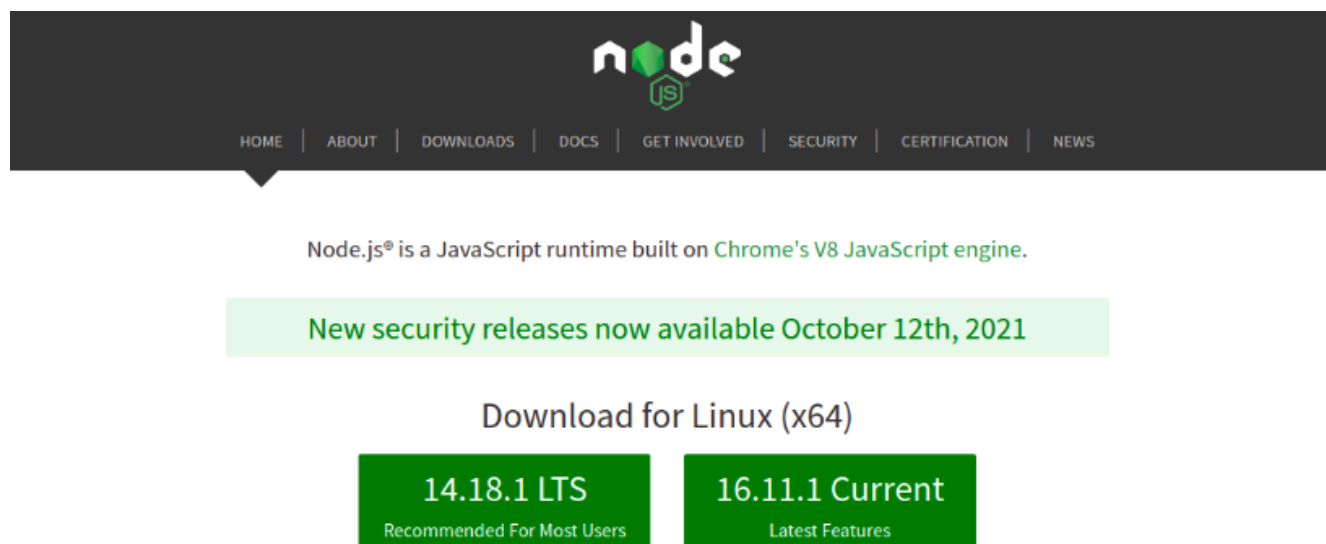
Su nombre viene de la composición de JavaScript (JS) + XML = (JSX) y corresponde a la sintaxis propuesta en React JS para escribir elementos de interfaz web compuestos por JavaScript + HTML, donde el HTML sigue las normas de código bien formado XML.

3.1. Instalando React JS

Las siguientes instrucciones presentan el proceso de instalación para sistemas operativos Linux versión Ubuntu 20.04.3 LTS y Windows. En ambos casos, primero se necesita instalar el servidor denominado Node y el manejador de paquetes npm.

- a) Ingrese a la página oficial de Node.js: <https://nodejs.org/en/>.

Figura 9. Sección de descargas en el sitio web de Node.js



b) Ingrese a la pestaña DOWNLOADS.

Figura 10. Página principal del sitio web de Node.js



Windows Installer
node-v14.18.1-x86.msi



macOS Installer
node-v14.18.1.pkg



Source Code
node-v14.18.1.tar.gz

Windows Installer (.msi)

Windows Binary (.zip)

macOS Installer (.pkg)

macOS Binary (.tar.gz)

Linux Binaries (x64)

Linux Binaries (ARM)

Source Code

32-bit	64-bit
32-bit	64-bit
64-bit	
64-bit	
64-bit	
ARMv7	ARMv8
node-v14.18.1.tar.gz	

Additional Platforms

Docker Image

Linux on Power LE Systems

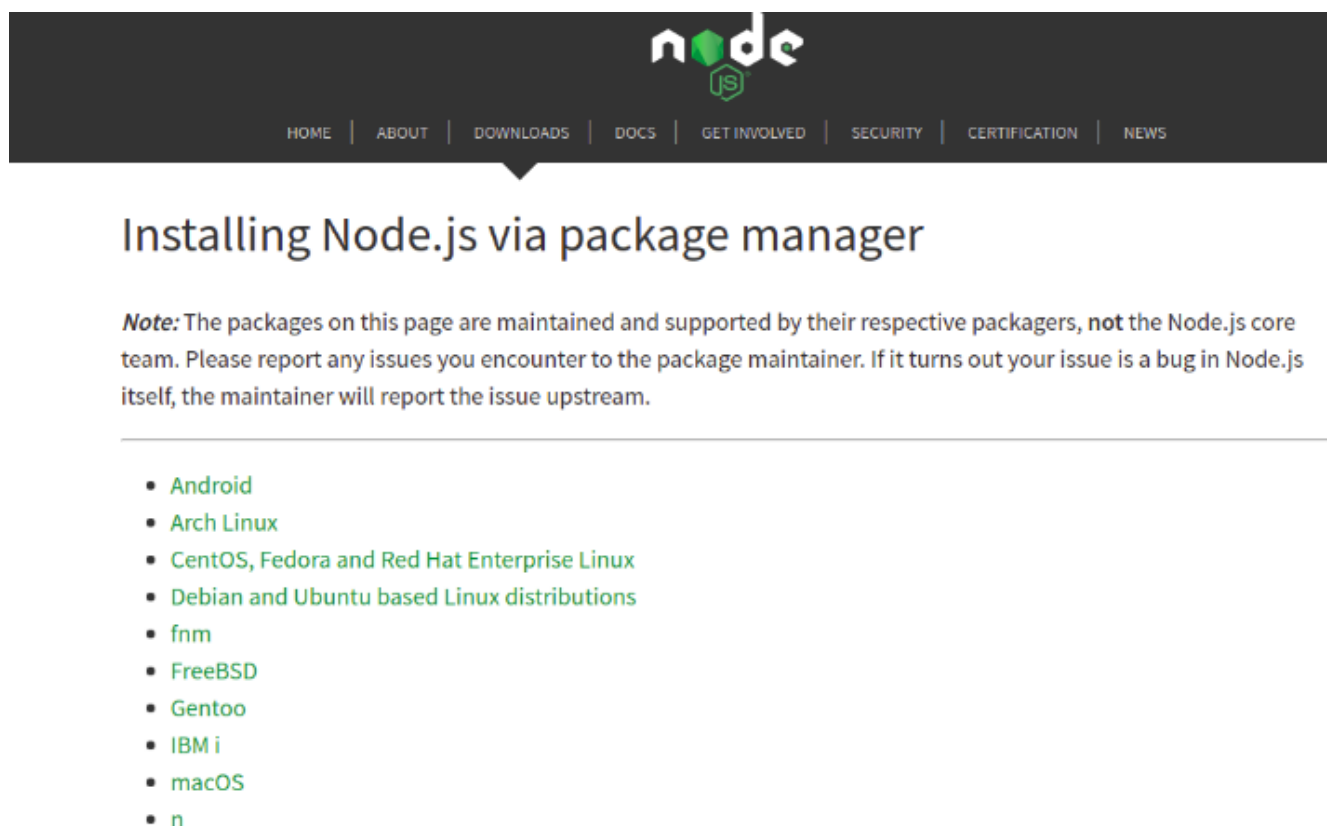
Linux on System z

AIX on Power Systems

Official Node.js Docker Image	
64-bit	
64-bit	
64-bit	

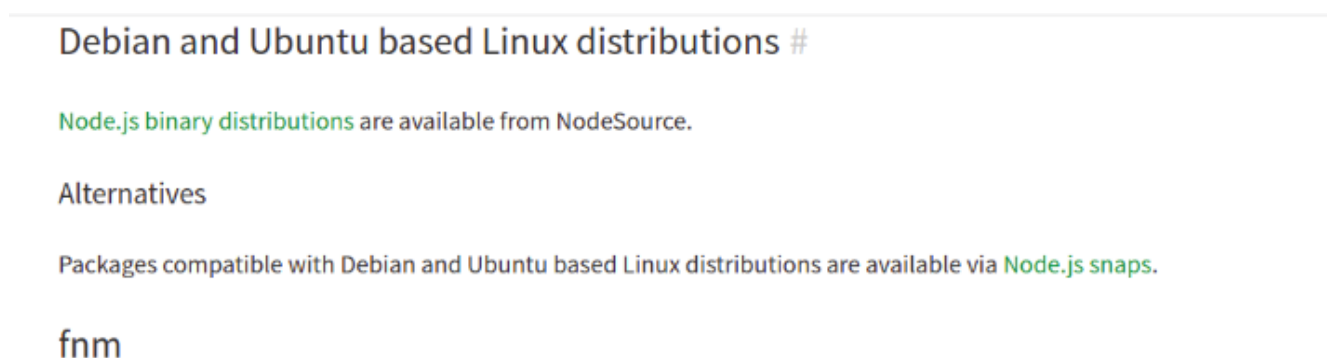
c) Ingrese a la opción Installing Node.js via package manager.

Figura 11. Información sobre la instalación de Node.js



d) Ingrese a la opción Debian and Ubuntu based Linux distributions.

Figura 12. Lista de sistemas operativos compatibles para la instalación de Node.js



e) Ingrese a la opción Node.js binary distributions.

Figura 13. Detalles específicos para distribuciones basadas en Debian y Ubuntu







PASSED

This repository contains documentation for using the [NodeSource Node.js](#) Binary Distributions via .rpm, .deb and Snap packages as well as their setup and support scripts.

If you are looking for NodeSource's low-impact Node.js performance monitoring platform, please [get started here](#).

Please file an issue if you are experiencing a problem or would like to discuss something related to the distributions.

Pull requests are encouraged if you have changes you believe would improve the setup process or increase compatibility across Linux distributions.

Table of Contents

- [Debian and Ubuntu based distributions](#) (deb)
 - [Installation instructions](#)
 - [Manual installation](#)
- [Enterprise Linux based distributions](#) (rpm)
 - [Installation instructions](#)
- [Tests](#)
- [FAQ](#)

f) Ingrese a la opción Installation instructions.

Figura 14. Página de NodeSource mostrando la compatibilidad de sus distribuciones binarias de Node.js

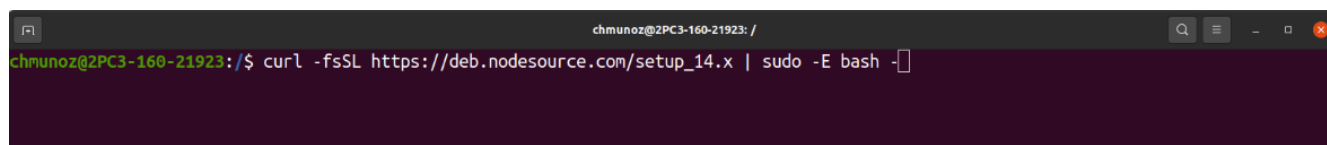
Node.js v14.x:

```
# Using Ubuntu
curl -fsSL https://deb.nodesource.com/setup_14.x | sudo -E bash -
sudo apt-get install -y nodejs

# Using Debian, as root
curl -fsSL https://deb.nodesource.com/setup_14.x | bash -
apt-get install -y nodejs
```

- g) Desde línea de comandos en Ubuntu, ingrese las siguientes instrucciones.
Si no tiene instalado curl, ingrese: `sudo snap install curl`.

Figura 15. Instrucciones de terminal para instalar Node.js en Ubuntu usando curl



```
chmunoz@2PC3-160-21923: /
chmunoz@2PC3-160-21923:/$ curl -fsSL https://deb.nodesource.com/setup_14.x | sudo -E bash -
```

- h) Vuelva a ejecutar: `curl -fsSL https://deb.nodesource.com/setup\_14.x`

Figura 16. Pantalla de terminal mostrando la ejecución del comando curl

```
chmunoz@2PC3-160-21923: ~/my-app
chmunoz@2PC3-160-21923:~/my-app$ npm run build
> my-app@0.1.0 build /home/chmunoz/my-app
> react-scripts build

Creating an optimized production build...
Compiled successfully.

File sizes after gzip:

 41.34 KB  build/static/js/2.8833cd6e.chunk.js
 1.62 KB  build/static/js/3.ff0311e9.chunk.js
 1.17 KB  build/static/js/runtime-main.1c420381.js
 591 B    build/static/js/main.6bc689e0.chunk.js
 556 B    build/static/css/main.a617e044.chunk.css

The project was built assuming it is hosted at /.
You can control this with the homepage field in your package.json.

The build folder is ready to be deployed.
You may serve it with a static server:

  npm install -g serve
  serve -s build

Find out more about deployment here:

  https://cra.link/deployment

chmunoz@2PC3-160-21923:~/my-app$
```

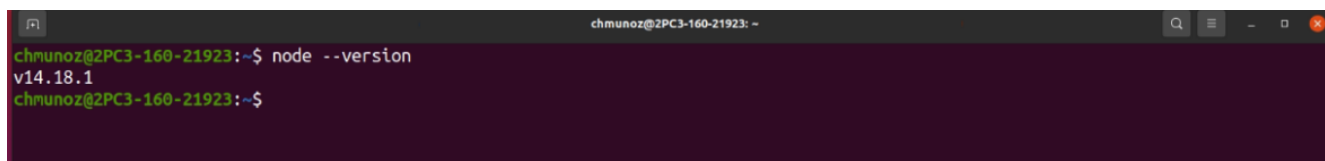
i) Luego ejecute el comando: `sudo apt-get install -y nodejs`.

Figura 17. Pantalla de terminal mostrando la compilación exitosa de una aplicación Node.js

```
chmunoz@2PC3-160-21923: /
chmunoz@2PC3-160-21923:/$ sudo apt-get install -y nodejs
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes NUEVOS:
 nodejs
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 278 no actualizados.
Se necesita descargar 25,0 MB de archivos.
Se utilizarán 122 MB de espacio de disco adicional después de esta operación.
Des:1 https://deb.nodesource.com/node_14.x focal/main amd64 nodejs amd64 14.18.1-deb-1nodesource1 [25,0 MB]
Descargados 25,0 MB en 9s (2.687 kB/s)
Seleccionando el paquete nodejs previamente no seleccionado.
(Leyendo la base de datos ... 190600 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../nodejs_14.18.1-deb-1nodesource1_amd64.deb ...
Desempaquetando nodejs (14.18.1-deb-1nodesource1) ...
Configurando nodejs (14.18.1-deb-1nodesource1) ...
Procesando disparadores para man-db (2.9.1-1) ...
chmunoz@2PC3-160-21923:/$
```

- j) Finalmente, compruebe la versión instalada (v14.18.1) ejecutando el siguiente comando: `node --version`.

Figura 18. Terminal mostrando la instalación exitosa de Node.js en Ubuntu



```
chmunoz@2PC3-160-21923: ~$ node --version
v14.18.1
chmunoz@2PC3-160-21923: ~$
```

A continuación, siga los siguientes pasos de Instalación sobre sistema operativo Windows.

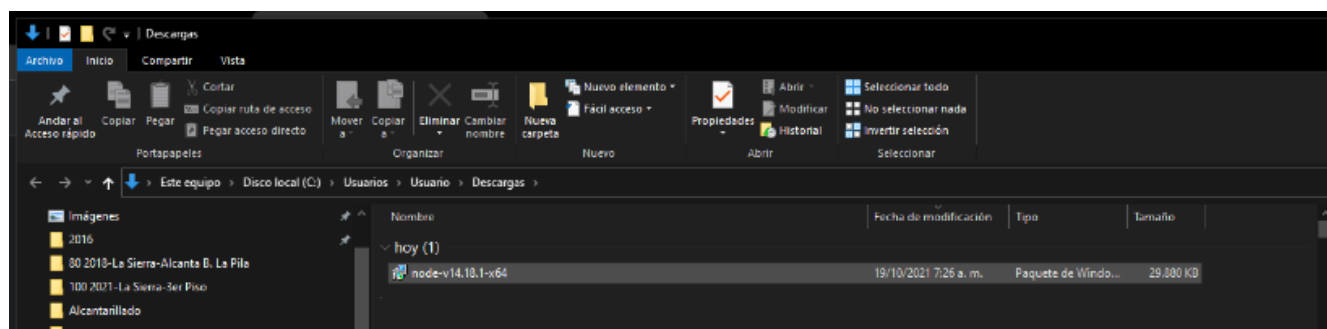
- **Paso 1.** Ingrese a Node >= 14.0.0 and npm >= 5.6 que corresponde a la página oficial de nodejs <https://nodejs.org/en/> y descargue la versión para windows de 64 bits.

Figura 19. Paso 1



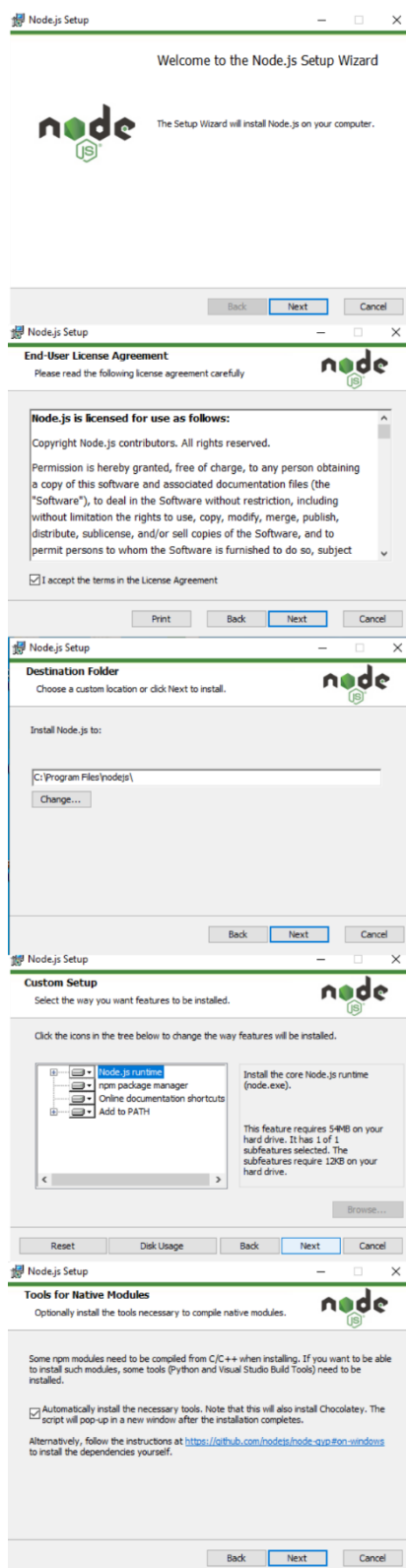
- **Paso 2.** Descargue en el sitio de su preferencia.

Figura 20. Paso 2



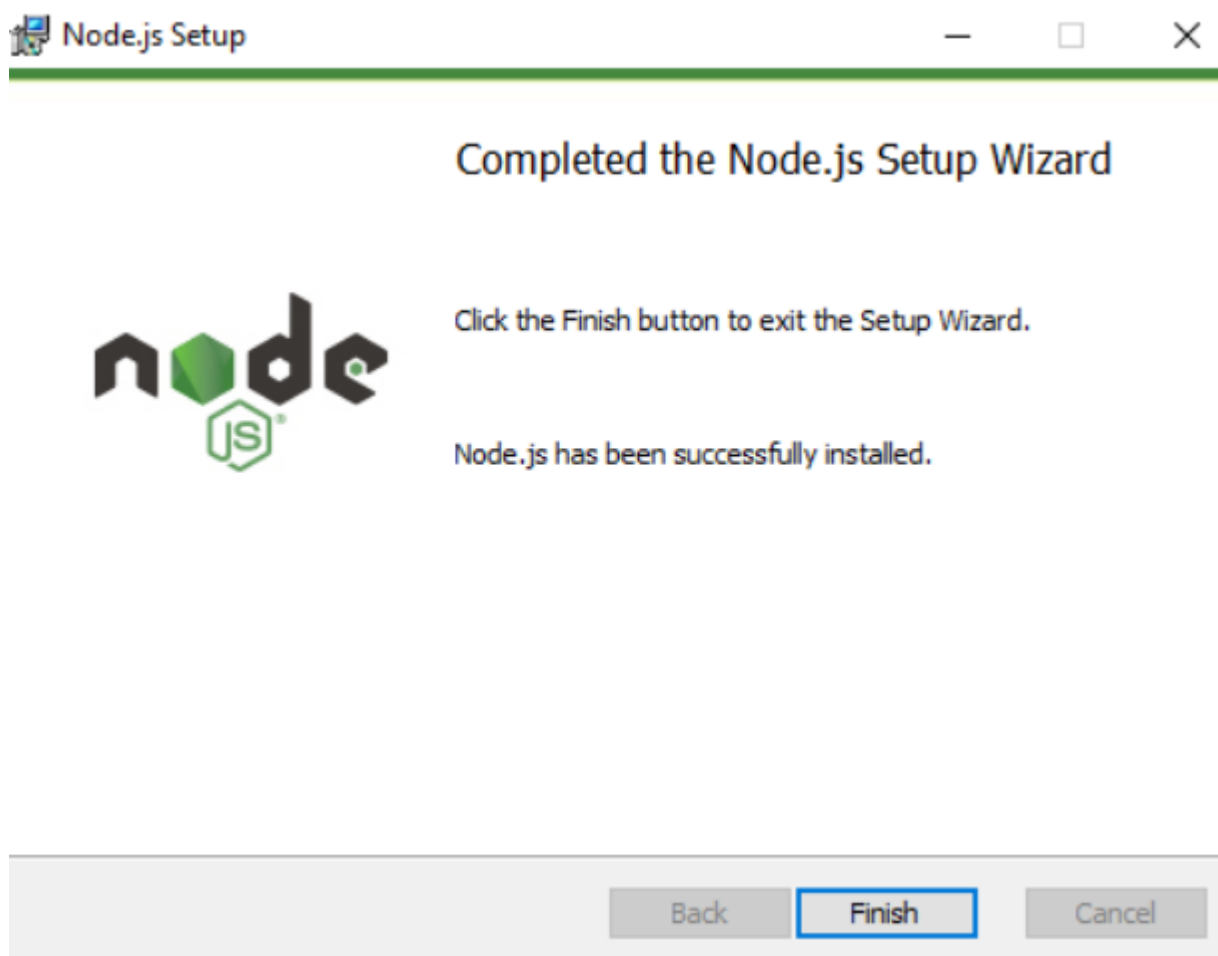
- **Paso 3.** Ejecute el archivo descargado como administrador de Windows y continúe el wizard haciendo clic en next en cada pantalla.

Figura 21. Paso 3



- **Paso 4.** Por último de clic en finish para terminar la instalación.

Figura 22. Paso 4



3.2. Crear un proyecto

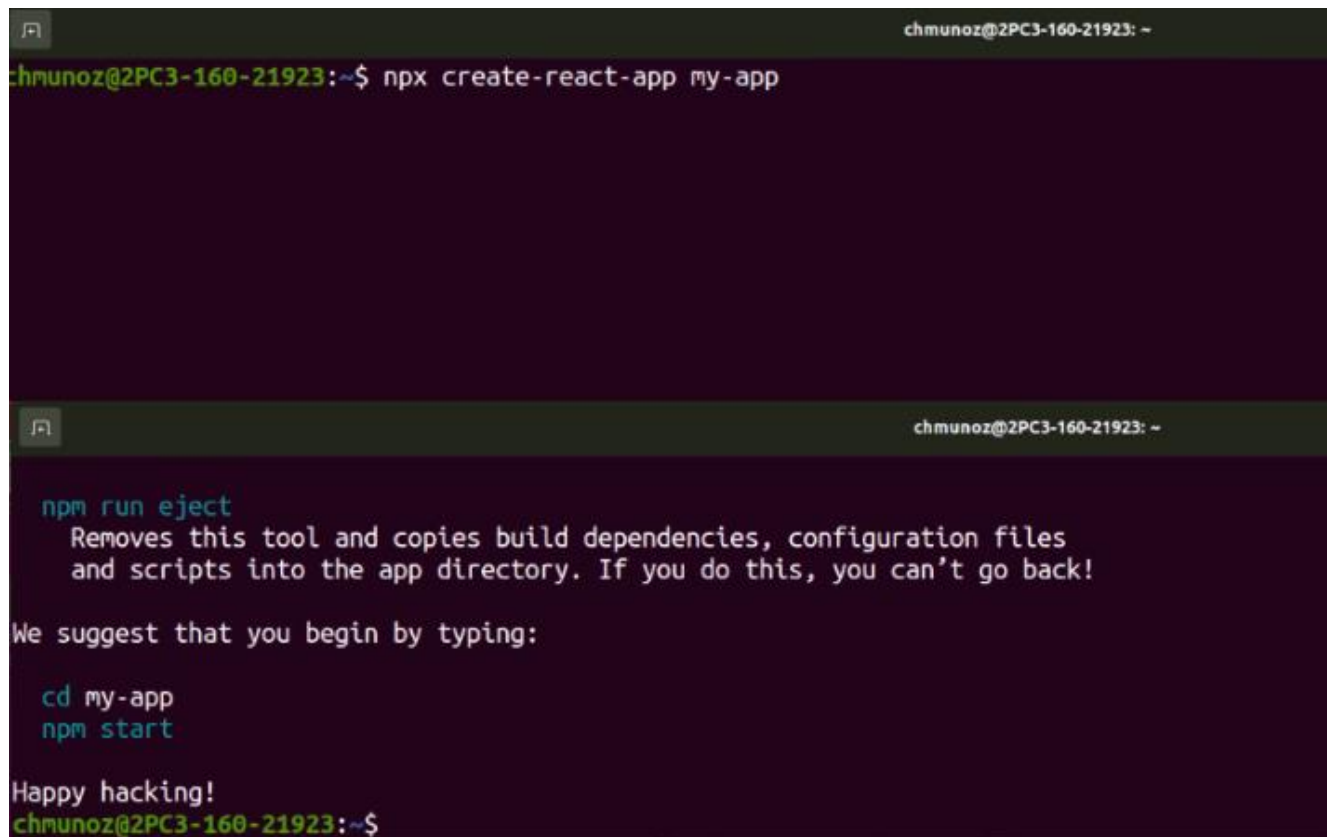
Una vez llevado a cabo el proceso de instalación del paso anterior de node y npm, se puede crear el proyecto React e ir ejecutando los siguientes comandos:

- `npx create-react-app my-app`
- `cd my-app`
- `npm start`

Iniciar la creación del proyecto, a través de la siguiente secuencia:

- **Paso 1.** Crear un comando Ubuntu: Desde la línea de comandos en Ubuntu, ejecute: `npx create-react-app my-app`.

Figura 23. Pantalla de terminal



```
chmunoz@2PC3-160-21923: ~$ npx create-react-app my-app

npm run eject
  Removes this tool and copies build dependencies, configuration files
  and scripts into the app directory. If you do this, you can't go back!

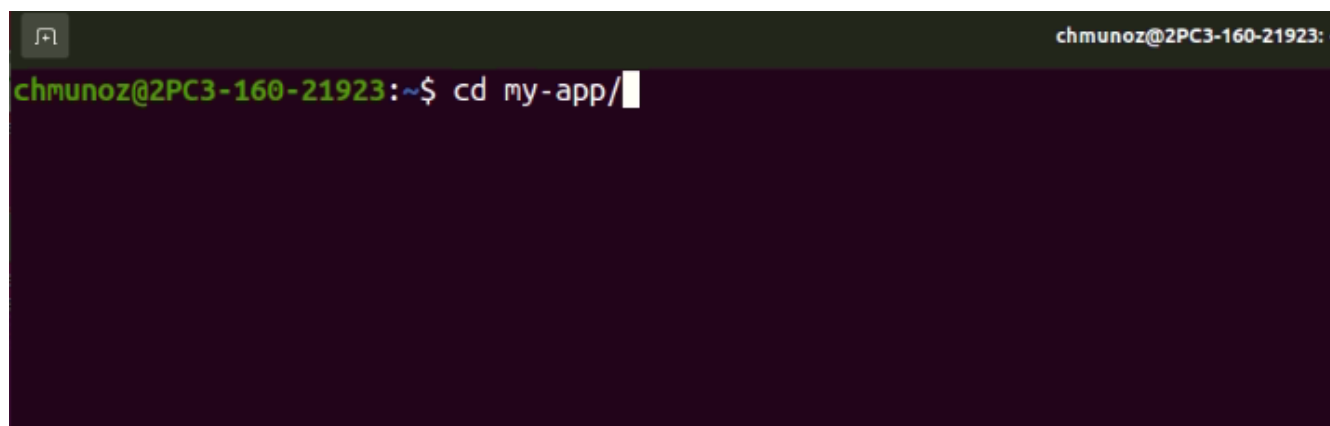
We suggest that you begin by typing:

  cd my-app
  npm start

Happy hacking!
chmunoz@2PC3-160-21923: ~$
```

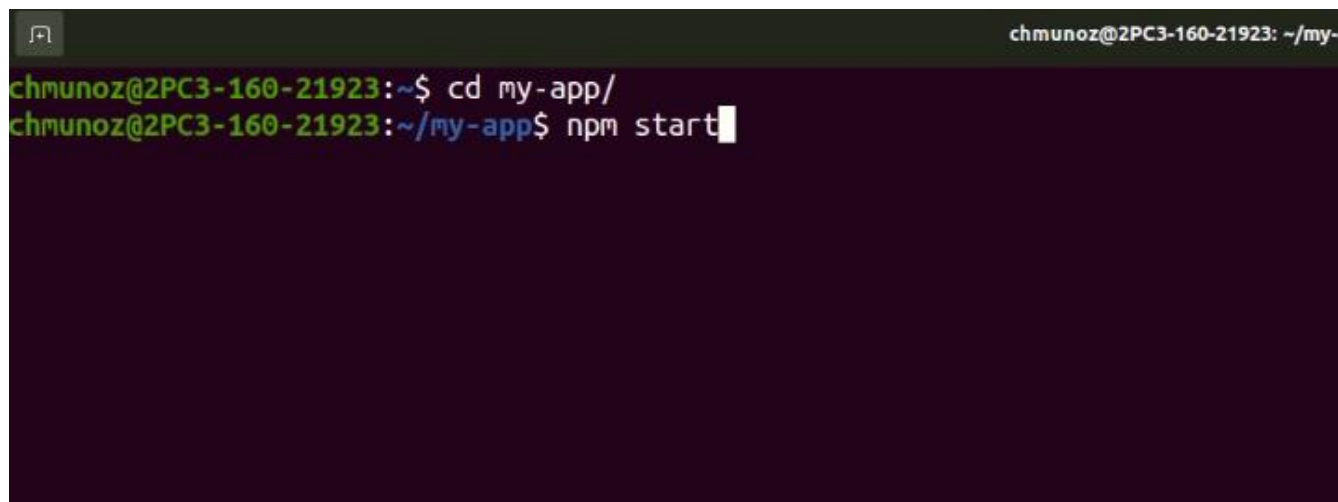
- **Paso 2.** Ejecución de `npx create-react-app my-app`: Una vez el sistema termine la ejecución del paso anterior, ejecute: `cd my-app`.

Figura 24. Pantalla de terminal después de navegar



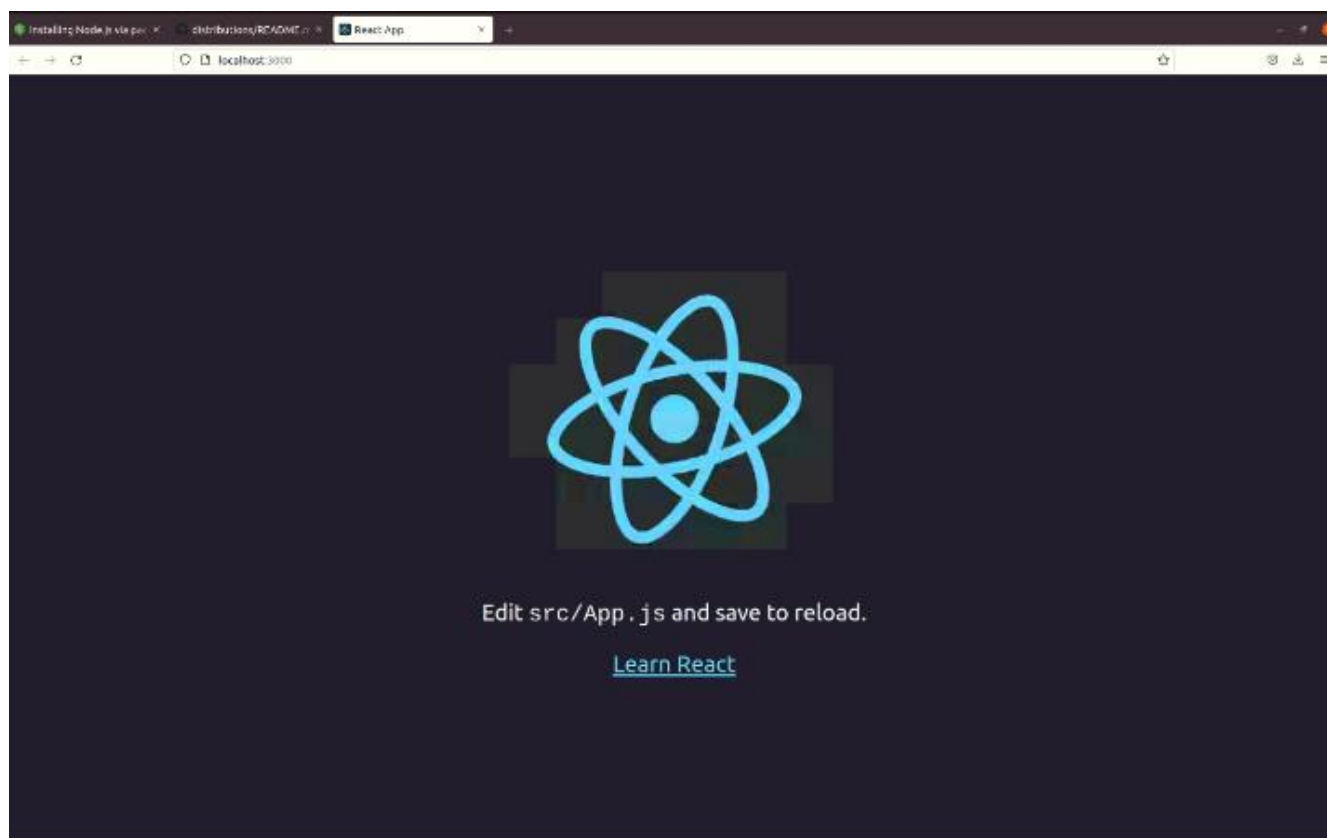
- **Paso 3.** Ejecute npm start para lanzar el proyecto: Dentro de la carpeta, ejecute: npm start para lanzar el proyecto.

Figura 25. Pantalla de terminal mostrando el comando npm start ejecutándose



- **Paso 4.** Proyecto React: El sistema lanza el primer proyecto React en el navegador como lo muestra la siguiente imagen.

Figura 26. Página inicial en el Navegador



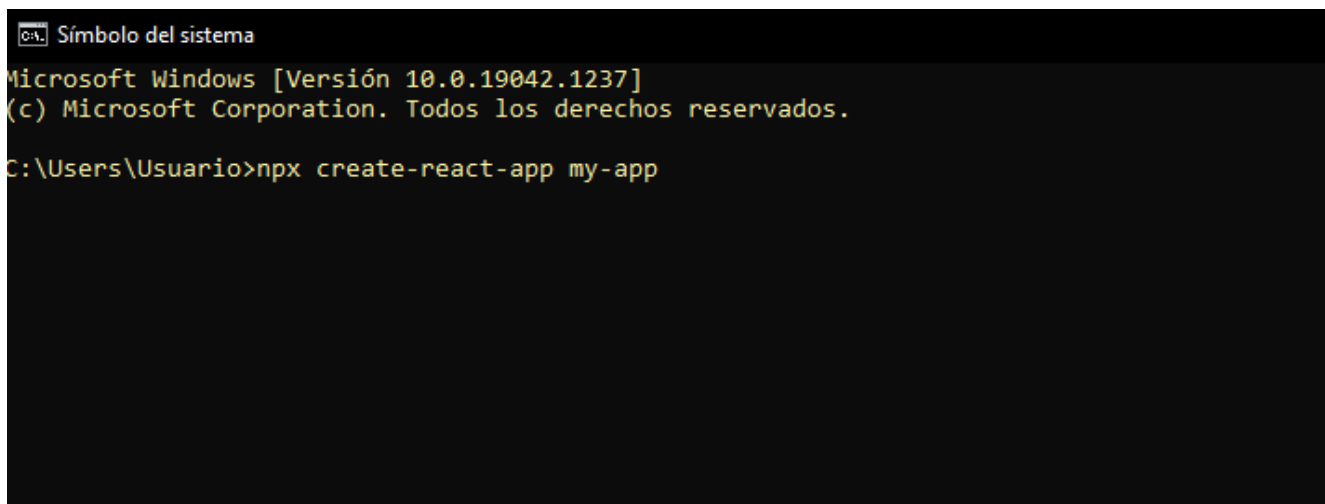
- **Paso 5.** Estructura del proyecto: Una vez finalizada la creación del proyecto, revise que la estructura del proyecto React esté conformada por el siguiente conjunto de carpetas y archivos.

Figura 27. Estructura del proyecto

```
my-app
├─ node_modules
│   └─ Conjunto de carpetas de los módulos de node
├─ package.json Archivo para gestionar las dependencias del proyecto
├─ package-lock.json
├─ public Carpeta en donde se encuentra recursos compartidos del proyecto
│   ├── favicon.ico
│   ├── index.html
│   ├── logo192.png
│   ├── logo512.png
│   ├── manifest.json
│   └─ robots.txt
├─ README.md Archivo de lectura y documentación del proyecto
└─ src Carpeta donde se encuentra el código fuente del proyecto
    ├── App.css
    ├── App.js
    ├── App.test.js
    ├── index.css
    ├── index.js
    ├── logo.svg
    ├── reportWebVitals.js
    └─ setupTests.js
```

- **Paso 6.** Crear un comando Windows: Desde la línea de comandos en Windows, ejecute: `npx create-react-app my-app`.

Figura 28. Pantalla de la consola de comandos de Windows



```

C:\> Símbolo del sistema

Microsoft Windows [Versión 10.0.19042.1237]
(c) Microsoft Corporation. Todos los derechos reservados.

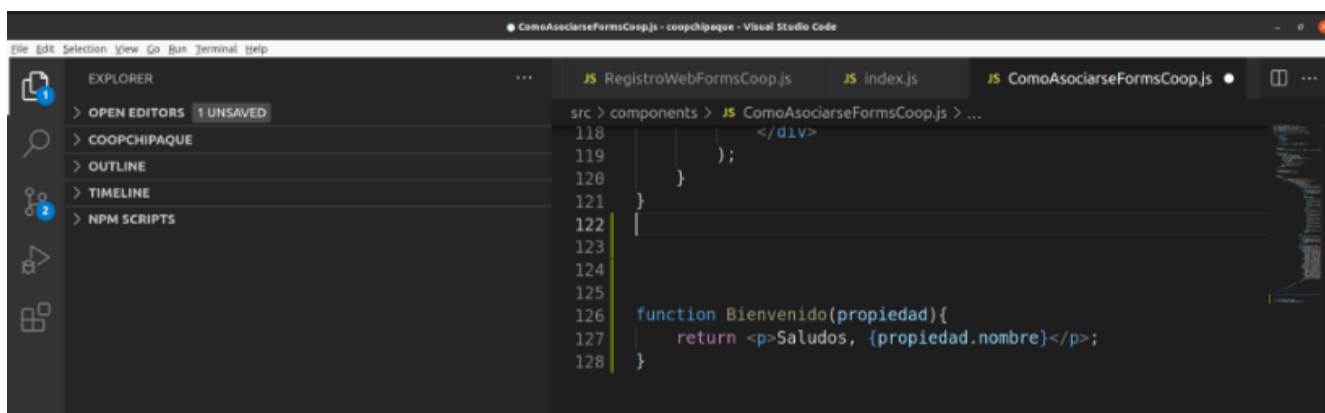
C:\Users\Usuario>npx create-react-app my-app
  
```

3.3. Propiedades y estados de los componentes

Un componente en React hace referencia a la posibilidad de definir de manera independiente una funcionalidad que puede ser reutilizable dentro del sistema. Esencialmente, estas funcionalidades se escriben como funciones JavaScript que reciben entradas denominadas propiedades y devuelven una forma de visualización.

Ejemplo de un componente denominado "Bienvenido" que recibe una propiedad y devuelve un párrafo de saludo.

Figura 29. Propiedades



```

src > components > JS ComoAsociarseFormsCoop.js > ...
118     </div>
119   );
120 }
121 }
122
123
124
125
126 function Bienvenido(propiedad){
127   return <p>Saludos, {propiedad.nombre}</p>;
128 }
  
```


Además, a un componente se le puede aplicar el concepto de estado en donde este, se define como la capacidad de especificar variables que pueden cambiar su valor en el tiempo, convirtiéndose en un comportamiento interno del componente, estos componentes se definen con el nombre de clase con la palabra reservada `class`.

```
class Clock extends React.Component {  
  
  constructor(props) {  
  
    super(props);  
  
    this.state = { date: new Date() };  
  
  }  
  
  render() {  
  
    return (  
  
      <div>  
  
        <h1>Hola, Mundo!</h1>  
  
        <h2>Es {this.state.date.toLocaleTimeString()}</h2>  
  
      </div>  
  
    );  
  
  }  
  
}
```

Eventos en React

Los eventos en React se manejan semejante a los del DOM se le pasa una función al controlador del evento.

```
function Bienvenido(propiedad) {  
    return <p>Saludos, {propiedad.nombre}</p>;  
}  
  
function activarCamaras(propiedad) {  
    return <p>Camara Uno, {propiedad.nombre}</p>;  
}
```

```
<button onClick={activarCamaras}>  
    Activar Camaras  
</button>
```

```
class Clock extends React.Component {  
    constructor(props) {  
        super(props);  
        this.state = { date: new Date() };  
    }
```

```
}  
  
render() {  
  return (  
    <div>  
      <h1>Hola, Mundo!</h1>  
      <h2>Es {this.state.date.toLocaleTimeString()}.</h2>  
    </div>  
  );  
}  
}
```

Listas y Keys

Las listas hacen referencia a un arreglo de datos, que puede ser un arreglo de objetos o de datos simples como caracteres, números, etc. Como toda lista en programación, React proporciona un método para extraer estos datos y mostrarlos en la página en la cual se está trabajando. A continuación, se presenta cómo se hace:

En React es necesario pasar arreglos de datos a listas para el manejo de algunos componentes lo cual se trabaja con la función `map` de la siguiente manera en donde se devuelve un elemento `<li>` por cada número.

Adicional al manejo de listas es muy importante tener en cuenta el identificador único de componentes para evitar errores en el DOM por lo tanto es necesario manejar el atributo key.

```
const numbers = [1, 2, 3, 4, 5];

const listItems = numbers.map((number) =>

  <li>{number}</li>

);

function NumberList(props) {

  const numbers = props.numbers;

  const listItems = numbers.map((number) =>

    <li key={number.toString()}>

      {number}

    </li>

  );

  return (

    <ul>{listItems}</ul>

  );

}
```

Formularios

En React el envío de formularios se comporta como en HTML, solo que aquí, el ingreso y cambio de valores en las entradas se deben manejar con componentes controlados por medio de estados.

```
class Clock extends React.Component {  
  
  constructor(props) {  
  
    super(props);  
  
    this.state = { date: new Date() };  
  
  }  
  
  render() {  
  
    return (  
  
      <div>  
  
        <h1>Hola, Mundo!</h1>  
  
        <h2>Es {this.state.date.toLocaleTimeString()}.</h2>  
  
      </div>  
  
    );  
  
  }  
  
}
```

Concepto de clases en React

Las clases son otra forma de definir un componente en React. Se caracterizan por tener propiedades y, a diferencia de las funciones, pueden definir estados propios sin utilizar el concepto de Hook. Dentro de los componentes de clase, existe una función constructora que se utiliza para definir el estado inicial del componente. Es muy importante llamar a la función `super()` dentro del constructor para que los parámetros funcionen adecuadamente, como puede verse en la siguiente imagen:

```
class Counter extends React.Component {  
  
  constructor(props) {  
  
    super(props);  
  
    this.state = { foo: 123, bar: 456 };  
  
  }  
  
  render() {  
  
    return (  
  
      <div>foo: {this.state.foo} bar: {this.state.bar}</div>  
  
    );  
  
  }  
  
}
```

Explorar el siguiente recurso y conocer los aspectos más relevantes relacionados con el concepto de clases en React:

- Los componentes permiten separar la interfaz de usuario en piezas independientes, reutilizables y pensar en cada pieza de forma aislada.
- React permite definir componentes como clases o funciones. Los componentes definidos como clases actualmente proporcionan una serie de características extra.
- La función `super()` dentro del constructor es un componente de React válido porque acepta un solo argumento de objeto "props" con datos y devuelve un elemento de React. Llamamos a dichos componentes "funcionales" porque literalmente son funciones JavaScript.
- Nombres de componentes: Comience siempre los nombres de componentes con una letra mayúscula. React trata los componentes que empiezan con letras minúsculas como etiquetas del DOM. Por ejemplo, `<div/>` representa una etiqueta div HTML pero `<Welcome/>` representa un componente y requiere que `Welcome` esté definido.

Funciones Hooks

Las funciones Hooks permiten a los componentes definidos por medio de funciones manejar variables de estado locales, similar a cómo lo hacen las clases. Los Hooks son funciones JavaScript y están sujetas a dos reglas:

- Solo llamar Hooks en el nivel superior. No se deben llamar Hooks dentro de loops, condiciones o funciones anidadas.
- Solo llamar Hooks desde componentes de función de React. No se deben llamar Hooks desde las funciones regulares de JavaScript.

Ejemplo de una función que utiliza el Hook useState:

```
import React, { useState } from 'react';

function Example() {

  // Declara una nueva variable de estado, que llamaremos "count"

  const [count, setCount] = useState(0);

  return (

    <div>

      <p>You clicked {count} times</p>

      <button onClick={() => setCount(count + 1)}>

        Click me

      </button>

    </div>

  );

}
```

Ciclos de vida de los componentes de React

Hace referencia a los métodos que se pueden utilizar por defecto (es decir, que forman parte de la librería de React). Cuando se ejecuta o se renderiza un componente en la línea de tiempo de ejecución del programa, estos métodos son:

Tabla 2. Métodos de ciclo de vida de React

Métodos de ciclo de vida de React	Descripción
Nombre del Método	Utilidad
componentWillMount()	Se ejecuta justo antes del primer pintado del componente.
componentWillReceiveProps(nextProps)	Se invoca cuando las propiedades se van a actualizar.
shouldComponentUpdate(nextProps, nextState)	Se invocará cuando se producen cambios de propiedades o cambios de estado.
componentWillUpdate(nextProps, nextState)	Este se invocará justo antes de que el componente vaya a actualizar su vista.
componentDidUpdate(prevProps, prevState)	Este se invocará justo después de que el componente vaya a actualizar su vista.
componentWillUnmount()	Se ejecuta en el momento que el componente se va a retirar del DOM.

Se puede revisar en el siguiente código cómo quedará el archivo:

```
import React from 'react';
```

```
export default class CicloVida extends React.Component {
```

```
constructor(props) {  
  console.log('Ejecutando constructor', ...args);  
  super(props);  
  this.state = {  
    estado: 'Inicializado en el constructor'  
  };  
}
```

```
componentDidMount() {  
  console.log('Ejecuta componentDidMount');  
}
```

```
componentWillUnmount() {  
  console.log('Ejecuta componentWillUnmount');  
}
```

```
componentDidUpdate(prevProps, prevState) {  
  console.log('Ejecuta componentDidUpdate. Anteriores propiedades y  
estado:', prevProps, prevState);
```

```
}
```

```
shouldComponentUpdate(nextProps, nextState) {
```

```
    console.log('Ejecuta shouldComponentUpdate. Próximas propiedades y  
estado:', nextProps, nextState);
```

```
    return true;
```

```
}
```

```
componentWillReceiveProps(nextProps) {
```

```
    console.log('Ejecuta componentWillReceiveProps con las propiedades  
futuras:', nextProps);
```

```
}
```

```
componentWillUpdate(nextProps, nextState) {
```

```
    console.log('Ejecuta componentWillUpdate. Próximas propiedades y  
estado:', nextProps, nextState);
```

```
}
```

```
componentDidCatch(error, info) {
```

```
    console.log('Ejecuta componentDidCatch:', error, info);
```

```
}

render() {

  console.log('Ejecuta render');

  return (

    <div>

      <h1>Ciclo de vida del componente</h1>

      <p>Estado: {this.state.estado}</p>

      <p>Propiedad: {this.props.propiedad}</p>

    </div>

  );

}
```

¡Atención! En React.js los componentes que no sean puros (todos los que se crean mediante clases o `React.createClass`) poseen algo conocido como el ciclo de vida. Este ciclo de vida, son una serie de funciones que se ejecutan en distintos momentos de la vida del componente y nos permiten realizar distintas acciones en estos momentos.

Ejecutar la aplicación

Para ejecutar una aplicación React JS, se utiliza el comando `npm start`. Para generar la versión de producción, se debe ejecutar el siguiente comando: `npm run build`.

```
> my-app@0.1.0 build /home/chmunoz/my-app
```

```
> react-scripts build
```

```
Creating an optimized production build...
```

```
Compiled successfully.
```

```
File sizes after gzip:
```

```
41.34 KB build/static/js/2.8833cde.chunk.js
```

```
1.62 KB build/static/js/3.ff0311e9.chunk.js
```

```
1.38 KB build/static/js/runtime~main.1c420381.js
```

```
1.31 KB build/static/js/main.68b9e600.chunk.js
```

```
506 B build/static/css/main.cfa70044.chunk.css
```

```
The project was built assuming it is hosted at /.
```

```
You can control this with the homepage field in your package.json.
```

The build folder is ready to be deployed.

You may serve it with a static server:

```
npm install -g serve
```

```
serve -s build
```

Find out more about deployment here:

```
https://cra.link/deployment
```

Este comando genera la carpeta build con la versión compilada y minificada del proyecto React SPA (Single Page Application). A continuación, se describirá cómo debe quedar la organización y el registro de este proceso.

```
build
```

```
├─ asset-manifest.json
```

```
├─ favicon.ico
```

```
├─ index.html
```

```
├─ logo192.png
```

```
├─ logo512.png
```

- |— manifest.json
- |— robots.txt
- └─ static
 - |— css
 - | |— main.a617e044.chunk.css
 - | |— main.a617e044.chunk.css.map
 - |— js
 - | |— 2.8833cd6e.chunk.js
 - | |— 2.8833cd6e.chunk.js.LICENSE.txt
 - | |— 2.8833cd6e.chunk.js.map
 - | |— 3.ff0311e9.chunk.js
 - | |— 3.ff0311e9.chunk.js.map
 - | |— main.6bc689e0.chunk.js
 - | |— main.6bc689e0.chunk.js.map
 - | |— runtime-main.1c420381.js
 - | |└─ runtime-main.1c420381.js.map
- └─ media
 - └─ logo.6ce24c58.svg

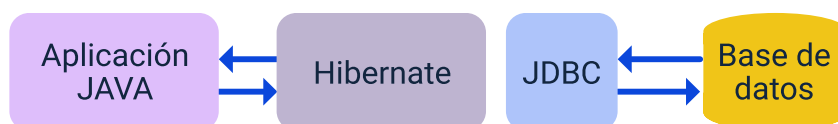
4. Framework Hibernate

Hibernate es software libre con licencia GNU LGPL que presta servicios de ORM para Java. Es un plugin que puede ser integrado al IDE de desarrollo en Java y facilita el proceso de mapeo de tablas en una base de datos relacional a objetos Java mediante el uso de archivos declarativos en formato XML o anotaciones.

“Hibernate utiliza JDBC para todos los procesos de comunicación con la base de datos, funcionando como una capa adicional de abstracción de las funcionalidades del JDBC. En una aplicación Java, podremos delegar la mayoría de las operaciones de bases de datos al framework de Hibernate, pero en segundo plano todo estará pasando por la API de JDBC” (Martinez, 2015).

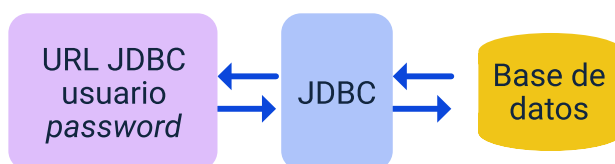
Así que, cuando se configura Hibernate para conectarse a una base de datos, lo que realmente se está haciendo es configurar Hibernate para el uso del JDBC, y es este quien realiza las operaciones. A continuación, se presenta el esquema general del funcionamiento:

Figura 30. Hibernate



Teniendo en cuenta que Hibernate debe usar el driver JDBC para la ejecución de las instrucciones en la base de datos, se debe construir un archivo de configuración en formato XML donde se especifique la URL del JDBC y las credenciales de acceso a la base de datos. En la siguiente figura se presenta el esquema general de funcionamiento en el proceso de configuración:

Figura 31. Hibernate archivo de configuración



Este archivo de configuración define un conjunto de propiedades que serán utilizadas para crear las sesiones mediante las cuales se inician un conjunto de transacciones u operaciones sobre la base de datos. Entre las propiedades más importantes se encuentran las siguientes:

Tabla 3. Propiedades Hibernate

Métodos de ciclo de vida de React	Descripción
connection.driver.class	Se ejecuta justo antes del primer pintado del componente.
connection.url	Se invoca cuando las propiedades se van a actualizar.
connection.username	Se invocará cuando se producen cambios de propiedades o cambios de estado.
connection.password	Este se invocará justo antes de que el componente vaya a actualizar su vista.
connection.pool_size	Este se invocará justo después de que el componente vaya a actualizar su vista.
Dialect	Se ejecuta en el momento que el componente se va a retirar del DOM.
show_sql	Se ejecuta en el momento que el componente se va a retirar del DOM.
current_session_context_class	Especifica el contexto de la sesión actual, por defecto se puede asignar thread.

Es importante tener en cuenta que este archivo de configuración de Hibernate debe estar ubicado en la raíz de la carpeta src del proyecto, de forma que pueda ser utilizado por Hibernate en cualquier contexto de la aplicación. A continuación, se muestra cómo debe verse un archivo de configuración de Hibernate completo, donde se establece una conexión con una base de datos MySQL en un servidor que se está ejecutando en la máquina local (localhost) y puerto 3306, con el usuario “root” y contraseña 12345:

```
<!DOCTYPE hibernate-configuration PUBLIC
```

```
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
```

```
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
```

```
<hibernate-configuration>
```

```
<session-factory>
```

```
<property
```

```
name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
```

```
<property
```

```
name="connection.url">jdbc:mysql://localhost:3306/prueba?useSSL=false&serverTime  
zone=UTC</property>
```

```
<property name="connection.username">root</property>
```

```
<property name="connection.password">12345</property>
```

```
<!-- JDBC connection pool settings ... using built-in test pool -->
```

```
<property name="connection.pool_size">1</property>

<!-- Select our SQL dialect -->

<property name="dialect">org.hibernate.dialect.MySQLDialect</property>

</session-factory>

</hibernate-configuration>

<!-- Echo the SQL to stdout -->

<property name="show_sql">true</property>

<!-- Set the current session context -->

<property name="current_session_context_class">thread</property>

</session-factory>

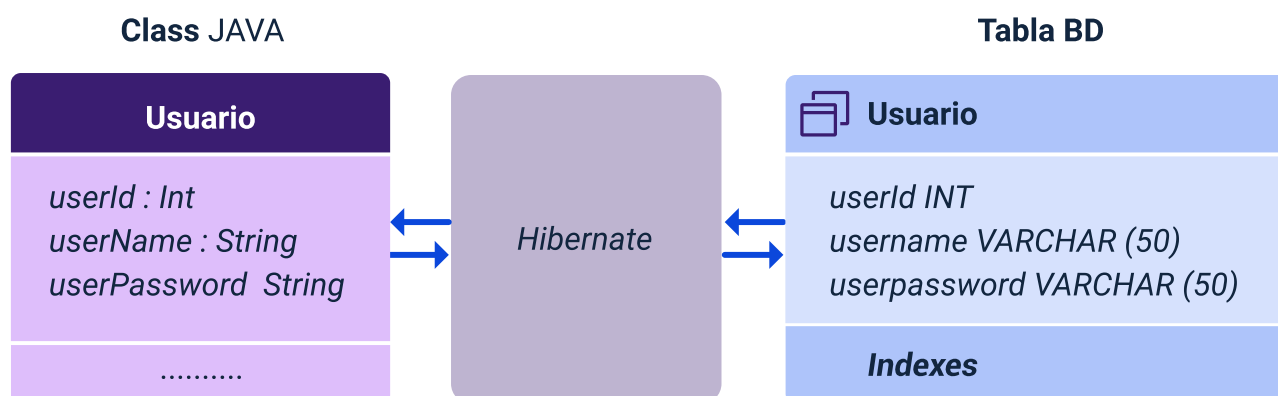
</hibernate-configuration>
```

Se invita a revisar en los anexos el archivo de configuración mostrado anteriormente para que pueda ser descargado y utilizado en la construcción de su proyecto, teniendo en cuenta que debe editar las propiedades de acuerdo con sus necesidades.

Luego de esto, se requiere definir todas las “Entity Classes” que serán utilizadas por Hibernate para realizar el proceso de mapeo hacia una base de datos relacional y viceversa. Una “Entity Class” básicamente es una clase Java tradicional que tiene las mismas características definidas en una tabla de la base de datos, es decir, es una imagen de la tabla, pero desde la perspectiva de la programación orientada a objetos.

Adicionalmente, tiene unas anotaciones especiales que le permiten a Hibernate realizar el trabajo adecuado.

Figura 32. Entity Classes



Las anotaciones pueden ser utilizadas al importar la librería de persistencia de JAVA `javax.persistence.*`, entre las más utilizadas encontramos:

Tabla 4. Anotaciones Java

Anotación	Significado
@Entity	Especifica que la clase definida en una Entity Class.
@Table(name = " ")	Permite especificar que la clase será mapeada con una tabla en la base de datos especificada en la propiedad name.
@Column(name = " ")	Permite especificar que el atributo por ser definido será mapeado con una columna especificada por la propiedad name. Cuando el nombre del atributo coincide con el nombre de la columna en la tabla se puede omitir la especificación de la propiedad name y solo dejar la anotación @Column.
@Id	Permite especificar que el atributo a definir corresponde con el identificador primario a ser mapeado en la tabla.

A continuación, se presenta un corto video donde se agrega el archivo de configuración de Hibernate al proyecto y se crea la clase Entity para la gestión de una tabla en una base de datos MySQL.

Video 5. Creación Clase Entity



[Enlace de reproducción del video](#)

Síntesis del video: Creación Clase Entity

En el video "Creación Clase Entity", se explica cómo configurar un proyecto para incluir un archivo de configuración de Hibernate y definir las clases de entidad necesarias para manipular una base de datos MySQL. Primero, se reutiliza un archivo de configuración previamente definido, el cual se coloca en la raíz del directorio de recursos del proyecto. Este archivo define propiedades esenciales para la conexión,

como el Driver Class y la URL de conexión. Luego, se crea una nueva clase de entidad en Java que refleja la estructura de la tabla usuario en la base de datos. La clase se organiza en un paquete llamado `com.ejemplo.entidad` y contiene los atributos `userID`, `username`, y `userPassword`. También se definen un constructor por defecto y un constructor parametrizado, además de los métodos `set` y `get` para cada atributo.

Posteriormente, se añaden anotaciones a la clase de entidad para especificar que es una entidad (`@Entity`) y se mapea a la tabla usuario utilizando la anotación `@Table`. Cada atributo se mapea a su columna correspondiente en la base de datos mediante las anotaciones `@Id` y `@Column`. Una vez definidas las clases de entidad, se pueden construir las operaciones de inserción, eliminación y actualización en la base de datos utilizando la configuración establecida. El video proporciona una guía detallada sobre cómo integrar Hibernate en un proyecto Java y definir clases de entidad para interactuar con una base de datos MySQL, permitiendo manipular los datos de manera eficiente y estructurada.

4.1. Operaciones básicas

Una vez se tienen registradas todas las dependencias, los archivos de configuración de Hibernate y las respectivas clases de entidad, se pueden utilizar las facilidades ofrecidas por Hibernate para la manipulación de la base de datos de forma transparente por medio de los procesos de mapeo que ofrece este framework.

Hibernate utiliza principalmente dos clases que debemos conocer y utilizar, las cuales se describen a continuación:

- **SessionFactory**

Esta clase lee el archivo de configuración de Hibernate y crea los objetos de sesión. Por ser una clase que genera objetos pesados por los procesos de los que es responsable solo debo crearlo una vez por toda la aplicación.

- **Session**

Esta clase envuelve todo lo correspondiente a una conexión JDBC y es el tipo de objeto que será usado para guardar o recibir objetos desde y hacia la base de datos. Es instanciado a partir de una clase SessionFactory. Los objetos creados de este tipo son más livianos por lo que pueden ser creados y desechados tantas veces como se necesite dentro de la aplicación.

En el siguiente fragmento de código se detalla cómo es el proceso de definición de los objetos SessionFactory y Session para poder realizar la manipulación de la base de datos:

```
// creación del session factory

SessionFactory factory = new Configuration()

    .configure("hibernate.cfg.xml")

    .addAnnotatedClass(Usuario.class)

    .buildSessionFactory();

// creación de la session a partir del Session factory

Session session = factory.getCurrentSession();
```

```
try {  
  
    // Ahora se puede usar el objeto session para la manipulación de la BD  
  
} finally {  
  
    // Se cierra el objeto session factory una vez se deja de utilizar  
  
    factory.close();  
  
}
```

Note que en el código anterior, para la creación del objeto `SessionFactory`, se indica el nombre del archivo de configuración de Hibernate en el método `configure()`. Si este parámetro no se incluyera, Hibernate buscará en el directorio raíz del proyecto un archivo que contenga el nombre por defecto de este tipo de archivos de configuración, el cual es `hibernate.cfg.xml`.

También note que en el método `addAnnotatedClass()` se indica la clase de entidad a ser utilizada.

Método Save

El método `save()` del objeto `Session` permite realizar el proceso de inserción de un nuevo registro en la base de datos enviando como parámetro un objeto de tipo entidad con sus valores. La forma correcta de realizar un proceso de inserción de un nuevo registro en una base de datos con un objeto de tipo sesión incluye los siguientes pasos:

- a) Iniciar una transacción por medio del método `beginTransaction()`.
- b) Ejecutar la lógica necesaria para crear un objeto de la clase de entidad con sus correspondientes valores.
- c) Invocar el método `save()` de la sesión enviando como parámetro el objeto de la clase entidad.
- d) Confirmar la transacción realizada por medio del método `commit()`.

```
import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import com.jonathan.hibernate.demo.entity.Usuario;

public class CrearUsuarioDemo {

    public static void main(String[] args) {

        // Crear SessionFactory

        SessionFactory factory = new Configuration()

            .configure("hibernate.cfg.xml")

            .addAnnotatedClass(Usuario.class)

            .buildSessionFactory();
```

```
// Crear sesión

Session session = factory.getCurrentSession();

try {

    System.out.println("Creando un usuario...");

    // Crear un objeto Usuario

    Usuario usuario1 = new Usuario("Francisco", "FFFFF");

    // Iniciar una transacción

    session.beginTransaction();

    // Guardar el objeto Usuario

    System.out.println("Guardando el nuevo usuario...");

    session.save(usuario1);

    // Confirmar la transacción

    session.getTransaction().commit();
```

```
        System.out.println("Proceso finalizado.");  
  
    } finally {  
  
        factory.close();  
  
    }  
  
}  
  
}
```

Método Get

El método `get()` del objeto `session` permite obtener un registro de la base de datos por medio de una consulta por su llave primaria. Este método recibe como parámetro la clase de entidad que será utilizada para hacer el mapeo del registro en la respuesta y un segundo parámetro que corresponde al valor a ser buscado en la llave primaria de la tabla. Este método ofrece como resultado un objeto de la clase de entidad especificada en la invocación del método.

La forma correcta de realizar un proceso de consulta de un registro particular por medio de su llave primaria y un objeto de tipo sesión incluye los siguientes pasos:

- a) Iniciar una transacción por medio del método `beginTransaction()`.
- b) Invocar el método `get()` de la sesión, enviando como parámetro la clase de tipo entidad para hacer el mapeo y el valor de la llave primaria a buscar. La invocación de este método requiere de un objeto compatible que reciba el resultado.

- c) Confirmar la transacción realizada por medio del método
`getTransaction().commit()`.

```
import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import com.jonathan.hibernate.demo.entity.Usuario;


public class CrearLeerDemo {


    public static void main(String[] args) {

        // Crear SessionFactory

        SessionFactory factory = new Configuration()

            .configure("hibernate.cfg.xml")

            .addAnnotatedClass(Usuario.class)

            .buildSessionFactory();


        // Crear sesión

        Session session = factory.getCurrentSession();
```

```
try {  
  
    // Iniciar una transacción  
  
    session.beginTransaction();  
  
  
    // Consulta de un usuario cuyo pk tiene el valor de 16  
  
    System.out.println("Consulta de un usuario");  
  
    Usuario resultado = session.get(Usuario.class, 16);  
  
    System.out.println("Resultado = " + resultado);  
  
  
    // Confirmar la transacción  
  
    session.getTransaction().commit();  
  
    System.out.println("Proceso finalizado.");  
  
} finally {  
  
    factory.close();  
  
}  
  
}
```

Método CreateQuery

Hibernate permite el proceso de creación de consultas usando HQL, el cual es una versión especial de Hibernate para establecer consultas SQL orientadas a objetos y que comprende nociones propias del paradigma orientado a objetos, como la herencia, el polimorfismo y las asociaciones. Es importante tener en cuenta que, aunque las palabras reservadas y sentencias de SQL no son sensibles a mayúsculas, los nombres de las clases y sus atributos usados en HQL sí lo son.

Para el caso en el que se requiera realizar consultas más avanzadas, se puede utilizar la potencia de HQL para establecer las características de los registros a ser consultados. Como este tipo de consulta puede producir más de un registro proveniente de la base de datos, es necesario usar el método `getResultList()` para mapear la respuesta a una lista de objetos Java. En la siguiente tabla se presentan algunas de las cláusulas HQL más comúnmente utilizadas. Se debe tener en cuenta que en HQL se hace referencia a las clases de entidad Java con sus atributos, los cuales son sensibles a mayúsculas. Estas sentencias son mapeadas a su estructura equivalente en la base de datos usando los nombres de tablas y atributos, los cuales ya no son sensibles a mayúsculas.

Tabla 5. Sentencias y equivalentes

Sentencia HQL	Equivalente en SQL
from Usuario	SELECT * FROM usuario
from Usuario u where u.userName = 'Raul'	SELECT * FROM usuario U WHERE u.username = 'Raul'
from DomesticCat cat	SELECT * FROM domesticCat Cat

Sentencia HQL	Equivalente en SQL
order by cat. name asc	ORDER BY Cat. name ASC
from Cat as cat	SELECT * FROM cat cat
inner join Cat. mate as mate	INNER JOIN cat_mate mate ON Cat. id = mate. id

A continuación, se presentan diferentes ejemplos haciendo uso de consultas por medio de HQL:

```
import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import com.javahibernate.hibernate.demo.entity.Usuario;
```

```
public class ConsultarUsuarios {

    public static void main(String[] args) {

        // Crear el SessionFactory

        SessionFactory factory = new Configuration()

            .configure("hibernate.cfg.xml")

            .addAnnotatedClass(Usuario.class)

            .buildSessionFactory();
```

```
// Crear el Session

Session session = factory.getCurrentSession();

try {

    // Consultar todos los usuarios de la tabla Usuario

    List<Usuario> usuariosRegistrados = session.createQuery("from
Usuario").list();

    for (Usuario usuario : usuariosRegistrados) {

        System.out.println(usuario);

    }

    // Consultar usuarios con nombre que comienza con 'J'

    usuariosRegistrados = session.createQuery("from Usuario u where
u.nombre like 'J%").list();

    for (Usuario usuario : usuariosRegistrados) {

        System.out.println(usuario);

    }

    // Consultar usuarios con apellido 'Ramirez'
```



```
usuariosRegistrados = session.createQuery("from Usuario u where
u.apellido = 'Ramirez']").list();

for (Usuario usuario : usuariosRegistrados) {

    System.out.println(usuario);

}

// Consultar usuarios con email que contiene 'javacode'

usuariosRegistrados = session.createQuery("from Usuario u where u.email
like '%javacode%'").list();

for (Usuario usuario : usuariosRegistrados) {

    System.out.println(usuario);

}

// Consultar usuarios con nombre 'Francisco' o que su password es
'12345'

usuariosRegistrados = session.createQuery("from Usuario u where
u.nombre = 'Francisco' OR u.password = '12345']").list();

for (Usuario usuario : usuariosRegistrados) {

    System.out.println(usuario);

}
```

```
    } finally {  
  
        factory.close();  
  
    }  
  
}  
  
}
```

Actualizaciones

El proceso de actualización es muy similar al proceso de consulta. Se puede hacer uso directo de los objetos obtenidos de las clases de entidad mediante los métodos modificadores de sus atributos (métodos set()) y confirmar las transacciones realizadas. También se puede hacer uso de createQuery(), el cual permite enviar consultas HQL un poco más elaboradas, pero asociándolo con el método executeUpdate() para que los cambios se reflejen en la base de datos una vez se confirma la transacción.

```
import java.util.List;  
  
import org.hibernate.Session;  
  
import org.hibernate.SessionFactory;  
  
import org.hibernate.cfg.Configuration;  
  
import com.javahibernate.demo.entity.Usuario;  
  
  
public class ConsultarUsuario {
```

```
public static void main(String[] args) {  
  
    // Crear el SessionFactory  
  
    SessionFactory factory = new Configuration()  
  
        .configure("hibernate.cfg.xml")  
  
        .addAnnotatedClass(Usuario.class)  
  
        .buildSessionFactory();  
  
  
    // Crear el Session  
  
    Session session = factory.getCurrentSession();  
  
  
    try {  
  
        // Iniciar una transacción  
  
        session.beginTransaction();  
  
  
  
        // Consultar un usuario por su clave primaria igual a 6  
  
        Usuario consultaUsuario = session.get(Usuario.class, 6);  
  
        consultaUsuario.setUserName("Nuevo Raul");  
  
        System.out.println("Usuario = " + consultaUsuario.getUserId() + "  
Actualizado");  
    }  
}
```

```
// Confirmar la transacción

session.getTransaction().commit();

System.out.println("Proceso finalizado");


    } finally {

        factory.close();

    }

}

}

import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import com.javahibernate.demo.entity.Usuario;

public class ActualizarUsuarios {
```

```
public static void main(String[] args) {  
  
    // Crear el SessionFactory  
  
    SessionFactory factory = new Configuration()  
  
        .configure("hibernate.cfg.xml")  
  
        .addAnnotatedClass(Usuario.class)  
  
        .buildSessionFactory();  
  
  
    // Crear el Session  
  
    Session session = factory.getCurrentSession();  
  
  
    try {  
  
        // Iniciar una transacción  
  
        session.beginTransaction();  
  
  
        // Actualizar la contraseña de todos los usuarios cuyo password actual sea  
'678'  
  
        System.out.println("Actualizando varios usuarios...");  
  
        session.createQuery("update Usuario u set u.userPassword = 'Z@Q#J!'  
where u.userPassword = '678'")
```

```
.executeUpdate();

// Confirmar la transacción

session.getTransaction().commit();

System.out.println("Proceso finalizado");

} finally {

    factory.close();

}

}
```

Método Delete

El método `delete()` del objeto `session` permite eliminar un registro de la base de datos mediante un objeto recibido como parámetro, que identifica el registro de la tabla a eliminar. La forma correcta de realizar un proceso de eliminación de un registro particular incluye los siguientes pasos:

- a) Iniciar una transacción por medio del método `beginTransaction()`.
- b) Utilizar un mecanismo para obtener una instancia de la clase entidad con el objeto que referencia el registro a eliminar, ya sea por medio del método `get()` o `createQuery()`.

- c) Invocar el método delete() del objeto session, pasando como parámetro el objeto que referencia el registro a eliminar.
- d) Confirmarla transacción realizada por medio del método getTransaction().commit().

```
import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import com.javahibernate.demo.entity.Usuario;

public class EliminarUsuarioDemo {

    public static void main(String[] args) {

        // Crear el SessionFactory

        SessionFactory factory = new Configuration()

            .configure("hibernate.cfg.xml")

            .addAnnotatedClass(Usuario.class)

            .buildSessionFactory();

        // Crear el Session

        Session session = factory.getCurrentSession();
```

```
try {  
  
    // Iniciar una transacción  
  
    session.beginTransaction();  
  
  
    // Eliminar un usuario cuyo id es 6  
  
    System.out.println("Eliminación de un usuario");  
  
    Usuario usuarioDelete = session.get(Usuario.class, 6);  
  
    session.delete(usuarioDelete);  
  
    System.out.println("Usuario " + usuarioDelete.getUserName() + "  
eliminado");  
  
  
    // Confirmar la transacción  
  
    session.getTransaction().commit();  
  
    System.out.println("Proceso finalizado");  
  
  
} finally {  
  
    factory.close();  
  
}
```



```
}  
  
}
```

```
import java.util.List;  
  
import org.hibernate.Session;  
  
import org.hibernate.SessionFactory;  
  
import org.hibernate.cfg.Configuration;  
  
import com.javahibernate.demo.entity.Usuario;  
  
public class EliminarUsuarioDemo {  
  
    public static void main(String[] args) {  
  
        // Crear el SessionFactory  
  
        SessionFactory factory = new Configuration()  
            .configure("hibernate.cfg.xml")  
            .addAnnotatedClass(Usuario.class)  
            .buildSessionFactory();  
  
  
        // Crear el Session
```

```
Session session = factory.getCurrentSession();

try {

    // Iniciar una transacción

    session.beginTransaction();

    // Eliminar todos los usuarios cuyo id es 6

    System.out.println("Eliminando el usuario de id = 6");

    session.createQuery("delete from Usuario u where u.userId =
6").executeUpdate();

    // Confirmar la transacción

    session.getTransaction().commit();

    System.out.println("Proceso finalizado");

} finally {

    factory.close();

}

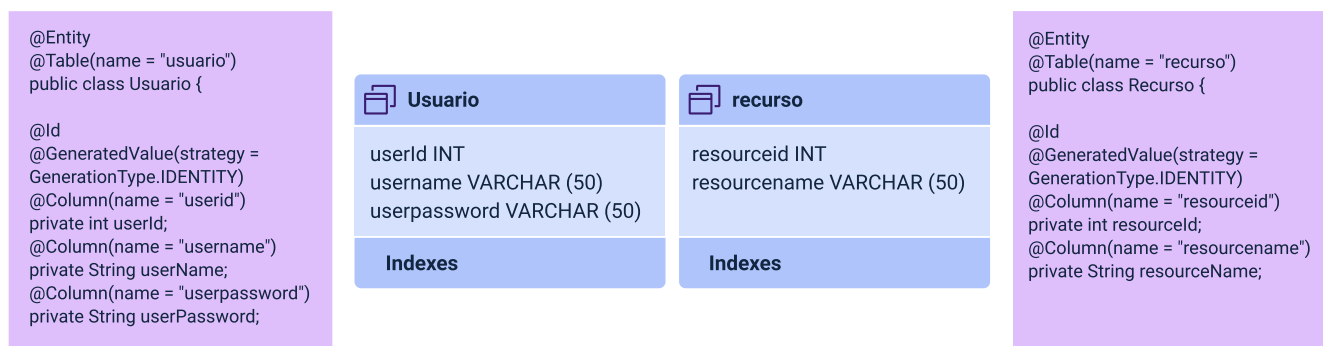
}
```

4.2. Asociaciones

Normalmente, al desarrollar una aplicación que gestione una base de datos, se requiere mucho más que una única tabla para la gestión de la información. Es común tener modelos de datos con múltiples tablas relacionadas con asociaciones de diferentes tipos y diferentes cardinalidades. Por lo tanto, es importante establecer en las clases de entidad este tipo de relaciones para que el proceso de mapeo sea coherente. Los tipos de asociaciones más comúnmente utilizadas son las que expresan relaciones uno a uno, relaciones uno a muchos, relaciones muchos a uno y relaciones muchos a muchos.

Para explicar cada uno de estos casos, utilizaremos como referencia el siguiente modelo de datos que ya fue mapeado de forma individualizada. A medida que se agregue una asociación de diferente tipo, se establecerán los cambios que deberían estar presentes, y que se detallan en la siguiente figura:

Figura 33. Asociaciones

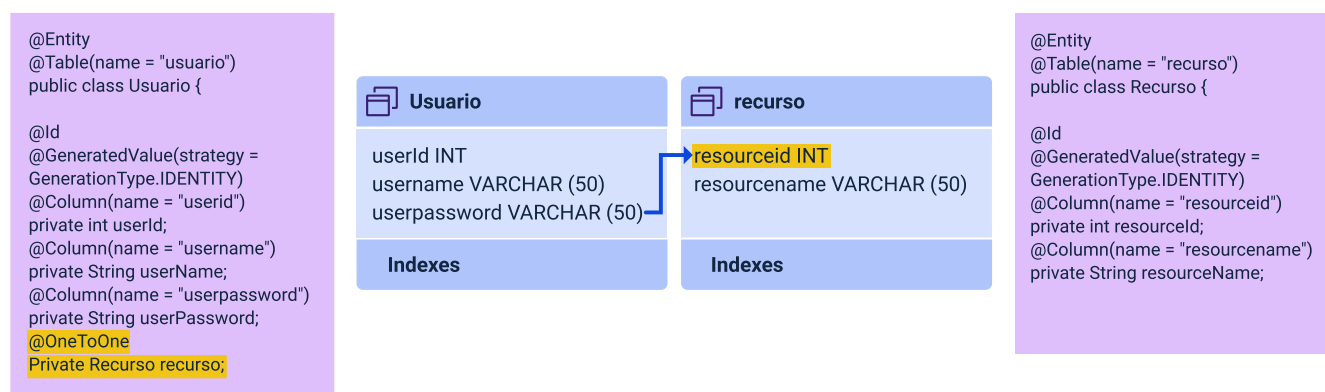


Relaciones OneToOne

En este tipo de relaciones, se quiere expresar un vínculo en el que un ejemplar de una tabla tiene relación con uno y solo un ejemplar de una segunda tabla. Desde el

punto de vista de las bases de datos, se creará una relación de integridad referencial vinculando una llave primaria con una llave foránea. Tomando como referencia el modelo de datos presentado anteriormente, supongamos que se desea indicar que un usuario tiene asociado un recurso. Para resolver este nuevo requerimiento, debemos modificar la clase de entidad de usuario para agregar un nuevo atributo que haga referencia al objeto recurso con el cual está relacionado y agregar la anotación correspondiente llamada OneToOne, de forma que Hibernate haga el mapeo correcto. El esquema se detalla a continuación en la siguiente figura:

Figura 34. Relaciones OneToOne

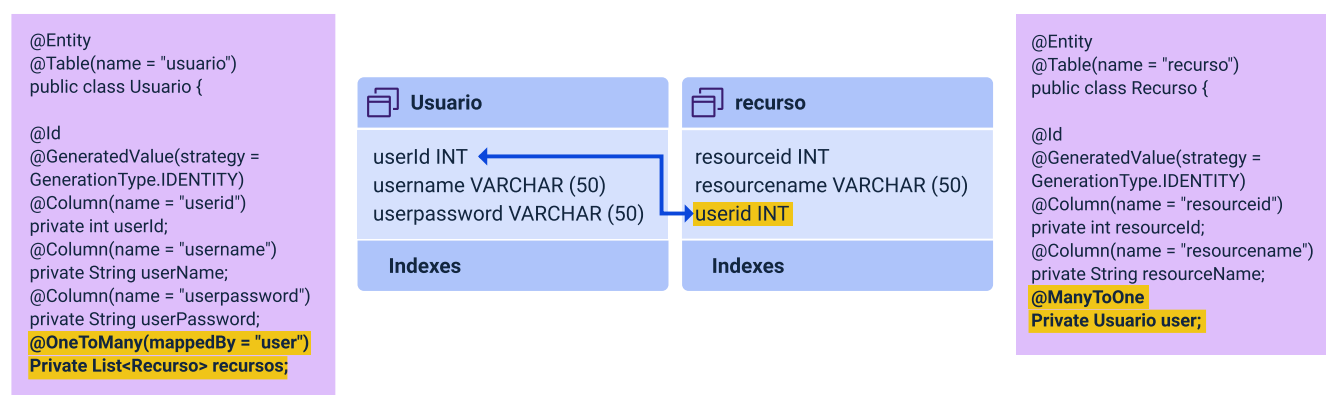


Mientras que en la base de datos se debe agregar un nuevo campo foráneo en la tabla usuario que haga referencia a la llave primaria de la tabla recurso para poder establecer la relación, en las clases de tipo entidad bastará con especificar el nuevo atributo del tipo de objeto correspondiente que referencia agregando la anotación de tipo OneToOne. Si el campo nuevo requerido en la tabla usuario no existe, el mapeador de Hibernate se encargará de agregarlo.

Relaciones OneToMany - ManyToOne

Las relaciones "OneToMany" tienen la particularidad de afectar cada una de las entidades vinculadas en la relación y es importante identificar en qué lugar es una relación "OneToMany" y en qué lugar la relación es "ManyToOne". Recordando las reglas de transformación del modelo lógico de una base de datos relacional, en este tipo de relaciones se indica que en la tabla del lado de la relación donde la cardinalidad máxima es muchos (*) deberá ser transferido como un campo foráneo la llave primaria de la tabla donde la cardinalidad máxima es 1, como se aprecia en la siguiente figura:

Figura 35. Relaciones OneToMany - ManyToOne



Para el caso de las clases de entidad, deberá especificarse en la clase donde la cardinalidad máxima es 1, un atributo del tipo lista de la clase de entidad con la que tiene un vínculo OneToMany. Este atributo será de tipo lista porque se está vinculando con varios. En el ejemplo, la entidad usuario se relaciona con varios recursos, es decir, un usuario puede estar vinculado con múltiples recursos.

Por otra parte, la clase de entidad donde la cardinalidad máxima es muchos deberá especificar un atributo de tipo objeto con la entidad vinculada, agregándole la anotación ManyToOne. En el ejemplo, un recurso estará siempre vinculado con un

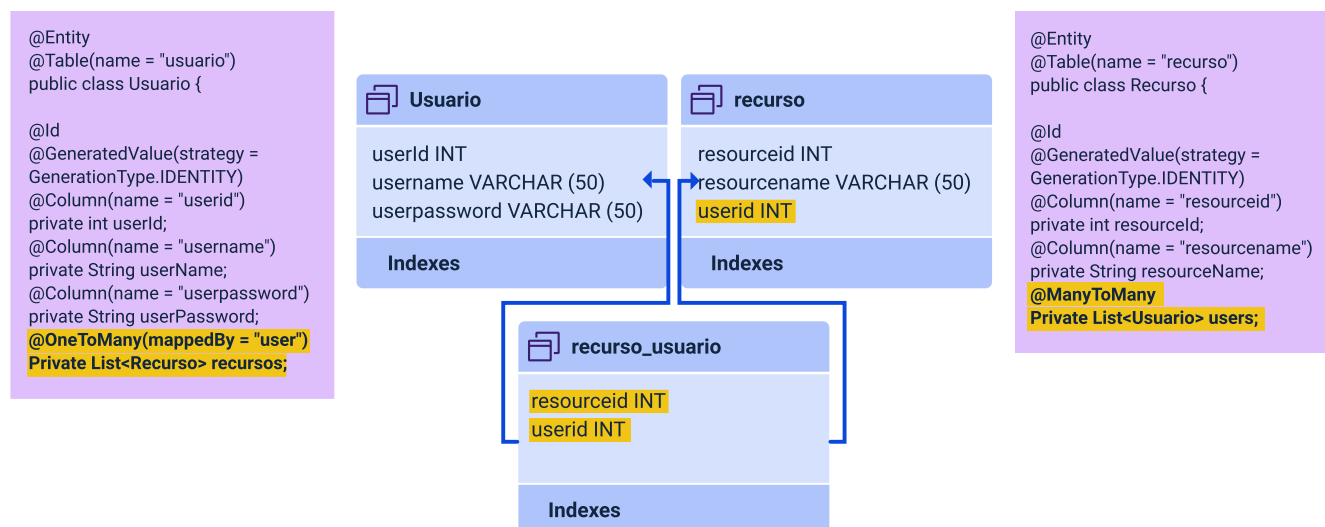
único usuario. Es importante anexar en la entidad de cardinalidad OneToMany el nombre del atributo en la entidad ManyToOne que será usado para el mapeo de la asociación por parte de Hibernate. En el ejemplo, se indica en la entidad Usuario que la relación OneToMany será mapeada usando el atributo user de la entidad ManyToOne.

Relaciones ManyToMany

Las relaciones ManyToMany obligan a la construcción de una nueva entidad de mapeo en la cual se alojarán las respectivas llaves primarias de las entidades vinculadas por medio de la relación ManyToMany. Para su correcta implementación, deberán agregarse en cada una de las clases entidad un objeto de tipo lista de la entidad a la cual está relacionada, etiquetada con la anotación ManyToMany. Adicionalmente, en una de las clases de entidad se deberá especificar el objeto de mapeo del otro extremo que será utilizado por Hibernate para realizar la asociación.

En el ejemplo siguiente, se establece una relación ManyToMany entre la clase Usuario y la clase Recurso, por lo cual en cada una de ellas se agrega la correspondiente lista del tipo de entidad a la cual se asocian con la anotación ManyToMany. En el caso de la clase Usuario, se agrega el atributo de tipo lista de Recursos llamado recursos para indicar que un usuario puede estar vinculado con muchos recursos. Por otro lado, en la clase Recurso se agrega una lista de tipo Usuario para indicar que un recurso puede estar vinculado con múltiples usuarios. Adicionalmente, la relación entre usuario y recurso se mapea con el objeto users presente en la clase Recurso. La relación entre recurso y usuario no se mapea explícitamente, por lo que generará la tabla intermedia recurso_usuario, lo cual cumple las reglas de transformación de un modelo lógico para una relación de muchos a muchos.

Figura 36. Relaciones ManyToMany



4.3. Ciclo de vida

Los objetos manipulados por Hibernate para el proceso de gestión de la persistencia pasa por cuatro estados diferentes, a continuación, se describe cada uno de ellos:

- **Estado transitorio**

Corresponde al primer estado de un objeto de entidad y sucede cuando instanciamos y manipulamos el objeto de tipo entidad sin conectarlo por medio de la sesión a un registro de la base de datos. Así, cualquier manipulación de este objeto no afecta la base de datos. En este estado, un objeto de tipo entidad se comporta como cualquier otro objeto JAVA.

- **Estado persistente**

Corresponde al estado que logra el objeto una vez se conecta con una sesión de Hibernate, lo cual ocurre cuando se invoca, por ejemplo, una operación de tipo save() o cuando el objeto fue producto de realizar un

proceso de consulta a la base de datos por un método `get()`. Todos los cambios que se realizan sobre un objeto en estado persistente son observados por Hibernate y podrán eventualmente realizar modificaciones en la base de datos.

- **Estado independiente**

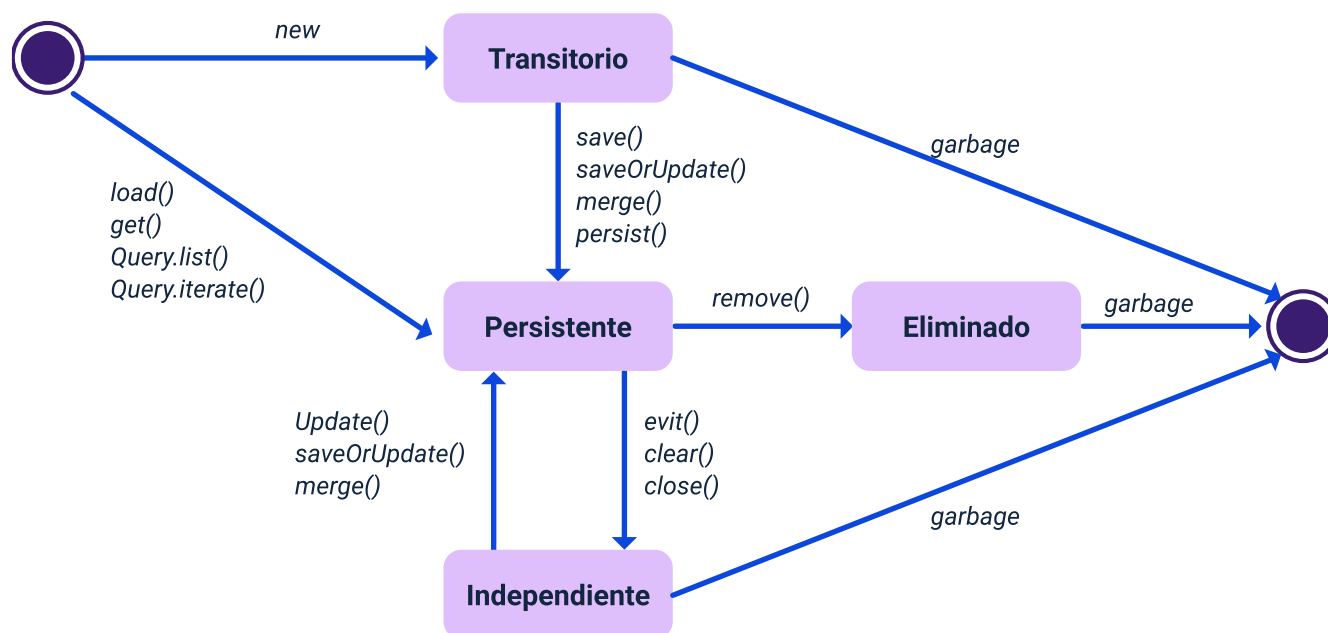
A este estado pasan los objetos en estado persistente una vez se realiza un cierre de sesión y en este momento son dejados de observar por Hibernate y cualquier cambio no será propagado en la base de datos. Un objeto independiente puede volver a ser persistente una vez se vuelva a abrir una nueva sesión y se use uno de los siguientes métodos: `save()` o `merge()`.

- **Estado eliminado**

Es el último estado en el ciclo de vida de un objeto de tipo entidad y ocurre cuando el objeto en cuestión es eliminado de la base de datos por medio del método `delete()`. Cualquier cambio realizado sobre este objeto no afectará la base de datos.

A continuación, se presenta un resumen de los estados del ciclo de vida y sus transiciones completas con todos los posibles métodos.

Figura 37. Ciclo de vida



5. Framework Spring

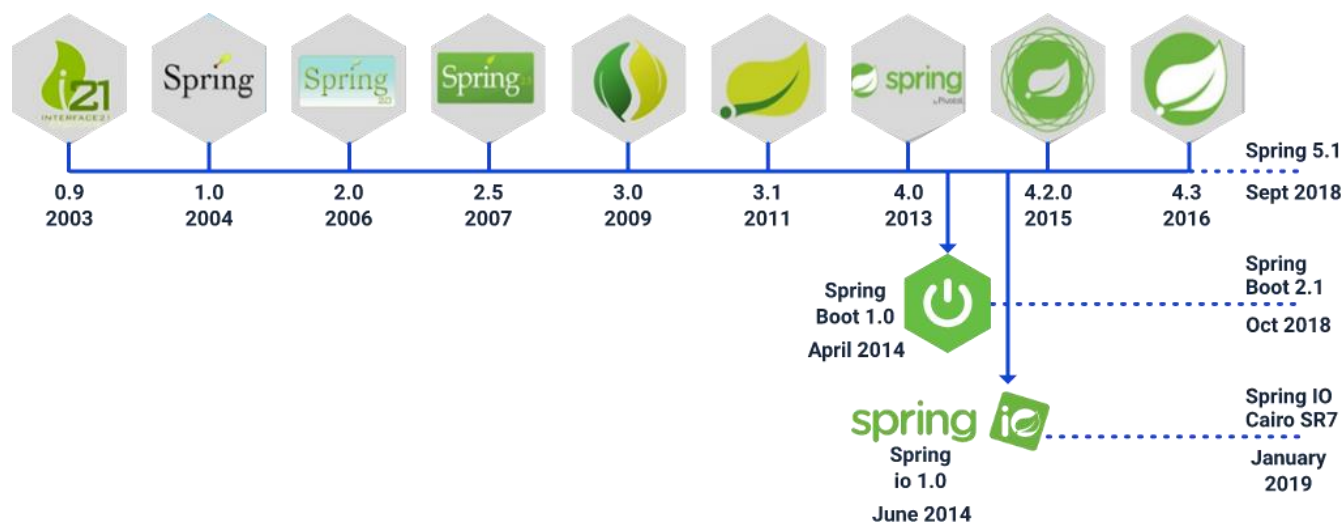
Spring es un framework de código abierto que facilita la creación de aplicaciones empresariales de alto rendimiento en Java. Además de Java, tiene soporte para los lenguajes Groovy y Kotlin. La estructura que lo compone es modular, ofreciendo flexibilidad para implementar una variedad de arquitecturas dependiendo de los requerimientos de las aplicaciones, haciendo que la programación sea lo más ágil posible, rápida y muy segura.

Historia

Rod Johnson, creador de Spring, se dio cuenta de que necesitaban una herramienta para simplificar y facilitar la construcción de aplicaciones JEE utilizando EJB. Escribió el libro *Expert One-on-One J2EE Development*, que es un conjunto de buenas prácticas para la creación de aplicaciones empresariales. En el año 2004, lanzó la primera versión 1.2.6 de Spring, marcando el inicio de J2EE tradicional (Johnson, 2004).

La siguiente figura presenta las versiones de Spring Framework:

Figura 38. Versiones de Spring Framework



A continuación, se presentan algunas de las características clave que hacen de Spring una opción popular entre los desarrolladores:

- **Flexibilidad**

Contiene un grupo flexible de librerías y extensiones de terceros de Spring, facilitando a los desarrolladores construir aplicaciones fácilmente.

- **Inyección de dependencia (DI)**

Ofrece un patrón que permite ayudar en el acoplamiento bajo. Esto significa que cuando existen dependencias entre las clases y se necesita crear un objeto de esta clase, Spring suministra este objeto, obteniendo beneficios en la modularidad, escalabilidad y dependencia de clases.

- **POJOs (Plain Old Java Objects)**

Facilita un sencillo desarrollo por medio de los POJOs.

- **ORM (Object Relational Mapping)**

Suministra un fácil acceso a datos por medio del ORM.

- **Estructura y módulos**

Spring integra una variedad de módulos que se pueden integrar para el desarrollo de aplicaciones.

A continuación, se detalla lo que se puede crear con Spring Framework: aplicaciones de consola, aplicaciones de escritorio, aplicaciones web, aplicaciones móviles, servicios SOAP y REST. Spring está orientado esencialmente al back-end.

- **Microservicios**

Permite crear y ejecutar microservicios en producción a escala.

- **Nube**

Ejecución de servicios en la nube

- **Aplicaciones web**

Creación rápida de aplicaciones web.

- **Reactive**

Permite la creación de proyectos reactivos basados en mensajes.

- **Servlets**

Permite la ejecución de código impulsado por eventos y se encarga del escalado la se tridad del lado del servidor.

- **Batch**

Aplicaciones de procesamiento de datos en lote.

5.1. Introducción

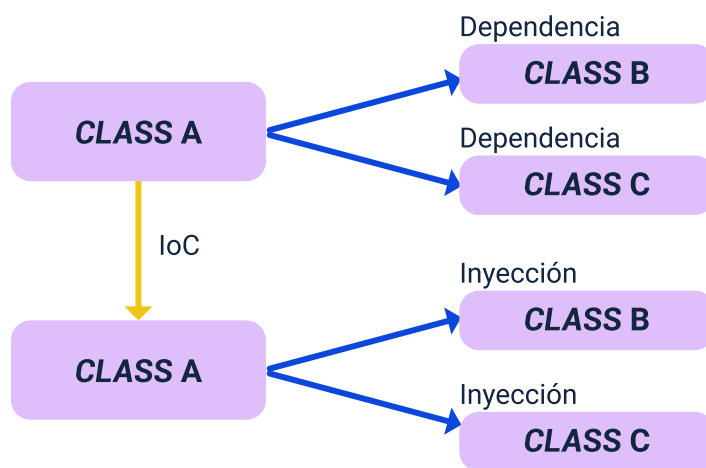
El framework Spring, básicamente con su inyección de dependencias, se encarga de construir objetos y configurar todas las dependencias que estos necesiten. En Spring, estos objetos creados son llamados beans, y se anotan en las clases de la

siguiente manera: @Component, @Service, @Repository, etc. Uno de los principios principales de Spring es el de Inversión de Dependencia (DIP), el cual establece que los módulos de alto nivel no deben depender de los de bajo nivel. Esto significa que se debe hacer referencia a las abstracciones más que a las implementaciones, evitando depender de una clase o implementación en particular.

La implementación de este principio se llama Inversión de Control (IoC), que se encarga de desacoplar el código utilizando eventos. Este principio también se conoce como el principio de Hollywood: "no nos llames; nosotros te llamaremos". Consiste en una manera de programar donde la aplicación interviene en lugar del código. Para implementar el IoC, se necesita un agente externo llamado contenedor, que es el encargado de verificar el flujo de la aplicación. Los IoC utilizan Service Locator, eventos y DI (inyecciones de dependencias), enfocando el Spring Framework en la DI.

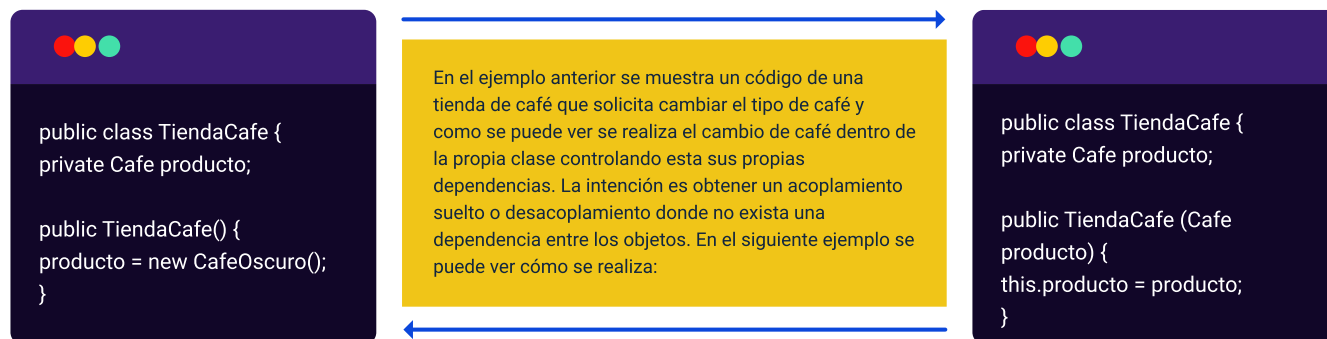
Inyección de dependencias: es un patrón de diseño de software, subtipo de inversión de control, cuyo objetivo es tener un código fácil de mantener o desacoplado. Su función es determinar lo que los objetos necesitan.

Figura 39. Principio inversión de controlguración



A continuación, se presenta un ejemplo de código donde los objetos dependen entre sí:

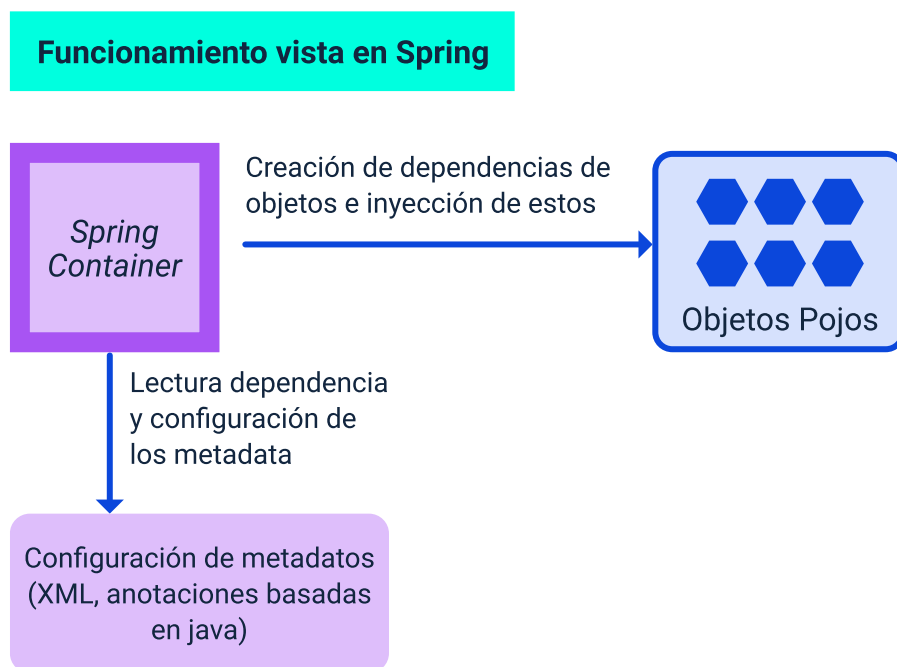
Figura 40. Acoplamiento hermético



En el código anterior, se obtuvo el objeto desde el exterior y que el objeto puede ser cualquier tipo de café, sin necesidad de modificar la clase `TiendaCafe`. Resumiendo, la inversión de control es un patrón de programación para obtener un acoplamiento flexible, logrando que los objetos independientes no sean responsables de sus dependencias, sino que las reciban desde afuera. El contenedor de IoC recoge los metadatos que vienen de un archivo XML o código Java y recibe las indicaciones de los objetos que debe crear, enlazar, configurar y destruir por medio de POJOs (Objetos simples de Java), creando objetos por medio del proceso que recibe el nombre de Spring Beans y resultando en objetos preparados para la inyección de dependencias.

La siguiente figura describe el funcionamiento de una vista abstracta de Spring.

Figura 41. Vista abstracta de Spring



Módulos del Framework

Spring Framework es un marco de trabajo muy amplio y brinda cerca de 20 módulos, los cuales se asocian en el contenedor principal: accesos e integración de datos, web, AOP (Programación orientada a aspectos), instrumentación, mensajes y pruebas (Spring, 2021).

Para trabajar con Spring se necesita tener instalados los siguientes requerimientos:

- **Un entorno de desarrollo (IDE).**

Se puede seleccionar cualquiera de los relacionados en la tabla No 6.

- **Un JDK (Kit de desarrollo de Java).**

Se puede descargar cualquiera de los relacionados en la tabla No 7.

Tabla 6. Entornos para desarrollo en Spring

Entornos de desarrollo	Enlace
IntelliJ IDEA	https://www.jetbrains.com/idea/
Spring Tools	https://spring.io/tools
Visual Studio Code	https://code.visualstudio.com/docs/languages/java
Eclipse	https://www.eclipse.org/downloads/packages/
NetBeans	https://netbeans.apache.org/

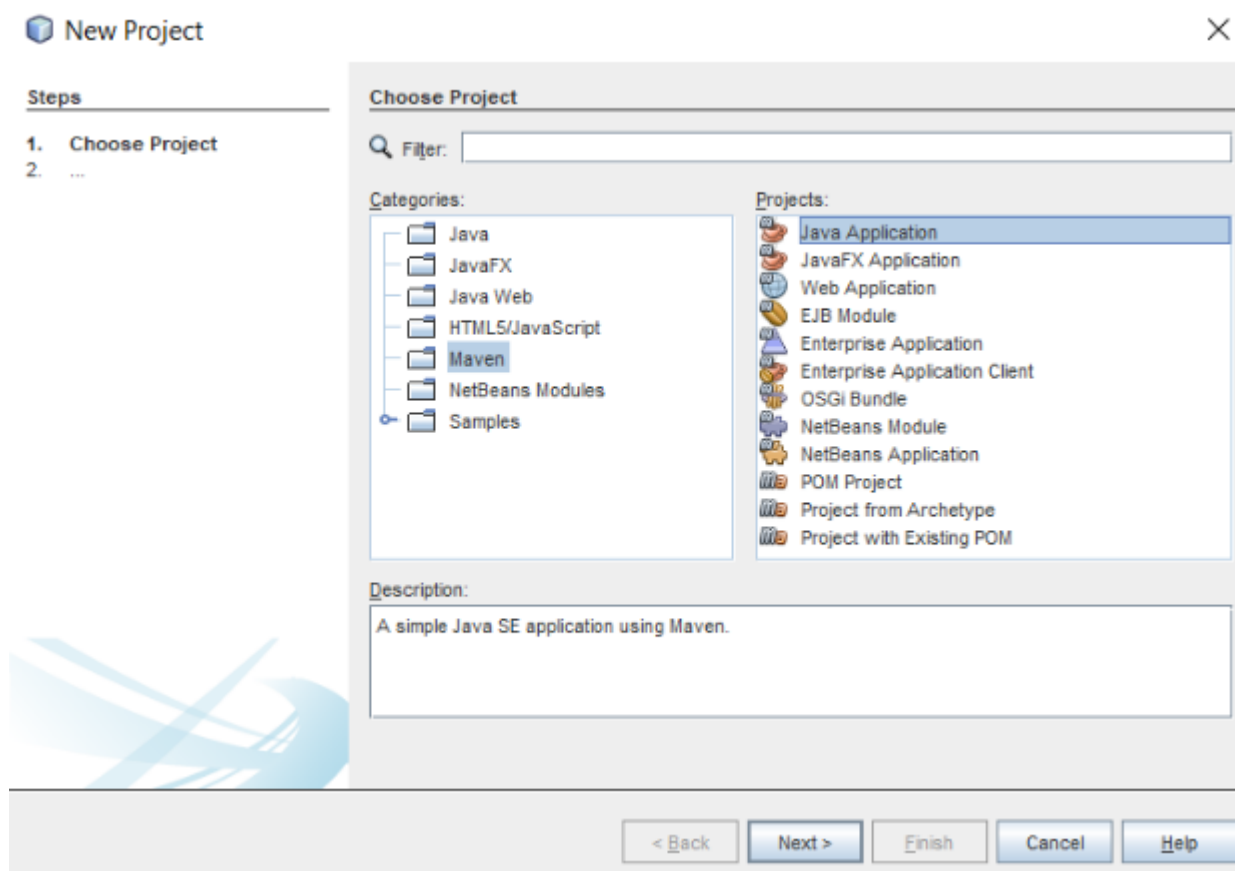
Tabla 7. JDK para desarrollo en Spring

JDK	Enlace
JDK Oracle	https://www.oracle.com/java/technologies/downloads/
AdoptOpenJDK	https://adoptopenjdk.net/

Para construir aplicaciones en Spring Framework necesitamos:

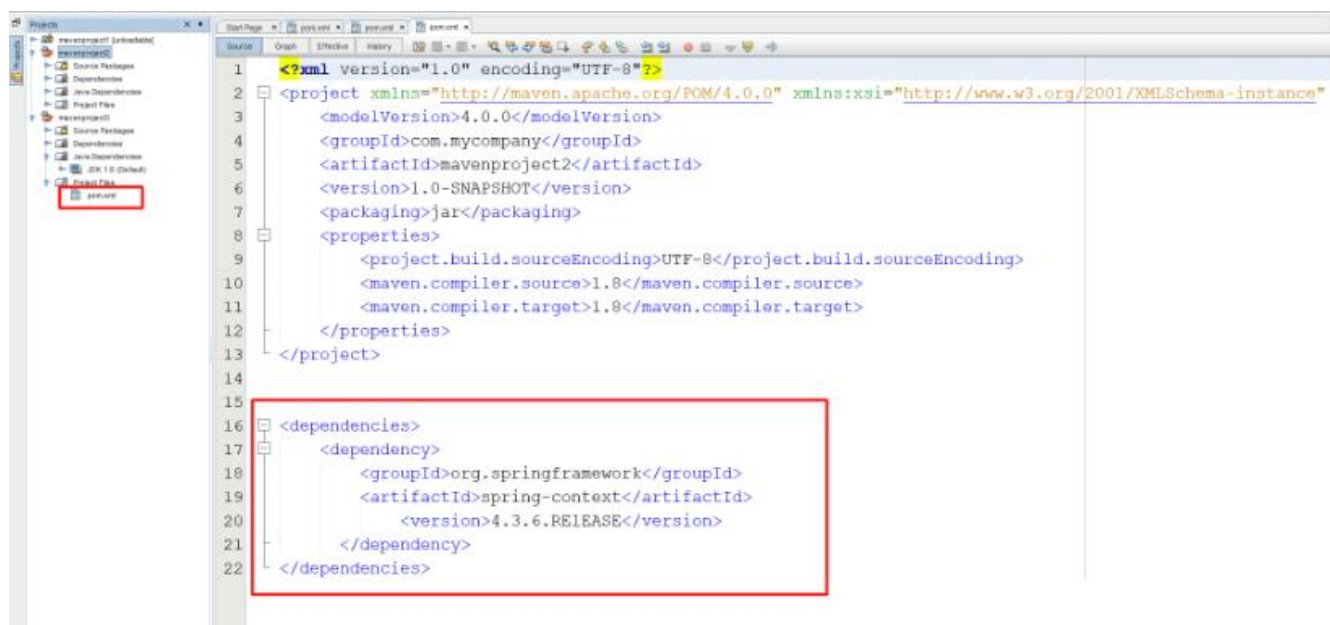
- Elegir Jars con Maven: lo primero que se debe hacer es crear un proyecto Maven o Gradle y realizar la descarga de las dependencias necesarias.

Figura 42. Ventana de creación de un nuevo proyecto en un IDE



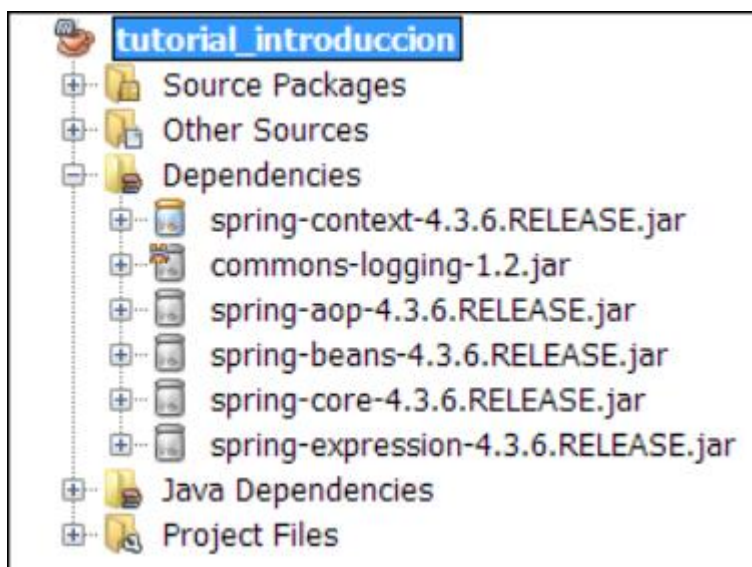
- Configuración de las dependencias en el archivo pom.xml.

Figura 43. Pantalla de un archivo XML de Maven



- Las dependencias se deben listar en la carpeta Dependencias.

Figura 44.



Creación de la aplicación

A continuación, se presenta como crear la aplicación:

- **Creación de la Aplicación**

Para el ejemplo se creará un programa con el ejemplo de HolaPlaneta.

```
public interface HolaPlaneta {  
  
    void enviarSaludo ();  
  
}
```

```
public class HolaPlanetaImpl implements HolaPlaneta {  
  
    @Override  
  
    public void enviarSaludo () {  
  
        System.out.println("Hola Planeta estamos en Spring Framework");  
  
    }  
  
}
```

- **Configurar archivo XML**

Una vez se tenga la aplicación, se realizará la configuración en el archivo springXMLconfig.

```
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans-4.3.xsd">  
  
    <bean id="saludaService" class="sena.spring.introduccion.HelloPlanetaImpl"/>  
  
</beans>
```

```
</beans>
```

- **Ejemplo de código**

A continuación, se presenta un nuevo ejemplo de código explicando más a detalle el uso del Contenedor en Spring. Para el siguiente ejemplo se tiene un proyecto llamado “Sena” el cual tiene una clase Aprendiz con los siguientes atributos:

```
package com.sena
```

```
public class AprendizBean {
```

```
    private String primernombre;
```

```
    private String apellidos;
```

```
    private String cedula;
```

```
    public String getPrimerNombre () {
```

```
        return primernombre;
```

```
    }
```

```
    public void setPrimerNombre (String nombre) {
```

```
        this.primernombre = nombre;
```

```
}
```

```
public String getApellidos () {
```

```
    return apellidos;
```

```
}
```

```
public void setApellidos (String apellidos) {
```

```
    this.apellido = apellidos;
```

```
}
```

```
public String getCedula () {
```

```
    return cedula;
```

```
}
```

```
public void setCedula (String cedula) {
```

```
    this.cedula = cedula;
```

```
}
```

```
public void verAprendiz () {
```

```
System.out.println(primernombre+" "+apellidos+", es la actual estudiante");  
  
}
```

```
}
```

- **Gestión de objetos**

Al utilizar este proyecto Sena, Spring framework cuenta con un archivo XML que gestiona objetos (Spring IOC Container). Para este proyecto, recibe el nombre de springconfig.xml y se ve de la siguiente manera.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>  
<beans xmlns="http://www.springframework.org/schema/beans" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/dtd/spring-beans-2.0.dtd">  
    <bean id="aprendiz" class="com.sena.AprendizBean">  
        <property name="primernombre" value="Zulema">  
            </property>  
        <property name="apellidos" value="Leon">  
            </property>  
        <property name="cedula" value="25.311.165">  
            </property>  
    </bean>  
</beans>
```

- **Configuración del archivo**

En la configuración del archivo, se ve que la etiqueta bean referencia la clase AprendizBean.java. Además, el atributo class dirige a la dirección del proyecto y

cuenta con un atributo id que permite agregar un nombre para poder identificarlo. Las demás etiquetas como property indican los atributos de la clase AprendizBean y con la etiqueta value se pueden agregar valores directos. Por medio de los métodos get y set, el Spring Container obtiene acceso y puede modificar los objetos de la clase AprendizBean. Con los archivos de configuración XML, Spring facilita una clase de listas por cada uno de los beans existentes para poder hacer llamado de ellos y así poder manipularlos desde otras clases sin tener que realizar varias instancias.

```
package com.sena
```

```
import org.springframework.beans.factory.BeanFactory;
```

```
import org.springframework.beans.factory.xml.XmlBeanFactory;
```

```
import org.springframework.core.io.ClassPathResource;
```

```
import org.springframework.core.io.Resource;
```

```
public class SenaLogic{
```

```
    public static void main(String[ ] args){
```

```
        Resource recurso = new ClassPathResource("springconfig.xml");
```

```
        BeanFactory factory = new XmlBeanFactory(recurso);
```

```
AprendizBean aprendizBean = (AprendizBean)factory.getBean("aprendiz");
```

```
aprendizBean.verAprendiz();
```

```
}
```

```
}
```

- **Implementación de Spring**

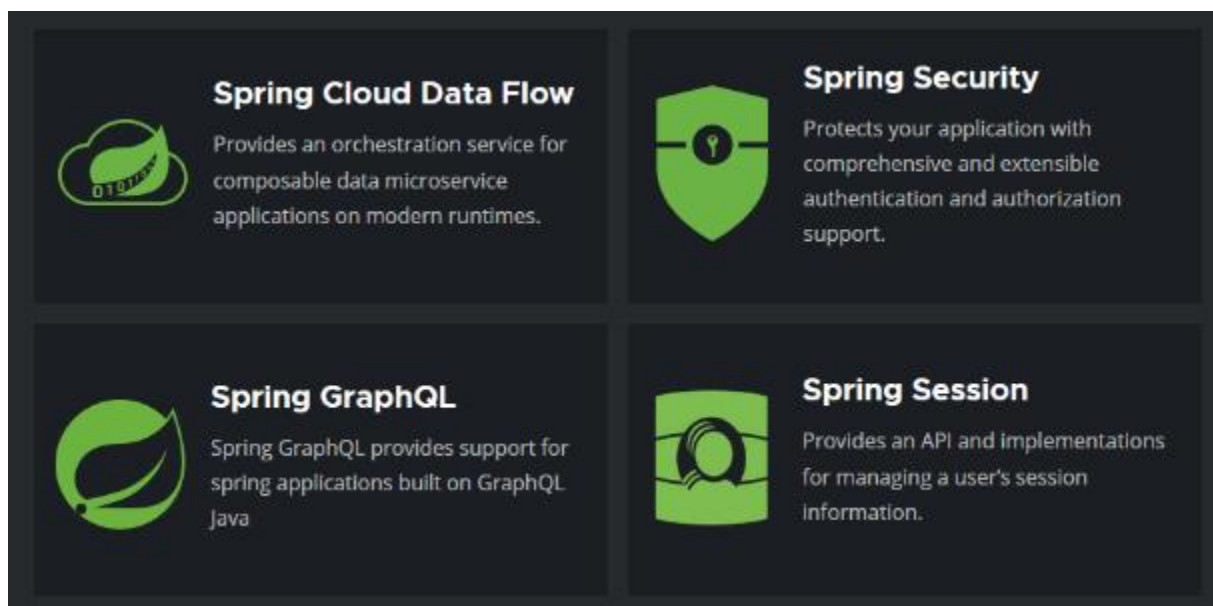
En el siguiente código se presenta la implementación de Spring del control de los objetos del proyecto Sena. La salida que muestra se logró realizando las configuraciones en el archivo XML. La utilidad del uso de Spring se evidencia en proyectos grandes con varias clases y varios objetos ya que facilita un manejo mucho más eficiente.

Zulema Leon, es la actual estudiante

5.2. Spring Data

Spring ofrece una variedad de proyectos independientes que se pueden articular en un proyecto principal. Cada proyecto permite resolver una necesidad especial del negocio; algunos de ellos se ilustran en la siguiente figura.

Figura 45. Spring



Uno de los proyectos más utilizados es Spring Data, ya que es uno de los módulos más grandes de Spring debido a que cuenta con muchos submódulos. Su objetivo principal es proporcionar una serie de herramientas que permiten realizar integraciones muy sencillas con los diferentes entornos de almacenes de datos disponibles. Los datos que se utilizan pueden proceder de orígenes tanto de bases de datos relacionales como no relacionales.

Las funcionalidades de Spring Data son:

- Poderosos repositorios y uso personalizado de mapeo de objetos.
- Creación de consultas dinámicas fundamentadas en los nombres que tienen los métodos.
- Plantillas de clases que brindan propiedades elementales que hacen más fácil utilizar los repositorios.

- Campos que hacen referencia a fecha de creación y modificación de las clases.
- Sencilla integración con Spring Framework por medio de la configuración de clase o archivos XML.

Módulos de Spring Data

Los módulos oficiales de Spring Data son:

- **Spring Data Commons.**
Brinda infraestructura compartida para módulos de Spring.
- **Spring Data JDBC.**
Implementa repositorios fundamentados en JDBC.
- **Spring Data JPA.**
Implementa repositorios fundamentados en JPA.
- **Spring Data Gemfire.**
Implementa GemFire para aplicaciones baja latencia.
- **Spring Data KeyValue.**
Implementa repositorios para datos de clave-valor.
- **Spring Data LDAP.**
Implementa repositorios para soporte de LDAP.
- **Spring Data MongoDB.**
Soporte para documentos y repositorios de MongoDB.
- **Spring Data Rest.**
Implementa repositorios Restful.

- **Spring Data Redis.**

Fácil acceso a Redis desde Spring.

- **Spring Data para Apache Cassandra.**

Implementación para aplicaciones orientadas a datos.

- **Spring Data para Apache Geode.**

Implementación para aplicaciones orientadas a datos.

A continuación, se presenta un ejemplo de un mapeo para JPA, que es uno de los submódulos de Spring Data. En el ejemplo, se desarrolla la clase Persona, la cual representa una tabla en la base de datos llamada Persona:

- a) La creación de la tabla se especifica mediante la anotación `@Entity` y en el atributo `name` se coloca el nombre de la tabla.
- b) Por medio de la anotación `@Column` se asigna cada uno de los campos de la tabla de la base de datos a los atributos de la clase Persona con la etiqueta "name" se hace referencia al nombre de la columna de la tabla.

```
@Entity(name = "persona")
```

```
public class Persona {
```

```
    @Column(name = "id")
```

```
    private Long id;
```

```
    @Column(name = "persona_nombre")
```

```
private String nombre;  
  
@Column(name = "persona_apellido")  
  
private String apellido;  
  
@Column(name = "persona_telefono")  
  
private String telefono;  
  
}
```

Al extender de la clase CrudRepository, se facilita el uso de métodos como findByNombre, que genera una consulta de todas las personas con el parámetro nombre. En bases de datos relacionales, esta operación se traduce en SELECT * FROM persona WHERE nombre=?. Este enfoque reduce considerablemente las sentencias SQL, permitiendo obtener consultas dinámicas de forma más eficiente.

5.3. Spring Boot y MySQL

Spring Boot es una infraestructura liviana que suprime en gran parte la tarea de configurar las aplicaciones fundamentadas en Spring. A diferencia de Spring Framework, Spring Boot realiza una simplificación en cuanto a configuración y el despliegue de las aplicaciones.

Las características de Spring Boot son:

- Es intuitivo: permite crear aplicaciones rápidamente gracias a sus valores predeterminados.

- Permite crear aplicaciones que son independientes de Spring.
- Incluye los servidores de Undertow, Tomcat o Jetty.
- Usualmente realiza la configuración automática de librerías de Spring o de otros.
- No requiere configuraciones de archivos XML.
- Agiliza el desarrollo.

Spring Boot cuenta con dos mecanismos que permiten la simplificación de los procesos: el contenedor de aplicaciones integrado y los starters.

Contenedor de aplicaciones integrado

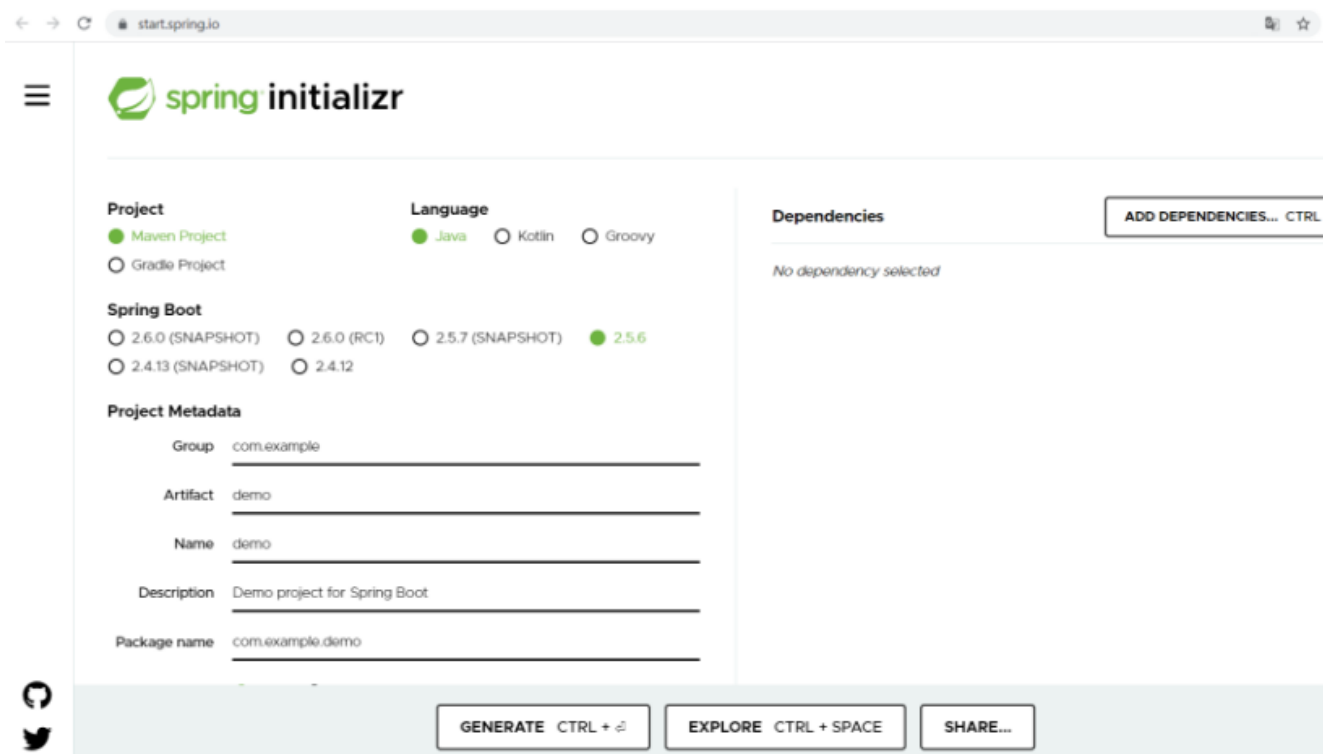
La compilación de las aplicaciones web se realiza como un archivo .jar desplegado en un servidor Tomcat, facilitando la distribución de las aplicaciones. Anteriormente, se trabajaba con .war, que si se requiere, se puede seguir utilizando.

Starters

Son los encargados de brindar una variedad de dependencias que se agregan al proyecto; estas pueden ser de terceros o de Spring. Los starters por defecto traen una configuración de valores predeterminados.

Otra de las ventajas de Spring Boot es que ofrece un mecanismo que genera la estructura del proyecto automáticamente. Este inicializador crea el archivo pom.xml, el main de la aplicación y un archivo de test para verificar que la aplicación funcione correctamente.

Figura 46. Inicializador de Spring



The screenshot shows the Spring Initializr web application at start.spring.io. The interface is divided into several sections:

- Project:** Radio buttons for ☒ Maven Project and ☐ Gradle Project.
- Language:** Radio buttons for ☒ Java, ☐ Kotlin, and ☐ Groovy.
- Spring Boot:** Radio buttons for versions: ☐ 2.6.0 (SNAPSHOT), ☐ 2.6.0 (RC1), ☐ 2.5.7 (SNAPSHOT), ☒ 2.5.6, ☐ 2.4.13 (SNAPSHOT), and ☐ 2.4.12.
- Project Metadata:**
 - Group:**
 - Artifact:**
 - Name:**
 - Description:**
 - Package name:**
- Dependencies:** A section with the text "No dependency selected" and a button "ADD DEPENDENCIES... CTRL + +".

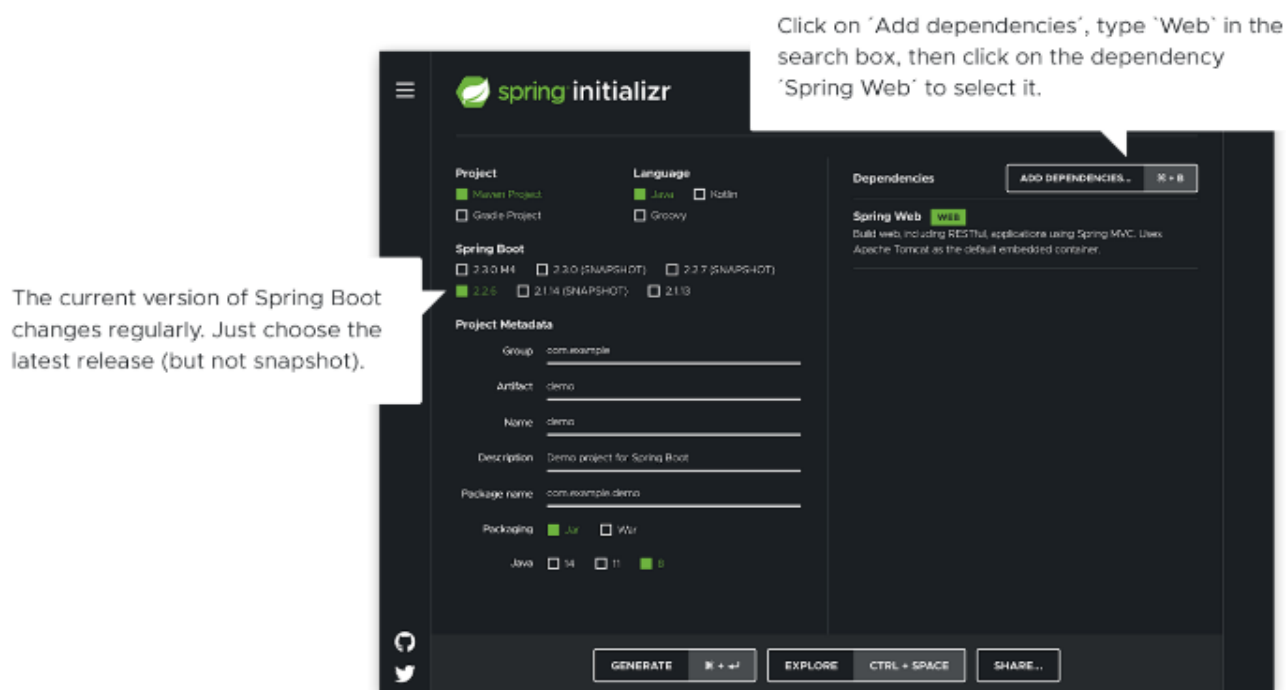
At the bottom, there are three buttons: "GENERATE CTRL + G", "EXPLORE CTRL + SPACE", and "SHARE...".

A continuación, se realizan los pasos para la creación de un proyecto en Spring Boot. Hay que recordar que se deben tener instaladas cualquiera de las herramientas de Entorno de desarrollo y JDK.

- **Comienza un nuevo proyecto**

Ingresar a start.spring.io y crear un proyecto web.

Figura 47. Spring Initializr



- **Ingresar el código**

Después de descargar el archivo que genera el inicializador, ejecute su IDE y abra el archivo. Una vez ubicado en la clase principal, copie el siguiente código para ejecutar un Hola mundo. Lo que hace el método hello() es recibir un parámetro de tipo string con el nombre de name y después concatenar este parámetro con la palabra "Hello". Esto quiere decir que si se ingresa un nombre "Zulema", la salida sería "Hello Zulema".

Figura 48. Fragmento de código en Java para una aplicación Spring Boot

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

    @GetMapping("/hello")
    public String hello(@RequestParam(value = "name", defaultValue = "World") String name) {
        return String.format("Hello %s!", name);
    }
}
```

- **Prueba del proyecto**

Ahora solo es probar el proyecto. Para esto, abra una consola (Windows + R), ubíquese en la carpeta del proyecto y ejecute el siguiente comando:

```
mvn spring-boot:run
```


Figura 49. Salida de terminal

```

demo ./mvnw spring-boot:run --quiet

  ____
 /  _ \
/  / \  \
/  /  __ \
/___/  /_/\
:: Spring Boot :: (v2.2.4.RELEASE)

2020-02-14 16:16:47.746 INFO 4838 --- [main] com.example.demo.DemoApplication : Starting DemoApplication
on Brians-MacBook-Pro.local with PID 4838 (/Users/bclozel/workspace/tmp/demo/target/classes started by bclozel in /Users/bclozel/workspace/tmp/demo)
2020-02-14 16:16:47.748 INFO 4838 --- [main] com.example.demo.DemoApplication : No active profile set, falling back to default profiles: default
2020-02-14 16:16:48.272 INFO 4838 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2020-02-14 16:16:48.279 INFO 4838 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2020-02-14 16:16:48.279 INFO 4838 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.30]
2020-02-14 16:16:48.323 INFO 4838 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2020-02-14 16:16:48.324 INFO 4838 --- [main] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization completed in 532 ms
2020-02-14 16:16:48.438 INFO 4838 --- [main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
2020-02-14 16:16:48.533 INFO 4838 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s) 8080 (http) with context path ''
2020-02-14 16:16:48.535 INFO 4838 --- [main] com.example.demo.DemoApplication : Started DemoApplication in 1.006 seconds (JVM running for 1.248)

```

Lo que hace es ejecutar el servidor Tomcat que ya viene por defecto dentro de Spring Boot. Ahora se debe ingresar al navegador y escribir "localhost" con el puerto 8080, y se presentará la ejecución de la aplicación. En la siguiente videoclase se explica de forma más detallada cómo utilizar Spring Boot:

Video 6. Spring boot



Enlace de reproducción del video

Síntesis del video: Spring boot

En el video "Spring Boot", se explica cómo crear una aplicación sencilla utilizando Spring Boot. El proceso se detalla desde la instalación de las herramientas necesarias hasta la ejecución de un proyecto básico.

Primero, se mencionan los entornos de desarrollo compatibles, como IntelliJ, Spring Tools, Visual Studio Code, Eclipse y NetBeans. También se recomienda instalar el JDK de Java, preferiblemente OpenJDK. El tutorial utiliza Spring Tools para este ejemplo.

Una vez descargado e instalado Spring Tools, se crea un nuevo proyecto seleccionando "Spring Starter Project". Se nombra el proyecto y se elige Maven como gestor de dependencias. Luego, se agrega la dependencia "Spring Web" para desarrollar un servicio web.

El proyecto genera automáticamente un archivo pom.xml con todas las dependencias necesarias. Spring Boot integra un servidor web (Tomcat), eliminando la necesidad de configuraciones adicionales.

En el código del proyecto, se crea un método llamado Hello que imprime un saludo. Se usan anotaciones de Spring Boot para definir un controlador REST. Una vez configurado, el proyecto se ejecuta mediante clic derecho y seleccionando "Run as Spring Boot App".

Finalmente, se accede al navegador web, se ingresa localhost/hello, y se obtiene el saludo "Hola mundo". El video destaca cómo Spring Boot facilita el desarrollo enfocado en la lógica del negocio sin preocuparse por configuraciones complejas.

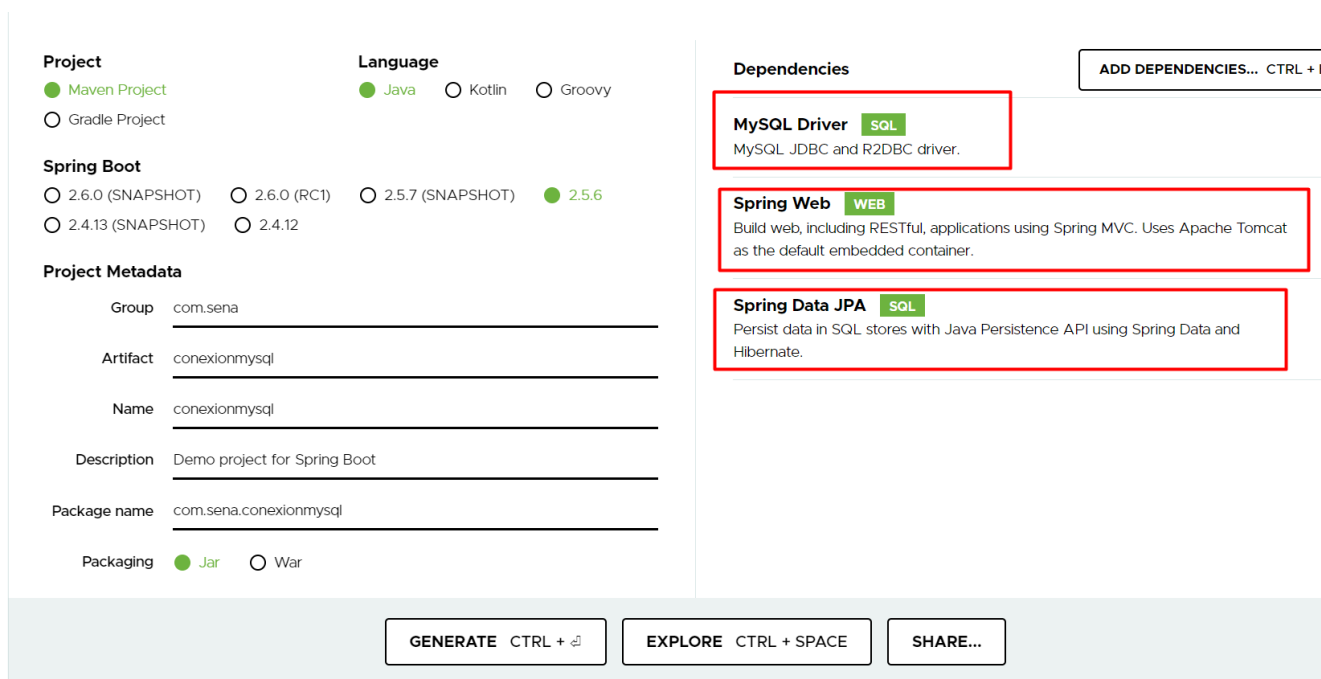
Spring Boot y MySQL

Para tener una aplicación web completa, es necesario almacenar información. Por esta razón, es importante realizar conexiones con bases de datos. En este ejemplo, se detalla cómo realizar la conexión con la base de datos MySQL.

Los requerimientos fundamentales son tener instalado MySQL, el entorno de desarrollo y seguir estos pasos:

- **Paso 1.** Declarar dependencias. JPA, MySQL JDBC Driver, Web. Como se mostró en el ejemplo anterior, por medio del inicializador se pueden seleccionar las dependencias requeridas, como se muestra en la siguiente imagen.

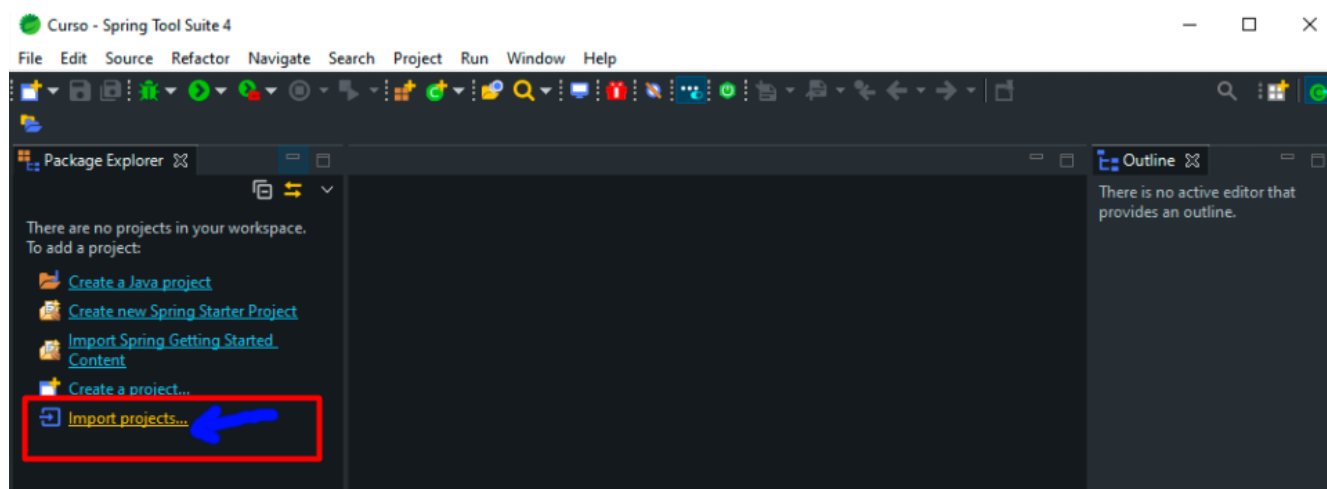
Figura 50. Vista del IDE Eclipse



The screenshot shows the Spring Initializr web interface. On the left, under 'Project', 'Maven Project' is selected. Under 'Language', 'Java' is selected. Under 'Spring Boot', version '2.5.6' is selected. The 'Project Metadata' section contains the following fields: Group (com.sena), Artifact (conexionmysql), Name (conexionmysql), Description (Demo project for Spring Boot), Package name (com.sena.conexionmysql), and Packaging (.Jar). On the right, the 'Dependencies' section shows three selected dependencies: 'MySQL Driver' (SQL), 'Spring Web' (WEB), and 'Spring Data JPA' (SQL). At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and 'SHARE...'.

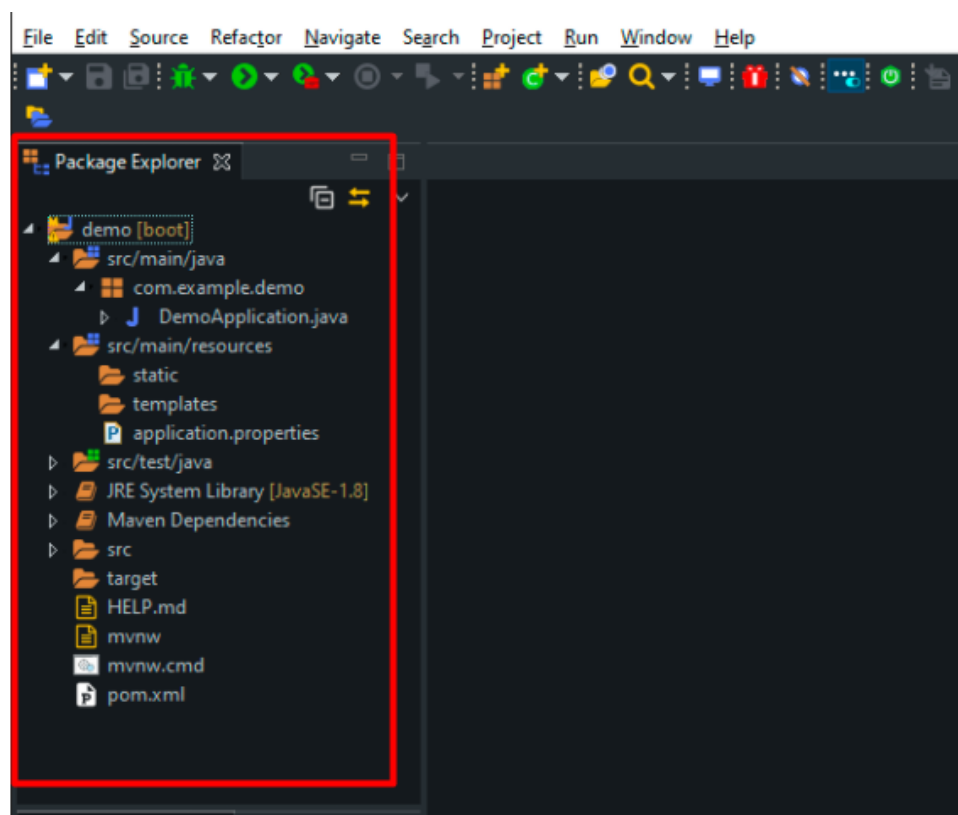
- **Paso 2.** Después de agregar las dependencias, clic en GENERATE y descargará un archivo zip el cuál se debe descomprimir y después importar el proyecto en el IDE, en este caso Spring Tool Suite.

Figura 51. Captura de pantalla del Spring Initializr



- **Paso 3.** Importar el proyecto. Una vez importado el proyecto, se puede ver cómo queda la estructura.

Figura 52. Vista del "Package Explorer" en Eclipse



El paso siguiente es abrir el archivo “application.properties” y configurar todas las propiedades de la conexión así:

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/demo?useUnicode=true&characterEncoding=utf8&createDatabaseIfNotExist=true
```

```
spring.datasource.username=root
```

```
spring.datasource.password=password
```

```
spring.jpa.show-sql=true
```

A continuación, se explicará cada línea del anterior código que va en el archivo “application. properties”.

Línea 1: permite obtener los metadatos de la base de datos.

Línea 2: se muestra la url de la conexión con el motor de base de datos.

Línea 3: Se especifica el nombre del usuario de la base de datos para la conexión

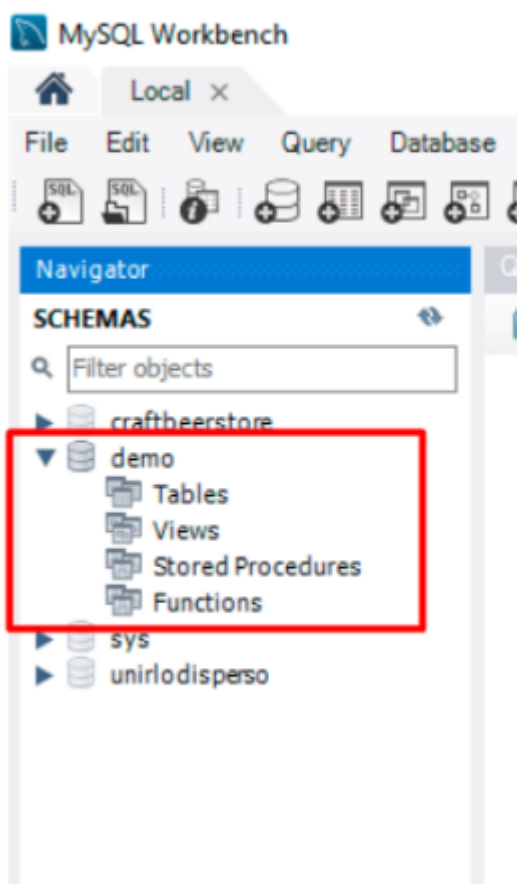
Línea 4: Se especifica clave de la base de datos para la conexión

Línea 5: la propiedad true permite utilizar las consultas JPA a realizar en la base de datos.

Lo que queda es ejecutar la aplicación y comprobar que no existan problemas con la conexión a la base de datos. Para ejecutar, dar clic derecho en el archivo Java, seleccionar la opción >Run As y >Spring Boot App.

Si todo está bien, ahora solo queda ir al motor de bases de datos y verificar si se creó el esquema.

Figura 53. Ejecución de la aplicación



5.4. Validaciones

Es muy importante realizar validaciones cuando se están desarrollando aplicaciones web en Java para impedir que se ingresen datos que afecten al programa. En Spring viene incluido el hibernate-validator en el paquete spring-boot-starter-web, que permite validar valores de los campos beans.

```
<dependency>
```

```
  <groupId>org.springframework.boot</groupId>
```

```
  <artifactId>spring-boot-starter-web</artifactId>
```

```
<version>2.0.0.RELEASE</version>
```

```
</dependency>
```

Para el ejemplo, se realiza la creación de un bean con unas anotaciones que permiten la validación de ingreso de un usuario.

```
@Data
```

```
@NoArgsConstructor
```

```
public class Usuario {
```

```
    @NotBlank(message = "Debe ingresar un nombre de usuario")
```

```
    private String nombreUsuario;
```

```
    @NotBlank(message = "Debe ingresar un valor correcto para la edad")
```

```
    @Pattern(regexp = "[0-9]{1,2}$", message = "Edad incorrecta")
```

```
    private String edad;
```

```
    @AssertFalse(message = "Debe ser falso")
```

```
    private Boolean esfalso;
```

```
    @Pattern(regexp = "[0-9]{4}-[0-9]{2}-[0-9]{2}$", message = "No es correcto el  
formato de la fecha")
```



```
private String fechanacimiento;  
  
}
```

Una vez se crean las anotaciones, se debe crear el controlador.

```
@Controller  
  
public class UserController {  
  
    @RequestMapping("/usuario")  
  
    public void create(@RequestBody @Validated Usuario usuario) {  
  
        System.out.println("Hola: " + usuario.getNombreUsuario());  
  
    }  
  
}
```

Una vez se realice el controlador, se puede realizar la prueba enviando una solicitud POST.

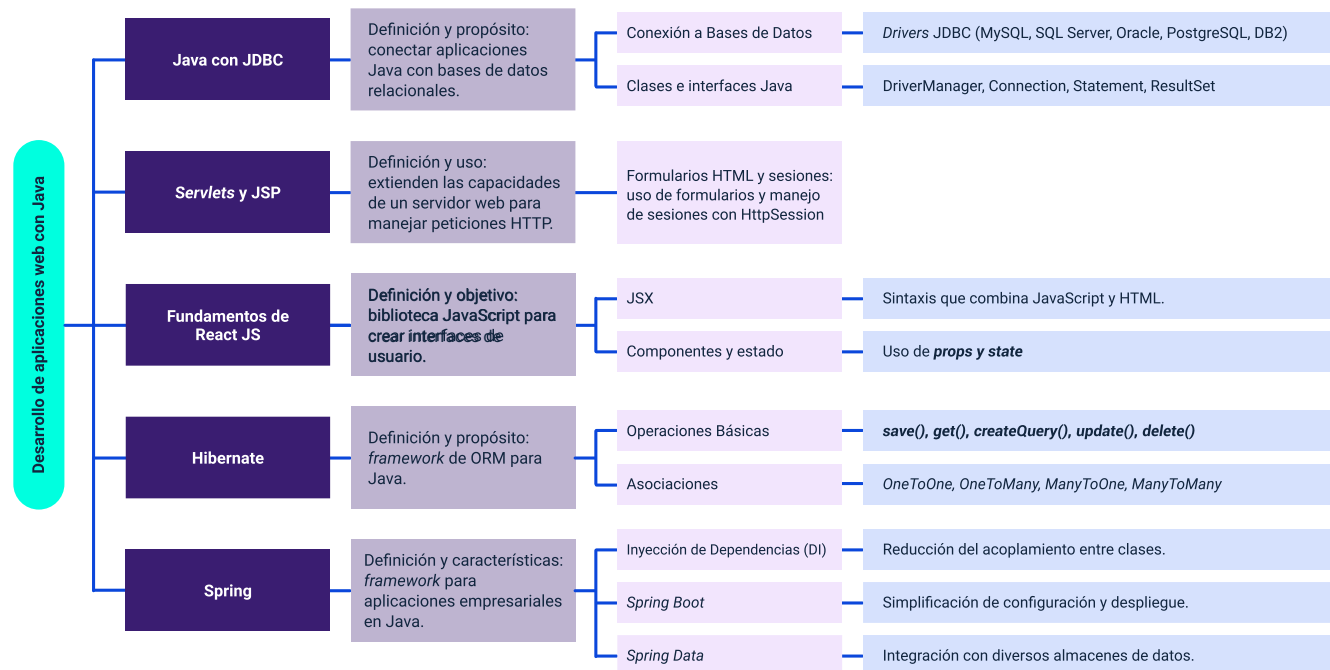
```
{"nombreUsuario": "Ana", "edad": 120, "esFalso": true, "nacimiento": "21010-  
21-12"}
```

La verificación que mostraría el resultado es:

- No es correcto el formato de la fecha.
- Debe ser falso.
- Debe ingresar un valor correcto para la edad.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Instalando React	Editor HTML gratuito en línea limpiador y convertidor. (2021, 27 de octubre). Editor HTML gratuito en línea limpiador y convertidor.	Página web	https://htmled.it/or/
Instalando React	Cubic Factory. (2021, 27 de octubre). Cubic Factory.	Página web	https://www.cubicfactory.com/jeditor/
Propiedades y estados de los componentes	CodeSandbox. (2021, 27 de octubre). CodeSandbox: Online Code Editor and IDE for Rapid Web Development.	Página web	https://codesandbox.io/
Crear un proyecto	React Playground. (2021, 27 de octubre). React Playground.	Página web	https://playcode.io/react
Hibernate	Hibernate Community Documentation. (s. f.). Capítulo 15. HQL: El lenguaje de consulta de Hibernate.	Página web	https://docs.jboss.org/hibernate/orm/3.5/reference/es-ES/html/queryhql.html

Glosario

API: conjunto de métodos ofrecidos por una librería que puede ser utilizado por otro software por medio de un conjunto de reglas establecidas.

CGI: interfaz de entrada común se refiere a un sistema de comunicación que le indica al servidor web cómo enviar y recibir datos de una aplicación de servidor a un cliente.

CSS3: hojas de estilo en cascada versión 3.

DOM: modelo de objetos de un documento HTML.

ES6: abreviación del estándar ECMA SCRIPT. Versión 6.

Framework: es un marco o esquema de trabajo que define un conjunto de utilidades implementadas para un contexto particular para ser reutilizadas por los desarrolladores y así facilitar y acelerar el desarrollo de ciertas actividades propias de la construcción de software.

Groovy: lenguaje de programación efectuado sobre la plataforma JAVA el cual permite realizar APIs de JAVA.

HQL: lenguaje de consulta parecido al SQL implementado por el framework Hibernate.

HTML: HyperText Markup Language, lenguaje de Marcas de Hipertexto.

JAVASCRIPT: lenguaje de programación.

JDBC: API para JAVA que permite el acceso y manipulación de bases de datos relacionales.

JDK: kit de Desarrollo de Java.

JPA: API de persistencia de JAVA.

JSX: sintaxis propuesta en React JS para escribir elementos de interfaz web.

Kotlin: lenguaje de programación de tipado estático, sencillo y fácil de entender y aprender.

NODE.js: es un servidor de aplicaciones.

NPM: es un manejador de paquetes de sistema operativo.

Plugin: programas pequeños que se anexan a otros para ofrecer alguna funcionalidad nueva.

SPA: tipo de aplicación Web.

URL: localizador uniforme de recursos, facilita la recuperación de información determinada en internet, se escribe en el navegador para ingresar a una página determinada.

Referencias bibliográficas

Berzal, F., Cubero, J., & Cortijo, F. (2007). Desarrollo profesional de aplicaciones web con Asp. net.

https://books.google.es/books?hl=es&lr=&id=J1d_9l6zIAIC&oi=fnd&pg=PA3&dq=servlests+y+jsp+fernando+berzal&ots=GSR9oMYn8M&sig=YaCnuz6ed8E_kEo_G1CPHH-KjjU

Capítulo 15. HQL: El lenguaje de consulta de Hibernate. (s. f.). Hibernate Community Documentation. <https://docs.jboss.org/hibernate/orm/3.5/reference/es-ES/html/queryhql.html>

Gallagher, J. (2021, 4 de mayo). Los lenguajes de programación más populares. Career Karma. <https://careerkarma.com/blog/top-programming-languages-2021/>

Gómez, M., & Cervantes, J. (2017). Introducción a la Programación Web con Java: JSP y Servlets JavaServer Faces.

Johnson, R. (2004). Expert one-on-one J2EE design and development. John Wiley & Sons.

Martínez, P. E. (2015). Hibernate. Persistencia de objetos en JEE (1ª ed.). Ra-Ma.

Node. (2021). Node JS Server. <https://nodejs.org/en/>

React JS. (2021). React - A JavaScript library for building user interfaces. <https://reactjs.org/>

Spring. (2021). Spring makes Java simple. <https://spring.io>

Statistics Times. (2021). Principales idiomas informáticos. STATISTICSTIMES. <https://statisticstimes.com/tech/top-computer-languages.php>

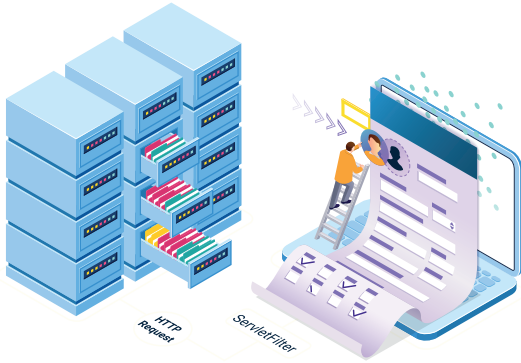
Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Carlos Hernán Muñoz Carvajal	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Zulema Yidney León Escobar	Experta Temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Juan Daniel Polanco Muñoz	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Jhon Jairo Urueta Álvarez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Alvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia



Clases e interfaces Java

Construcción de aplicaciones con JAVA



Además de tener a disposición el Driver JDBC correspondiente, para la base de datos que se desea conectar, se requiere de un conjunto adicional de clases e interfaces Java para hacer uso del driver, para lo cual es necesario importar los paquetes `java.sql` o `javax.sql`.

A continuación, se describen algunas de las clases más importantes dentro de estas librerías para gestionar correctamente una base de datos por medio de JDBC.

DriverManager:



Es una clase estática que permite la gestión del driver JDBC que esté disponible para la aplicación. Por medio del método `getConnection` crea una instancia de tipo `Connection` mediante la cual se realiza el flujo de envío y recibo de información hacia y desde la base de datos destino.

La clase `DriverManager` debe estar informada de los controladores JDBC disponibles, es decir que adicional a descargar el Driver JDBC y agregarlo al proyecto JAVA, es necesario registrar wdicho Driver para que sea cargado en memoria y pueda ser reconocido por el `DriverManager`. Para hacer este proceso se utiliza la siguiente sentencia:

```
Class.forName("string");
```

El parámetro esperado por la sentencia anterior que está marcado en rojo debe corresponder con el nombre de la clase que va a ser cargada e instanciada, teniendo en cuenta que previamente ya se ha descargado el Driver correspondiente y que fue anexado al proyecto. Para el caso de un Driver JDBC para MySQL deberá usar el comando:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```



Connection:

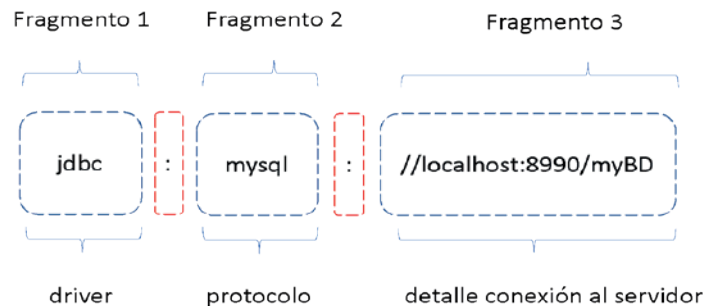


Es una interfaz Java que permite establecer una conexión con una base de datos específica por medio de la cual se puede extraer información de la descripción de tablas, procedimientos almacenados y ejecutar sentencias SQL. Se instancia un objeto de este tipo invocando el método `getConnection` del `DriverManager` al cual se le debe pasar como parámetros las credenciales (usuario y contraseña) del usuario de base de datos destino y la cadena de conexión correspondiente.

La cadena de conexión es un *String* que tiene un formato específico de tres fragmentos conformado por el nombre del driver, el protocolo del driver y los detalles de conexión al servidor de base de datos. Adicionalmente se requieren las credenciales de acceso al servidor.



Figura 1 Cadena de conexión



Un ejemplo para crear un objeto conexión a una base de datos Mysql es el siguiente:

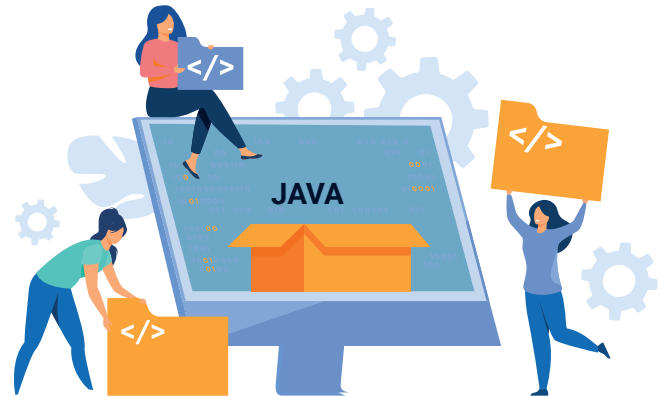
```
String usuario = "root";
String password = "123@S";
String url = "jdbc:mysql://localhost:3306/prueba";
...
Connection conexion =
    DriverManager.getConnection(url,usuario,password);
```

Statement:



Es una interfaz java usada para la ejecución de sentencias SQL estáticas y retornar los resultados que son producidos por estas sentencias. Una instancia de este tipo se obtiene de la ejecución del método `createStatement()` desde una instancia de un objeto de tipo `Connection`.

```
Statement st = conexion.createStatement();
```



Una instancia de tipo `Statement` puede invocar un gran conjunto de métodos disponibles para la ejecución de sentencias SQL, la tabla 2 muestra algunos de los métodos más utilizados:

Tabla 1 Métodos comúnmente usados de un `Statement`.

Método y parámetros	Descripción	Retorno
<code>execute(String sql)</code>	Ejecuta la sentencia SQL entregada como parámetro la cual puede retornar múltiples resultados	Boolean
<code>executeQuery(String sql)</code>	Ejecuta la sentencia SQL entregada como parámetro y retorna un objeto de tipo <code>ResultSet</code> con los resultados.	<code>ResultSet</code>
<code>executeUpdate(String sql)</code>	Ejecuta la sentencia SQL entregada como parámetro la cual puede ser de tipo INSERT, UPDATE o DELETE o una sentencia SQL que no tiene retorno como por ejemplo una sentencia DDL	int

ResultSet:



Es un tipo especial de dato que representa un conjunto de datos de tipo tabular el cual es ideal para referenciar los resultados obtenidos por medio de una consulta que retorna un conjunto de datos estructurados como por ejemplo los de una consulta `SELECT`.

Un `ResultSet` es un apuntador de tipo `Cursor` que está direccionado a la primera posición de toda la estructura tabular que está referenciando.

Figura 2 `ResultSet`

`ResultSet:`



123001	Jose Miguel	Perez Torres	Admin
123002	Maria Jose	Muñoz Trujillo	User
123003	Eduardo	Ante Garcia	User
123004	Michael Steven	Cabezas Paz	Salesman

Conjunto de datos obtenidos desde una sentencia `SELECT`

El método next():



Además de desplazarse una posición de forma lineal dentro de la estructura también devuelve como resultado un valor Booleano para informar si el desplazamiento realizado encontró o no un conjunto de datos válidos, así es posible entonces determinar cuándo se alcanza el final de los elementos. Algunos métodos útiles del ResultSet se muestran en la tabla 2.



Tabla 2 Métodos comúnmente usados de un ResultSet

Método	Descripción	Retorno
isFirst()	Indica si el cursor se encuentra en la primera fila del objeto ResultSet	Boolean
isLast()	Indica si el cursor se encuentra en la última fila del objeto ResultSet	Boolean
first()	Mueve el cursor a la primera fila del objeto ResultSet	Boolean
last()	Mueve el cursor a la última fila del objeto ResultSet	Boolean
next()	Mueve el curso a la fila siguiente del objeto ResultSet	Boolean
previous()	Mueve el cursor a la fila previa del objeto ResultSet	Boolean
getInt(String columnLabel)	Obtiene el valor de la columna especificada como parámetro para la fila en la que actualmente está posicionado el cursor, obtiene el valor en formato Int por lo que se requiere que el valor que se encuentre en la columna corresponda con un valor Int.	Int
getInt(int columnIndex)	Obtiene el valor de la columna especificada como parámetro tomando como referencia el número de su posición dentro de la estructura para la fila en la que actualmente está posicionado el cursor, obtiene el valor en formato Int por lo que se requiere que el valor que se encuentre en la columna corresponda con un valor Int.	Int
getDouble(String columnLabel)	Obtiene el valor de la columna especificada como parámetro para la fila en la que actualmente está posicionado el cursor, obtiene el valor en formato Double por lo que se requiere que el valor que se encuentre en la columna corresponda con un valor Double.	Double
getDouble(int columnIndex)	Obtiene el valor de la columna especificada como parámetro tomando como referencia el número de su posición dentro de la estructura para la fila en la que actualmente está posicionado el cursor, obtiene el valor en formato Double por lo que se requiere que el valor que se encuentre en la columna corresponda con un valor Double.	Double
getString(String columnLabel)	Obtiene el valor de la columna especificada como parámetro para la fila en la que actualmente está posicionado el cursor, obtiene el valor en formato String por lo que se requiere que el valor que se encuentre en la columna corresponda con un valor String.	String
getString(int columnIndex)	Obtiene el valor de la columna especificada como parámetro tomando como referencia el número de su posición dentro de la estructura para la fila en la que actualmente está posicionado el cursor, obtiene el valor en formato String por lo que se requiere que el valor que se encuentre en la columna corresponda con un valor String.	String

Figura 2 Conjunto de datos obtenidos desde una sentencia SELECT

ResultSet: →

userId	userName	userLastName	userRol:
123001	Jose Miguel	Perez Torres	Admin
123002	Maria Jose	Muñoz Trujillo	User
123003	Eduardo	Ante Garcia	User
123004	Michael Steven	Cabezas Paz	Salesman

Conjunto de datos obtenidos desde una sentencia SELECT

ResultSet.getString(2) → Ante Garcia

ResultSet.getInt("userId") → 123003

Es común encontrar estructuras cíclicas apoyadas en el método next para realizar el recorrido completo y manipulación de los elementos obtenidos con un ResultSet. A continuación, se muestra un ejemplo:

```
resultSet.next();
do{
    ...
    //Acciones para procesar cada una de las filas

    System.out.println(resultSet.getInt("userid")+ " : " +
resultSet.getString("username"));

}while(resultSet.next());
```

Construcción API

Breve descripción:

En este componente formativo se abordarán los conceptos relacionados con la configuración del entorno de desarrollo como espacio de trabajo para la API REST. Se revisarán las buenas prácticas de los estándares de codificación para el desarrollo de la API, la cual será construida con las tecnologías de Node.js, Express y MongoDB y se abordará el renting de la API creada, a través de la herramienta Postman.

Junio 2024

Tabla de contenido

Introducción	3
1. La internet y la web	4
1.1. Conceptos sobre el entorno de desarrollo integrado - IDE.....	4
1.2. Instalación del Visual Studio Code.....	6
1.3. Instalación y configuración del Node.js y npm	6
1.4. Instalación de módulo	7
2. Codificación de la API REST con Node.js	10
2.1. Buenas prácticas en la codificación de Node.js	10
3. Testing de la API REST con Postman.....	13
Síntesis	14
Material complementario	15
Glosario	16
Referencias bibliográficas	18
Créditos	19

Introducción

En este componente formativo se estudiarán los conceptos y pasos para el desarrollo de una API REST, utilizando las tecnologías del back-end como Node.js, Express y el sistema de bases de datos no relacional como lo es MongoDB. De esta forma se inicia con la puesta a punto del entorno de desarrollo con Visual Studio Code, el cual será el espacio de trabajo donde se construirá la API. Adicionalmente, se comprenderá la importancia del uso de los comandos (en la terminal) para la instalación y configuración de paquetes necesarios para el correcto funcionamiento del proyecto; de igual manera, se ilustran algunas buenas prácticas de desarrollo y herramientas que harán más prolijo el código.

Una vez construida la API se inicia con un testing a través de la herramienta Postman, que es muy utilizada para este tipo de desarrollos. Así, se corroboran las respuestas y solicitudes de un tipo de proyecto de esta naturaleza.

1. La internet y la web

Para todo proyecto software es recomendable alistar las herramientas necesarias y mínimas para iniciar con un desarrollo. En ese sentido, dependiendo del alcance, la tecnología y/o lenguaje se sugiere definir muy bien este tipo de recursos para realizar el ejercicio. En este orden de ideas se abordarán algunos conceptos fundamentales.

1.1. Conceptos sobre el entorno de desarrollo integrado - IDE

Como primera medida, es necesario tener en cuenta la definición de un entorno de desarrollo integrado IDE, que es un espacio de trabajo basado en tecnologías y software, que sirve de plataforma para el diseño de aplicaciones que componen una serie de herramientas para el desarrollador de software. Un IDE cuenta con las siguientes características:

- **Editor de código fuente**

Es un editor de texto que ayuda a escribir en el lenguaje de programación definido; esta herramienta apoya la sintaxis o construcción de instrucciones, para que se realicen de la manera correcta. Una de sus funcionalidades más importantes es resaltar ciertas partes del código que se están desarrollando, para tener una referencia sobre el flujo o línea lógica del desarrollo de la aplicación. De manera adicional, hay editores que ayudan a autocompletar ciertas funciones ya preestablecidas del lenguaje.

- **Automatización de compilaciones locales**

Es un recurso que permite automatizar tareas sencillas e iterativas como parte de la creación de una compilación local del software para su uso por parte del desarrollador.

- **Depurador**

“Programa que sirve para probar otros programas y mostrar la ubicación de un error en el código original de forma gráfica” (Red Hat, 2021).

Además, hay entornos que ayudan con varias opciones de la sintaxis para corregir el código.

Para el caso de este componente formativo se utilizará el IDE Visual Studio Code por las siguientes razones:

- Visual Studio Code resalta las palabras clave del código en diferentes colores para ayudar a identificar fácilmente los patrones de codificación y aprender más rápido.
- A medida que se codifica, Visual Studio Code brinda sugerencias para completar líneas de código y soluciones rápidas para errores comunes.
- Se aprovecha porque este entorno tiene incorporada la terminal, lo cual hace mucho más fácil el desarrollo, sobre todo con las tecnologías de Node.js, porque trae los paquetes necesarios para el proyecto.

En el proceso de alistamiento del entorno para iniciar el proyecto de desarrollo de la API, se deberá contar con los siguientes recursos:

- Visual Studio Code
- Node.js

- Express.js
- MongoDB

1.2. Instalación del Visual Studio Code

Para la instalación se deben seguir los siguientes pasos (para este caso se ha tomado como plataforma el sistema operativo Windows 10):

- **Paso 1**

Descargar el instalador de la página principal para Visual Studio Code en Windows.

Enlace: <https://code.visualstudio.com/docs?dv=win>

- **Paso 2**

Una vez descargado, se debe correr el instalador y seguir con el wizard o paneles de instalación. Luego dar clic en finalizar.

Por defecto, el Visual Studio Code se instala en la siguiente ruta:

C:\users\{username}\AppData\Local\Programs\Microsoft VS Code

1.3. Instalación y configuración del Node.js y npm

Node.js está considerado como un entorno de ejecución de JavaScript orientado a eventos asíncronos, de igual manera, está diseñado para crear aplicaciones network escalables y pensado en principio como tecnología back-end (node.js.org, 2021).

Los pasos para instalar el Node.js son los siguientes:

- **Paso 1**

Se ingresa a la página oficial, se selecciona el sistema operativo, y hace clic en descargar.

Enlace: <https://nodejs.org/en/download>

- **Paso 2**

Una vez descargado, se debe correr el instalador y seguir con el wizard o paneles de instalación.

- **Paso 3**

Posterior a la instalación, se verifica que tanto el Node.js como el npm estén instalados, para eso se utiliza la línea de comandos en la terminal del Visual Studio Code: `node -v` o `npm -v`.

1.4. Instalación de módulo

Un paquete en Node.js contiene todos los archivos que se necesitan para instalar un módulo; los módulos son bibliotecas de JavaScript que se pueden incluir en el proyecto. Instalar módulos es una de las principales tareas que debe aprender a hacer al comenzar con el administrador de paquetes Node. El comando para instalar un módulo en el directorio actual es el siguiente:

```
$ npm install < módulo >
```

```
$ npm i < módulo >
```

El primer módulo a instalar es Express, el cual es el framework web más popular de Node y proporciona mecanismos para (Mozilla, 2021).

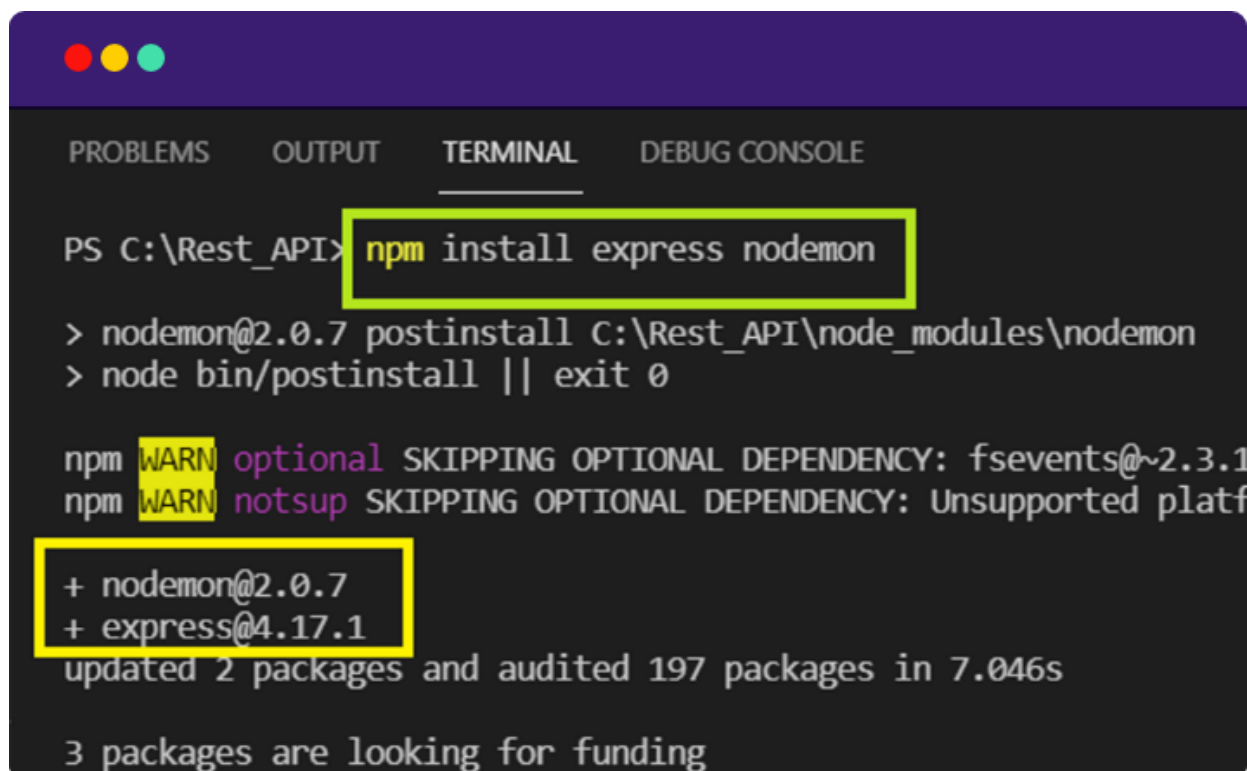
- Escritura de manejadores de peticiones con diferentes verbos HTTP, en diferentes caminos URL (rutas).
- Integración con motores de renderización de “vistas”, para generar respuestas mediante la introducción de datos en plantillas.

- Establecer ajustes de aplicaciones web como qué puerto usar para conectar, y la localización de las plantillas que se utilizan para renderizar la respuesta.
- Añadir el procesamiento de peticiones “middleware” adicional en cualquier punto dentro de la tubería de manejo de la petición.

El segundo módulo para instalar es nodemon, que es una utilidad que monitorea los cambios en el código fuente que se está desarrollando y de manera automática reinicia el servidor. Es una herramienta muy útil para el desarrollo de aplicaciones en Node.js (Gómez, 2017).

A continuación, se muestra la línea de comando que se debe ejecutar desde la terminal del Visual Studio Code.

Figura 1. Comando para la instalación de Express y nodemon



```
PS C:\Rest_API> npm install express nodemon

> nodemon@2.0.7 postinstall C:\Rest_API\node_modules\nodemon
> node bin/postinstall || exit 0

npm WARN optional SKIPPING OPTIONAL DEPENDENCY: fsevents@~2.3.1
npm WARN notsup SKIPPING OPTIONAL DEPENDENCY: Unsupported platf

+ nodemon@2.0.7
+ express@4.17.1
updated 2 packages and audited 197 packages in 7.046s

3 packages are looking for funding
```

Hasta esta parte se ha abordado la instalación y configuración del espacio de trabajo para iniciar con el desarrollo de la API. No obstante, es necesario revisar las buenas prácticas de codificación para que el desarrollo no solo sea prolijo en su estructura, sino que también siga estándares de codificación, permitiendo que el código tenga un alto desempeño.

2. Codificación de la API REST con Node.js

En este tema, se inicia con el desarrollo de la API REST, por lo que se verá la codificación de cada uno de los archivos que componen el proyecto. Para esto, es importante tener en cuenta dos cosas, la primera es realizar buenas prácticas de codificación, permitiendo que el código quede legible y mucho más limpio al momento de documentar y la segunda, es entender la estructura de directorios del proyecto, la cual va a dar el orden al proceso de búsqueda e indexación de las rutas y recursos que se necesitarán al momento del desarrollo.

2.1. Buenas prácticas en la codificación de Node.js

El objetivo de seguir estándares de codificación es alcanzar un código mantenible y se considera mantenible cuando el código es legible, consistente, predictivo y documentado. En la práctica, la corrección de errores y modificación del código desarrollado de un producto software toma demasiado tiempo y se llega algunas veces a que no es posible realizar del todo el desarrollo, por lo que se toma la decisión de iniciar de cero, debido a las malas prácticas. (Choque, 2016).

A continuación, se indican las buenas prácticas enunciadas por uno de los mejores servicios de control de versiones y desarrollo de software colaborativo basado en Git, llamada GitLab. Esta comunidad sigue de igual manera estándares como Google JavaScript Style Guide (Whyte, 2021).

a) Código en general

- Utilizar el punto y coma (;) como final de instrucción. Mejora la lectura y evita conflictos con herramientas de verificación de código.

- Para delimitar y diferenciar cadenas de texto, se recomienda utilizar comillas simples con excepción de JSON.
- La codificación en la sintaxis para los nombres de variables o procedimientos es lowerCamelCase, la cual establece que la letra inicial debe ser minúscula y las palabras siguientes deben tener la inicial en mayúscula, y el resto en minúscula, por ejemplo, var logFileName.
- La codificación en la sintaxis de las clases e interfaces es UpperCamelCase que establece que la letra inicial debe ser mayúscula y las palabras siguientes deben tener la inicial en mayúscula, y el resto en minúscula.
- Evitar escribir prefijos del tipo de dato a las variables. Se recuerda que las variables en JavaScript son sin tipo, es decir, que toman su tipo de dato dependiendo del valor.

b) Bloques de código

- Las líneas de código que tienen alguna relación deben estar agrupadas sin espacio entre sus líneas.
- La cantidad máxima de caracteres que debe tener una línea de código debería ser de 120.
- Los bloques de código que son alusivos a las clases, procedimientos y estructuras de control, deben estar identados con 4 espacios, que es lo mismo a una tabulación.

c) Procedimientos

La cantidad de líneas que se recomiendan para un procedimiento es de 20, salvo casos excepcionales. En el caso que un procedimiento supere las 20

líneas, se debe evaluar la división del proceso en procesos más pequeños.

Se debe tener en cuenta que un procedimiento debe realizar una tarea.

d) Manejo de errores y excepciones

- Como una buena práctica los errores y/o excepciones posibles, deben ser capturados (catch) y se debe lanzar (throw) un mensaje preciso, con el fin de evitar la finalización abrupta del programa.
- En caso de que el error requiera alguna acción por parte del usuario, como puede ser continuar, cancelar u otras opciones, el sistema debe informar al usuario sobre las consecuencias de cada elección.

e) Comentarios

- Los comentarios forman parte de la documentación de un proyecto, de hecho, esta buena práctica hace parte de la base del manual técnico de un proyecto de software.
- Su principal ventaja es que, a partir de ellos, se logra reducir el costo de transferencia de conocimientos entre equipos de desarrollo, dado que es posible intervenir de manera más rápida en la continuación del desarrollo.

Desarrollo de la API REST

Lo invitamos a revisar el PDF “**Desarrollo API REST**”, en el cual se describen de manera sucinta, los pasos requeridos para el desarrollo de la API REST con Node.js, que encuentra en la carpeta Anexos.

3. Testing de la API REST con Postman

En la definición de la arquitectura de despliegue de aplicaciones y servicios, es muy importante la definición de los componentes desde el punto de vista de la plataforma tecnológica que serán utilizados en el momento de iniciar el proceso de desarrollo y en el de hacer el despliegue o puesta en producción, para lo cual el equipo de desarrollo deberá elegir qué tipo de plataforma tecnológica será la utilizada.

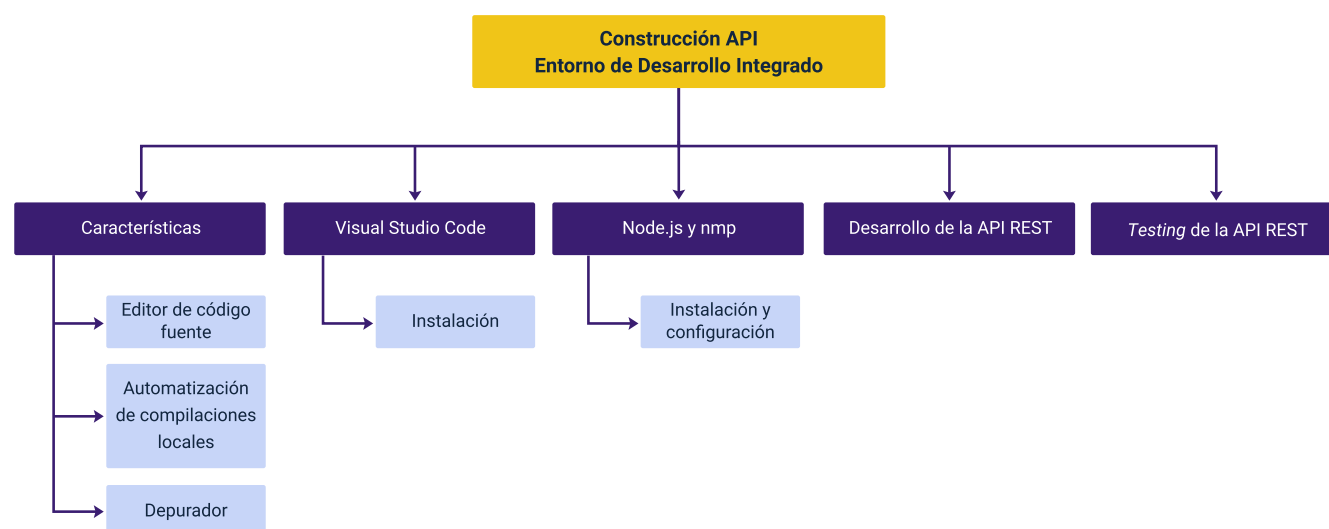
Si bien la plataforma tecnológica exige la selección de varios componentes como sistema operativo, servidor web, gestor de base de datos y lenguaje de programación, en la actualidad se debe tener claro qué tipo de despliegue se requiere para la construcción local del producto en las máquinas de los equipos de desarrollo y para el despliegue en producción (generalmente debe ser el mismo).

Instalación Postman

Lo invitamos a revisar el PDF “**Instalación herramienta Postman**”, en el cual se explica su instalación, que encuentra en la carpeta Anexos.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Entidades de información digital	Coding, G. (2020). Cómo instalar Node.js en Windows 2024 [video]. YouTube.	Video	https://www.youtube.com/watch?v=BgtB31gXkoA
Información digital	Dominicode. (2018). "Postman en Español: ¡El tutorial definitivo para el éxito en las pruebas API! [video]. YouTube.	Video	https://www.youtube.com/watch?v=qsejysrhjiU
Entidades de información digital	DiSimone, K. (2020). mLab to Atlas for existing NS users [video]. YouTube.	Video	https://www.youtube.com/watch?v=hXSLBACjYQo

Glosario

Indentación: dentro del contexto de desarrollo de software indentar es un término que significa mover un bloque de código hacia la derecha insertando espacios o tabulaciones, para así separarlo del margen izquierdo y distinguirlo mejor del texto adyacente, es decir, organizar mejor el código. La indentación se considera un principio fundamental de buena práctica de la programación de software.

JSON: Java Script Object Notation (JSON) es un formato basado en texto estándar que se utiliza para representar datos estructurados en la sintaxis de objetos de JavaScript. Se utiliza principalmente para transmitir datos en aplicaciones web (como es el caso de enviar algunos datos desde el servidor al cliente, por lo tanto, estos datos pueden ser mostrados en páginas web o viceversa).

Localhost: en el contexto de las redes de computadoras, localhost es una computadora que ejecuta un programa. La computadora que se tiene al frente funciona también como un servidor virtual; este modo es comúnmente utilizado para realizar pruebas en el desarrollo web.

Manejadores o handlers de eventos: en la programación con JavaScript un evento por sí mismo carece de utilidad; para que los eventos resulten útiles o aplicables se debe asociar funciones o código JavaScript a cada evento. De esta forma, cuando se produce un evento de manera inmediata se ejecuta el código indicado, por lo que la aplicación responde ante cualquier evento que se produzca durante su ejecución. Las funciones o código JavaScript que se realizan para cada evento se denominan "manejador de eventos".

Middleware: es un bloque de código que se ejecuta entre la petición que hace el usuario (request) hasta que esa petición llega al servidor. De esta manera, son funciones que tienen acceso al objeto de solicitud (req), el objeto de respuesta (res) y la siguiente función de middleware en el ciclo de solicitud-respuesta de la aplicación.

Referencias bibliográficas

Chrome web store. (2021). Postman.

<https://chrome.google.com/webstore/detail/postman/fhbjgbiflinjbdggehcddcbncdddo mop?hl=es-419>

Express. (2021). Direccionamiento básico. <https://expressjs.com/es/starter/basic-routing.html>

Google. (s.f.). Google TypeScript Style Guide.

<https://google.github.io/styleguide/tsguide.html>

Gómez, D. (2017). Nodemon. Vortexbird.

<https://vortexbird.com/nodemon/#:%7E:text=Nodemon%20es%20una%20utilidad%20que,desarrollo%20de%20aplicaciones%20en%20nodejs>

López, A. (2021). Qué es Postman y para qué sirve. OpenWebinars.

<https://openwebinars.net/blog/que-es-postman/>

Mozilla. (2021). Await. MDN Web Docs.

<https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Operators/await>

Mozilla. (2021). Introducción a Express/Node. MDN Web Docs.

https://developer.mozilla.org/es/docs/Learn/Server-side/Express_Nodejs/Introduction

Nodejs.org. (2021). Acerca de Node.js. Node.js. <https://nodejs.org/es/about/>

npm. (2021). About npm. <https://www.npmjs.com/about>

Red Hat. (2021). El concepto de IDE.

<https://www.redhat.com/es/topics/middleware/what-is-ide>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Zulema Yidney León Escobar	Experta Temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Jonathan Guerrero Astaiza	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Ana Catalina Córdoba Sus	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Yerson Fabián Zárate Saavedra	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Luis Jesús Pérez Madariaga	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Jaime Hernán Tejada Llano	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <title>My perfect website</title>
5   <meta charset="utf-8" />
6
7   <link rel="preconnect" href="//s3.mysite
8   <link rel="preconnect" href="//www
9
10  <meta name="viewport" content="width
11
12    <script>
13      var mytag = mytag || {};
14      mytag.cmd = mytag.cmd ||
15      (function(){
16        var gads = document
17        gads.async = true
18        gads.type = "text"
```

Desarrollo de la API REST

Análisis y desarrollo de *software*
Servicio Nacional de Aprendizaje - SENA

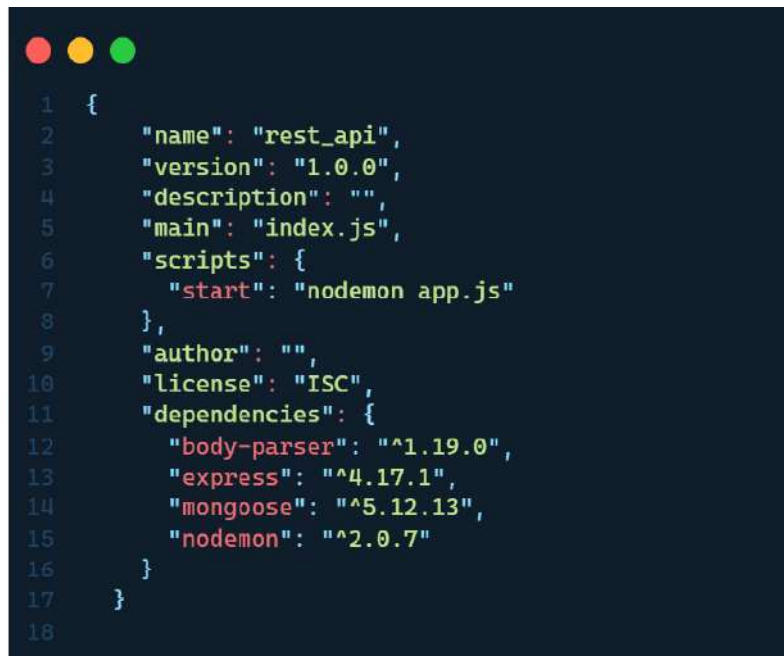
Desarrollo de la API REST

A continuación, se describen de manera sucinta los pasos requeridos para el desarrollo de la API Rest con Node.js

01 Configuración de package.json

Se configura el package.json para implementar el nodemon, con el fin de que el servidor funcione más rápido, como se muestra en la figura 1.

Figura 1. Código fuente del archivo package.json



```
1 {
2   "name": "rest_api",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "start": "nodemon app.js"
8   },
9   "author": "",
10  "license": "ISC",
11  "dependencies": {
12    "body-parser": "^1.19.0",
13    "express": "^4.17.1",
14    "mongoose": "^5.12.13",
15    "nodemon": "^2.0.7"
16  }
17 }
18
```

Referencia Nota. SENA (2021).

02 Creación del archivo app.js

Se crea un nuevo archivo llamado app.js como se muestra en la figura 2, en el cual se implementa lo siguiente:

- 1 Se importa la librería de Express.
- 2 Se configura el listening del servidor, es decir, se precisa el puerto por donde va a escuchar.

- 3** Se crea la ruta por defecto a manera de prueba, que es la URL por donde el servidor responde las solicitudes que se le hacen. La definición de la ruta tiene la siguiente estructura:

```
app.METHOD(PATH, HANDLER)
```

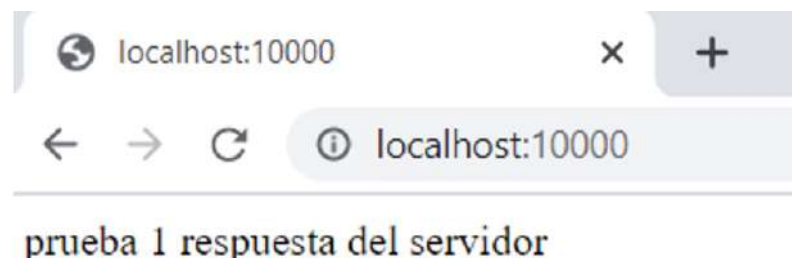
Figura 2. Ejemplo de conexión entre 10 equipos o terminales

```
1 // Iniciamos el módulo express con el fin de dar inicio al servidor, evitando varias configuraciones
2 const express = require('express'); // importamos el paquete
3 const app = express();
4 /* SE CREAN LAS RUTAS*/
5 app.get('/', (req, res) => {
6   res.send('prueba 1 respuesta del servidor');// Ruta por defecto
7 } );
8 //Primero se configura cómo va escuchar el servidor las peticiones
9 app.listen(10000);
```

Referencia Nota. SENA (2021).

Se comprueba la ruta iniciando el servidor con el comando `npm start` desde la terminal del Visual Studio Code. Una vez realizado esto, se debe abrir el browser de la preferencia e inmediatamente se ingresa la dirección local por el puerto donde se está escuchando, que para el caso de este ejemplo es el puerto 10000. A continuación, se presenta la figura que ejemplifica este paso.

Figura 3. Browser con la ruta y puerto donde funciona el servicio



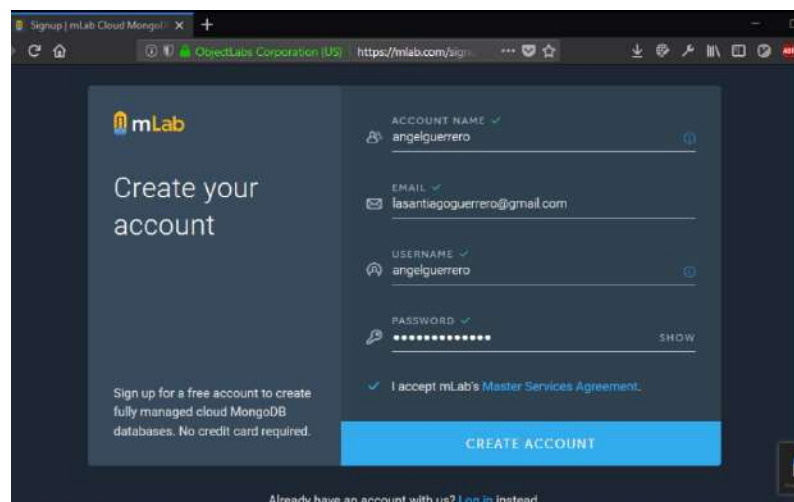
Como se observa el servidor está activo escuchando desde el puerto 10000.

03 Instalación de mongoose y mLab MongoDB

Ahora se necesita una base de datos para almacenar la data que se quiere enviar o registrar, para este caso se utilizará MongoDB. Al respecto, esta tecnología se diferencia de otras porque no guarda los datos en tablas tal y como se hace en las bases de datos relacionales, MongoDB en cambio guarda estructuras de datos en formato BSON (una especificación similar a JSON) con un esquema dinámico. En ese sentido, esto permite que la integración de los datos en ciertas aplicaciones sea más fácil y rápida.

En este orden de ideas, una de las formas para utilizar una base de datos MongoDB es con la herramienta mongoose que establece la conexión a la base de datos MongoDB y además permite realizar el mapeo de los esquemas realizados en MongoDB y utilizarlos.

De manera que se instala mongoose desde la terminal de Visual Studio Code así: tiene la siguiente estructura:



```
$ npm install mongoose
```

Una vez instalado mongoose, se utiliza mLab para el cual crea una base de datos gratuita de MongoDB, es decir, es un hosting para bases de datos MongoDB:

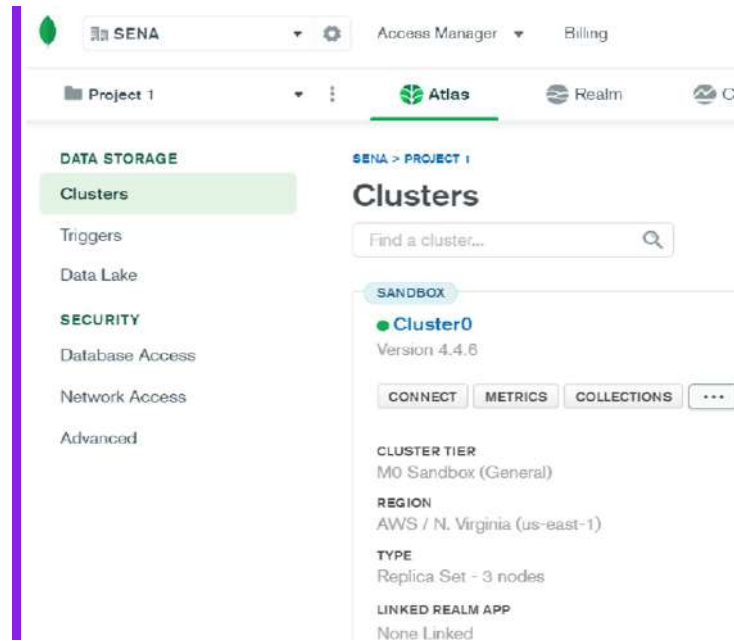
 **Enlace web.** <https://mlab.com/>

Para la creación de la base de datos y del usuario de prueba se debe seguir la siguiente guía:

 **Enlace web.** <https://docs.mlab.com/>

A continuación, se presenta la interfaz del hosting de bases de datos mLab /Atlas.

Figura 4. Interfaz de mLab funcionando



Referencia Nota. SENA (2021).

Una vez creada la base de datos se utiliza mongoose para realizar la conexión. Esta codificación se realiza igualmente en el archivo app.js. como se muestra en la siguiente figura.

Figura 4. Código fuente del archivo app.js (conexión BD)

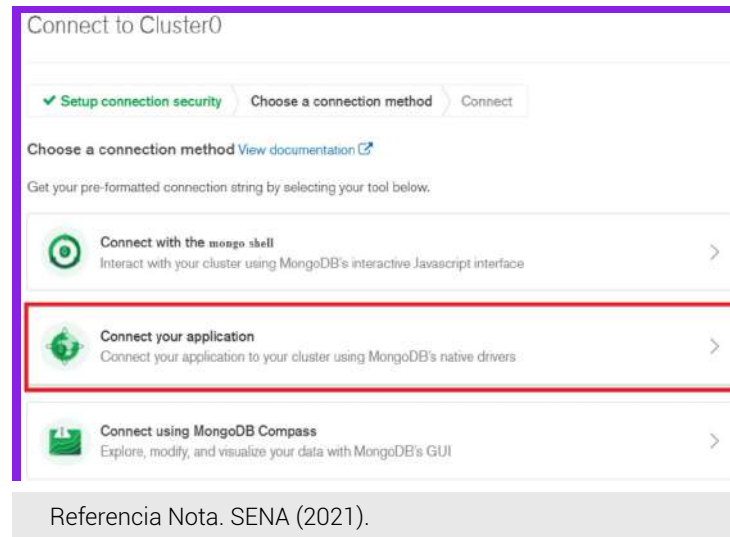
```

1 // Iniciamos el módulo express con el fin de dar inicio al se
nvidon, evitando varias configuraciones
2 const express = require('express'); // importamos el paquete
3 const app = express();
4 const mongoose = require('mongoose');
5 /* SE CREAN LAS RUTAS*/
6 app.get('/', (req, res) => {
7   res.send('prueba 1 respuesta del servidor');// Ruta por de
fecto
8 } );
9 //Conexión a la BD
10 mongoose.connect('mongodb+srv://Sena:Slypherin23@cluster0.klxq
7.mongodb.net/miBD?retryWrites=true&w=majority',
11 {useNewUrlParser: true }, () =>{
12   console.log('Si hay conexión a la BD');
13 });
14 //Primero se configura cómo va escuchar el servidor las peti
ciones
15 app.listen(10000);

```

El código de la conexión se puede generar a través de la plataforma mLab, Por lo tanto, se puede elegir el modo de conexión. Para el caso del ejemplo se elige la opción "Connect your application" como se muestra en la siguiente figura.

Figura 6. Panel de opción de conexión a la base de datos ofrecido por mLab/Atlas



Es importante anotar que las solicitudes que se envían y reciben deben estar en formato JSON para que sean procesadas de manera correcta, por lo que es necesario convertir o "parsear" a este formato la información que vaya por el método POST. Por lo tanto, se hace necesario instalar el siguiente paquete:

```
$npm install body-parser
```

El archivo app.js queda de la manera como se muestra en la siguiente figura.

Figura 7. Código fuente del archivo app.js aplicando el body parser

```

1 // Iniciamos el módulo express con el fin de dar inicio al ser
  vidor, evitando varias configuraciones
2 const express = require('express'); // importamos el paquete
3 const app = express();
4 const mongoose = require('mongoose');
5 const bodyParser = require('body-parser');
6
7 // llamar al body-parser
8 app.use(bodyParser.json());
9 // Importar las Rutas
10 const postRoute = require('./routes/post');
11 app.use('/servicios', postRoute);
12 //Middlewares: Es la llamada a una función cuando se present
  a un evento en una ruta específica
13 /*app.use('/servicios', () => {
14     console.log('Corriendo la middleware');
  })*/

```

```

1  });
2  /*
3  /* SE CREAM LAS RUTAS*/
4
5  app.get('/', (req, res) => {
6    res.send('prueba 1 respuesta del servidor');// Ruta po
7    // defecto
8  });
9  /*
10 app.get('/servicios', (req, res) => {
11   res.send('prueba servicios');// Ruta por defecto
12 });
13 /*
14 //Conexión a la BD
15 mongoose.connect('mongodb+srv://Sena:Slypherin23@cluster0.
16 klxq7.mongodb.net/miBD?retryWrites=true&w=majority',
17 {useNewUrlParser: true }, () =>{
18   console.log('Si hay conexión a la BD');
19 });
20 //Primero se configura cómo va escuchar el servidor la
21 s peticiones
22 app.listen(10000);

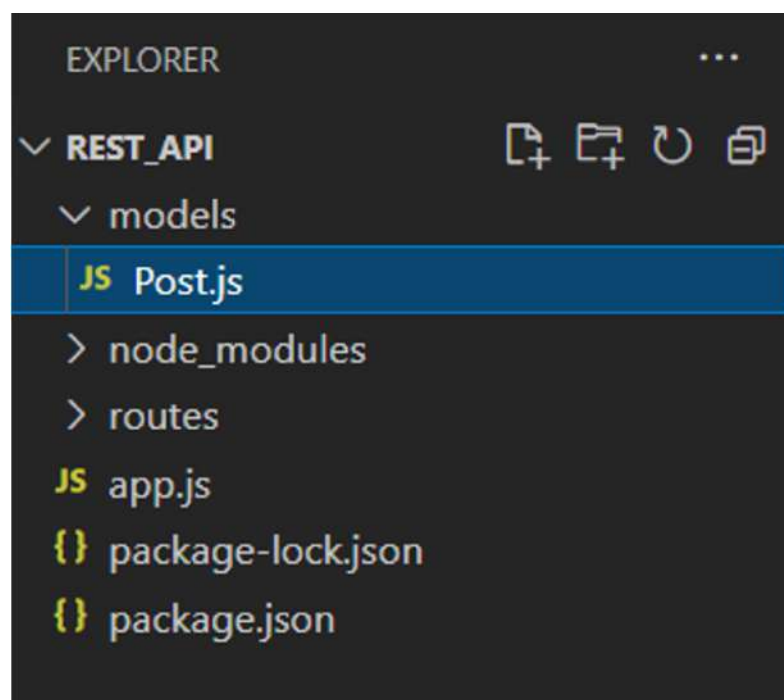
```

Referencia Nota. SENA (2021).

04 Creación del modelo

Ahora se necesita crear los datos de prueba para que se puedan almacenar en la base de datos, por lo tanto, se utilizará mongoose para generar los modelos. En ese sentido, en la estructura de directorios del proyecto se crea una nueva carpeta llamada models como se muestra a continuación.

Figura 8. Estructura de directorios (carpeta models)



Ahora se necesita una base de datos para almacenar la data que se quiere enviar o registrar, para este caso se utilizará MongoDB. Al respecto, esta tecnología se diferencia de otras porque no guarda los datos en tablas tal y como se hace en las bases de datos relacionales, MongoDB en cambio guarda estructuras de datos en formato BSON (una especificación similar a JSON) con un esquema dinámico. En ese sentido, esto permite que la integración de los datos en ciertas aplicaciones sea más fácil y rápida.



En este orden de ideas, una de las formas para utilizar una base de datos MongoDB es con la herramienta mongoose que establece la conexión a la base de datos MongoDB y además permite realizar el mapeo de los esquemas realizados en MongoDB y utilizarlos.

De manera que se instala mongoose desde la terminal de Visual Studio Code así:

Figura 9. Código fuente del archivo post.js (modelo)

```

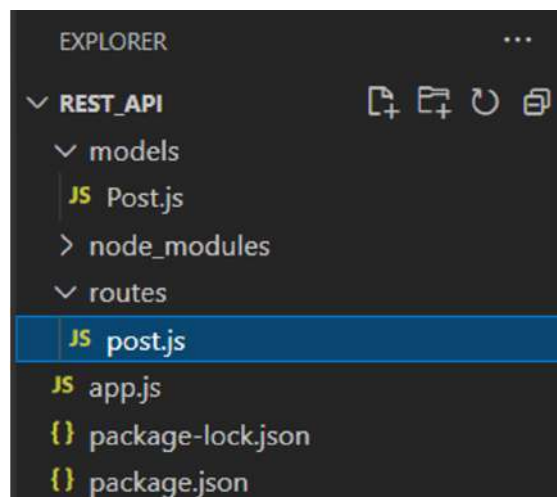
1  const mongoose = require('mongoose');
2  const PostSchema = mongoose.Schema({
3    title: {
4      type: String,
5      require:true
6    },
7    description: {
8      type: String,
9      require:true
10   },
11   date: {
12     type: Date,
13     default: Date.now
14   }
15 });
16 module.exports = mongoose.model('Post',PostSchema);

```

05 Configuración de las rutas

Se define la ruta para el modelo, para esto se crea una carpeta llamada routes en la raíz de la estructura del proyecto y se genera un archivo post.js como se muestra en la siguiente figura.

Figura 10. Estructura de directorios (carpeta routes)



Referencia Nota. SENA (2021).

En el archivo `post.js` se definen las acciones o tratamiento a cada uno de los datos. Esto se hace a partir de las rutas o direccionamiento que hace referencia a la determinación de cómo responde una aplicación a una solicitud de cliente en un determinado punto final, que es una URL (o una vía de acceso) y un método de solicitud HTTP específico (GET, POST, etc.) Es en esta instancia que se invocan acciones para guardar, extraer, eliminar o visualizar datos, a partir del direccionamiento. A continuación, se presenta el código.



Figura 10. Código fuente del archivo `post.js` (rutas)

```

1  router.post('/', async (req, res) => {
2
3      //console.log(req.body); Se utiliza para la respuesta
4      a del post en consola
5      const post = new Post({
6          title: req.body.title,
7          description: req.body.description
8      });
9      try {
10         const savedPost = await post.save(); // método que
11         e guarda en la BD
12         res.json(savedPost);
13     } catch (error) {
14         res.json({message: error});
15     }
16 });
17
18 /**
19  * Bloque para mostrar solo un post por el Id
20  */
21
22 router.get('/:postId', async (req, res) => {
23     try {
24         const post = await Post.findById(req.params.postId);
25         // encuentra por id
26         res.json(post);
27     } catch (error) {
28         res.json({message: error});
29     }
30 });
31
32 /**
33  * Bloque para borrar un post
34  */
35
36 router.delete('/:postId', async (req, res) => {
37     try {
38         const removedPost = await Post.remove({_id: req.params.postId});
39         //borra
40         res.json(removedPost);
41     } catch (error) {
42         res.json({message: error});
43     }
44 });
45
46 /**
47  * Actualizar un post
48  */

```



```

1  router.post('/', async (req,res) => {
2
3      //console.log(req.body); Se utiliza para la respuesta
   a del post en consola
4      const post = new Post({
5          title: req.body.title,
6          description: req.body.description
7      });
8      try {
9          const savedPost = await post.save(); // método qu
   e guarda en la BD
10         res.json(savedPost);
11
12         } catch (error) {
13             res.json({message: error});
14         }
15     });
16
17     /**
18     * Bloque para mostrar solo un post por el Id
19     */
20
21     router.get('/:postId', async (req, res) => {
22         try {
23             const post = await Post.findById(req.params.postId);
24             // encuentra por id
25             res.json(post);
26         } catch (error) {
27             res.json({message: error});
28         }
29     });
30
31     /**
32     * Bloque para borrar un post
33     */
34
35     router.delete('/:postId', async (req, res) => {
36         try {
37             const removedPost = await Post.remove({_id: req.p
   arams.postId}); //borra
38             res.json(removedPost);
39         } catch (error) {
40             res.json({message: error});
41         }
42     });
43
44     /**
45     * Actualizar un post
46     */
47
48     router.patch('/:postId', async (req, res) => { //patch pa
   ra actualizar
49         try {
50             const updatePost = await Post.updateOne( // actualiz
   a de uno en uno
51                 {_id: req.params.postId},
52                 {$set: {title: req.body.title}});
53             res.json(updatePost);
54
55         } catch (error) {
56             res.json({message: error});
57         }
58     });
59
60     module.exports = router; //devuelve como módulo lo que s
   e le asigna a route
61

```

Referencia Nota. SENA (2021).

La clase `express.router` sirve para crear manejadores o `handlers` de rutas montables y modulares. Una instancia `router` es un sistema que invoca una función cuando se presenta algún evento (`middleware`) y además presenta un direccionamiento completo; por este motivo, a menudo se conoce como una "miniaplicación" (Express, 2021).

La palabra clave `async` se añade a las funciones para indicar que devuelvan una **promesa** en lugar de devolver directamente el valor. Adicionalmente, esto permite que las funciones síncronas puedan evitar cualquier potencial de sobrecarga que viene con correr el soporte por usar `async`. De esta manera, cuando una función es declarada `async` con solo añadir la manipulación necesaria, el motor de JavaScript puede optimizar el programa por sí mismo (Mozilla, 2021).

La expresión `await` provoca que la ejecución de una función `async` de una pausa hasta que una promesa sea terminada o rechazada, y vuelve de nuevo a la ejecución de la función `async` después de finalizada. Al regreso de la ejecución, el valor de la expresión `await` se da por una promesa terminada (Mozilla, 2021).

Referencias bibliográficas

Mozilla. (2021). `Await`. MDN Web Docs.



Enlace web. <https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Operators/await>

Servicios web con PHP

Breve descripción:

Los servicios web, también conocidos como web services, son un conjunto de protocolos que facilitan la comunicación entre dispositivos, permitiendo intercambiar información (datos). Un servicio web tiene una interfaz que oculta los detalles de implementación, para que se pueda utilizar independientemente de la plataforma de hardware o software en la que se implementa, e independientemente del lenguaje de programación en el que está escrito.

Julio 2024

Tabla de contenido

Introducción	4
1. Métodos o funciones	6
2. Clases y objetos	8
3. Integración de aplicaciones (XML, SOAP, WSDL y UDDI).....	12
3.1. XML Extensible Markup Language.....	12
Estructura	13
Etiquetas o Tags	15
Atributos.....	16
3.2. SOAP Simple Object Access Protocol.....	19
3.3. WSDL Lenguaje de Descripción de Servicios Web	21
Estructura de WSDL.....	21
3.4. UDDI Universal Description, Discovery, and Integration.....	22
3.5. REST Representational State Transfer	23
3.6. JSON JavaScript Object Notation	24
4. Creación de servicios	31
Creación de servicios.....	31
5. Conexiones a SQL.....	33
Grupo de sentencias de SQL.....	34

6.	Servicios en PHP	35
7.	Aplicación SOAP UI	37
	Uso de la herramienta SOAP UI	37
	Síntesis	38
	Material complementario	39
	Glosario	40
	Referencias bibliográficas	41
	Créditos	43

Introducción

En el mundo de la programación y desarrollo de software, la organización y estructuración del código son fundamentales para asegurar la eficiencia y facilidad de mantenimiento de las aplicaciones. Este documento aborda conceptos esenciales como métodos o funciones, clases y objetos, y la integración de aplicaciones mediante estándares como XML, SOAP, WSDL y UDDI. Además, se explora el proceso de creación de servicios, destacando su desarrollo, ejecución y consumo. Estos temas son cruciales para los desarrolladores que buscan construir sistemas robustos y escalables, capaces de interactuar con diversas aplicaciones y plataformas. A través de una comprensión clara y concisa de estos elementos, se sientan las bases para la creación de soluciones tecnológicas que respondan a las necesidades actuales del mercado.

La correcta utilización de métodos o funciones permite segmentar el trabajo en tareas más pequeñas y manejables, facilitando su reutilización y mantenimiento. Por otro lado, la definición clara de clases y objetos es fundamental para la organización del código, proporcionando una estructura lógica que mejora la eficiencia del desarrollo y la claridad del programa. Estos conceptos no solo optimizan el proceso de programación, sino que también permiten una mejor colaboración entre equipos de desarrollo, ya que el código se vuelve más legible y comprensible.

La integración de aplicaciones mediante XML, SOAP, WSDL y UDDI es un componente vital en la creación de servicios web modernos. Estos estándares permiten que diferentes aplicaciones se comuniquen e interactúen entre sí, independientemente de las plataformas o lenguajes de programación utilizados. Al dominar estos procesos, los desarrolladores pueden crear servicios que sean accesibles y utilizables por una

amplia gama de sistemas, asegurando la interoperabilidad y ampliando el alcance y la funcionalidad de las aplicaciones. Esta integración es esencial para construir soluciones tecnológicas que sean tanto versátiles como adaptables a las cambiantes demandas del mercado.

Le deseamos muchos éxitos en este proceso de aprendizaje.

1. Métodos o funciones

De acuerdo con Gonzáles y Pelissier (2002), los métodos o funciones permiten segmentar el trabajo que hace un programa en subtareas o tareas más pequeñas, enfocadas en un fin específico, y se pueden utilizar cuantas veces se necesite. Estas están separadas del programa principal, pero aportan a su objetivo.

El uso de funciones da la capacidad de agrupar varias instrucciones de código bajo un solo nombre. Con el nombre de la función, es posible invocarla las veces que se necesite, permitiendo utilizar esta función repetidamente según sea necesario.

Ejemplo:

La función se define con la siguiente sintaxis:

```
<?php
```

```
function media_aritmetica($a, $b){ //Aquí definimos la función y la llamamos  
media_aritmetica y las variables a y b
```

```
    $media=($a+$b)/2; //escribimos la fórmula de la media, en este caso de 2  
números divididos entre 2
```

```
    return $media; // aquí la función retorna el valor de la media
```

```
}
```

```
echo media_aritmetica(7,9),"<br>"; // aquí ingresamos los números, en este caso  
7 y 9
```



```
echo media_aritmetica(300,500),"<br>"; ?> //aquí ingresamos otros dos números  
300 y 500
```

Nota: el símbolo “//” quiere decir que no hace parte del código, se utiliza para hacer comentarios sobre el código, por lo cual se pone en color azul.

La función devuelve como resultados los números 8 y 400.

En el ejemplo anterior, se describe la sintaxis de una función que calcula la media aritmética de dos o más números, pero es necesario recordar qué es la media aritmética. Según Paz (s. f.), “La media aritmética o promedio simple se calcula sumando los valores de interés y dividiendo entre el número de valores sumados”. Para el caso anterior, se suman 2 números y el resultado se divide entre 2, así:

$$\frac{a + b}{2}$$

Los autores, además, dicen que estos tienen 3 tipos de visibilidad: Public “Pública”, Private “Privada”, o Protected “Protegidas”, si no se especifica, se sobreentiende que es privada.

2. Clases y objetos

Las clases son contenedores de información basados en atributos y en métodos de construcción, los cuales pueden modificar y consultar; son suficientes para representar objetos con los cuales se procesa información y se usan para hacer objetos que tienen un mismo comportamiento, estado e identidad.

Ejemplo:

Se tiene una clase “personas” (Mario, Margarita y Alejandro), donde el comportamiento de las personas es leer, trabajar, jugar, etc.; pueden estar en estado despierto o dormido; sus propiedades pueden ser color de ojos, género, estado civil, etc.

Su sintaxis sería así:

```
class Persona {  
  
# Propiedades  
  
# Métodos  
  
}
```

- **Objeto:** el objeto es una instancia de una clase, la cual puede crear varias instancias de la misma clase.
- **Ejemplo:** existe una sola clase “Persona”, pero muchos objetos de tipo persona pueden ser instancias de esta clase.

A continuación, revise la diferencia entre clases y objetos con una analogía:

- **Una clase**

Es como el plano de un apartamento, define la forma del apartamento sobre el papel, con las relaciones entre las diferentes partes claramente definidas y planeadas, a pesar de que el apartamento no exista.

- **Un objeto**

Es el apartamento real construido de acuerdo a ese modelo; es decir, un objeto es una instancia de la clase.

Ejemplo:

En el siguiente ejemplo, se observará el código de un objeto.

```
<?php
```

```
class Persona // Se define la clase Persona
```

```
{
```

```
    private $nombre; // Declaración del campo. Este debe ser de carácter privado.
```

```
    // Aquí adelante se utiliza un constructor, que es un método que lleva el  
    nombre de la clase y que será el responsable de construir el objeto.
```

```
    function Persona($nom)
```

```
{
```

```
        $this->nombre = $nom; // this hace referencia a la instancia actual, y se le  
        asigna el valor al constructor->
```

```
}
```

```
function setNombre($n) { // Métodos accesorios
```

```
    $this->nombre = $n; // Métodos de escritura
```

```
}
```

```
function getNombre() { // Métodos de lectura
```

```
    return $this->nombre;
```

```
}
```

```
public function Saluda($saludo) { // Métodos se define como público, solo es  
un mensaje
```

```
    echo $saludo."<br>";
```

```
}
```

```
}
```

```
$objPersona = new Persona("Mario Meneses"); // Se define el nombre del  
Objeto
```

```
echo "<p>El objeto <span style='color:red; font-  
weight:bold'>objPersona</span> se llama " .
```

```
$objPersona->getNombre()."</p>"; // se imprime el texto "el objeto se llama  
con estilos de color"
```

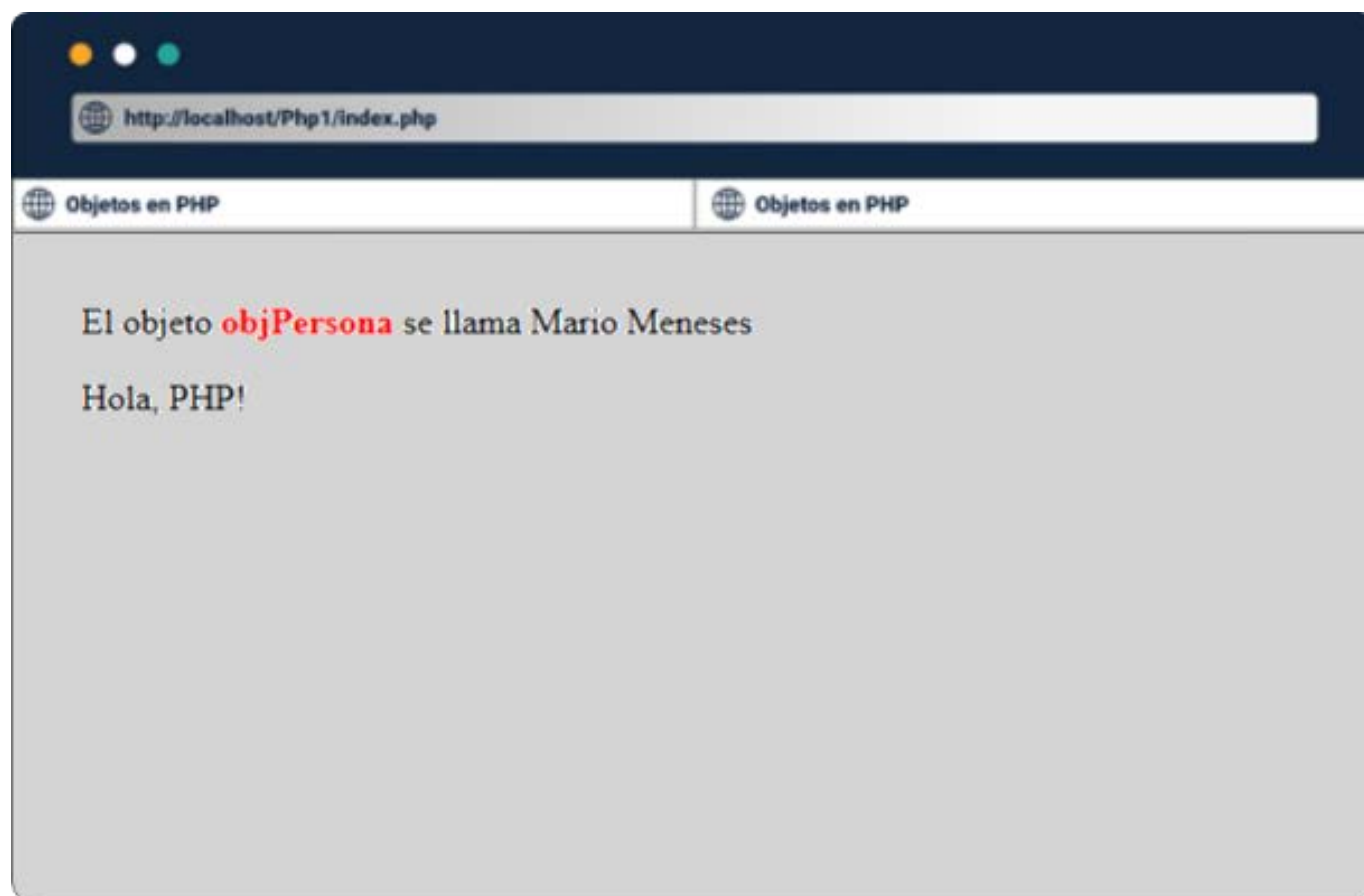
```
$objPersona->Saluda("Hola, PHP!"); // se imprime el nombre escrito
```

```
?>
```

Resultado

Si se ejecutara el código, daría este resultado.

Figura 1. Resultado del ejemplo



3. Integración de aplicaciones (XML, SOAP, WSDL y UDDI)

Los servicios web permiten la nueva generación de aplicaciones basadas en Internet. Estos servicios admiten la comunicación de Internet de aplicación a aplicación, es decir, las aplicaciones en diferentes ubicaciones de la red se pueden integrar para funcionar como si fueran parte de un único sistema de software grande. Ejemplos de aplicaciones que los servicios web hacen posibles incluyen transacciones comerciales automatizadas y acceso directo (sin navegador) a dispositivos de escritorio y de mano, a las reservas, el comercio de acciones y los sistemas de seguimiento de pedidos.

Han surgido varios estándares clave que, juntos, forman la base de los servicios web: XML (Extensible Markup Language), WSDL (Web Services Definition Language), SOAP (Simple Object Access Protocol) y UDDI (Universal Description, Discovery, and Integration). Además, se ha especificado ebXML (Electronic Business XML) para facilitar la integración automatizada de procesos comerciales entre socios comerciales.

3.1. XML Extensible Markup Language

“XML (Extensible Markup Language) es un lenguaje de marcas que define un conjunto de reglas para la codificación de documentos, se utiliza para estructurar información en cualquier fichero que contenga texto y es muy usado en el medio, debido a que es un estándar abierto y libre, creado por el consorcio World Wide Web, W3C”. Bianco (2005)

Además, el lenguaje XML proporciona una estructura para definir elementos, crear un formato y generar un lenguaje personalizado. Estos archivos están conformados por dos partes: prolog y body.

- **PROLOG**

Son metadatos administrativos, como declaración XML, instrucción de procesamiento opcional, declaración de tipo de documento y comentarios.

- **BODY**

Se compone de dos partes: estructural y de contenido (presente en los textos simples).

El XML es de uso sencillo, por lo cual se utiliza en varios servicios web, haciendo posible una forma independiente de almacenar datos para que puedan ser compartidos por diferentes aplicaciones, logrando una óptima compatibilidad entre dispositivos.

Estructura

El prólogo va en la primera línea de código. Aquí se declara que el documento es XML y qué versión se está utilizando. Vea el siguiente ejemplo:

Ejemplo:

```
<?xml version="1.0"?>
```

Declaración normal, sirve para cualquier documento, solo especifica la versión.

```
<?xml version="1.0" encoding="ISO-8859-1" standalone="yes"?>
```

Aquí está la versión y también se agrega el tipo de codificación y el standalone, que se refiere a que es un documento autónomo.

En el cuerpo es donde va la programación. Hay que tener en cuenta un elemento principal llamado raíz, dentro del cual se encuentran el resto de los elementos. Este elemento es el “padre” de todos los demás elementos, y de él se derivan las ramas del árbol hasta el nivel más bajo.

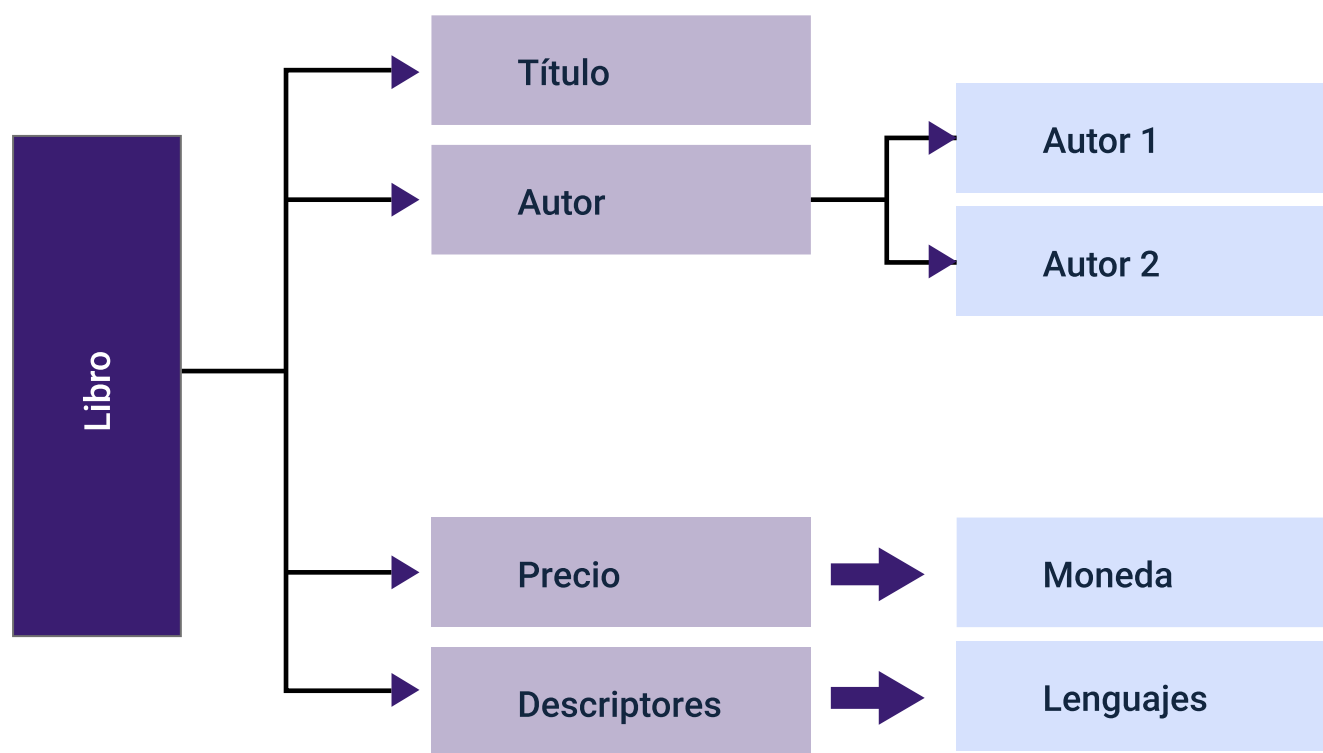
Hay que tener en cuenta que XML es muy diferente de HTML, por ejemplo:

Tabla 1. Diferencias XML-HTML

XML	Título de col2
<code><libro></code> <code><autor>Mario Meneses</autor></code> <code><titulo>Guía de Aprendizaje XML </titulo></code> <code><precio moneda="pesos">100.000 </precio></code> <code></libro></code>	<code>Mario Meneses</code> <code><i>Guía de Aprendizaje XML</i></code> <code>precio: 100.000 \$ </code>
XML Utiliza etiquetas para definir el contenido y significado de esta información.	HTML se centra en colocar etiquetas para representar información.

Para hacer un XML, se debe tener en cuenta la estructura de árbol, la cual es jerárquica. Debe haber un elemento raíz del cual se desprenden todos los demás. Estos no se pueden superponer entre ellos; al contrario, deben estar anidados. Por ejemplo, de la siguiente forma:

Figura 2. Estructuras



Etiquetas o Tags

En el anterior ejemplo, se observa que hay elementos en los cuales se describen los datos que se contienen, así:

Ejemplo:

En el ejercicio, se puede identificar que:

`<titulo>Guía de Aprendizaje XML</titulo>`

- Cuando se inicia una etiqueta, siempre se deben abrir así: `<titulo>`
- Cuando se termina de describir un dato, siempre se debe cerrar así:
`</titulo>`

Atributos

Un atributo sirve para proporcionar información extra sobre el elemento que lo contiene. Los atributos siempre están dentro de las etiquetas de apertura:

Ejemplo:

En el ejercicio, se puede observar que:

```
<precio moneda="euros">30</precio>
```

```
<precio moneda="pesos">80.000</precio>
```

El atributo está entre comillas dobles, pero también puede ir entre comillas sencillas. En este caso específico, indica que la moneda de un libro es en euros, mientras que el otro es en pesos.

```
<autor>Mario Meneses</autor>
```

```
<titulo>Guía de Aprendizaje XML</titulo>
```

```
<precio moneda="euros">30</precio>
```

```
<autor>Eduardo Benavides</autor>
```

```
<titulo>XML con ejemplos</titulo>
```

```
<descriptor>lenguajes</descriptor>
```

```
<descriptor>Programación</descriptor>
```

```
<precio moneda="euros">30</precio>
```

```
<precio moneda="pesos">80.000</precio>
```

Revisemos el siguiente ejercicio:

```
<Familia>
```

```
<Padres>
```

```
<Padre nombre = "Mario"></Padre>
```

```
<Madre nombre = "Margarita"></Madre>
```

```
</Padres>
```

```
<Hijos>
```

```
<Nombre> "Samuel" genero="Masculino"</Nombre>
```

```
<Nombre> "Ana" genero="Femenino"</Nombre>
```

```
</Hijos>
```

```
<Mascotas>
```

```
<gato> nombre="Bills"</gato>
```

```
<perro> nombre="Vegeta"</perro>
```

```
</Mascotas>
```

```
</Familia>
```

Es posible probar y editar este código fuente de una manera muy sencilla:

- **Paso 1:**

Abrir el programa Notepad++

- **Paso 2:**

Pegar o escribir el código.

- **Paso 3:**

Dar clic en guardar como, seleccionar una ubicación y, en el nombre, escribir un nombre+.xml, así: Familia.xml

- **Paso 4:**

Guardar los cambios.

- **Paso 5:**

Ahora, al archivo que guardó, se le da clic derecho y se abre con el navegador Google Chrome, y debe aparecer en el navegador así:

```
<Familia>
```

```
  <Padres>
```

```
    <Padre nombre = "Mario"></Padre>
```

```
    <Madre nombre = "Margarita"></Madre>
```

```
  </Padres>
```

```
  <Hijos>
```

```
    <Nombre> "Samuel" genero="Masculino"</Nombre>
```

```
    <Nombre> "Ana" genero="Femenino"</Nombre>
```

```
  </Hijos>
```

```
  <Mascotas>
```

```
    <gato> nombre="Bills"</gato>
```

```
    <perro> nombre="Vegeta"</perro>
```

</Mascotas>

</Familia>

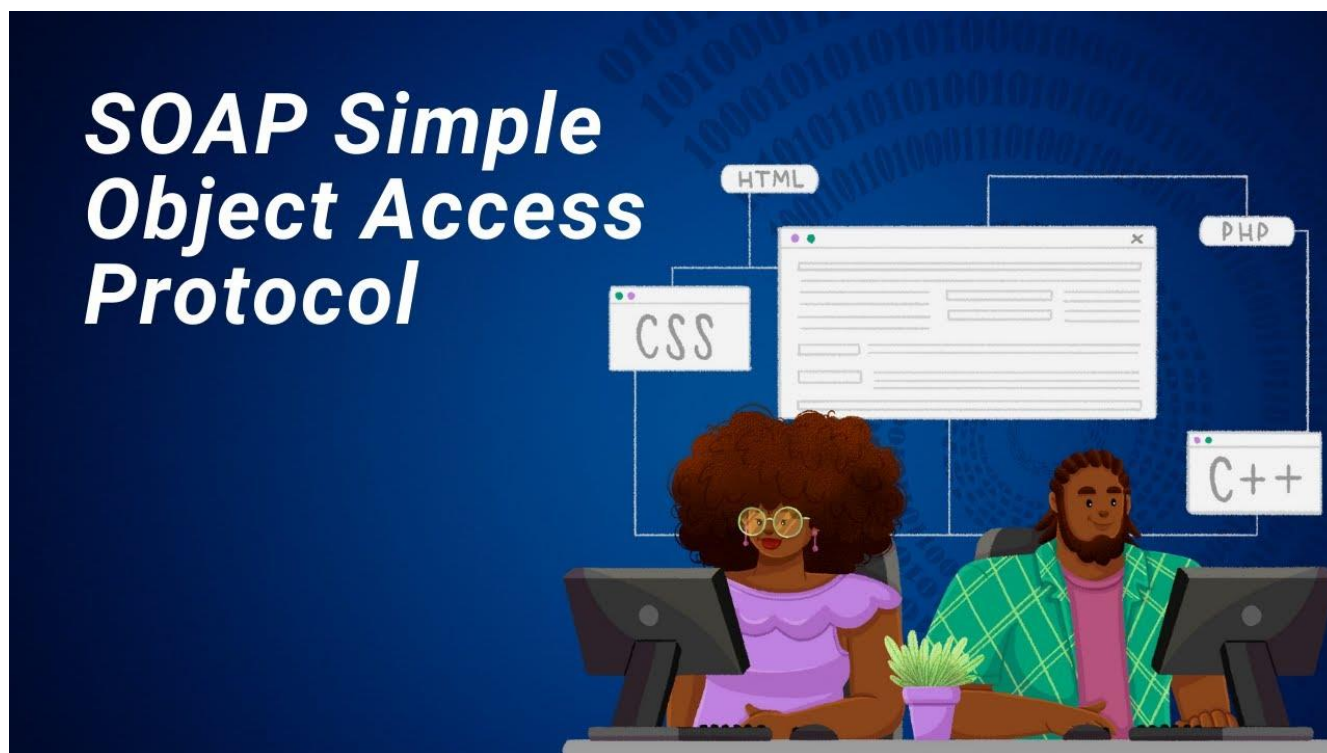
De acuerdo con Bianco (2005), hay que recordar que:

- Para editar el código, existen muchos programas, como: Microsoft Word, bloc de notas, Notepad++ y un sin número de editores.
- Para visualizar, es mejor utilizar los navegadores Google Chrome o Mozilla Firefox, ya que otros pueden tener problemas de compatibilidad.
- XML hace distinción de la escritura en mayúsculas y minúsculas, por lo cual hay que tener cuidado en la escritura.

3.2. SOAP Simple Object Access Protocol

SOAP es un acrónimo de Protocolo Simple de Acceso a Objetos. Es un protocolo de mensajería, basado en XML, para intercambiar información entre computadoras, y es una aplicación de la especificación XML. Profundice con el siguiente video:

Video 1. SOAP Simple Object Access Protocol



[Enlace de reproducción del video](#)

Síntesis del video: SOAP Simple Object Access Protocol

SOAP, sus siglas hacen referencia a Simple Object Access Protocol, es un protocolo estándar que define cómo dos objetos en diferentes procesos, deben comunicarse por medio de intercambio de datos XML. Las tiendas en línea, los buscadores y las empresas que ofrecen productos en línea aplican a este protocolo, el cual se utiliza desde los años 90, es la conexión entre un cliente navegador de internet y los servicios de un servidor.

La función básica de SOAP, es comunicar aplicaciones realizando la llamada a procedimiento remoto RPC e implementando un esquema requerimiento respuesta

equivalente a una arquitectura cliente servidor en la cual el cliente inicia el proceso y el servidor responde al requerimiento, el cliente envía un mensaje request vía http que es generado por la aplicación servidora y una respuesta que es enviada a la aplicación cliente vía http.

3.3. WSDL Lenguaje de Descripción de Servicios Web

Según Gutiérrez (2016), es un lenguaje por medio del cual un servicio web describe, entre otras cosas, qué hace o qué funcionalidad implementa. Es el lenguaje de la interfaz pública para los servicios web. Es una descripción basada en XML de los requisitos funcionales necesarios para establecer una comunicación con los servicios web.

Estructura de WSDL

Este archivo tiene una estructura jerárquica, por lo tanto, la información está anidada. Tiene dos tipos de descripciones:

- **Descripciones abstractas**
Se refiere a la funcionalidad del servicio.
- **Descripciones concretas**
Comunican datos específicos, como el protocolo de transmisión.

El mismo autor resalta que WSDL recurre a 6 elementos principales:

- **Paso 1:**
Types. Tipos de datos.
- **Paso 2:**
Message. Descripción de los datos a transferir.

- **Paso 3:**

Interface. Operaciones abstractas que describen la comunicación entre el servidor y el cliente (aún se llamaba portType en una versión anterior del estándar).

- **Paso 4:**

Binding. Información sobre el protocolo de transporte utilizado.

- **Paso 5:**

Endpoint. Información sobre la interfaz de comunicación, generalmente, en forma de un URI (aún se denominaba port en una versión anterior del estándar).

- **Paso 6:**

Service. Puntos de acceso del servicio web.

Por medio del WSDL, el cliente tiene toda la información necesaria para acceder a un servicio web, el beneficio es que todos los sistemas poseen el mismo lenguaje, lo que hace que los servicios web sean multiplataforma.

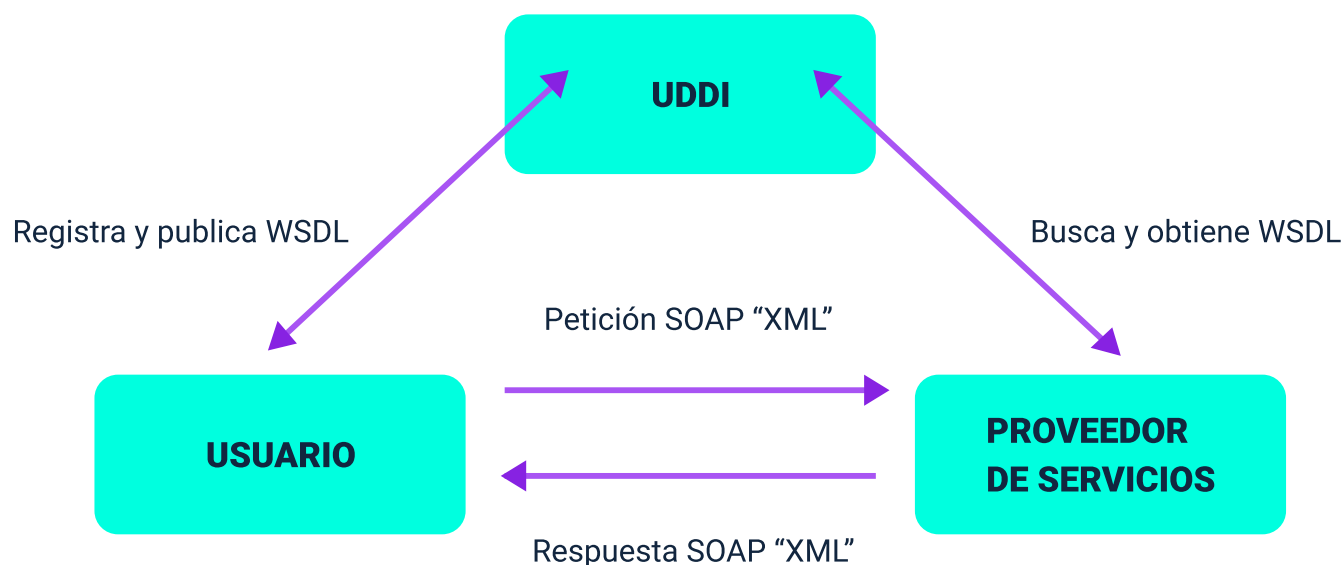
3.4. UDDI Universal Description, Discovery, and Integration

Para IBM Corporation (s. f.), UDDI (Universal Description, Discovery, and Integration) define un modo de publicar y encontrar información sobre servicios web. Incluye un esquema XML para mensajes SOAP que define un conjunto de documentos para describir información de empresas y servicios, un conjunto común de API para consultar y publicar información en los directorios, y una API para duplicar entradas de directorio entre nodos UDDI iguales.

UDDI tiene dos funciones:

- Es un protocolo basado en SOAP que define cómo se comunican los clientes con los registros UDDI.
- Es un conjunto de registros duplicados globales en particular.

Figura 3. Protocolo UDDI



3.5. REST Representational State Transfer

El protocolo REST (Representational State Transfer) es una arquitectura cliente-servidor, en la cual un servicio es visto como un recurso que es identificado a través de una dirección URL, por medio de la cual puede ser accedido o consumido. Para acceder a estos servicios web, se hace uso de mensajes en formato simple, los cuales se intercambian entre cliente y servidor.

Este define, a partir de HTTP, cuatro métodos:

- **GET**

Es usado para enviar la representación de un recurso o servicio al cliente “consulta información”.

- **PUT**

Este es usado para transferir el estado de un cliente al recurso “actualiza un registro”.

- **DELETE**

Borra un recurso específico.

- **POST**

Es utilizado para enviar una entidad a un recurso en específico, causando a menudo un cambio en el estado o efectos secundarios en el servidor “crear un nuevo registro”.

Para hacer uso de esta comunicación, se emplean los siguientes lenguajes o formatos: XML, HTML y JSON. JSON es el tipo de mensajes más difundido en diferentes servicios propios de redes sociales (Facebook y Twitter) y comunidades en Internet. Cada mensaje intercambiado contiene la información necesaria para el funcionamiento del servicio, de tal forma que, para cada servicio, el cliente y el servidor conocen el formato o protocolo interno de los mensajes.

3.6. JSON JavaScript Object Notation

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos. Es de fácil lectura y escritura para los usuarios y fácil de analizar y generar por parte de los dispositivos. Además, se basa en un subconjunto del lenguaje de

programación JavaScript. Es compatible con la gran mayoría de lenguajes de programación, lo que lo hace el más popular y usado en los web services. (IBM Corporation, 2020).

La cardinalidad, también llamada multiplicidad, indica la cantidad de elementos o instancias de una entidad A que se relacionan con una instancia de una entidad B y viceversa. Esta puede ser de cuatro tipos:

- **Uno a uno (única)**

Una instancia de la entidad A se relaciona con una sola instancia de la entidad B.

- **Uno a varios (múltiple)**

Una instancia de la entidad A se relaciona con múltiples instancias de la entidad B.

- **Varios a uno (múltiple)**

Múltiples instancias de la entidad A se relacionan con una sola instancia de la entidad B.

- **Varios a varios (múltiple)**

Múltiples instancias de la entidad A se relacionan con múltiples instancias de la entidad B.

A continuación, se presentan los siguientes ejemplos:

Ejemplo 1

Se va a crear un objeto con el nombre negocio con cardinalidad única:

Figura 4. Objeto con cardinalidad única

Cliente:

- ☐ Nombre: String
- ☐ Apellido: String
- ☐ Dirección: Address
- ☐ Teléfono: String

Address:

- ☐ *Street Address*: String
- ☐ *City*: String
- ☐ *State*: String
- ☐ *Postal Code*: int

Se asignan los siguientes valores para las propiedades del objeto negocio:

a) Objeto:

- Cliente

b) Propiedad:

- nombre
- apellido
- streetAddress
- city
- state
- codigopostal
- telefonos[0]
- telefonos[1]

c) Valor:

- Alejandro
- Meneses
- Calle 5 con carrera 26
- Santa Marta

- Magdalena
- 760004
- 50158182332
- 61745896258

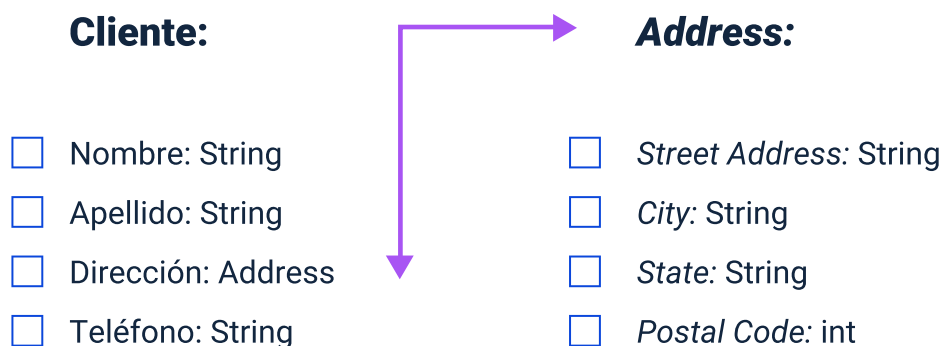
El formato Json quedaría de la siguiente forma:

```
{  
  
  "nombre": "Alejandro",  
  
  "apellido": "Meneses",  
  
  "address": {  
  
    "streetAddress": "Calle 5 con carrera 26",  
  
    "city": "Santa Marta",  
  
    "state": "Magdalena",  
  
    "postalCode": 760004  
  
  },  
  
  "telefonos": [  
  
    "50158182332",  
  
    "61745896258"  
  
  ]  
  
}
```

Ejemplo 2

Se va a crear un objeto con el nombre negocio con cardinalidad múltiple:

Figura 5. Objeto con cardinalidad múltiple



También, se asignan los siguientes valores para las propiedades del objeto negocio:

a) Objeto:

- Address []
- Address []

b) Propiedad:

- nombre
- apellido
- streetAddress
- city
- state
- postalCode
- telefonos[0]
- telefonos[1]

c) Valor:

- Alejandro
- Meneses
- Calle 5 con carrera 26
- Santa Marta
- Magdalena
- 760004
- Calle 18 No. 25-10
- Popayán
- Cauca
- 761115
- 50158182332
- 61745896258

El formato Json quedaría de la siguiente forma:

```
{  
  
  "nombre": "Alejandro",  
  
  "apellido": "Meneses",  
  
  "address": [  
  
    {  
  
      "streetAddress": "Calle 5 con carrera 26",  
  
      "city": "Santa Marta",
```

```
"state": "Magdalena",  
  
"codigopostal": 760004  
  
},  
  
{  
  
"streetAddress": "Calle 18 No 25-10",  
  
"city": "Popayán",  
  
"state": "Cauca",  
  
"postalCode": 760004  
  
}  
  
],  
  
"phoneNumbers": [  
  
"50158182332",  
  
"61745896258"  
  
]  
  
}
```


4. Creación de servicios

Indican que es una tecnología que establece unos estándares y protocolos que se utilizan para intercambiar datos entre distintas aplicaciones, plataformas y con diferentes o iguales lenguajes de programación. Por lo general, se utilizan estándares abiertos, lo que comúnmente se llama interoperabilidad; las organizaciones OASIS y W3C son los comités responsables de la arquitectura y la reglamentación de los servicios web. Sayago et al. (2019)

La función es permitir que los dispositivos interactúen entre sí y presenten información dinámica al usuario, basándose en una arquitectura de referencia estándar, que hace posible su combinación para realizar integraciones entre las mismas aplicaciones.

Creación de servicios

Para la creación de servicios, se debe seguir el siguiente proceso:

- **Desarrollo de servicios**

Se realiza en la parte de servicios o en la nube, y este tipo de desarrollo también se conoce como backend. Esta parte de la programación no es visible para el usuario y no se ejecuta en los dispositivos o navegadores del cliente. Aquí es donde se provee el servicio para que los clientes lo consuman (Recalde, 2019).

- **Ejecución de servicios**

Se realiza en la parte de servicios o en la nube, y este tipo de desarrollo también se conoce como backend. Esta parte de la programación no es visible para el usuario y no se ejecuta en los dispositivos o navegadores del

cliente. Aquí es donde se provee el servicio para que los clientes lo consuman (Recalde, 2019).

- **Consumo de servicios**

Una vez el servicio es creado, es el turno de los usuarios para consumirlo. Por ello, se desarrollan o programan aplicaciones del lado del frontend para consumir dichos servicios; esto consiste en la conversión de datos en una interfaz gráfica, para que el usuario pueda interactuar con la información.

5. Conexiones a SQL

Para iniciar con este tema, es preciso recordar que SQL (Structured Query Language) es un lenguaje de consulta estructurado, definido como la base de las bases de datos relacionales con el lenguaje de alto nivel estándar.

De acuerdo con García (2003), SQL agrupa tres tipos de sentencias con objetivos particulares, en los siguientes lenguajes:

- DDL (Lenguaje de Definición de Datos)
- DML (Lenguaje de Manipulación de Datos)
- DCL (Lenguaje de Control de Datos)

El grupo de sentencias de SQL soporta la definición y declaración de los objetos de la base de datos, tales como:

DATABASE: Base de datos misma.

PROCEDURE: Los procedimientos almacenados.

TRIGGER: Los disparadores.

DEFAULT: Valores por defecto.

CREATE, ALTER y DROP INDEX: Índices.

VIEW: Vistas.

TABLE: Tablas.

Grupo de sentencias de SQL

Para el ejemplo que se realizará más adelante, en el cual se unirán todos los temas, se utilizará phpMyAdmin, que es una herramienta escrita en PHP cuya función es administrar MySQL a través de páginas web, utilizando cualquier navegador.

6. Servicios en PHP

PHP es uno de los lenguajes de código abierto más utilizados y adecuados para el desarrollo de aplicaciones y servicios web; este puede ser introducido en HTML. Por ser un lenguaje gratuito, millones de páginas y portales web están creados con PHP. Su función es muy sencilla: el usuario utiliza aplicaciones o navegadores para realizar una solicitud a un servidor, el servidor recibe la petición, organiza la información solicitada y responde enviando la consulta por medio de una página web o aplicación "normal", pero su creación es "dinámica".

Tabla 2. PHP

Páginas estáticas	Petición	Respuesta
Páginas dinámicas	Petición	Procesa, prepara y da respuesta

El proceso de los servicios PHP está estructurado de la siguiente manera:

- **Creación de servicios en PHP**

Los servicios web son una integración de aplicaciones web, básicamente se trata de un servidor que expone un segmento de su funcionalidad para que sus clientes (dispositivos, aplicaciones, computadores u otros servidores) utilicen sus servicios. En su gran mayoría, los servicios se constituyen en JavaScript, que también es un lenguaje de programación, pero igualmente funciona de buena manera con PHP.

- **Implementación de servicios en PHP**

García (2003) describe que, para que un cliente tenga acceso a la información de un servicio, lo implemente y active el método en el

servidor, hay un sistema de catálogo que, de manera predeterminada, efectúa todos los servicios web y sus métodos. De esta forma, con las credenciales adecuadas, un cliente tiene acceso a todas las funciones de los servicios web.

Al momento de implementar un servicio web, se actualiza el WSDL, archivo de Lenguaje de Descripción de Servicios Web. Hay que recordar que cada servicio web tiene su propio WSDL, el cual contiene información sobre los servicios implementados en ese momento y también las firmas de métodos.

- **Consumo de métodos**

El consumo de un servicio, en este caso en RESTful, es tan sencillo como enviar un pedido en HTTP y procesar su respuesta.

Ejemplo:

```
$res = file_get_contents("https://www.sena.edu.co/es-  
co/formacion/Paginas/Estudie-en-el-SENA.aspx");
```

Ejercicio práctico creación de servicios web

En el siguiente ejercicio, se observarán en detalle los temas que se han abordado en cuanto a servicios web. Se trabajará desde el punto de vista del creador de servicios y el cliente o consumidor de servicios:

- XAMPP
- Editor de código fuente de licencia libre como = Notepad++ o NetBeans
- Navegador web = Chrome o Internet Explorer

7. Aplicación SOAP UI

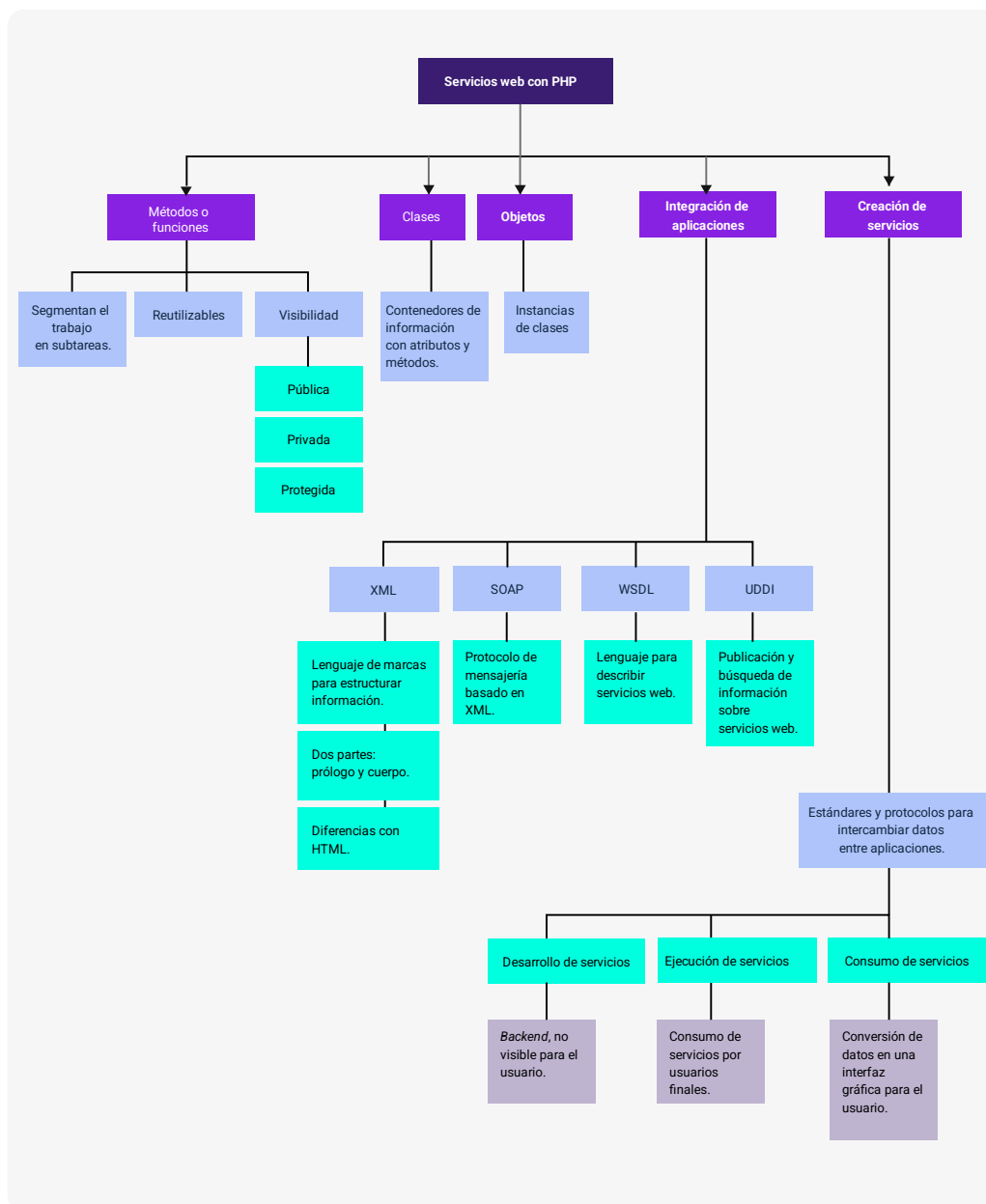
Según la organización SoapUI (s. f.), la aplicación SOAP UI es una herramienta muy versátil que sirve para probar, simular y obtener el código de aplicaciones web service y la transferencia de estado representacional. Esta herramienta está generada en código Java, funciona sobre SOAP, JMS, JDBC, API de REST, GraphQL y muchos servicios más, además de ser muy útil para que los futuros desarrolladores aprendan a crear API.

Uso de la herramienta SOAP UI

Para probar un servicio web existente, se utilizan recursos y directorios de web service que se pueden acceder de manera libre, o diferentes páginas que proveen servicios, como almacenes, bibliotecas, supermercados, entre otros. Para esto, es necesario saber qué tipo de proyecto se utilizará en SOAP UI, ya sea REST o SOAP.

Síntesis

A continuación, se muestra un mapa conceptual con los elementos más importantes desarrollados en este componente.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Integración de aplicaciones (XML, SOAP, WSDL y UDDI)	IBM (s. IBM Corporation. (s. f.). UDDI (Universal Description, Discovery, and Integration).	Página web	https://www.ibm.com/docs/es/rsas/7.5.0?topic=standards-universal-description-discovery-integration-uddi
Servicios en PHP	Meneses, M. (2021a). Creación archivos PHP en el localhost [Video]. YouTube.	Video	https://youtu.be/9MXAQGGxCN4
Servicios en PHP	Meneses, M. (2021b). Web Services programación Front End Consumo de servicios web [Video]. YouTube.	Video	https://youtu.be/MV3dSZx5iTo
Aplicación SOAP UI	SoapUI. (s. f.). Accelerating API Quality Through Testing.	Página web	https://www.soapui.org/

Glosario

API: la interfaz de programación de aplicaciones es un conjunto de subrutinas, funciones y procedimientos que ofrece cierta biblioteca para ser utilizada por otro software como una capa de abstracción.

BackEnd: es la parte del desarrollo web que se encarga de que toda la lógica de una página web funcione. Se trata del conjunto de acciones que pasan en una web pero que no vemos, como, por ejemplo, la comunicación con el servidor.

FrontEnd: el desarrollo web FrontEnd consiste en la conversión de datos en una interfaz gráfica, para que el usuario pueda ver e interactuar con la información de forma digital, usando HTML, CSS y JavaScript.

REST: la Transferencia de Estado Representacional, o REST, es un estilo de arquitectura software para sistemas hipermedia distribuidos, como la World Wide Web.

Referencias bibliográficas

Álvarez, J. (2017). Entorno de programación intencional basado en XML.

Universidad Politécnica de Madrid.

https://oa.upm.es/49793/1/PFC_JOSE_ANTONIO_ALVAREZ_PEREZ.pdf

Bianco, P. (2005). Desarrollo de Aplicaciones Basadas en XML Web Service para Dispositivos Móviles con Microsoft. NET Compact Framework. Universidad de Belgrano.

Chanchí, G., Campo, W., Amaya, J. y Arciniegas, J. (2011). Esquema de servicios para Televisión Digital Interactiva, basados en el protocolo REST-JSON. Cuadernos de Informática, 6(1), p. 233-240.

<http://seer.ufrgs.br/cadernosdeinformatica/article/view/v6n1p233-240>

Eslava, V. (2013). El nuevo PHP. Conceptos avanzados. Bubok.

García, A. (2003). Manual práctico de SQL.

<https://www.lawebdelprogramador.com/cursos/archivos/ManualPracticoSQL.pdf>

Gonzáles, S. y Pelissier, C. (2002). Programación con PHP. Universidad Técnica Federico Santa María.

<http://profesores.elo.utfsm.cl/~agv/elo330/2s02/projects/pelissier/informe.pdf>

Gutiérrez, A. (2016). Elaboración de un servicio web para el registro de operaciones entre clientes:(infraestructura de fibra óptica NEBA de Telefónica). Universidad Carlos III de Madrid.

IBM Corporation. (2015). IBM integration Bus 10.0.0.

<https://www.ibm.com/docs/es/integration-bus/10.0?topic=ssmkhh-10-0-0-com-ibm-etools-mft-doc-bi12017—htm>

IBM Corporation. (2020). Formato Json (JavaScript Object Notation).

<https://www.ibm.com/docs/es/baw/20.x?topic=formats-javascript-object-notation-json-format>

IBM Corporation. (s. f.). UDDI (Universal Description, Discovery, and Integration).

<https://www.ibm.com/docs/es/rsas/7.5.0?topic=standards-universal-description-discovery-integration-uddi>

Ortiz, A., Otón, S. y Barchino, R. (2005). Learning Objects universal publishing and location Architecture using Web Services. Universidad de Alcalá.

https://www.researchgate.net/publication/267217723_Learning_Objects_universal_publishing_and_location_Architecture_using_Web_Services

Paz, K. (s. f.). Media aritmética simple. Universidad Rafael Landívar.

https://fgsalazar.net/LANDIVAR/ING-PRIMERO/boletin07/URL_07_BAS01.pdf

Sayago, J., Flores, E. y Recalde, A. (2019). Análisis comparativo entre los estándares orientados a servicios web SOAP, REST y GraphQL. Revista Antioqueña de las Ciencias Computacionales y la Ingeniería de Software (RACCIS), 9(2), p. 10-22.

<http://dx.doi.org/10.5281/zenodo.3592004>

SoapUI. (s. f.). Acelerando la calidad de la API a través de pruebas.

<https://www.soapui.org/>

Villate, J. (2001). Introducción al XML. Universidad de Oporto.

https://villate.org/publications/Villate_2001_XML.pdf

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Jonathan Guerrero Astaiza	Experta Temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Mario Fernando Meneses Calvache	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluable Instruccionale	Centro de Servicios de Salud - Regional Antioquia
Carlos Julián Ramírez Benítez	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edwin Sneider Velandia Suarez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Construcción aplicación web

Breve descripción:

En este componente formativo se realizará una aplicación web completa (Back-end y Front-end). Se utilizarán tecnologías del lado del servidor como Node.js, MongoDB, Express y del lado del cliente AngularJS y Materialize. La aplicación consumirá un API REST con el propósito de hacer el registro de empleados en una empresa.

Julio 2024

Tabla de contenido

Introducción	4
1. Construcción del back-end de la aplicación web	6
1.1. Instalación y configuración de herramientas de Node.js	6
1.2. Desarrollo del index.js	7
1.3. Configuración e instalación de la base de datos MongoDB	12
1.4. Configuración de database.js	14
1.5. Configuración de las rutas del proyecto con Node.js.....	16
1.6. Desarrollo de controladores para la API REST	19
1.7. Realización de pruebas de la API REST con Postman	24
2. Construcción del front-end de la aplicación web	29
2.1. Instalación y configuración de AngularJS	29
2.2. Generación de código con ng.....	31
2.3. Incorporación de Materialize al proyecto web	34
2.4. Configuración de app.component.html	35
2.5. Configuración de los módulos app.module.ts	36
2.6. Configuración del modelo empleado.ts	37
2.7. Creación del servicio empleado.service.ts	39
2.8. Desarrollo del componente empleados.component.ts	41

2.9. Desarrollo de la interfaz HTML empleados.component.html	44
2.10. Probando la aplicación completa.....	47
Síntesis	54
Material complementario.....	55
Glosario	56
Referencias bibliográficas	57
Créditos.....	58

Introducción

El desarrollo de aplicaciones web modernas requiere la integración de diversas tecnologías y herramientas que permitan crear soluciones robustas, escalables y eficientes. En este componente, se detalla el proceso de construcción de una aplicación web utilizando la arquitectura MEAN, que comprende MongoDB, Express.js, Angular y Node.js. Este conjunto de tecnologías permite desarrollar aplicaciones completas del lado del servidor (back-end) y del cliente (front-end), facilitando la creación de sistemas integrados.

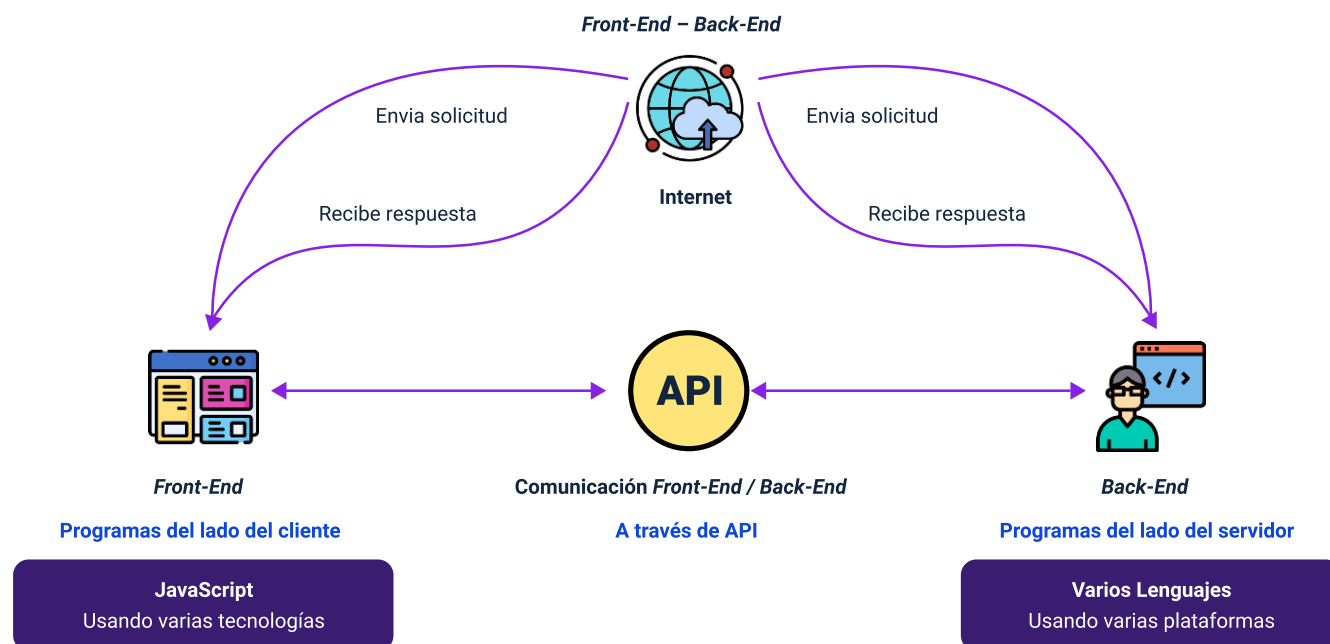
El enfoque principal de este proyecto es la gestión de empleados, abarcando desde la configuración inicial de herramientas y la estructura del proyecto, hasta el desarrollo de controladores, modelos y rutas para la API REST. Además, se explica cómo configurar y utilizar Angular para la creación de componentes y servicios en el front-end, asegurando una interacción fluida y dinámica con el servidor.

Cada sección del componente proporciona instrucciones detalladas y ejemplos de código que guían paso a paso el proceso de desarrollo. Desde la instalación de Node.js y Express, la configuración de la base de datos MongoDB, hasta la integración de Materialize para mejorar la interfaz de usuario, se cubren todos los aspectos necesarios para construir una aplicación web funcional y profesional.

Esto permitirá tener un mayor conocimiento sobre las tecnologías web actuales, especialmente en el uso de las técnicas del lenguaje, los recursos para probar y testear la aplicación, las nuevas formas de sintaxis de algunos lenguajes, los componentes y módulos que harán el trabajo más fácil a los desarrolladores. En últimas palabras, este

componente apoya de manera significativa el primer paso para generar competencias y habilidades con el fin de generar nuevas aplicaciones web.

Figura 1. Imagen con el diagrama que ilustra la comunicación entre FrontEnd y BackEnd



1. Construcción del back-end de la aplicación web

Se inicia con una estructura de proyecto MEAN (Mongo, Express, Angular y Node.js) para el proyecto que abarca tanto el front-end como el back-end. Para la instalación de las primeras herramientas como Node.js y Express.

1.1. Instalación y configuración de herramientas de Node.js

Se utiliza Visual Studio Code (VS Code) con la siguiente estructura de directorios para el back-end, detallada a continuación:

GESTION-EMPLEADOS

 backend

 controllers

 models

 routes

 database.js

 index.js

node_modules

package-lock.json

package.json

Como se presentó anteriormente, se crea la carpeta GESTION-EMPLEADOS, que será la principal o raíz del proyecto. Dentro de esta, se crea la carpeta back-end, en la cual se crean otros directorios y archivos. De tal forma, esta será la estructura que apoyará el desarrollo de la base de la aplicación. Una vez creadas las carpetas y

archivos, como controllers, models, routes, database.js e index.js, se abre la terminal del VS Code y se digita el siguiente comando para crear un proyecto Node.js: npm init -yes

PROBLEMS | OUTPUT | TERMINAL | DEBUG CONSOLE

Windows PowerShell

Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Prueba la nueva tecnología PowerShell multiplataforma

<https://aka.ms/pscore6>

PS D:\gestion-empleados> npm init --yes

Una vez creado el proyecto, se requieren todas las bondades del framework de Node.js, para lo cual es necesario Express. Express generará una serie de componentes que ayudarán mucho al desarrollo de esta aplicación. Por lo tanto, se instala Express con la siguiente línea de comandos desde la terminal del VS Code: npm install express.

1.2. Desarrollo del index.js

El siguiente paso es iniciar con el desarrollo del archivo principal del back-end, que se llama index.js como se presenta a continuación:

```
const express = require('express')
```

```
const morgan = require('morgan');
```

```
const cors = require('cors');
```

```
const app = express(); // la constante app tendrá ahora todo el funcionamiento del servidor
```

```
const { mongoose } = require('./database'); // no se quiere todo el archivo sino la
conexión/** * Se crea una REST API, es la manera de decirle al servidor que reciba y
envíe datos */
```

```
// Configuraciones
```

```
app.set('port', process.env.PORT || 3000);
```

```
app.use(morgan('dev'));
```

```
app.use(express.json()); // método que ayuda a convertir el código para que el
servidor pueda entender lo que viene del cliente.
```

```
app.use(cors({origin: 'http://localhost:4200'})); // método para comunicar con el
cliente
```

```
// rutas de nuestro servidor
```

```
app.use('/api/empleados',require('./routes/empleado.routes'));
```

```
// Iniciando el servidor
```

```
app.listen(app.get('port'), () => { // esta es una mejor manera de configurar el
puerto
```

```
console.log('server activo en el puerto', app.get('port'));
```

```
});
```

A continuación, se presentan los siguientes pasos para el desarrollo del index.js:

a) Desarrollo del archivo principal del #[em back-end]

El siguiente paso es iniciar con el desarrollo del archivo principal del back-end, que se llama index.js.

b) Código del archivo index.js

Se explica el código del archivo index.js para el proyecto. A medida que avance este componente, se explicará cada línea de código que se requiera ejecutar.

c) Requerimiento de Express

Se inicia con el requerimiento de Express y se almacena en una constante en JavaScript llamada express. De esta forma, se tendrá acceso a todo el funcionamiento del servidor con la línea de código: `const app = express();`.

d) Configuración del puerto

Una vez creada la constante app, se pasa a la configuración del puerto por donde va a escuchar y recibir el servidor todo tipo de solicitudes por parte del cliente, según se muestra en la siguiente línea: `app.set('port', process.env.PORT)`

e) Uso de set

Set es para crear una variable que va a ser accedida desde cualquier parte de la aplicación. La estructura es la siguiente: (nombre var, valor). process es porque cuando se despliegue la app no se va a tener la opción de definir el puerto, sino que el mismo servicio de la nube ayudará a definirlo, en caso de que el puerto 3000 esté ocupado.

f) Instalación de Nodemon

Con el fin de revisar los cambios en el servidor de manera automática, se tiene planeada la instalación de algunas herramientas que harán más fácil el desarrollo sobre Node.js, por lo que se instala Nodemon como una utilidad que monitorea de manera constante los cambios en el código fuente que se está desarrollando y de manera automática reinicia el servidor. La línea de código utilizada en la terminal de VS Code es: `npm install nodemon -D`.

g) Configuración de Nodemon

Esto significa que Nodemon será instalado como una dependencia de desarrollo y no como dependencia general del proyecto. Se configura el archivo `package.json` de acuerdo a las instrucciones.

h) Iniciar el servidor

Se inicia a ejecutar el servidor con la siguiente línea de comando: `npm run dev`.

Estructura `package.json`:

```
{  
  
  "name": "gestion-empleados",  
  
  "version": "1.0.0",  
  
  "description": "",  
  
  "main": "index.js",  
  
  "scripts": {
```

```
"dev": "nodemon BackEnd/index.js"

},

"keywords": [],

"author": "",

"license": "ISC",

"dependencies": {

  "cors": "^2.8.5",

  "express": "^4.17.1",

  "mongoose": "^5.12.14",

  "morgan": "^1.10.0"

},

"devDependencies": {

  "nodemon": "^2.0.7"

}

}
```

Ahora, se construyen las Middleware dentro del archivo index.js, encargadas de procesar los datos, es decir, cuando se solicitan o envían al servidor. El servidor debe entender los datos que le llegan del cliente (AngularJS toma los datos del cliente y los envía en formato JSON) y es por eso que se necesita algún tipo de conversión. Por lo

tanto, se requiere del uso del formato JSON ejecutando la siguiente instrucción:

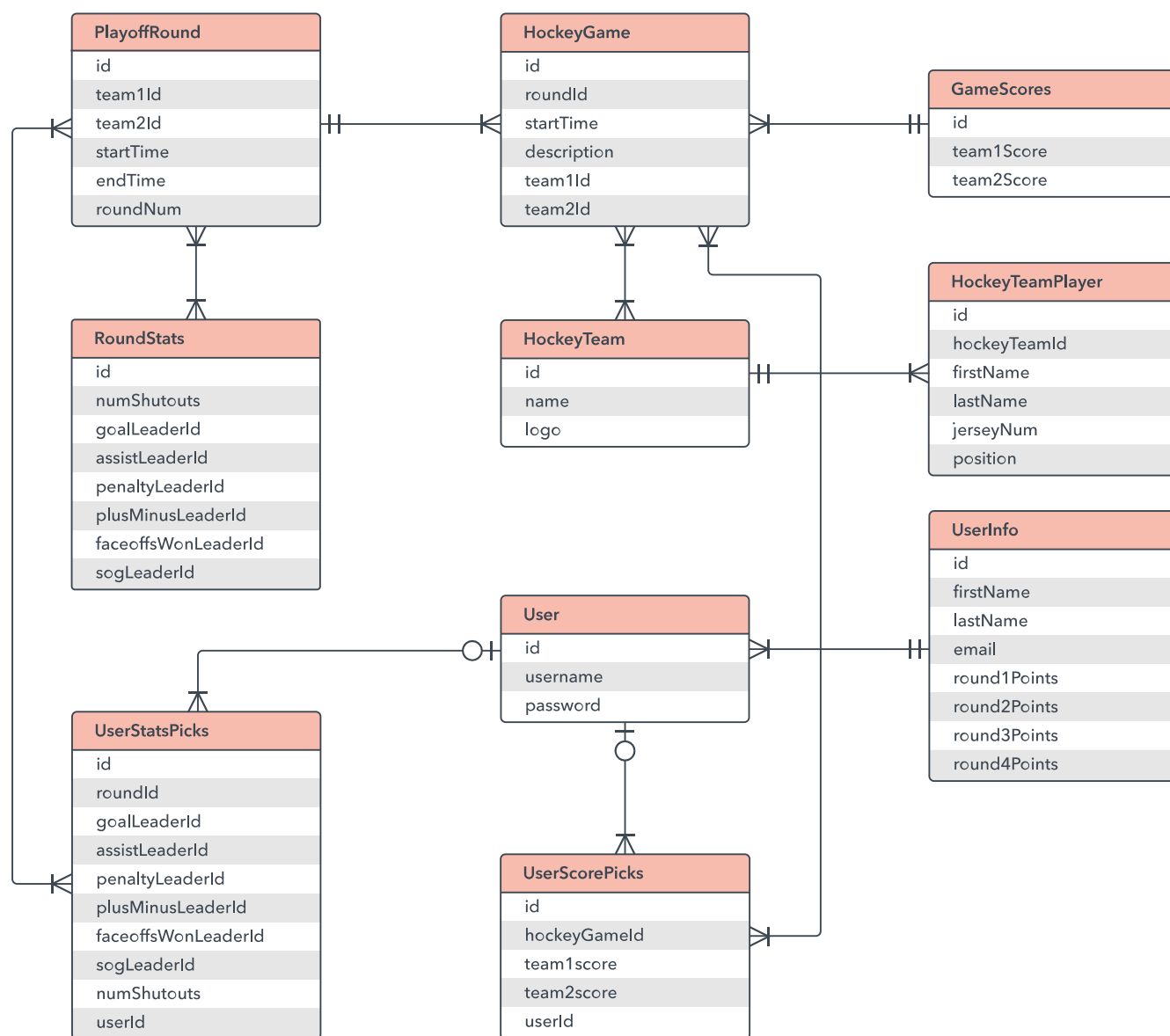
```
app.use(express.json());
```

Para registrar la transacción de mensajes por consola, se puede utilizar una herramienta útil llamada Morgan, que ayuda a verificar en consola lo que el usuario está solicitando. Esta herramienta es opcional para el desarrollo de la aplicación, no obstante, se deja la línea de comando para su instalación y utilización: `npm install morgan`. `app.use(morgan('dev'));`

1.3. Configuración e instalación de la base de datos MongoDB

A continuación, se realizará la conexión de la aplicación del servidor a la base de datos, por lo que se inicia configurando e instalando los recursos necesarios antes de desarrollar. En ese sentido, se instala MongoDB de manera persistente en el equipo de desarrollo. La siguiente figura representa de manera general el diagrama entidad-relación de una base de datos.

Figura 2. Diagrama entidad-relación



De manera opcional, se puede instalar la extensión de MongoDB dentro del entorno de VS Code, permitiendo una mayor integralidad en el desarrollo. Estas herramientas facilitan la celeridad en el proceso de construcción de sistemas de software. Por lo tanto, en el material de apoyo se dejará información para acceder a la guía de instalación de esta extensión, denominada extensión VS Code y MongoDB.

localhost:27017 connected

- admin
- config
- empleados
- local

Posteriormente, para realizar la conexión a la base de datos y definir la estructura de los datos de la aplicación a desarrollar, conocida técnicamente como esquemas, se instala la herramienta Mongoose con la siguiente línea de comando desde la terminal de VS Code: `npm install mongoose`

1.4. Configuración de database.js

Con todas las herramientas necesarias para la conexión de la aplicación a la base de datos, se empieza a desarrollar y configurar el archivo `database.js`

Con respecto al código del archivo `database.js`, se indica que es necesario que el módulo se conecte a MongoDB, por eso se utiliza Mongoose. Al llamarse el módulo, se almacena en una constante. Además, se define la dirección de la conexión de la base de datos en una constante denominada URI, y se utiliza una promesa para obtener la información de la base de datos y corroborar si hay conexión o no.

```
const mongoose = require('mongoose');  
  
const URI = 'mongodb://localhost/empleados';  
  
mongoose.connect(URI)  
  
  .then(db => console.log('DB is connected'))
```

```
.catch(err => console.error(err));
```

```
module.exports = mongoose;
```

Por ser un módulo que se utiliza en toda la aplicación, se exporta. De esta forma, la constante Mongoose devolverá la conexión. De manera adicional, en el archivo index.js se invoca o llama el archivo database.js, especificando que lo que se requiere es la conexión. Solo resta probar si la conexión está activa, por lo que se inicia el servidor con la línea de comando npm run dev, como se indicó anteriormente.

A continuación, se presenta el resultado de la terminal del VS Code:

```
[nodemon] restarting due to changes...
```

```
[nodemon] starting `node backend/index.js`
```

```
(node:78784) DeprecationWarning: current URL string parser is deprecated, and  
will be removed in a future version. To use the new parser, pass option {  
useNewUrlParser: true } to MongoClient.connect.
```

```
(Use `node --trace-deprecation ...` to show where the warning was created)
```

```
(node:78784) [MONGODB DRIVER] Warning: Current Server Discovery and  
Monitoring engine is deprecated, and will be removed in a future version. To use  
the new Server Discover and Monitoring engine, pass option {  
useUnifiedTopology: true } to the MongoClient constructor.
```

```
server activo en el puerto 3000
```

```
DB is connected
```

1.5. Configuración de las rutas del proyecto con Node.js

El siguiente paso es configurar las rutas o URL por donde se van a enviar o recibir los datos, por lo que se crea un archivo llamado `empleado.route.js` dentro de la carpeta `routes`. En este primer apartado, se desea revisar el funcionamiento de las rutas, por lo que, a manera de prueba, se presenta el siguiente código:

```
/**
 * Vamos a crear rutas del servidor
 * creamos un módulo por eso utilizamos express
 * vamos a utilizar como nuestra rest api para
 * enviar y recibir datos en formato json
 */
const express = require('express');
const router = express.Router();

/* generamos un ejemplo cuando le soliciten algo al span servidor por el método
GET */
router.get('/', (req, res) => {
  res.json({
    status: 'API REST funcionando'
  });
})
```

```
module.exports = router;
```

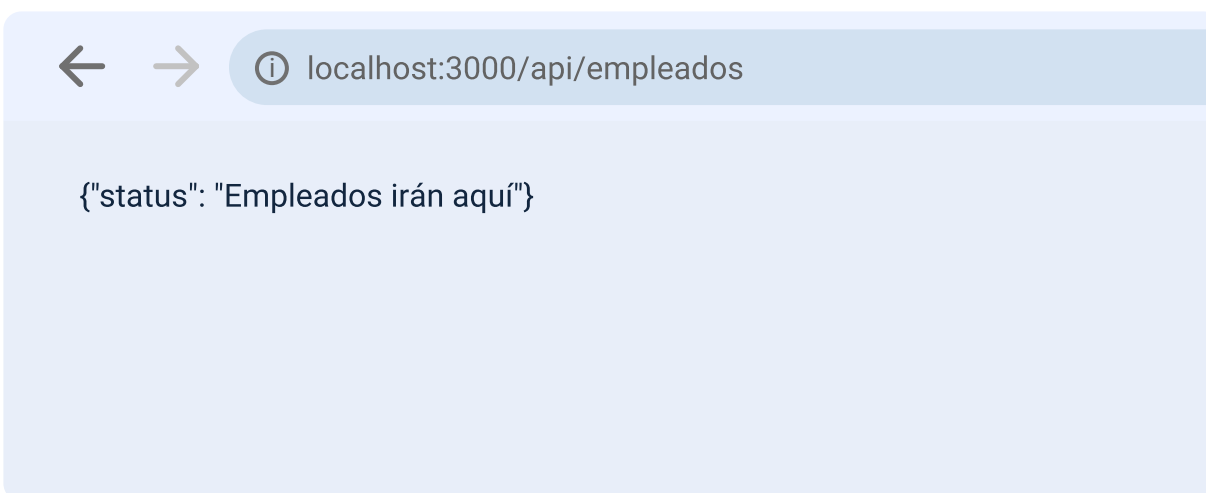
De manera adicional, se puede observar el archivo `index.js`, en el cual se establece la ruta por donde se van a recibir o enviar los datos desde el servidor. Para este caso de estudio, será <http://localhost:3000/api/empleados>

Para la definición de esta ruta, es necesario requerir el archivo `empleado.routes.js` ejecutando la siguiente instrucción:

```
app.use('/api/empleados', require('./routes/empleado.routes'));
```

Al hacer el llamado en el servidor, se vería de la siguiente manera:

Figura 3. Respuesta del servidor



Ya visto un ejemplo del funcionamiento de las rutas, es necesario comprender su comportamiento. Cada solicitud debe ir asociada a una ruta, y esta a su vez a un intercambio de información o de datos, es decir, de interacción con la base de datos. En ese sentido, es más que necesaria la construcción del modelo `empleado.js` con datos específicos para los empleados dentro de la carpeta `models`.

Para construir el modelo empleado.js, primero se debe crear un esquema que defina la estructura de los datos que se almacenarán en la base de datos. Aquí un ejemplo de cómo podría verse el archivo empleado.js:

```
const mongoose = require('mongoose');
```

```
const {Schema} = mongoose;
```

```
const EmpleadoSchema = new Schema({
```

```
  name: {type: String, require: true},
```

```
  position: {type: String, require: true},
```

```
  office: {type: String, require: true},
```

```
  salary: {type: Number, require: true},
```

```
});
```

```
module.exports = mongoose.model('Empleado', EmpleadoSchema);
```

Se requiere Mongoose y se almacena en una constante denominada de la misma forma, que se utilizará para definir los esquemas de datos. Para esto, se empieza a modelar con la información, como nombre y cargo, entre otros. En ese sentido, Mongoose le indica a MongoDB cómo va a lucir la estructura de empleados.

1.6. Desarrollo de controladores para la API REST

Para mantener una estructura organizada y facilitar el crecimiento de la aplicación, se recomienda el uso de controladores. Los controladores son responsables de definir los métodos y acciones necesarios para las rutas de la aplicación. En este caso, se creará el archivo `empleado.controller.js` dentro de la carpeta `controllers`.

A continuación, se presenta un ejemplo de cómo podría estructurarse el controlador para manejar las operaciones básicas de la API REST:

```
/**  
  
 * Se coloca el controlador como un objeto y luego se exporta como  
  
 * se requiere primero el modelo empleado  
  
 */  
  
const Empleado = require('../models/empleado');  
  
const empleadoCtrl = {};  
  
/**  
  
 * DEFINO LOS MÉTODOS  
  
 */  
  
// Obtener todos los empleados  
  
empleadoCtrl.getEmpleados = async (req, res) => {  
  
  const empleados = await Empleado.find();  
  
  res.json(empleados);  
}
```



```
}

// Crear empleados

empleadoCtrl.createEmpleados = async (req, res) => {

  const empleado = new Empleado(req.body);

  await empleado.save();

  res.json({

    'status': 'Empleado guardado'

  });

}

// Conseguir un único empleado

empleadoCtrl.getUnicoEmpleado = async (req, res) => {

  const empleadoUnico = await Empleado.findById(req.params.id);

  res.json(empleadoUnico);

}


// Actualizar empleado

empleadoCtrl.editarEmpleado = async (req, res) => {

  const { id } = req.params;

  const empleadoEdit = {
```

```
name: req.body.name,  
position: req.body.position,  
office: req.body.office,  
salary: req.body.salary  
};  
  
await Empleado.findByIdAndUpdate(id, {$set: empleadoEdit}, {new: true});  
res.json({status: 'Empleado Actualizado'});  
  
}  
  
// Eliminar empleado  
  
empleadoCtrl.eliminarEmpleado = async (req, res) => {  
  await Empleado.findByIdAndDelete(req.params.id);  
  res.json({status: 'Empleado Eliminado'});  
}  
  
  
// exporto el módulo  
  
module.exports = empleadoCtrl;
```

El desarrollo de controladores para la API REST se realiza:

- **Explicación del código**

Se inicia explicando de manera sucinta el código presentado anteriormente. En primera instancia, se requiere el modelo realizado denominado empleado.js, por lo cual se almacena en una constante Empleado.

- **Uso de funciones JavaScript**

En cada uno de los métodos se utilizan funciones nativas y nuevas de JavaScript. Anteriormente, se utilizaban funciones de tipo callback que existían como parámetros de los métodos, después aparecieron las promesas con la función then, pero ahora existen funciones que están dedicadas a realizar de manera más eficiente el procesamiento de los datos y mejorar los tiempos de respuesta como lo son async y await que hacen parte de la última versión de JavaScript.

- **Método obtener empleados**

Para el método de obtener empleados se utiliza la función async, la cual entrega en formato JSON todo lo que encuentre de empleados. Se utiliza la función await, porque se espera que tarde un poco en buscar. Se utiliza uno de los métodos intrínsecos del modelo Empleado, que en este caso es find().

- **Método crear empleado**

El método de crear empleado tiene una naturaleza similar al anterior. En este método se crea una nueva instancia de empleado new Empleado. Luego, se guarda ese nuevo registro con save(). Es importante notar que hasta el momento no se ha creado la base de datos en MongoDB, pero al almacenar un registro, se crea la base de datos.

- **Método editar empleado**

Para el método de editar empleado se presenta una sintaxis interesante. Primero, se escribe de otra forma para capturar el id del usuario que se quiere actualizar { id }. Además, se crea una constante empleadoEdit para pasarle los datos traídos del cliente y finalmente se utiliza await para esperar en la acción de encontrar por id y actualizar a través de la función set, que se utilizará para modificar los datos en última instancia.

- **Exportación del módulo**

Finalmente, al crear un módulo, se exporta para que sea utilizado en cualquier parte de la aplicación.

A continuación, se definen las rutas, por lo que se configura y desarrolla la lógica de esta sección en el archivo empleados.routes.js dentro de la carpeta routes:

```
plaintext
```

```
/**
```

```
*
```

```
* creamos un módulo por eso utilizamos express
```

```
*/
```

```
const express = require('express');
```

```
const router = express.Router();
```

```
const empleadoCtrl = require('../controllers/empleado.controller');
```

```
router.get('/', empleadoCtrl.getEmpleados); // Rutas más limpias (obtener  
empleados)  
  
router.post('/', empleadoCtrl.createEmpleados); // guardar  
  
router.get('/:id', empleadoCtrl.getUnicoEmpleado); // obtiene un único empleado  
  
router.put('/:id', empleadoCtrl.editarEmpleado); // Actualizar datos (uno a la vez)  
  
router.delete('/:id', empleadoCtrl.eliminarEmpleado);  
  
module.exports = router;
```

Con respecto al código anterior, se observa que se requiere el uso del controlador, por lo que se almacena en una constante llamada `empleadoCtrl`, y de esta forma se pueden utilizar todos los métodos de este, es decir, la gestión o CRUD sobre el empleado.

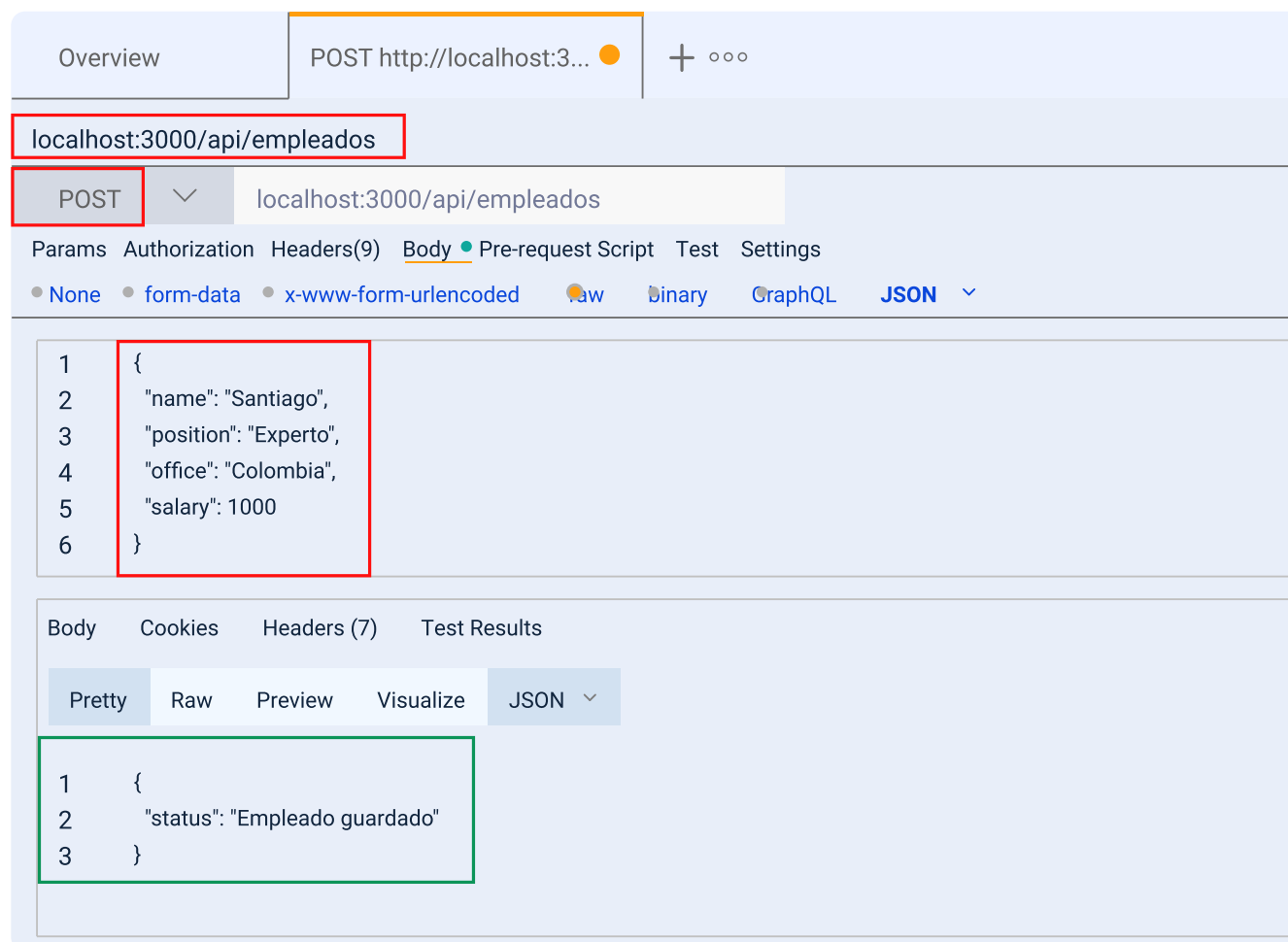
En ese sentido, se observa que el objeto `router` llama a cada uno de los métodos por defecto de petición HTTP como GET, POST, PUT, entre otros, asociados a los métodos realizados con el controlador. De manera que se tienen rutas mucho más limpias y apoyadas con la buena práctica del uso del controlador.

1.7. Realización de pruebas de la API REST con Postman

Una vez finalizada la construcción del modelo, controlador y rutas, se procede a realizar las pruebas respectivas para evaluar el funcionamiento de la API REST. Para ello, se utilizará la herramienta Postman.

Se comienza insertando datos utilizando el método POST, el cual llamará a la ruta para crear empleados.

Figura 4. Pruebas con Postman

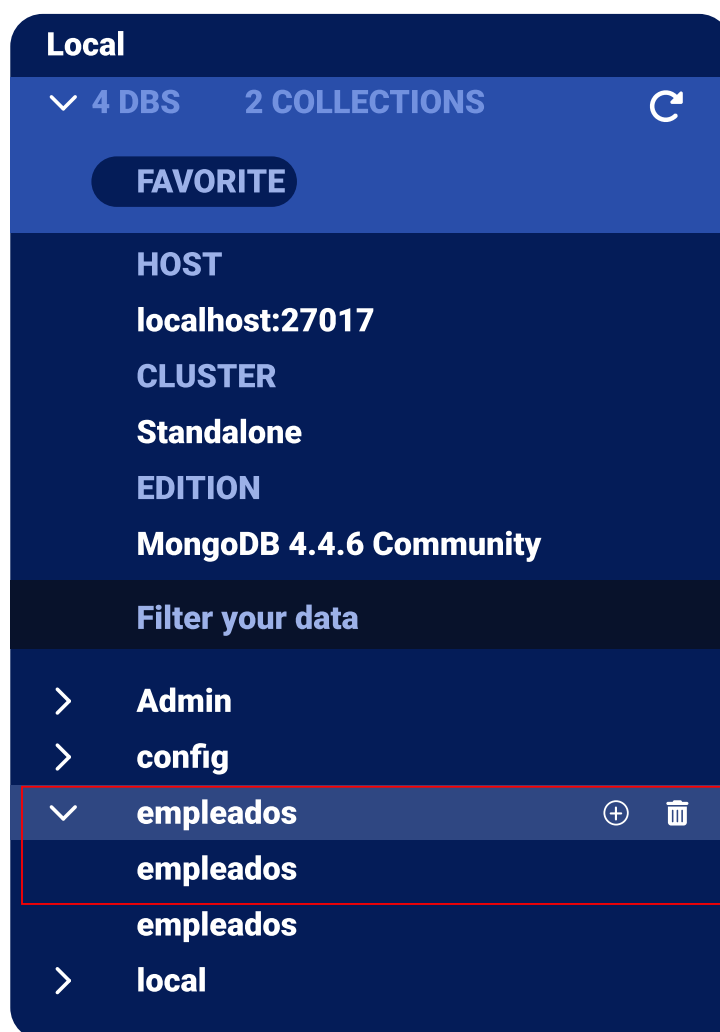


En la figura anterior se pueden apreciar los datos que se envían al servidor, dado que Postman permite realizar esa emulación del cliente. Los datos que se envían están acordes al esquema definido en el modelo de empleado.js, como lo son name, position, office y salary. La respuesta en consola, que se programa como status, es que el Empleado ha sido guardado.

En este orden de ideas, es necesario revisar la base de datos y verificar que efectivamente se ha guardado la información y si realmente MongoDB la ha creado.

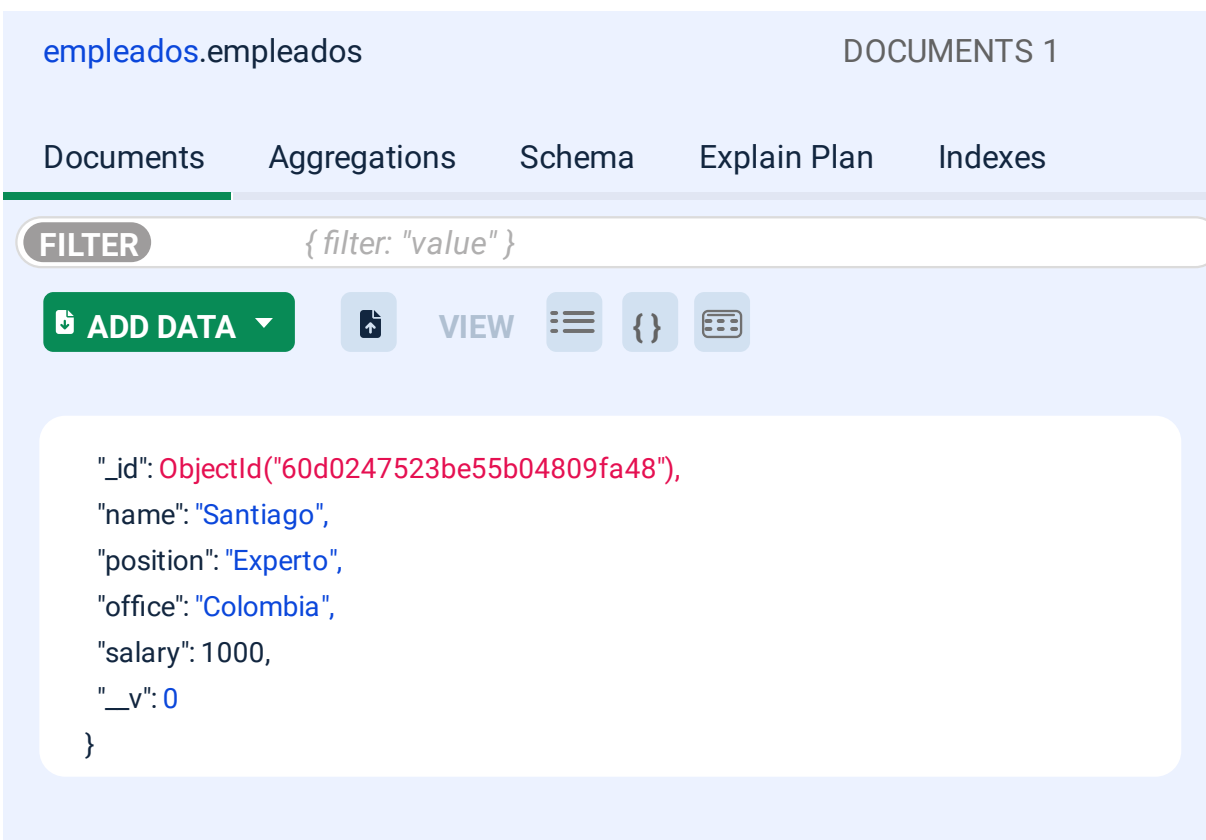
En la siguiente figura se presenta la imagen que representa la interfaz gráfica de MongoDB y el resultado de verificación de la creación de la base de datos.

Figura 5. Panel MongoDB



La siguiente figura muestra el resultado de la verificación de un registro de empleado almacenado en MongoDB.

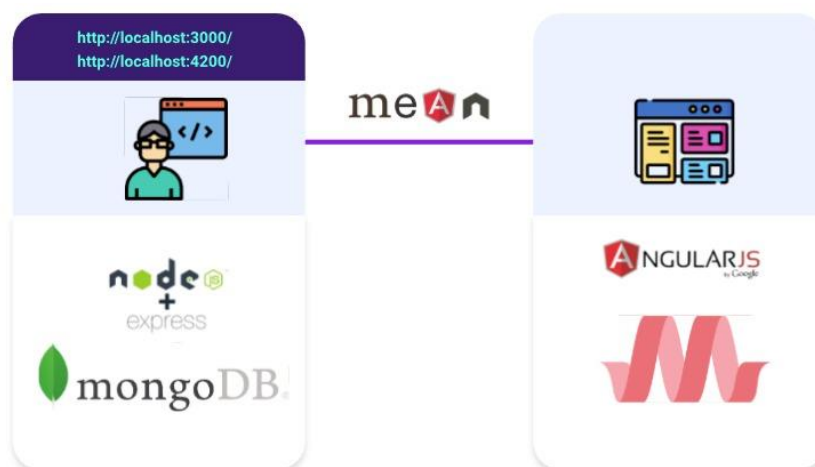
Figura 6. Registro de empleado almacenado en MongoDB



Como es evidente, la API REST funciona correctamente. Se ha creado la base de datos empleado y se tiene almacenado el primer registro del empleado Santiago. De igual forma, se confirma que los demás métodos HTTP, como GET, PUT o DELETE, también funcionan. Por lo tanto, se realiza la invitación a probar la API con Postman.

Hasta este punto se ha realizado el desarrollo del back-end, por lo que se tiene la base de datos lista para proceder con la construcción del front-end, que será el que interactuará con el servidor. A continuación, se presenta una figura que ilustra la estructura general del proyecto.

Figura 7. Resumen estructura del proyecto web



2. Construcción del front-end de la aplicación web

Para esta sección se plantea generar una sola vista, que corresponde al formulario de registro de empleados. Por esta razón, no es necesario generar sketch o maquetación para evaluar la navegación entre distintos componentes. No obstante, se emplean buenas prácticas de diseño amparadas en las técnicas y tecnologías como Materialize, que dará alcance a varias reglas de estilo para los sistemas interactivos de software.

plaintext

```
PS D:\gestion-empleados> npm install -g @angular/cli
```

```
npm WARN deprecated request@2.88.2: request has been deprecated, see  
https://github.com/request/request/issues/3142
```

```
npm WARN deprecated har-validator@5.1.5: this library is no longer supported
```

```
npm WARN deprecated uuid@3.4.0: Please upgrade to version 7 or higher. See  
https://v8.dev/blog/math-random for details.
```

2.1. Instalación y configuración de AngularJS

A continuación, se presenta la estructura de directorios del proyecto Angular creado con el comando `ng new frontend`:

GESTION-EMPLEADOS

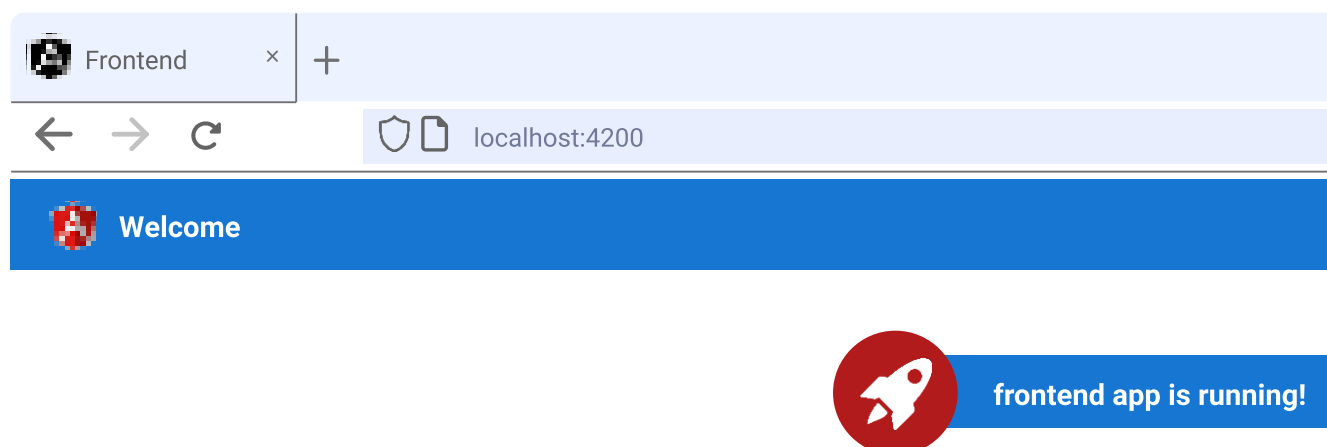
├── backend

├── frontend

```
| ├── node_modules
| ├── src
| ├── .browserslistrc
| ├── .editorconfig
| ├── .gitignore
| ├── angular.json
| ├── karma.conf.js
| ├── package-lock.json
| ├── package.json
| ├── README.md
| ├── tsconfig.app.json
| ├── tsconfig.json
| └── tsconfig.spec.json
```

A continuación, se debe ingresar a la carpeta del front-end e iniciar el servidor digitando en la terminal del VS Code la siguiente línea de comandos: `ng serve`. Es importante recordar que para ingresar por la terminal a la carpeta de front-end se hace con el comando `cd`, es decir, `cd Frontend/`. Una vez dentro de la carpeta front-end se ejecuta el comando `ng serve`, lo que inicia un servidor de desarrollo para Angular. En ese sentido, el servidor se inicializa en el puerto 4200 (<http://localhost:4200/>), como se representa en la siguiente figura:

Figura 8. Estado servidor Front-end



2.2. Generación de código con ng

Después de iniciar el servidor Angular, se revisa la estructura de directorios de este framework. Aunque hay muchos archivos, es necesario concentrarse en la carpeta ubicada en la ruta frontend/src/app. En la carpeta app se realizará todo el código necesario para el proyecto.

Adicionalmente, para dar coherencia al proyecto, dentro de la carpeta app se crean tres carpetas más: models, services y components, para tener bien definido el proyecto.

GESTION-EMPLEADOS

└─ backend

└─ frontend

| └─ node_modules

| └─ src

|| └─ app

```
| | | └─ components
| | | └─ models
| | | └─ services
| | | └─ app-routing.module.ts
| | | └─ app.component.css
| | | └─ app.component.html
| | | └─ app.component.spec.ts
| | | └─ app.component.ts
| | | └─ app.module.ts
```

Haciendo uso de una de las características del framework, se genera el código ingresando a las carpetas que se acaban de crear y en cada una se ejecutan las siguientes líneas de código respectivamente:

- **Generar componente empleados**

Dentro de la carpeta components se genera un componente llamado empleados con Angular CLI: `ng g c empleados`.

- **Crear clase empleado**

Dentro de la carpeta models se crea la clase empleado con: `ng g class empleado`.

- **Crear servicio empleado**

Dentro de la carpeta services se crea un servicio llamado empleado con la siguiente línea: ng g s empleado.

Para mayor precisión con relación a las instrucciones ejecutadas, a continuación se presenta la evidencia de las operaciones realizadas:

```
PS D:\gestion-empleados\frontend> cd .\src\app\components\
```

```
PS D:\gestion-empleados\frontend\src\app\components> ng g c empleados
```

```
CREATE src/app/components/empleados/empleados.component.html (24 bytes)
```

```
CREATE src/app/components/empleados/empleados.component.spec.ts (647  
bytes)
```

```
CREATE src/app/components/empleados/empleados.component.ts (287 bytes)
```

```
CREATE src/app/components/empleados/empleados.component.css (0 bytes)
```

```
UPDATE src/app/app.module.ts (498 bytes)
```

```
empleados.component.css
```

```
empleados.component.html
```

```
empleados.component.spec.ts
```

```
empleados.component.ts
```

2.3. Incorporación de Materialize al proyecto web

Se incorpora Materialize al proyecto mediante la inclusión de los CDN (red de distribución de contenidos) de hojas de estilos, JavaScript e íconos en el documento index.html del Front-end, por lo que la ruta al archivo es FrontEnd/src/index.html

```
html

<!doctype html>

<html lang="en">

<head>

  <meta charset="utf-8">

  <title>FrontEnd</title>

  <base href="/">

  <meta name="viewport" content="width=device-width, initial-scale=1">

  <link rel="icon" type="image/x-icon" href="favicon.ico">

  <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/css/materialize.min.css"
>

  <link href="https://fonts.googleapis.com/icon?family=Material+Icons"
rel="stylesheet">

</head>

<body>
```

```
<app-root></app-root>

<script
src="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/js/materialize.min.js"></s
cript>

</body>

</html>
```

2.4. Configuración de app.component.html

Ahora se inicia con la modificación del archivo app.component.html, dado que en este archivo se llaman los componentes de la aplicación que, para el caso de estudio propuesto, se denomina empleados.

```
html

<nav class="nav-wrapper blue">

  <div class="container">

    <a href="/" class="brand-logo">GESTIÓN DE EMPLEADOS</a>

  </div>

</nav>

<div class="container p-4">

  <app-empleados></app-empleados>

</div>
```


El código se evidencia que se adiciona la etiqueta <app-empleados>, indicando que se hará uso de ese componente.

2.5. Configuración de los módulos `app.module.ts`

A continuación, se configura uno de los archivos más importantes del proyecto, el cual se encarga de cargar los módulos de la aplicación. Es necesario entender la naturaleza de la aplicación que se está desarrollando, ya que esto permitirá definir qué módulos son necesarios para el óptimo funcionamiento de la aplicación.

En este caso de estudio, la aplicación permitirá la adición de empleados utilizando un API REST, por lo que es evidente el uso de un formulario en la interfaz, además de la comunicación que debe tener el front-end con el servidor.

typescript

```
import { NgModule } from '@angular/core';

import { BrowserModule } from '@angular/platform-browser';

import { FormsModule } from '@angular/forms';

import { HttpClientModule } from '@angular/common/http';

import { AppRoutingModule } from './app-routing.module';

import { AppComponent } from './app.component';

import { EmpleadosComponent } from
'./components/empleados/empleados.component';
```

```
@NgModule({
```

```
declarations: [  
  
  AppComponent,  
  
  EmpleadosComponent  
  
],  
  
imports: [  
  
  BrowserModule,  
  
  FormsModule,  
  
  AppRoutingModule,  
  
  HttpClientModule  
  
],  
  
providers: [],  
  
bootstrap: [AppComponent]  
  
})  
  
export class AppModule { }
```

2.6. Configuración del modelo empleado.ts

Una vez realizada la configuración en los módulos de la aplicación, se inicia la construcción del modelo de la aplicación. La codificación se realiza dentro del archivo `empleado.ts`, cuya ruta es `models/empleado.ts`.

Dentro de este archivo se define la estructura de datos que está asociada a lo que se describió en el back-end.

typescript

```
export class Empleado {  
  
    constructor(_id = "", name = "", position = "", office = "", salary = 0) {  
  
        this._id = _id;  
  
        this.name = name;  
  
        this.position = position;  
  
        this.office = office;  
  
        this.salary = salary;  
  
    }  
  
    _id: string; // Sub guión id porque los datos van a venir de MongoDB  
  
    name: string;  
  
    position: string;  
  
    office: string;  
  
    salary: number;  
  
}
```

En el código anterior, se definen los parámetros que serán utilizados para el envío de datos hacia el servidor; por lo tanto, se comienza con la inicialización de las variables a través de un constructor y con la declaración del tipo de datos de cada una.

2.7. Creación del servicio empleado.service.ts

El objetivo general de esta parte del proyecto es agregar los datos ya definidos a la REST API. Para ello, se enfoca en generar el servicio necesario, desarrollando el archivo empleado.service.ts. Este servicio permite definir los métodos reutilizables en cualquier parte de la aplicación. En este caso, se trata de funciones que gestionan los datos, como agregar, eliminar y editar, entre otros.

A continuación, se presenta el código del archivo empleado.service.ts.

typescript

```
import { Injectable } from '@angular/core';

import { HttpClient } from '@angular/common/http';

import { Empleado } from '../models/empleado';

@Injectable({
  providedIn: 'root'
})

export class EmpleadoService {

  selectedEmpleado: Empleado;
```

```
empleados: Empleado[];

readonly URL_API = 'http://localhost:3000/api/empleados';

constructor(private http: HttpClient) {

    this.selectedEmpleado = new Empleado();

    this.empleados = [];

}

getEmpleados() {

    return this.http.get(this.URL_API);

}

postEmpleado(Empleado: Empleado) {

    return this.http.post(this.URL_API, Empleado);

}

putEmpleado(Empleado: Empleado) {

    return this.http.put(this.URL_API + `/${Empleado._id}`, Empleado);

}

deleteEmpleado(_id: string) {

    // Solo se necesita el id, no todo lo del empleado

    return this.http.delete(this.URL_API + `/${_id}`); // utilizamos el método delete

}
```

```
}
```

En el código anterior se resalta la importancia de haber definido primeramente el módulo `empleado.ts`, ya que esto establece una estructura clara de datos. Por esta razón, se importa el modelo y se asegura la comunicación con el servidor a través de la modificación del archivo `app.modules.ts` con la incorporación del módulo `HttpClient`.

Además, se establece como atributo de solo lectura la URL de la API a la cual se conectará el cliente, en este caso, la URL del servidor ya configurada en la primera sección de este componente es <http://localhost:3000/empleados>.

A continuación, se procede con la definición de los métodos para la gestión de los datos de empleado, generando así cuatro métodos: `getEmpleados`, `postEmpleados`, `putEmpleado` y `deleteEmpleado`. Para el caso de estudio, se analiza el método `postEmpleado`, que recibe como parámetro un objeto de tipo `Empleado`. El retorno es la adición de un nuevo empleado (se envían todos los datos) a través del método HTTP POST:

```
return this.http.post(this.URL_API, Empleado);
```

Es fundamental asegurarse de que en cada método se envíe la URL del servidor y los datos necesarios en relación con el método correspondiente.

2.8. Desarrollo del componente `empleados.component.ts`

Definidos los servicios, es necesario abordar la lógica del componente. A continuación, se enfoca en el archivo `empleados.component.ts`. El código del archivo se presenta a continuación:

```
import { Component, OnInit } from '@angular/core';
```

```
import { EmpleadoService } from '../services/empleado.service';

import { NgForm } from '@angular/forms';

import { Empleado } from '../models/empleado';

declare var M: any;

@Component({
  selector: 'app-empleados',
  templateUrl: './empleados.component.html',
  styleUrls: ['./empleados.component.css'],
  providers: [EmpleadoService]
})

export class EmpleadosComponent implements OnInit {

  constructor(public empleadoService: EmpleadoService) { }

  ngOnInit(): void {

  }
}
```

```
agregarEmpleado(form?: NgForm) {  
  
    this.empleadoService.postEmpleado(form?.value)  
  
    .subscribe(res => {  
  
        this.resetForm(form);  
  
        M.toast({ html: 'Guardado satisfactoriamente' });  
  
    });  
  
}  
  
  
resetForm(form?: NgForm) {  
  
    // Limpiar el formulario, recibe un formulario como parámetro  
  
    if (form) {  
  
        form.reset();  
  
        this.empleadoService.selectedEmpleado = new Empleado();  
  
    }  
  
}  
  
}
```

En el código anterior se destaca la importancia de haber creado y configurado previamente los módulos y archivos para la construcción de este componente. Para su funcionamiento adecuado, es necesario contar con EmpleadoService, el modelo Empleado y el módulo ya definido como NgForm.

Una vez se importan estos módulos, se enfocan los esfuerzos en los métodos. Para el caso de estudio, se destaca el método de agregar empleado. Este método toma como parámetro la existencia de datos del formulario y, si es verdadero, llama al método del servicio empleado (previamente construido) `PostEmpleado`, entregando los valores para que sean enviados al servidor y almacenados en la base de datos de MongoDB.

Además, como ejemplo de este componente, se añade un plus: en cada adición de datos en el formulario, este se puede limpiar automáticamente mediante la función `resetForm`, que resetea o limpia los campos del formulario utilizando la función intrínseca `reset()`.

Finalmente, para mejorar la interfaz, se agrega un complemento de Materialize llamado `Toast`, que no es más que una notificación que se utiliza para indicar al usuario que el empleado se ha guardado correctamente.

2.9. Desarrollo de la interfaz HTML `empleados.component.html`

Después de definir la lógica del componente empleado, lo único que falta es la construcción del formulario. Por lo tanto, se enfoca ahora en el desarrollo del archivo `empleados.component.html`, que se presenta a continuación:

```
<div class="container">

  <div class="row">

    <div class="col s5">

      <div class="card">

        <div class="card-content">
```

```
<form #empleadoForm="ngForm"
(ngSubmit)="agregarEmpleado(empleadoForm)">

  <div class="row">

    <div class="input-field col s12">

      <input type="text" name="name" #name="ngModel"
[(ngModel)]= "empleadoService.selectedEmpleado.name" placeholder="Ingrese su
nombre">

    </div>

    <div class="input-field col s12">

      <input type="text" name="position" #position="ngModel"
[(ngModel)]= "empleadoService.selectedEmpleado.position" placeholder="Ingrese su
cargo">

    </div>

    <div class="input-field col s12">

      <input type="text" name="office" #office="ngModel"
[(ngModel)]= "empleadoService.selectedEmpleado.office" placeholder="Ingrese su
lugar de trabajo">

    </div>

    <div class="input-field col s12">
```

```
        <input type="text" name="salary" #salary="ngModel"
[(ngModel)]= "empleadoService.selectedEmpleado.salary" placeholder="Ingrese su
salario">

    </div>

    <div class="card-action">

        <div class="input-field col s12">

            <button class="btn right"
(click)="resetForm(empleadoForm)">Limpiar</button>

            <button class="btn">Guardar</button>

        </div>

    </div>

</div>

</div>

</form>

</div>

</div>

</div>

<div class="col s7">

</div>

</div>

</div>
```

En el código anterior se presenta la estructura base de un formulario en HTML, el cual se adhiere a clases y reglas provistas por Materialize para mejorar la apariencia de la interfaz. Además, se muestran cuatro inputs en los cuales el usuario debe ingresar datos, y al final se presentan dos botones: uno para enviar el formulario y otro para limpiarlo. A continuación, se destaca una línea de código específica:

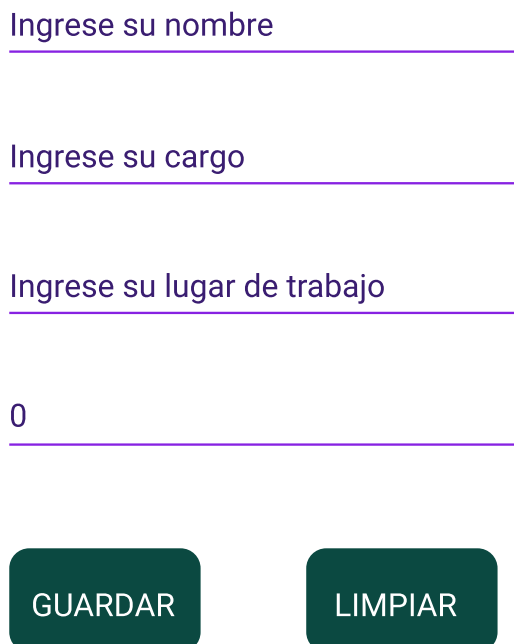
```
<form #empleadoForm="ngForm" (ngSubmit)="agregarEmpleado(empleadoForm)">
```

Esta línea define que todos los campos del formulario serán almacenados en una sola variable (empleadoForm), y estos campos serán procesados por el método agregarEmpleado, método que fue definido anteriormente en el componente de empleado.

2.10. Probando la aplicación completa.

Proyecto web terminado Finalizada la codificación de cada uno de los archivos del front-end, es necesario revisar cómo ha quedado la aplicación. Sin embargo, este paso no se realiza solo al final, sino que, debido a la concisión de este componente, se presenta en esta instancia. Como buena práctica, en cada bloque de desarrollo o línea se debe revisar el resultado obtenido, es decir, trabajar de manera paralela. Una muestra de la interfaz de la aplicación se ilustra en la siguiente figura:

Figura 9. Registro empleados



Formulario de registro de empleados con cuatro campos de texto y dos botones.

Ingrese su nombre

Ingrese su cargo

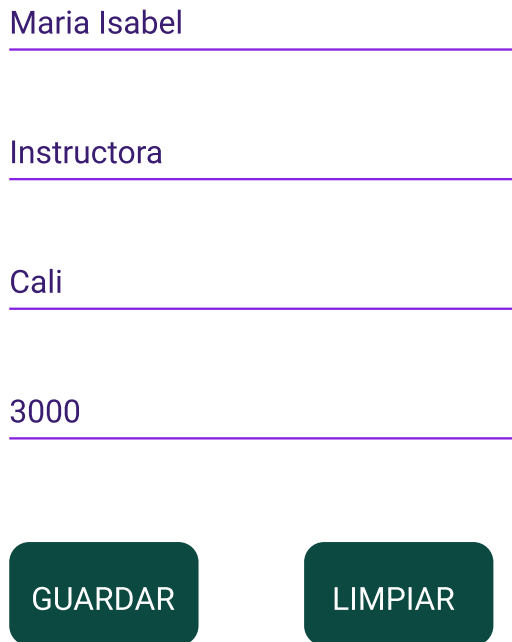
Ingrese su lugar de trabajo

0

GUARDAR LIMPIAR

En la muestra anterior de la interfaz se ilustra un formulario en el cual se solicitan los datos al usuario para el registro de un empleado. Es importante notar el uso de los placeholders en cada caja de texto, cuya función es guiar o indicar qué tipo de datos se deben registrar. De esta manera, la interfaz quedaría como se muestra en la siguiente figura:

Figura 10. Registro empleado



Maria Isabel

Instructora

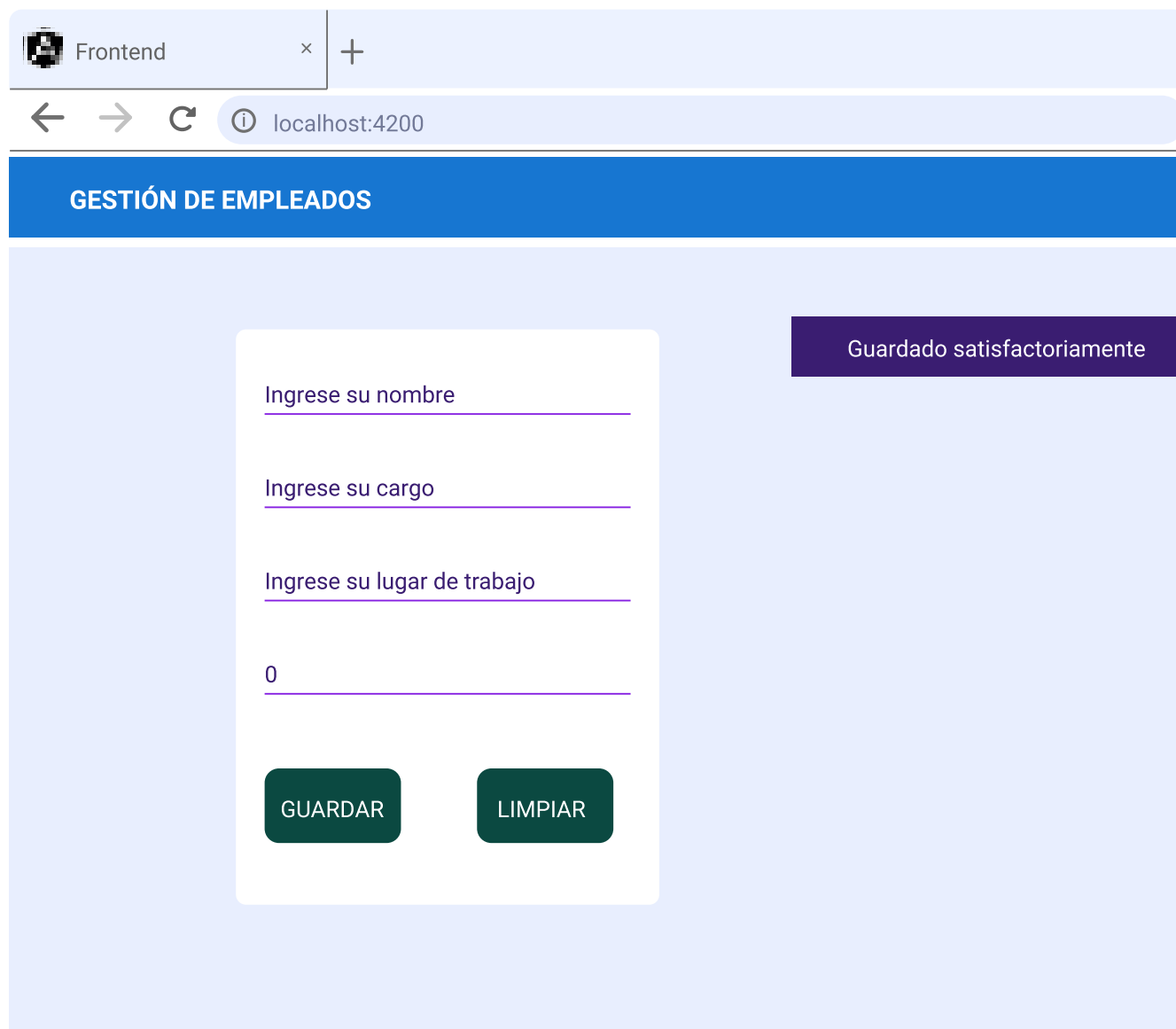
Cali

3000

GUARDAR LIMPIAR

Al momento de dar clic en guardar, se almacena la información y se muestra la notificación de Materialize Toast, indicando que el usuario se ha guardado exitosamente. Además, se limpia el formulario mediante la función `reset()`, tal como se aprecia en la figura.

Figura 11. Notificación Toast de Materialize



The screenshot shows a web browser window with a single tab titled 'Frontend'. The address bar displays 'localhost:4200'. The page has a blue header with the text 'GESTIÓN DE EMPLEADOS'. In the center, there is a white form with four input fields: 'Ingrese su nombre', 'Ingrese su cargo', 'Ingrese su lugar de trabajo', and a field containing the number '0'. Below these fields are two dark green buttons labeled 'GUARDAR' and 'LIMPIAR'. In the top right corner of the page, a dark purple toast notification box displays the text 'Guardado satisfactoriamente'.

Para verificar que los datos incorporados en el formulario se hayan almacenado correctamente en la base de datos empleado de MongoDB, se utilizará el panel de control de MongoDB, como se aprecia en los siguientes códigos:

```
json
```

```
{
```

```
"_id": ObjectId("60d2aeaa21a176987f0157485"),  
  
"name": "Julian",  
  
"position": "Ingeniero",  
  
"office": "Cali",  
  
"salary": 5000,  
  
"__v": 0  
}  
  
{  
  
  "_id": ObjectId("60d2aeaa21a176987f0157487"),  
  
  "name": "Julian",  
  
  "position": "Ingeniero",  
  
  "office": "Cali",  
  
  "salary": 5000,  
  
  "__v": 0  
}  
  
{  
  
  "_id": ObjectId("60d2aeaa21a176987f0157489"),  
  
  "name": "Jaime",  
  
  "position": "Oficial",
```



```
"office": "Juzgados",  
  
"salary": 8000,  
  
"__v": 0  
}  
  
{  
  
  "_id": ObjectId("60d64392eadf31b1d99e64f4"),  
  
  "name": "María Isabel",  
  
  "position": "Instructora",  
  
  "office": "Cali",  
  
  "salary": 3000,  
  
  "__v": 0  
}
```

Evidencia de registro en MongoDB:

```
{  
  
  "_id": ObjectId("60d64392eadf31b1d99e64f4"),  
  
  "name": "María Isabel",  
  
  "position": "Instructora",  
  
  "office": "Cali",  
  
  "salary": 3000,
```

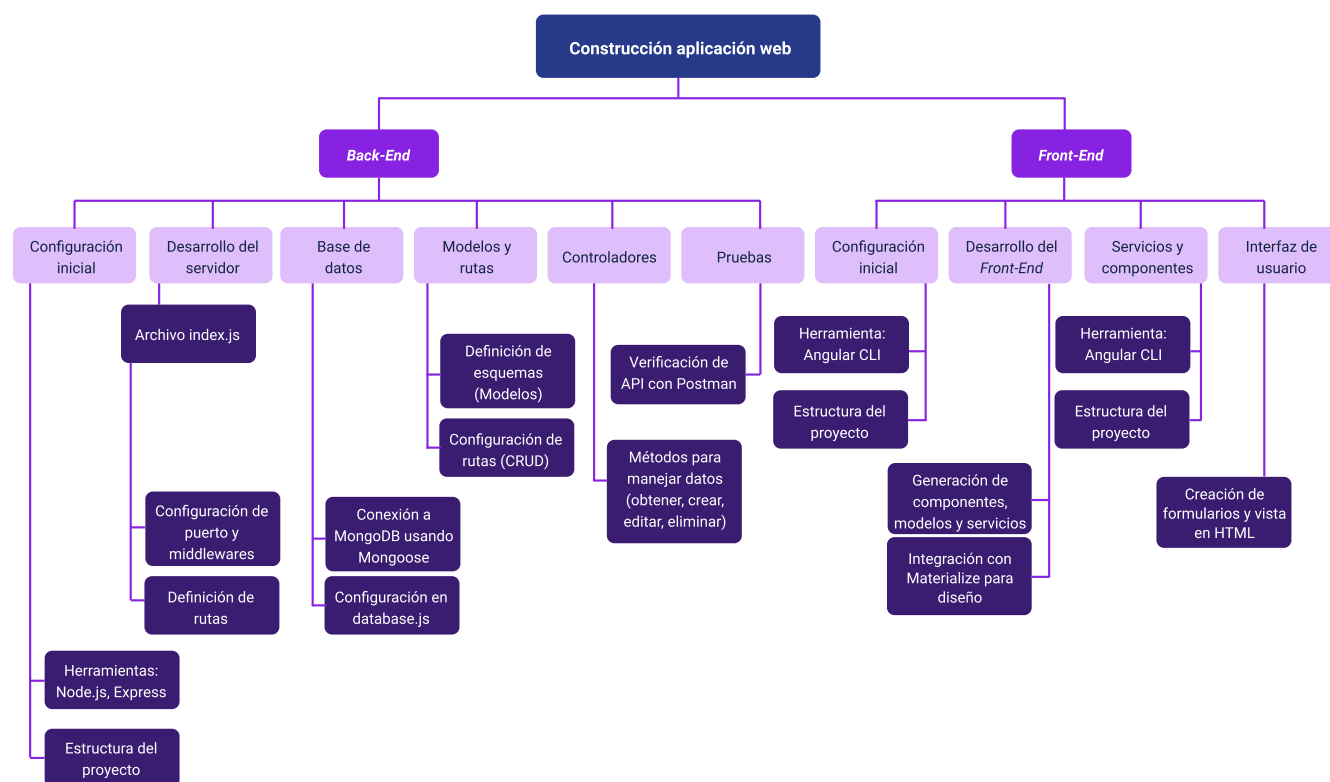
```
"__v": 0  
}
```

En los anteriores códigos se puede evidenciar que los datos se guardan correctamente en la base de datos, indicando que la aplicación desde el front-end funciona de manera idónea, así como la API REST del proyecto. Con esto se culmina este componente.

Se espera que haya sido de aprendizaje el uso de tecnologías tanto del lado del cliente como del lado del servidor. Este es uno de los ejercicios más completos y precisos que se pueden realizar en materia de esta disciplina del desarrollo web. En ese sentido, se exhorta a seguir estudiando más alternativas sobre este tipo de desarrollos que son muy demandados en la industria del software.

Síntesis

A continuación, se muestra un mapa conceptual con los elementos más importantes desarrollados en este componente.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Descripción Back-end y Fron-end	EDteam. (2020). ¿Qué es Backend y Front-end? (guía completa) [video]. YouTube.	Video	https://www.youtube.com/watch?v=50RbVujPPGs&feature=youtu.be
Instalación MongoDB	MongoDB. (2024). Install MongoDB Community Edition on Windows.	Instalador	https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-windows/
Instalación Node.js y Express	Coding, G. (2020). Cómo instalar Node.js en Windows 10 [video]. YouTube.	Video	https://www.youtube.com/watch?v=BgtB31gXkoA&feature=youtu.be
Guía de instalación Angular CLI	Angular. (2024). Installing Angular CLI.	Página web	https://angular.io/cli
Guía de instalación Postman	Limitless Minds. (2019). Instalación Postman y prueba de los métodos GET [video]. YouTube.	Video	https://www.youtube.com/watch?v=440yQGvINkk
Guía de Materialize	Materializecss. (2024). Getting Started - Materialize.	Página web	https://materializecss.com/getting-started.html
Guía extensión VS Code y MongoDB	Developer Soapbox. (2020). How to use Visual Studio Code as your MongoDB IDE [video]. YouTube.	Video	https://www.youtube.com/watch?v=wM7NJtQ0F6U
Proyecto web	GitLab. (2021). Gestion-empleados.	Software	https://gitlab.com/jonathanga/gestion-empleados

Glosario

Async: cuando se llama a una función `async`, se devuelve un elemento de tipo promesa o `promise`. Cuando la función `async` devuelve un valor, la promesa se resolverá con el valor devuelto. Si la función `async` genera una excepción o algún valor, la promesa se rechazará con el valor generado.

Await: una función `async` puede contener una expresión `await`, de hecho es una buena práctica. Para lo cual pausa la ejecución de la función asíncrona y espera la resolución de la promesa pasada y, a continuación, reanuda la ejecución de la función `async` y devuelve el valor resuelto.

CDN: una red de distribución de contenido. Es un tipo de sistema informático en red, el cual consiste en distribuir o repartir parte de la información en diferentes servidores. De esta forma el usuario recibe el contenido de la página del servidor más cercano.

Middleware: un `middleware` en Node.js es un bloque de código completo que se ejecuta entre la petición que hace el usuario (`request`) hasta que la petición llega al servidor.

Promise: el objeto `promise` (promesa) es usado para computaciones asíncronas. Así que una promesa representa un valor que puede estar disponible ahora o más adelante.

Referencias bibliográficas

Vortexbird. (2017). Nodemon. <https://vortexbird.com/nodemon/>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Santiago Muñoz de la Rosa	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluable Instruccionale	Centro de Servicios de Salud - Regional Antioquia
Andrés Felipe Herrera Roldán	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edwin Sneider Velandia Suárez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Jaime Hernán Tejada Llano	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Construir aplicativo según las características de la arquitectura Android

Breve descripción:

En este componente formativo se abordan los conceptos claves para el diseño y desarrollo de aplicaciones móviles nativas, junto con sus elementos, vistas, maquetación, entornos de desarrollo, lenguajes de programación.

Julio 2024

Tabla de contenido

Introducción	4
1. Estructura de un proyecto en Android Studio	5
Manifest	9
Java.....	10
Res.....	10
Gradle	13
2. Interfaz de usuario en Android.....	14
2.1. Layouts	16
LinearLayout.....	17
TableLayout	20
ConstraintLayout	24
2.2. Controles básicos.....	25
Botones	28
Texto e imágenes	29
Checkbox y Radiobutton.....	36
Controles de selección.....	40
ListView	43
3. Eventos Listeners	46

Crear el Button en el Layout	46
Declarar el tipo de view	48
Implementar el evento Listeners	49
4. Navegación (Intents)	51
Intent explícito.....	51
Intent implícito	52
5. Tareas asincrónicas	54
6. Persistencia de datos	56
6.1. Bases de datos local SQLite.....	56
6.2. Bases de datos en tiempo real.....	57
7. Multimedia	63
7.1. MediaPlayer	63
7.2. VideoView	65
Síntesis	68
Material complementario.....	69
Glosario	70
Referencias bibliográficas	71
Créditos.....	72

Introducción

Actualmente, los sistemas operativos para dispositivos móviles permiten la construcción de aplicaciones mediante el uso de un software llamado SDK. Tanto Android como iOS ofrecen sus respectivas herramientas para crear aplicaciones nativas. Las características de los dispositivos móviles, como GPS, cámara y navegación por internet, se aprovechan en el desarrollo de aplicaciones, permitiendo ideas interesantes y útiles para los usuarios.

El sistema operativo Android es compatible con la mayoría de los dispositivos y es, hoy en día, uno de los más utilizados en teléfonos móviles. El mercado de aplicaciones para Android está en constante crecimiento, y la presencia de empresas y sus aplicaciones en esta plataforma es prácticamente un requisito indispensable. Por lo tanto, el desarrollo de aplicaciones para este sistema puede convertirse en una excelente oportunidad laboral para muchos ingenieros.

Este componente formativo se enfoca en el desarrollo de aplicaciones nativas para la plataforma Android, utilizando su entorno de desarrollo oficial, Android Studio, y el lenguaje Java.

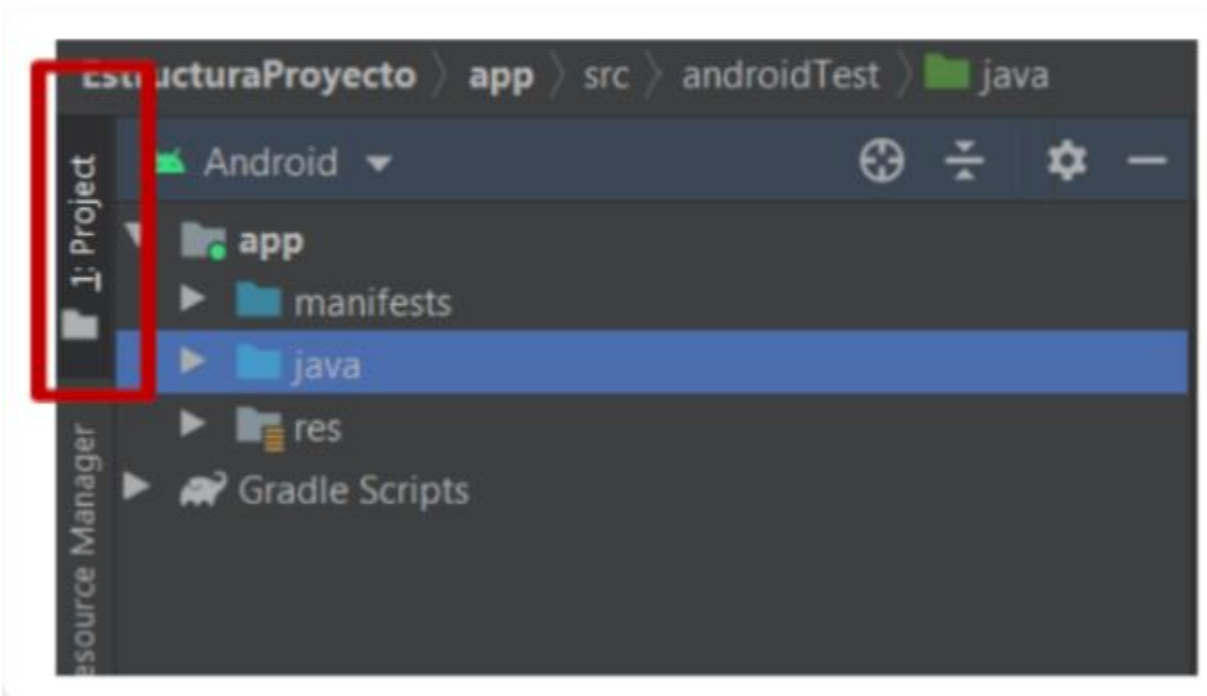
1. Estructura de un proyecto en Android Studio

Las actualizaciones en Android Studio avanzan rápidamente. Para este componente, los ejemplos se presentan para la versión 4.1.3.

En Android Studio, un proyecto contiene todos los elementos necesarios para el desarrollo de una aplicación: desde código fuente y recursos hasta código de prueba y configuraciones de compilación. Al iniciar un nuevo proyecto, Android Studio crea la estructura necesaria para todos los archivos y los organiza en la ventana Project, ubicada al lado izquierdo del IDE (accede a ella mediante View > Tool Windows > Project), o utilizando el acceso directo Alt + 1.

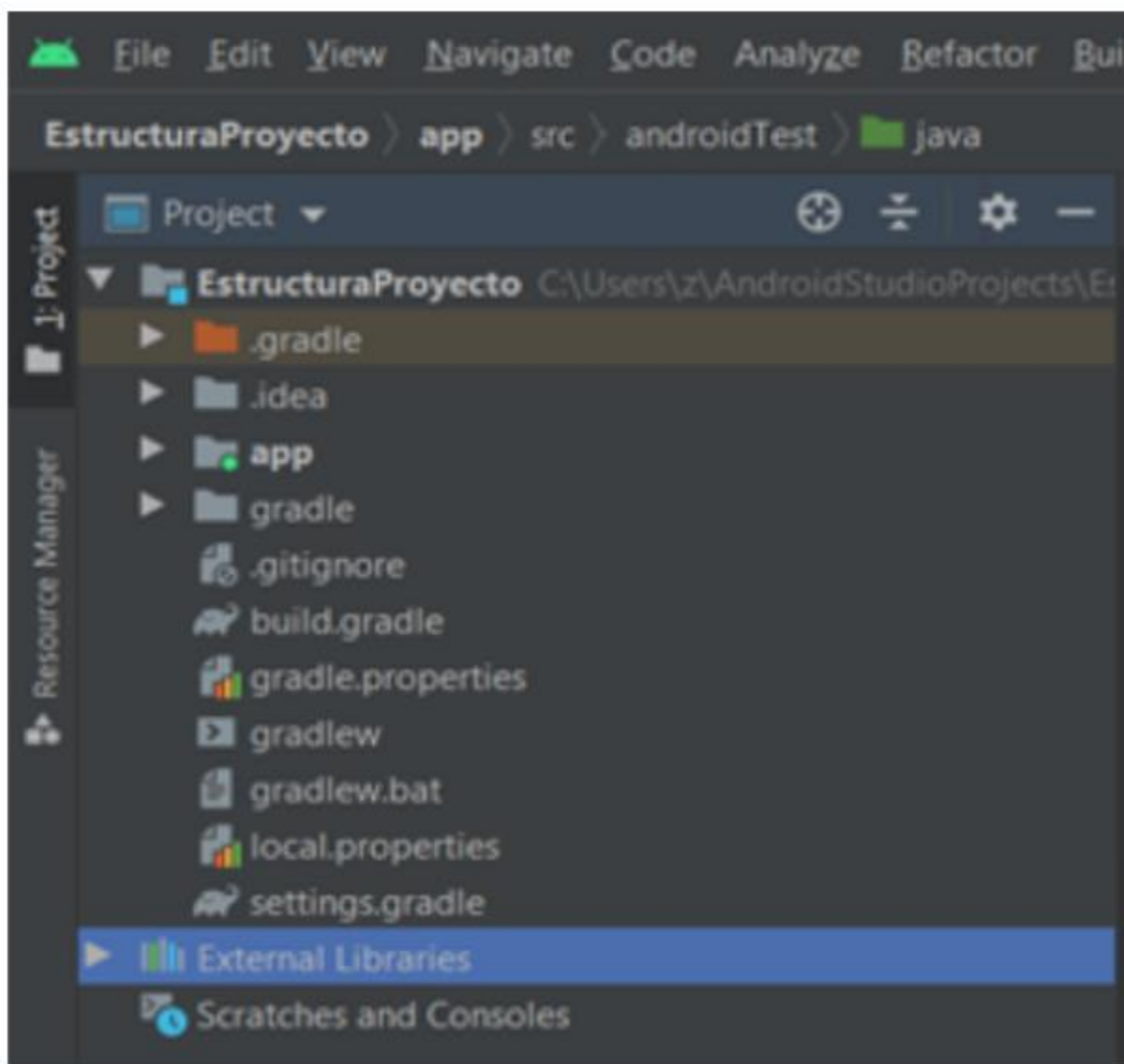
A continuación, se presenta la estructura:

Figura 1. Imagen estructura de proyecto en Android Studio



Ahora bien, las vistas de proyectos en Android, o la visualización completa de la estructura de archivos del proyecto, se realizan mediante la opción "Project" del menú desplegable en la parte superior de la ventana, como se ilustra a continuación.

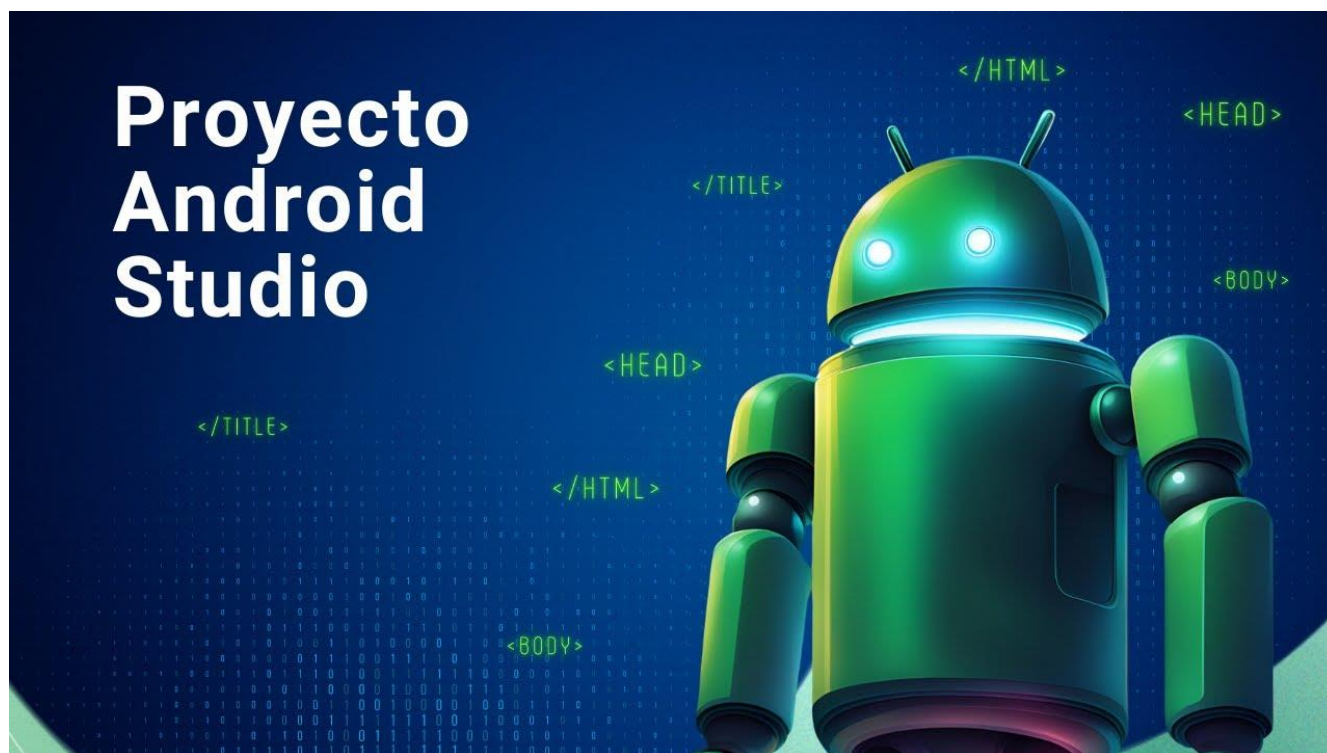
Figura 2. Vista estructura del proyecto



Los proyectos de Android Studio tienen unos componentes principales que se explican en el siguiente video, los cuales se distribuyen entre:

- Módulos
- Archivos de proyecto

Video 1. Proyecto Android Studio



[Enlace de reproducción del video](#)

Síntesis del video: Proyecto Android Studio

Estructura de un proyecto en Android Studio

Un proyecto en Android Studio está compuesto principalmente por módulos y por archivos del proyecto, en donde cada uno de estos cumple unas funciones principales, las cuales se describen brevemente a continuación.

Los Módulos: Contienen el grupo de archivos que permiten realizar la división del proyecto. Un proyecto puede contener uno o varios módulos. Estos se pueden trabajar de forma independiente para compilar, depurar o probar. El uso de los módulos se realiza principalmente para crear bibliotecas de código o conjuntos de código para diferentes dispositivos, como teléfonos o relojes inteligentes.

Para la creación de un módulo, lo hacemos a través del menú *File > New > New Module. Android ofrece diferentes tipos de módulos, como módulos de función o módulos de bibliotecas. Por ejemplo, seleccionaremos el módulo para relojes inteligentes, presionamos "Next", colocamos el nombre del módulo, presionamos "Next" nuevamente, seleccionamos el tipo de actividad y, por último, damos clic en "Finalizar". Así, se crea el módulo.

Archivos del proyecto: Dentro de cada uno de estos módulos, Android organiza los siguientes grupos principales de archivos:

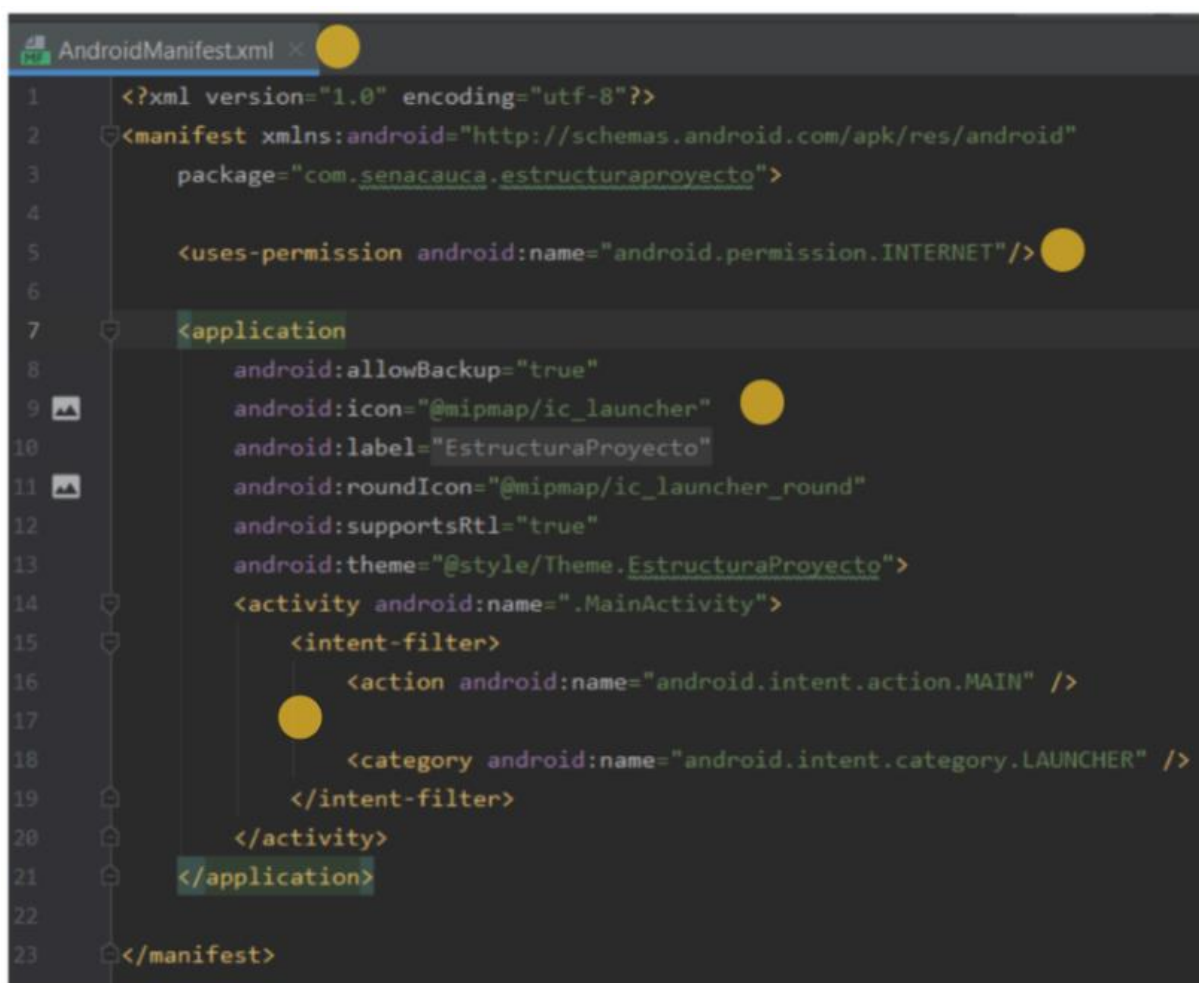
- Manifest: Contiene el archivo "AndroidManifest.xml", que se encarga de la configuración de íconos, etiquetas, temas y permisos de la aplicación.
- Java: Subdividido en directorios, contiene todos los archivos de código fuente Java.
- Res: En él se encuentran todos los recursos de la aplicación, como diseños XML, cadenas de texto, imágenes, entre otros, que se encuentran divididos en subdirectorios.

Manifest

En el manifest se encuentra la información esencial de una aplicación para las herramientas de desarrollo de Android, el sistema operativo Android y Google Play. El manifest es un archivo en lenguaje XML donde se definen aspectos principales de la aplicación, como su identificación (nombre, ícono, estilos), sus componentes (pantallas, servicios) y los permisos necesarios para su ejecución.

A continuación, se presentan algunos de los elementos más importantes que se deben declarar en el archivo manifest:

Figura 3. Archivo manifest



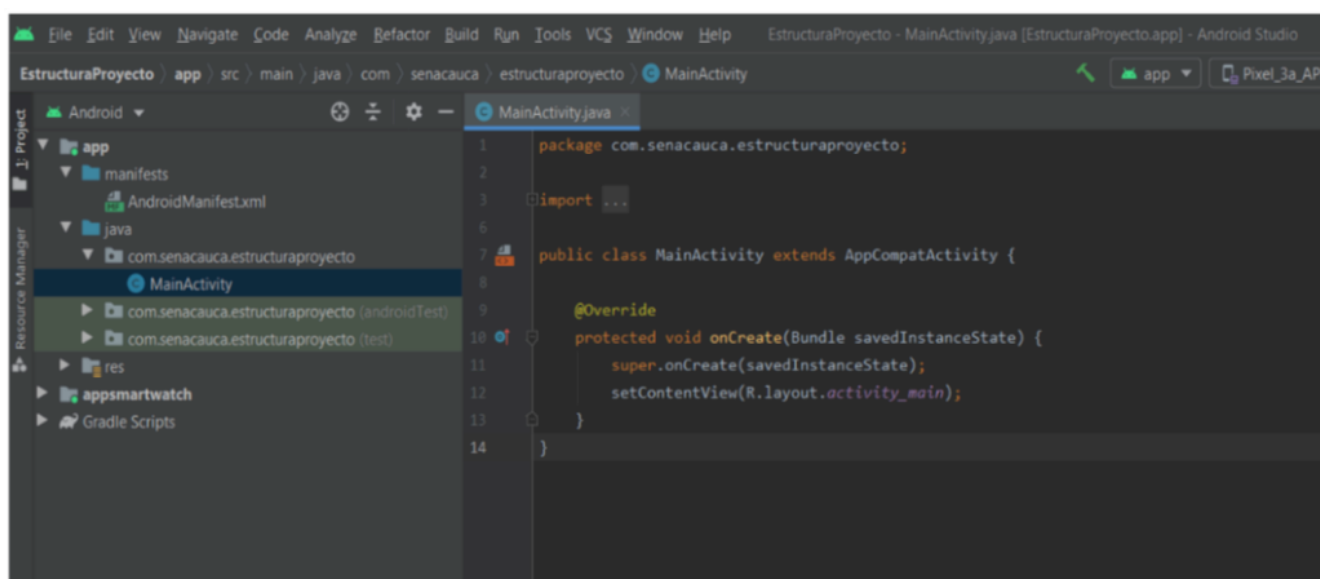
```
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3    package="com.senacauca.estructuraproyecto">
4
5    <uses-permission android:name="android.permission.INTERNET"/>
6
7    <application
8        android:allowBackup="true"
9        android:icon="@mipmap/ic_launcher"
10       android:label="EstructuraProyecto"
11       android:roundIcon="@mipmap/ic_launcher_round"
12       android:supportsRtl="true"
13       android:theme="@style/Theme.EstructuraProyecto">
14        <activity android:name=".MainActivity">
15            <intent-filter>
16                <action android:name="android.intent.action.MAIN" />
17
18                <category android:name="android.intent.category.LAUNCHER" />
19            </intent-filter>
20        </activity>
21    </application>
22
23 </manifest>
```


Java

La carpeta java contiene todo el código fuente de la aplicación, organizado en paquetes, clases, interfaces, POJOs (Plain Old Java Objects), entre otros.

Inicialmente, Android Studio crea el código básico de la pantalla principal de la aplicación (actividad o activity), que por defecto se llamará MainActivity, siempre bajo la estructura del paquete java definido durante la creación del proyecto.

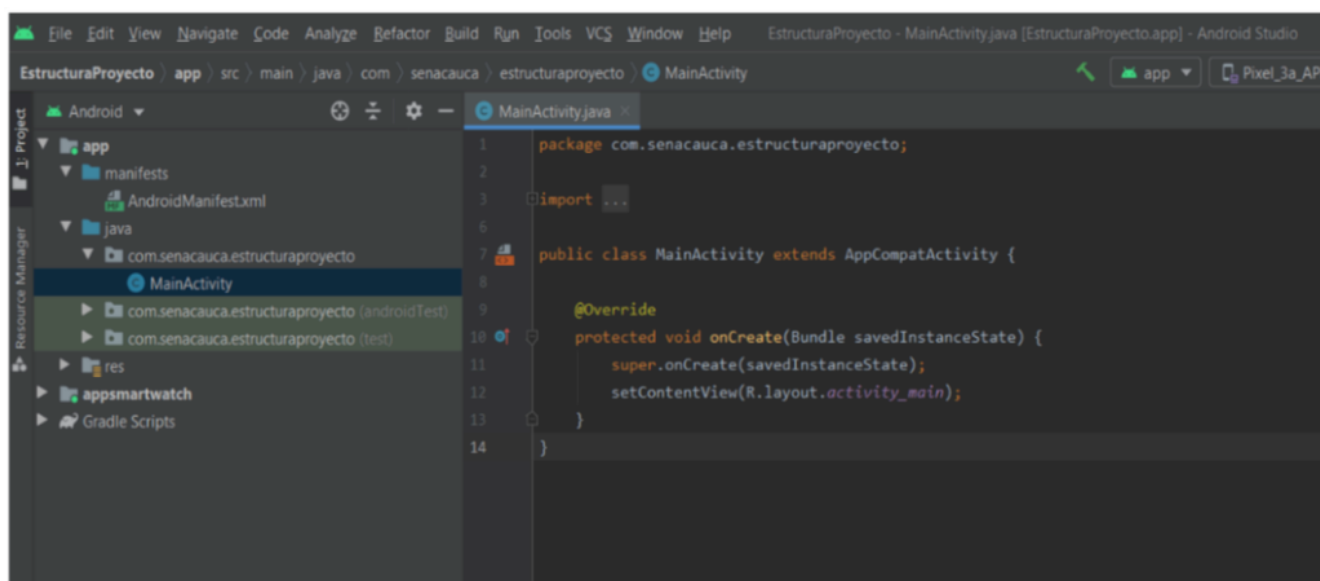
Figura 4. Clase MainActivity de la carpeta java



Res

En esta carpeta se encuentran todos los recursos del proyecto que no contienen código, como diseños XML, cadenas de texto de la interfaz de usuario y también imágenes de mapa de bits, organizados en subdirectorios.

Figura 5. Imagen estructura carpeta res



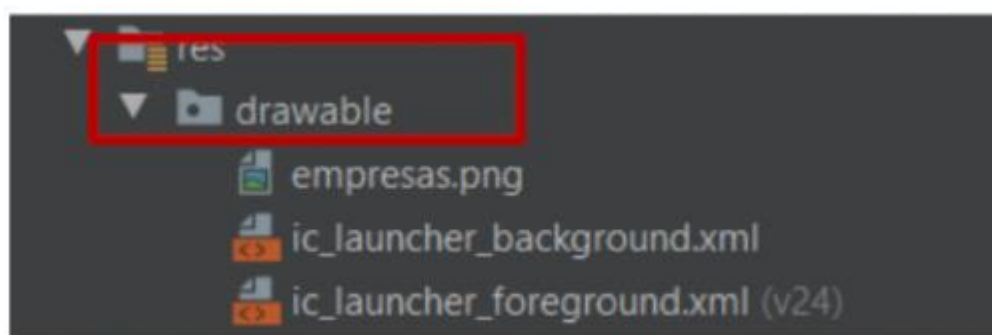
A continuación, se explican las subcarpetas:

- **Subcarpeta drawable**

/res/drawable: Contiene las imágenes que se van a usar en la aplicación.

Los formatos que soporta son (png, jpg, gif).

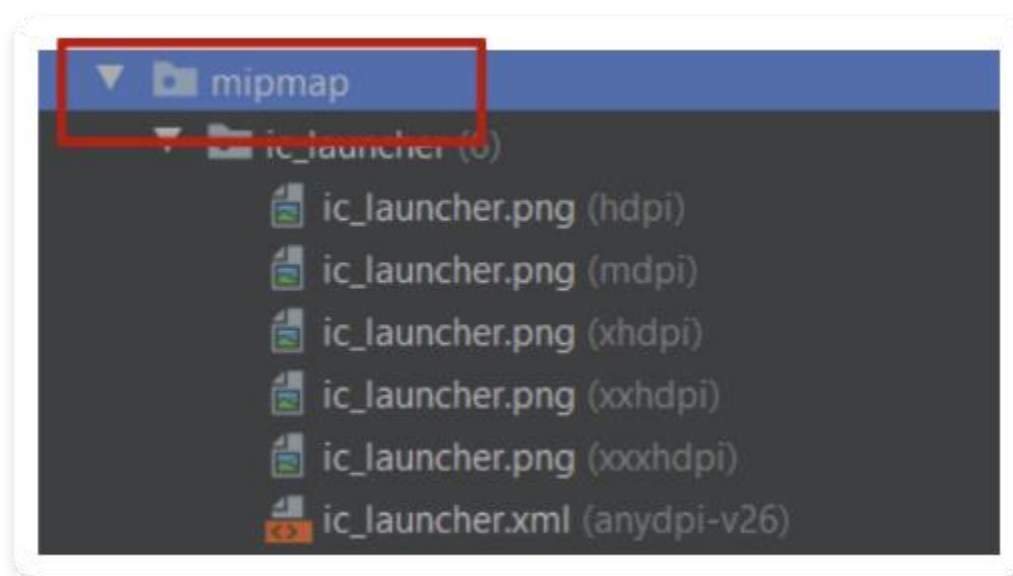
Figura 6. Explorador de archivos de Android Studio



- **Subcarpeta mipmap**

/res/mipmap: Contiene los íconos que se van a utilizar en la aplicación para las diferentes densidades de las pantallas existentes. Al igual que la carpeta drawable, se divide en subcarpetas dependiendo de la densidad de la pantalla.

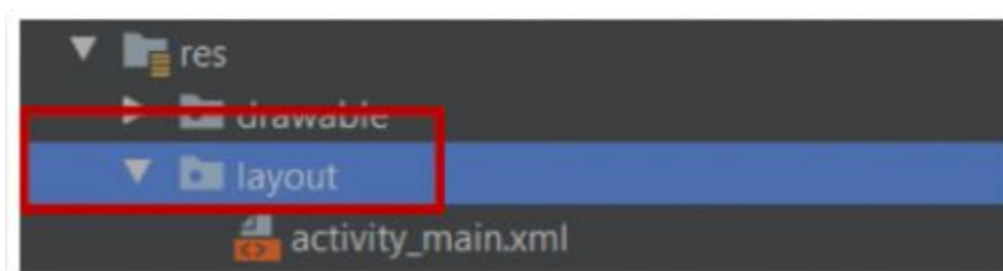
Figura 7. Subcarpeta mipmap



- **Subcarpeta layout**

/res/layout: En esta carpeta se define el diseño de la interfaz de usuario. El formato del archivo es en XML.

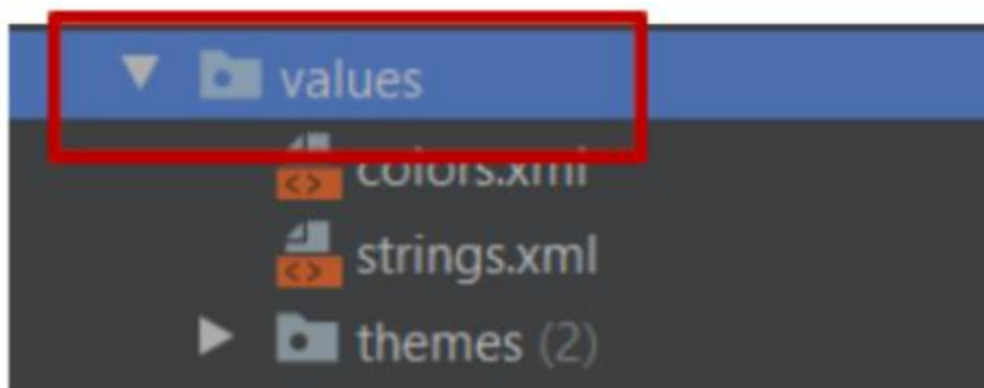
Figura 8. Subcarpeta layout



- **Subcarpeta values**

/res/values: Esta carpeta contiene archivos XML que contienen valores simples, como strings, temas y colores.

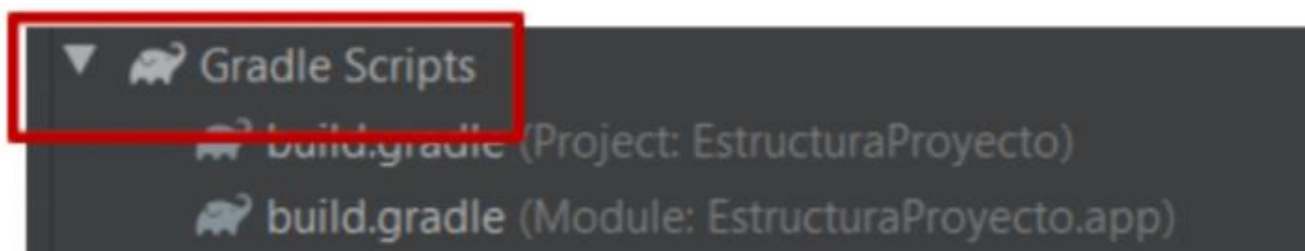
Figura 9. Subcarpeta values



Gradle

Gradle permite la compilación del proyecto. En este, es posible configurar la versión del SDK que Android utilizará para compilar, la versión mínima de Android que soportará la aplicación, referencias a las bibliotecas externas utilizadas, entre otros aspectos.

Figura 10. Gradle Scripts

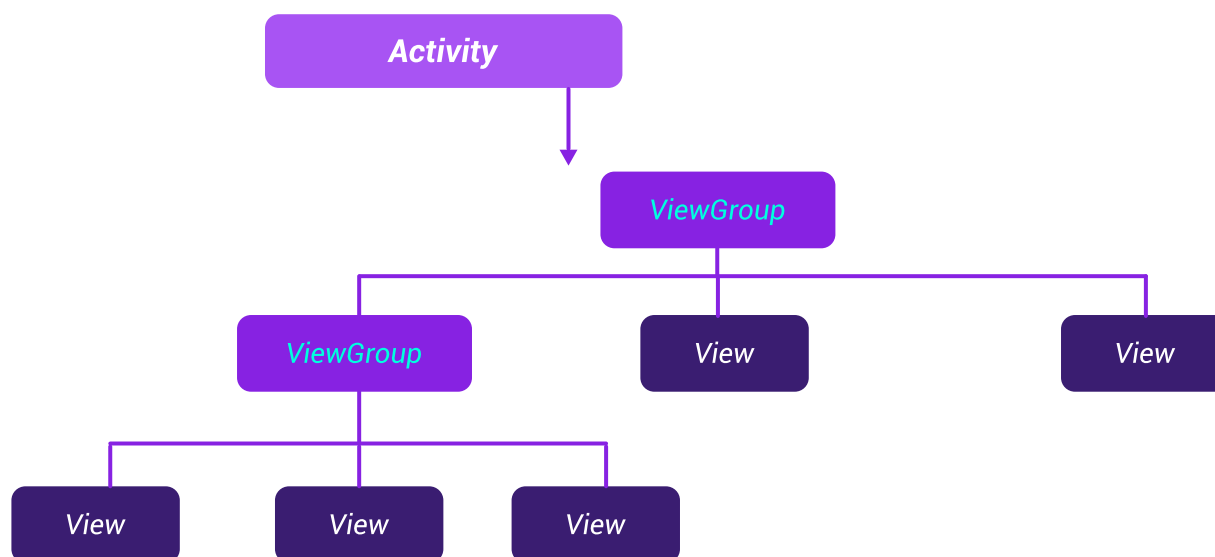


2. Interfaz de usuario en Android

La interfaz de usuario se define a través de la estructura de un diseño para la aplicación, como en el caso de una actividad en Android. La creación de los elementos del diseño se realiza utilizando una jerarquía de objetos View y ViewGroup. Los View son los elementos que el usuario puede interactuar con, como botones, textos e imágenes, mientras que un ViewGroup es un contenedor no visible que define la estructura del diseño de los View y otros objetos.

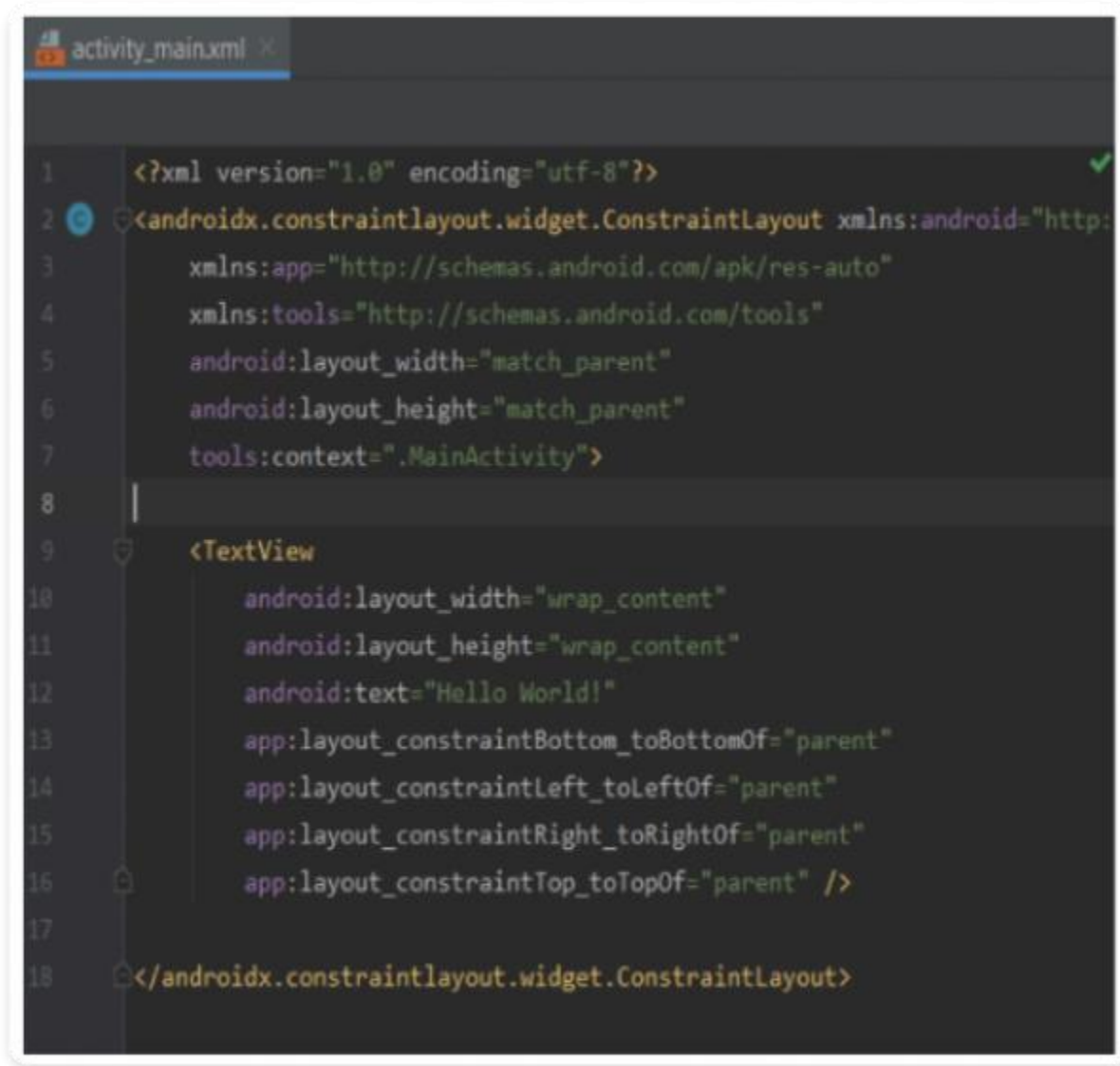
Los objetos ViewGroup se denominan Layout (diseños) y existen en varios tipos, dependiendo de la distribución y la posición de sus View. Ejemplos de estos son LinearLayout, RelativeLayout, entre otros.

Figura 11. Ilustración jerarquía de diseño



El diseño en Android se puede trabajar de dos maneras: en XML, o en el editor de diseño (mediante una interfaz de arrastrar y soltar).

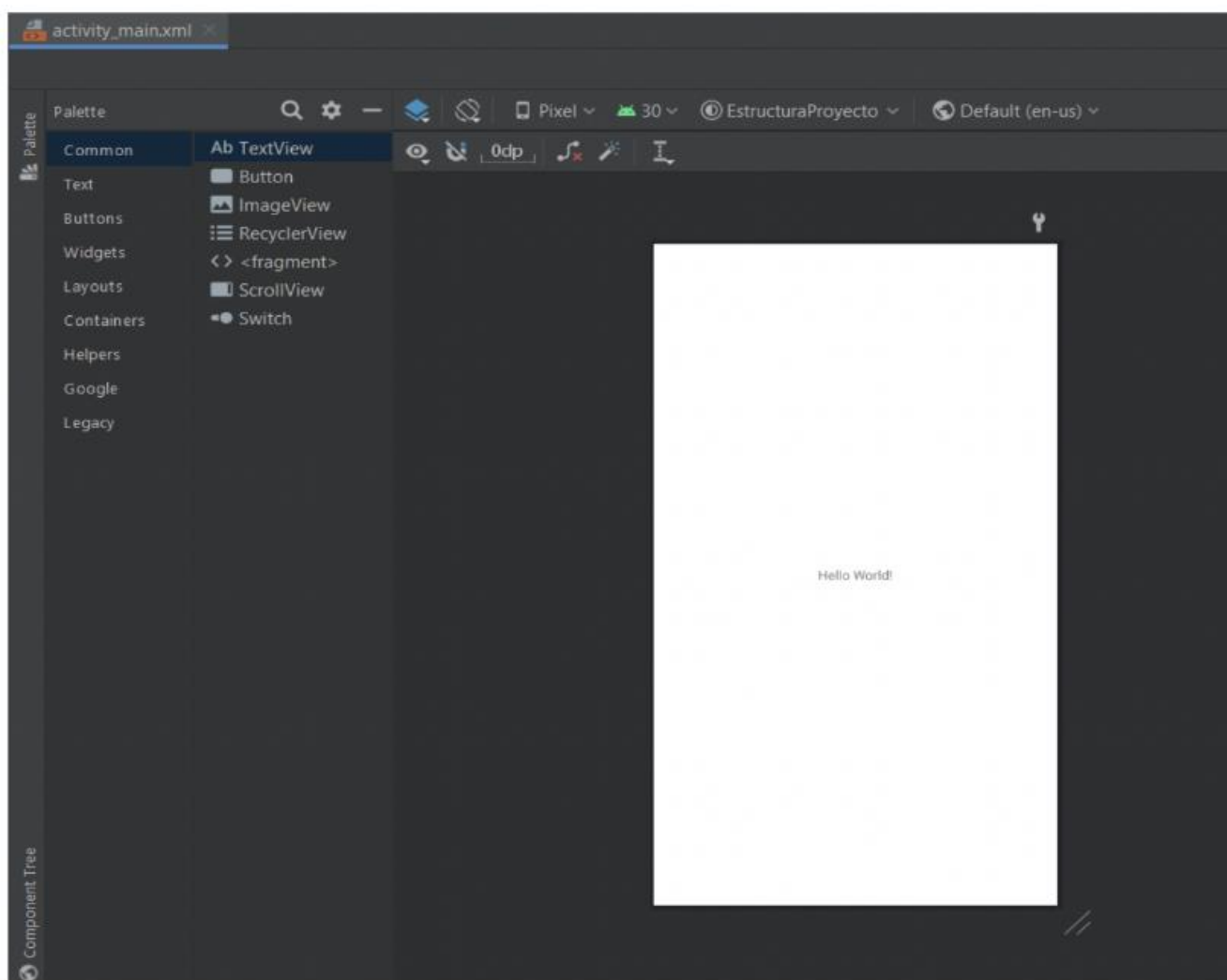
Figura 12. Vista diseño XML



The image shows a screenshot of an IDE with a file named 'activity_main.xml' open. The XML code is as follows:

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      tools:context=".MainActivity">
8
9      <TextView
10         android:layout_width="wrap_content"
11         android:layout_height="wrap_content"
12         android:text="Hello World!"
13         app:layout_constraintBottom_toBottomOf="parent"
14         app:layout_constraintLeft_toLeftOf="parent"
15         app:layout_constraintRight_toRightOf="parent"
16         app:layout_constraintTop_toTopOf="parent" />
17
18  </androidx.constraintlayout.widget.ConstraintLayout>
```

Figura 13. Vista diseño XML



2.1. Layouts

En Android Studio, los componentes visuales (como Button, EditText, TextView, etc.) se organizan mediante contenedores llamados Layout. Los Layouts son elementos no visuales que permiten controlar la organización, posición y dimensiones de los View que se crean dentro de ellos. Al igual que muchos otros componentes contenedores, son capaces de contener otros controles, es decir, se puede tener un contenedor dentro de otro contenedor.

Existen diferentes tipos de Layout, entre los más comunes están: LinearLayout, TableLayout y ConstraintLayout. A continuación, se profundiza en cada uno:

LinearLayout

Un LinearLayout dispone las vistas de la interfaz, una debajo de otra (orientación vertical) o una al lado de la otra (orientación horizontal). Sus propiedades principales son:

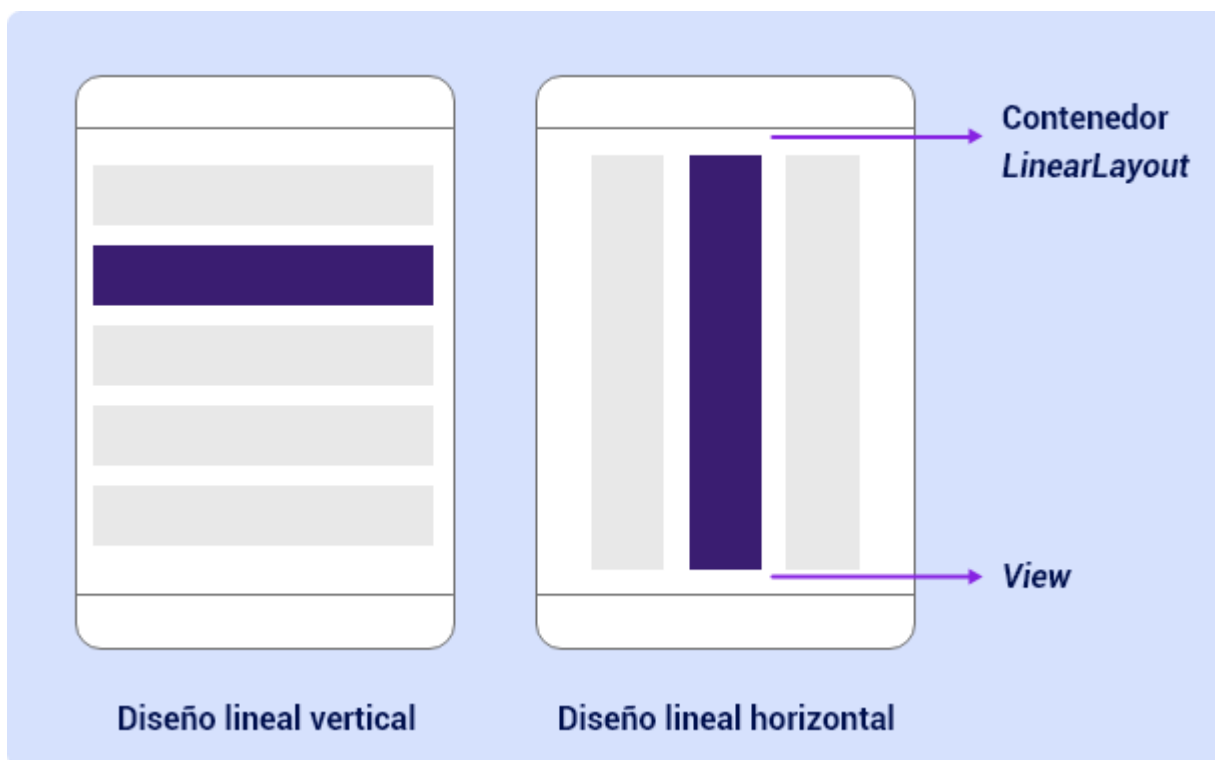
- a) Orientation: es la propiedad que define la disposición de sus vistas, ya sea horizontal o vertical.
- b) Layout_height: esta propiedad especifica el alto del ViewGroup (contenedor).
- c) Layout_width: esta propiedad especifica el ancho del ViewGroup.

Los valores que pueden tomar las propiedades height y width son:

- match_parent: para que ocupe el mismo tamaño de la pantalla.
- wrap_content: para que se adapte al tamaño de su contenido.
- Valores personalizados que se miden con dp (píxeles de densidad independiente).

Estas propiedades de height y width se aplican a todos los contenedores y vistas.

Figura 14. Diseño lineal vertical y diseño lineal horizontal en Android



```
<LinearLayout
```

```
xmlns:android="http://schemas.android.com/apk/res/android"
```

```
xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
xmlns:tools="http://schemas.android.com/tools"
```

```
android:orientation="vertical"
```

```
android:layout_width="match_parent"
```

```
android:layout_height="match_parent"
```

```
tools:context=".MainActivity">
```

```
<!-- Dentro del contenedor se agregan los view en este caso p Button -->
```

```
<Button  
  
    android:layout_width="match_parent"  
  
    android:layout_height="wrap_content"  
  
    android:layout_marginTop="45dp"  
  
    android:text="@string/boton1" />  
  
<Button  
  
    android:layout_width="match_parent"  
  
    android:layout_height="wrap_content"  
  
    android:text="@string/boton2" />  
  
</LinearLayout>  
  
<!-- Cierre del contenedor LinearLayout -->
```

El código XML para la creación de un layout de tipo `LinearLayout`: con orientación vertical, el cual contiene dos vistas de tipo `Button`.

- `<LinearLayout`: tipo de contenedor lineal.
- `orientation=vertical`: propiedad que define la ubicación de los views de forma vertical.
- `layout_width`: propiedad que define el ancho del contenedor.
- `layout_height`: propiedad que define el alto del contenedor >

`<Button`: etiqueta para crear un view de tipo `Button` (Botón).

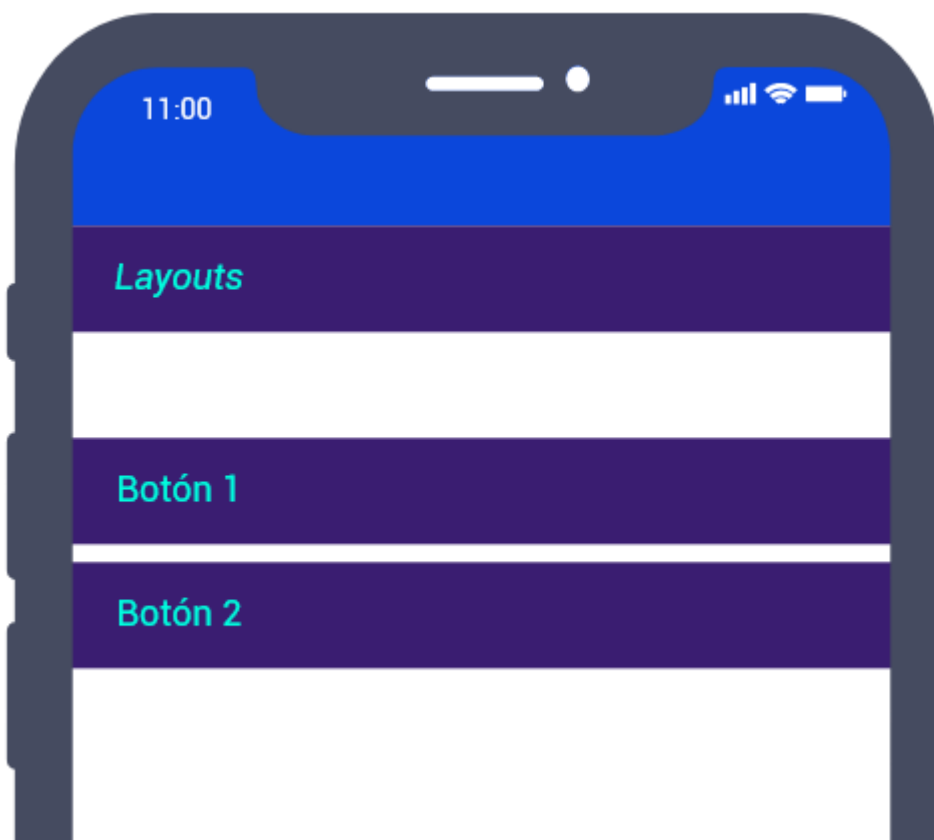
`layout_width`: propiedad que define el ancho del view.

`layout_height`: propiedad que define el alto del view.

`text`: propiedad que define el título del button />

El código anterior mostraría el LinearLayout de la siguiente forma:

Figura 15. LinearLayout



TableLayout

Es un contenedor ViewGroup que permite mostrar los elementos View en columnas y filas. Para crear las filas se utiliza el objeto TableRow. Cada fila puede tener una o más celdas, o ninguna, y cada celda se crea mediante un tipo de objeto View. Es

decir, las celdas de una fila pueden formarse por una diversidad de objetos View, como ImageView o TextView.

El siguiente código XML de diseño muestra dos filas con tres celdas en cada una, y una tercera fila con dos celdas:

```
<!-- Crea la tabla -->

<TableLayout

xmlns:android="http://schemas.android.com/apk/res/android"

android:layout_width="match_parent"

android:layout_height="match_parent"

android:stretchColumns="0,1,2">

<!-- Crea la fila -->

<TableRow>

<TextView android:text="Celda 1.1" /> <!-- Crea la Columna -->

<TextView android:text="Celda 1.2" /> <!-- Crea la Columna -->

<TextView android:text="Celda 1.3" /> <!-- Crea la Columna -->

</TableRow>

<!-- Crea la fila -->

<TableRow>

<TextView android:text="Celda 2.1" /> <!-- Crea la Columna -->
```

```
<TextView android:text="Celda 2.2" /> <!-- Crea la Columna -->
```

```
<TextView android:text="Celda 2.3" /> <!-- Crea la Columna -->
```

```
</TableRow>
```

```
<!-- Crea la fila -->
```

```
<TableRow>
```

```
<TextView android:text="Celda doble de ancho 3.1"
```

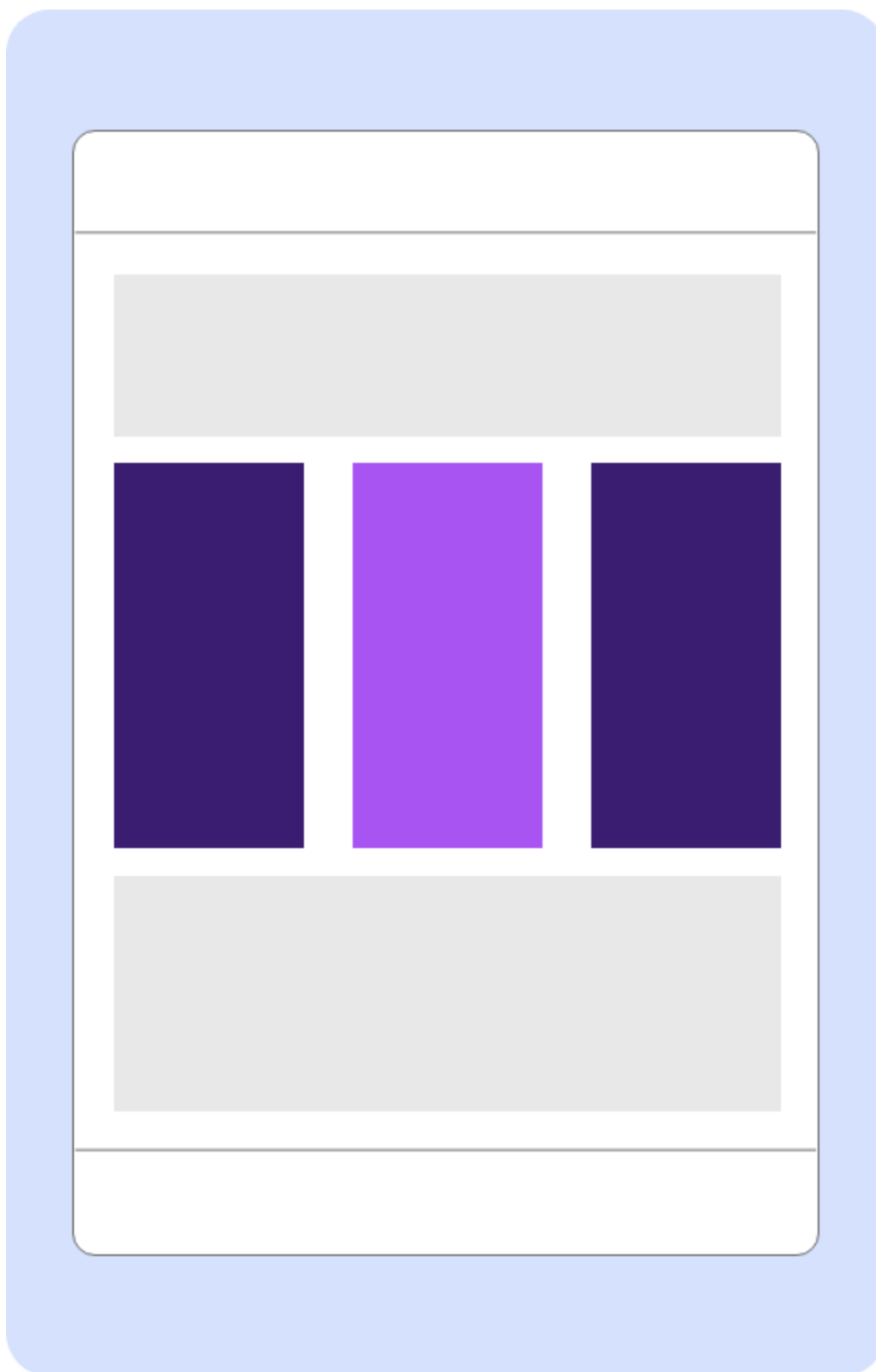
```
android:layout_span="2" />
```

```
<TextView android:text="Celda 3.2" />
```

```
</TableRow>
```

```
</TableLayout>
```

Figura 16. Diseño lineal horizontal



El código anterior mostraría el TableLayout de la siguiente forma:

Figura 17. TableLayout



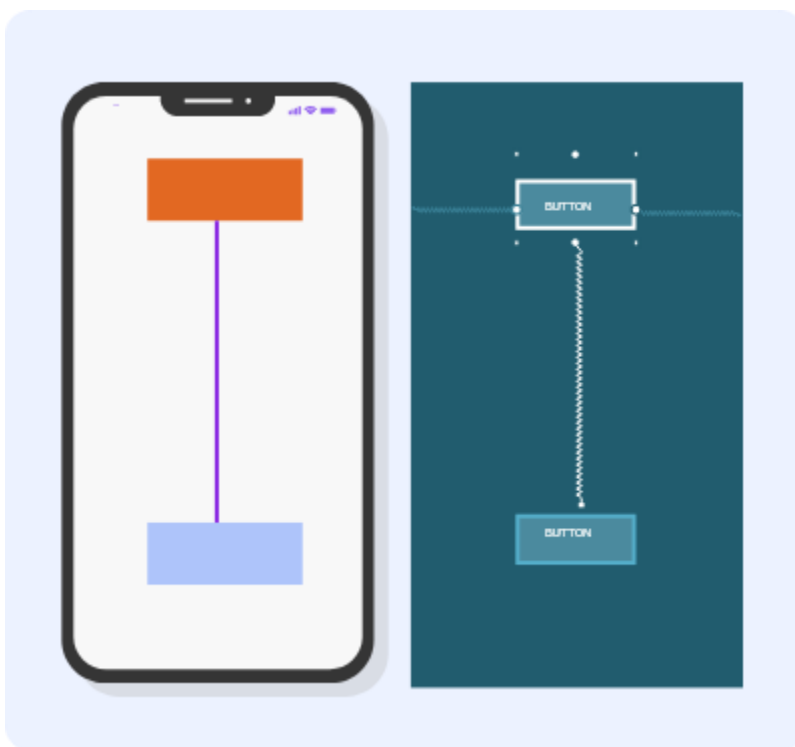
ConstraintLayout

ConstraintLayout es el contenedor más utilizado actualmente y es el que se configura por defecto al crear una actividad, ya que permite diseñar los view directamente desde las herramientas visuales del editor de diseño. Con este contenedor se puede crear todo el diseño arrastrando y soltando elementos, en lugar de editar el XML manualmente.

Cada posición de un view en un ConstraintLayout se define agregando al menos una restricción horizontal y una vertical. Cada restricción representa una conexión o alineación con otra vista. Estas restricciones determinan la posición de la vista a lo largo del eje vertical u horizontal, por lo que cada vista debe tener al menos una restricción para cada eje. Sin embargo, normalmente se requieren más restricciones para lograr un diseño preciso.

Vista de Diseño ConstraintLayout:

Figura 18. Vista de Diseño ConstraintLayout



2.2. Controles básicos

En Android Studio se encuentra una amplia variedad de controles que se pueden utilizar en la interfaz de usuario, tales como botones (Button), campos de texto (EditText), vista de imágenes (ImageView), casillas de verificación (CheckBox), botones de selección (RadioButton), etiquetas de texto (TextView), listas (ListView), entre otros.

Existen tres propiedades indispensables en la creación de los controles mencionados anteriormente:

a) Android: layout_width

Esta propiedad es común a todas las vistas y permite establecer el ancho que se utilizará. Este valor puede ser fijado en relación con el contenido que se visualizará o con la vista que lo contiene.

b) Android: layout_height

Esta propiedad es común a todas las vistas y permite establecer el ancho que se utilizará. Este valor puede ser fijado en relación con el contenido que se visualizará o con la vista que lo contiene.

c) Android: id

Define el identificador asociado a la vista para ser referenciado por código Java. La forma de crear un identificador es la siguiente:

- `android:id="@+id/myText"`
- El símbolo `@+id/` indica que se está generando un nuevo identificador, lo que provocará que la clase `R` del paquete `gen` del proyecto cree una referencia a este objeto, para que luego pueda ser manipulado por código.

Los valores que pueden tomar las propiedades de Android: `layout_width`, y Android: `layout_height` los cuales aplican para todos los controles son:

- a) **MATCH_PARENT**: Esto significa que la vista tendrá el tamaño de su contenedor (en este caso, la pantalla).
- b) **WRAP_CONTENT**: Esto significa que el tamaño de la vista se adaptará al contenido que tenga.

c) **PERSONALIZADO:** También se puede definir el alto y el ancho con valores personalizados usando la unidad de medida dp (píxeles independientes de densidad). Ejemplo:

- `android:layout_width="50dp"`
- `android:layout_height="200dp"`

Estas tres propiedades básicas son las principales en los componentes que se describen a continuación.

Figura 19. Controles básicos



Botones

Un botón (Button) es un elemento de la interfaz en el que el usuario puede tocar o hacer clic para realizar una acción. A continuación, se presenta un ejemplo de un Button con sus propiedades básicas:

Tabla 1. Propiedades básicas Button

Propiedades	Definición
android:layout_width	Define el ancho de la vista.
android:layout_height	Define el alto de la vista.
android:id	Define el identificador asociado a la vista para ser referenciado por código java.
android:text	Define el texto asociado a la vista.

XML Creación Button:

```
<Button  
  
    android:id="@+id/button"  
  
    android:layout_width="match_parent"  
  
    android:layout_height="wrap_content"  
  
    android:text="Boton" />
```

Y el botón quedaría como se presenta a continuación:

Figura 20. Vista diseño Button



Texto e imágenes

Uno de los controles más utilizados son los referentes a los textos e imágenes, ya que son con los que más interactúa el usuario. A continuación, se describen tres de los componentes básicos más imprescindibles para el desarrollo de nuestras aplicaciones: los textos o etiquetas (TextView), las imágenes (ImageView) y los cuadros de texto (EditText).

TextView: es un control utilizado para desplegar contenido textual generalmente estático. Tiene la propiedad de acoplarse al control que lo contenga y no captura el foco en ningún momento, por lo que generalmente no se programa ningún evento sobre él. Sus propiedades básicas son las siguientes:

Tabla 2. Propiedades básicas TextView

Propiedades	Definición
android:layout_width	Define el ancho de la vista.
android:layout_height	Define el alto de la vista.
android:id	Establece cuál es el contenido para mostrar por el TextView. Existen dos formas de asociar el texto a una vista, la primera forma y más simple es asociar el texto de forma directa: android:text="Hola Mundo" La segunda forma es por medio de archivo string XML ubicado dentro de la carpeta values de los recursos del proyecto y asociar cada uno de los textos con una llave respectiva. Ya luego se referencia el contenido por medio de este archivo XML de la siguiente forma: android:text="@string/boton2" El color de tinte de la imagen.
android: textColor	Esta propiedad nos permite definir cuál va a ser el color del texto.
android: textSize	Con esta propiedad podemos definir el tamaño de la fuente la unidad es sp. Ejemplo: android:textSize="20sp"
android: textStyle	Define el estilo de la fuente a usar, tiene los siguientes valores: android:textStyle="bold" android:textStyle="italic" android:textStyle="normal"
android: typeface	Esta propiedad me permite cambiar el tipo de fuente dentro de un conjunto de 4 posibles

Propiedades	Definición
	opciones: normal, sans, serif y monospace, se pueden descargar otro tipo de fuentes.

Ejemplo de TextView con sus propiedades básicas:

```
<TextView
    android:id="@+id/txtTexto"
    android:text="@string/control_de_texto"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textStyle="bold"
    android:textColor="@color/purple_500"
    android:typeface="monospace"
    android:textSize="40sp" />
```

El TextView quedaría como se presenta a continuación:

Figura 21. Vista diseño TextView



ImageView: es un control de imágenes que permite acceder a los recursos de imágenes (drawable o mipmap) y publicar en la aplicación. Su propiedad principal es `android:src`, que permite indicar la imagen a exponer del proyecto. Lo primero que se debe hacer es copiar la imagen que se quiere presentar en la carpeta drawable del proyecto (`/res/drawable`) y, después, en el XML se debe indicar la ruta de origen así: `android:src="@drawable/imagen"`.

Aparte de las propiedades de `width` y `height` que se encuentran en todos los view, están las siguientes propiedades básicas para el ImageView:

Tabla 3. Propiedades básicas ImageView

Propiedades	Definición
<code>android:src</code>	Establece la imagen del ImageView.

Propiedades	Definición
android:scaleType	Controla cómo se debe cambiar el tamaño o mover la imagen para que coincida con el tamaño de este ImageView.
android:tint	El color de tinte de la imagen.

Ejemplo de ImageView con sus propiedades básicas:

```
<ImageView  
  
    android:id="@+id/txtTexto"  
  
    android:layout_width="wrap_content"  
  
    android:layout_height="wrap_content"  
  
    android:src="@drawable/andy"  
  
    android:scaleType="centerInside" />
```

El ImageView quedaría como se presenta a continuación:

Figura 22. Vista diseño ImageView



EditText: es un elemento de la interfaz que permite al usuario ingresar y editar texto. Una de sus propiedades más funcionales es `inputType`, que define el tipo de entrada que tendrá el cuadro de texto. El valor se aplica dependiendo de lo que se quiera ingresar: por ejemplo, un correo electrónico (`textEmailAddress`), edad (`number`), apellido (`text`), entre otros. La elección del tipo de entrada configura el tipo de teclado que se expondrá.

Tabla 4. Propiedades básicas de EditText

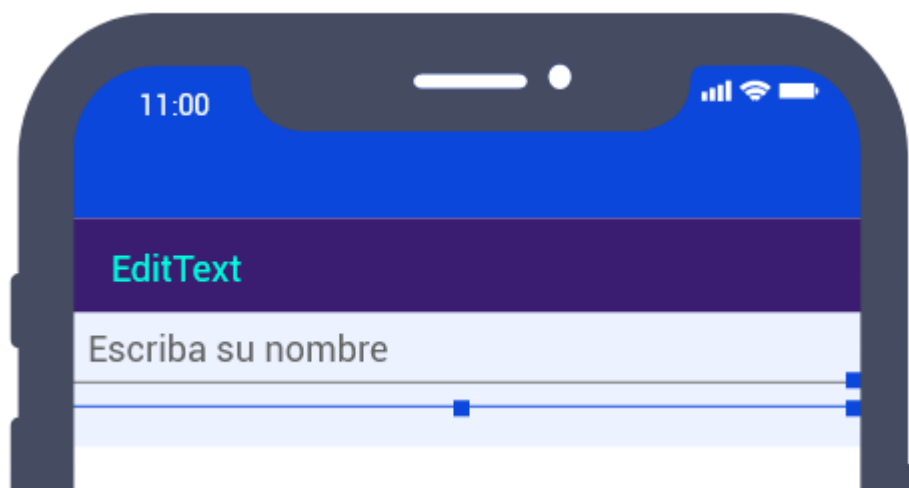
Propiedades	Definición
<code>android:inputType</code>	Establece el tipo de dato que se ingresará en el campo de texto. Para esta propiedad existen valores como: <code>text</code> , <code>number</code> , <code>textUri</code> , <code>textPassword</code> , <code>phone</code> , <code>date</code> , entre otros.
<code>android:hint</code>	Permite desplegar un texto sobre el control a manera de ayuda mientras el usuario aún no ingresa información. Se prefiere sobre la propiedad <code>text</code> .
<code>android:text</code>	Al igual que en <code>TextView</code> , se puede definir cuál es el texto que se mostrará en el control.

Ejemplo de EditText con sus propiedades básicas:

```
<EditText
    android:id="@+id/txtTexto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Escriba su Nombre"
    android:inputType="text" />
```

El EditText quedaría como se presenta a continuación:

Figura 23. Vista diseño EditText



Checkbox y Radiobutton

Otro de los controles más utilizados en las aplicaciones son los que permiten hacer una selección o un chequeo de alguna opción. Android cuenta con dos tipos: checkboxes y radiobuttons.

CheckBox: Las casillas de verificación permiten que el usuario seleccione una o más opciones de un grupo. Aparte de las propiedades de width y height que se encuentran en todas las vistas (view), las siguientes son básicas para el Checkbox:

Tabla 5. Propiedades básicas Checkbox

Propiedades	Definición
android:checked	Establece el estado actual del Checkbox.

Propiedades	Definición
android:text	Define el texto del Checkbox.
android:id	Define el identificador único asociado a la vista para ser referenciado por código Java.

Para definir un control de este tipo, se utiliza el siguiente código, que define dos checkboxes con los textos "español" e "inglés":

```

<CheckBox

android:id="@+id/txtTexto"

android:layout_width="match_parent"

android:layout_height="wrap_content"

android:text="@string/espa_ol"

android:checked="true" />

<CheckBox

android:id="@+id/txtTexto2"

android:layout_width="match_parent"

android:layout_height="wrap_content"

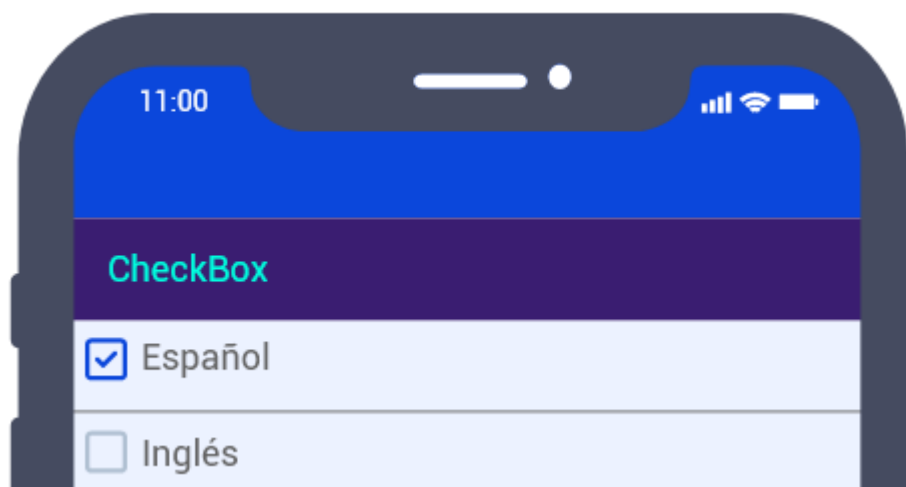
android:text="@string/ingles"

android:checked="false" />

```

Los checkbox quedarían como se presenta a continuación:

Figura 24. Vista diseño Checkbox



RadioButton: a diferencia del CheckBox, el control RadioButton permite realizar una selección única entre una lista de opciones. En Android, un grupo de botones RadioButton se define mediante un elemento RadioGroup, el cual contiene todos los elementos RadioButton necesarios.

Tabla 6. Propiedades básicas de RadioButton

Propiedades	Definición
android:checked	Establece el estado actual de selección por defecto del RadioButton.
android:text	Define el texto que se mostrará en el RadioButton.
android:id	Define el identificador único asociado a la vista para ser referenciado por código Java, al igual que en los otros controles.

Para definir un control de este tipo, se utiliza como componente padre un RadioGroup y como hijos dos RadioButton. El RadioGroup es el que permite que solo

sea seleccionable una sola opción; si se crean RadioButton sin un RadioGroup, todas las opciones serán seleccionables, lo que no es el propósito de este componente.

En las propiedades del RadioGroup se puede definir la orientación de los RadioButton, que puede ser vertical u horizontal.

El siguiente código presenta la creación de dos RadioButton con los textos de "femenino" y "masculino", los cuales están dentro de un RadioGroup con la propiedad de orientación vertical.

```
<RadioGroup  
  
    android:id="@+id/rg1"  
  
    android:layout_width="match_parent"  
  
    android:layout_height="wrap_content"  
  
    android:orientation="vertical">  
  
    <RadioButton  
  
        android:id="@+id/rb1"  
  
        android:layout_width="match_parent"  
  
        android:layout_height="wrap_content"  
  
        android:text="@string/femenino"  
  
        android:checked="true" />  
  
    <RadioButton  
  
        android:id="@+id/rb2"
```

```

android:layout_width="match_parent"

android:layout_height="wrap_content"

android:text="@string/masculino"

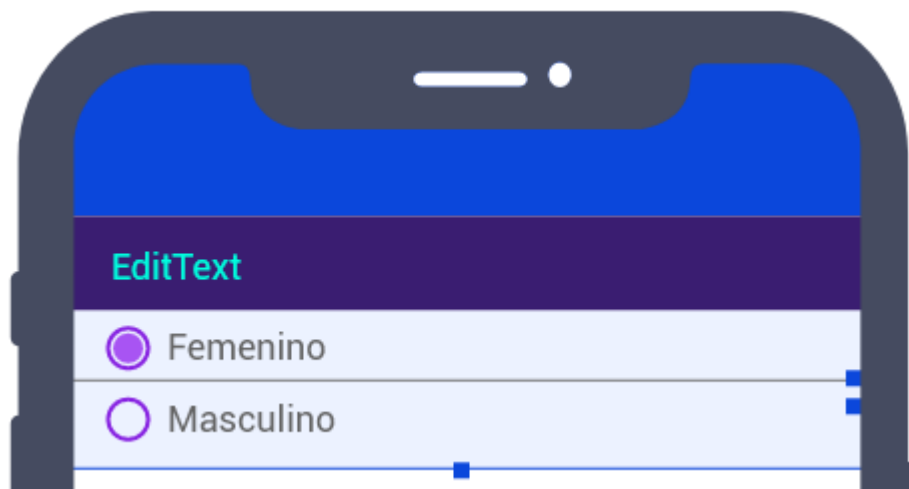
android:checked="false" />

</RadioGroup>

```

Los RadioButton quedarían como se presenta a continuación:

Figura 25. Vista diseño RadioButton



Controles de selección

Los diferentes controles que Android ofrece para seleccionar opciones dentro de una lista de posibilidades se llaman Spinner (para listas desplegadas) y ListView (para listas fijas).

Spinner

Este permite seleccionar un valor de un conjunto. Al hacer clic sobre él, se presenta una especie de lista emergente con todas las opciones disponibles. Su propiedad más importante es `android:entries="@array/lista"`, que permite fijar el contenido de datos que se quiere mostrar en la lista.

Para añadir una lista de este tipo, se deben seguir los pasos que se mencionan a continuación:

- a) En los recursos `/res/values/strings.xml`, se debe crear un arreglo de strings (string-array) con los valores que se quieren presentar en el spinner.

```
<resources>

<string name="app_name">RadioButton</string>

<string-array name="ciudades">

<!-- Se crea el Arreglo, en este caso se coloca el nombre de ciudades -->

<item>Popayán</item>

<item>Cali</item>

<item>Bogotá</item>

<item>Cartagena</item>

<item>Medellín</item>

<item>Pasto</item>

</string-array> <!-- Cierre del arreglo -->
```


</resources>

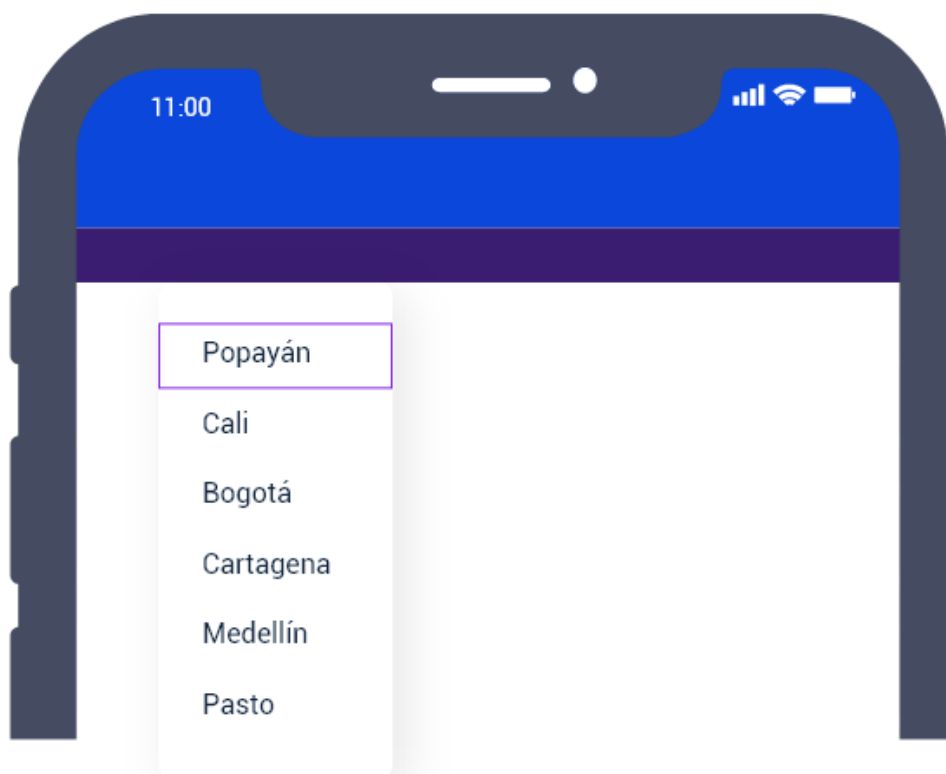
- b) Ahora en el /res/layout se crea el siguiente código para crear el Spinner y enlazar el string-array a través de la propiedad entries como se presenta:

```
<Spinner  
    android:id="@+id/spCiudades"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:entries="@array/ciudades" />
```

El código enseña la creación de un Spinner con sus propiedades básicas height, width y id, y la propiedad entries, donde se relaciona el array "ciudades" creado en los recursos.

El Spinner se presentaría de la siguiente manera:

Figura 26. Vista Diseño Spinner ciudades



ListView

El ListView permite mostrar al usuario una lista de opciones seleccionables directamente sobre el propio control, sin listas emergentes, como en el caso del control Spinner. Si existen más opciones de las que se pueden mostrar en el control, se podrá hacer scroll sobre la lista para acceder al resto de los elementos.

Para crear el ListView, al igual que en el Spinner, también se debe crear el arreglo de elementos que se quiere mostrar en la lista. Para este ejemplo, se utilizará el mismo arreglo de ciudades que se usó para el Spinner.

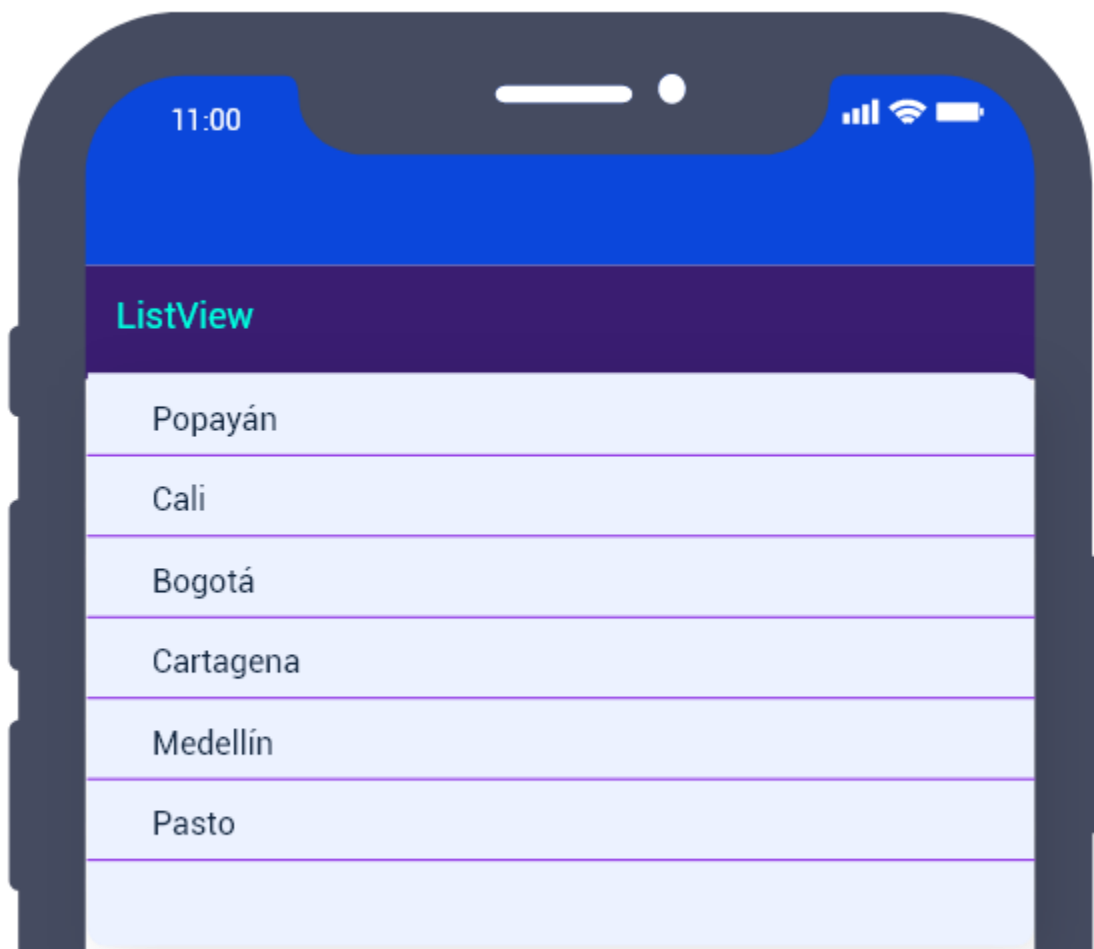
Cuando los elementos que se quieren mostrar en la lista o en el Spinner no son estáticos, se crea el arreglo desde el código Java.

Ejemplo de creación de un ListView:

```
<ListView  
  
    android:layout_gravity="center"  
  
    android:id="@+id/listaCiudades"  
  
    android:layout_width="wrap_content"  
  
    android:layout_height="wrap_content"  
  
    android:entries="@array/ciudades" />
```

El código describe la creación de un ListView, utilizando las mismas propiedades que el Spinner, y la propiedad entries, donde se relaciona el array "ciudades" creado en los recursos.

Figura 27. Lista diseño ListView ciudades



3. Eventos Listeners

En Android, la interacción del usuario con la aplicación se gestiona a través de eventos. Por ejemplo, cuando se toca una vista (como un botón), se llama al método `onTouchEvent()` del objeto, utilizando los objetos de escucha de eventos para detectar la interacción del usuario.

Los Event Listeners (objetos de escucha de eventos) son interfaces de la clase `View` que contienen un método de llamada, el cual será invocado cuando la vista registre una interacción del usuario con el elemento de la interfaz gráfica. Esto significa que los `View`, como `Button`, `ImageView`, `RadioButton` y `Checkbox`, al interactuar con el usuario, el objeto escuchará y llamará al método correspondiente, ejecutando una acción determinada.

La ejecución de los eventos se realiza en el código Java ubicado en la carpeta `/java`.

Para fijar un `Listeners`, se debe implementar y agregar los métodos del evento correspondiente a cada `View` mediante el método `setOn`. Para realizar el ejemplo del evento `Listeners` de un `Button`, se deben seguir algunos pasos:

Crear el `Button` en el `Layout`

Crear el `Button` en el `Layout` con su propiedad `ID`, ya que es la que nos va a permitir referenciarlo desde el código Java. Se muestra la creación de un botón en el `layout` llamado `activity_main.xml`. El código XML tiene un contenedor de tipo `LinearLayout` con su propiedad vertical y el `Button` es creado con sus propiedades básicas. La propiedad que se necesita para referenciar es la del `id`. El nombre del identificador que se colocó para el ejemplo fue `btn clic`, y en la vista de diseño se puede

ver cómo queda el botón. Estas son las diferentes vistas en las que se puede trabajar en el layout:

- **Code:** permite visualizar solo el código XML.
- **Split:** permite visualizar el código y el diseño como se ve en la figura.
- **Design:** permite visualizar solo el editor de diseño.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_marginTop="80dp"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/btn clic"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/click" />

</LinearLayout>
```

Declarar el tipo de view

En la clase Java, declarar el tipo de view y referenciarlo. La clase en Java está compuesta por un método principal que se llama onCreate. Todo el código que esté dentro de este método se ejecuta inmediatamente se corra la aplicación. Cada clase tiene relacionado su layout en el método setContentView, por lo tanto, se debe referenciar los view en cada clase que lo contenga. En el ejemplo, la clase Java MainActivity contiene el layout activity_main que es donde se creó el Button.

```
import android.os.Bundle;

import android.widget.Button;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button btn clic; // Declarar el botón

    @Override

    protected void onCreate(Bundle savedInstanceState) { // Método principal

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        /* Para referenciar el Button que está en el layout

        se hace por medio del Id y con el método findViewById */

        btn clic = findViewById(R.id.btn clic);

        /* En la variable btn clic quedan almacenadas las propiedades del botón
```

```
para poder manejar el evento listener */  
  
}  
  
}
```

Implementar el evento Listeners

Implementar el evento listener: En la figura se ve la implementación del evento `setOnClickListener` para que cuando el usuario dé clic en el botón se ejecute una instrucción. El código debe escribirse en el método `onClick`.

Los View tiene diferentes métodos escuchadores, en la tabla que se encuentra a continuación se pueden observar los más utilizados:

- **ListView**. `SetOnItemClickListener`
- **Spinner**. `SetOnItemSelectedListener`.
- **ImageView**. `setOnClickListener`
- **RadioButton**. `setOnClickListener`

```
public class MainActivity extends AppCompatActivity {  
  
    Button btnclic; // Declarar el botón  
  
    @Override  
  
    protected void onCreate(Bundle savedInstanceState) { // Método principal  
  
        super.onCreate(savedInstanceState);  
  
        setContentView(R.layout.activity_main);  
  
        /* Para referenciar el Button que está en el layout
```



```
se hace por medio del Id y con el método findViewById */  
  
btn clic = findViewById(R.id.btn clic);  
  
/* En la variable btn clic quedan almacenadas las propiedades del botón  
para poder manejar el evento listener */  
  
btn clic.setOnClickListener(new View.OnClickListener() {  
  
    @Override  
  
    public void onClick(View v) {  
  
        // Aquí se colocan las instrucciones que se quieran ejecutar al dar clic en el botón  
  
    }  
  
});  
  
}  
  
}
```

4. Navegación (Intents)

La navegación entre pantallas en Android se realiza a través de los Intents. El Intent es básicamente una intención de realizar una acción; es la forma de comunicación en Android que permite la interacción entre componentes de la misma aplicación o entre diferentes aplicaciones (Activities, Services, Broadcasts, etc.).

Existen dos tipos de Intents:

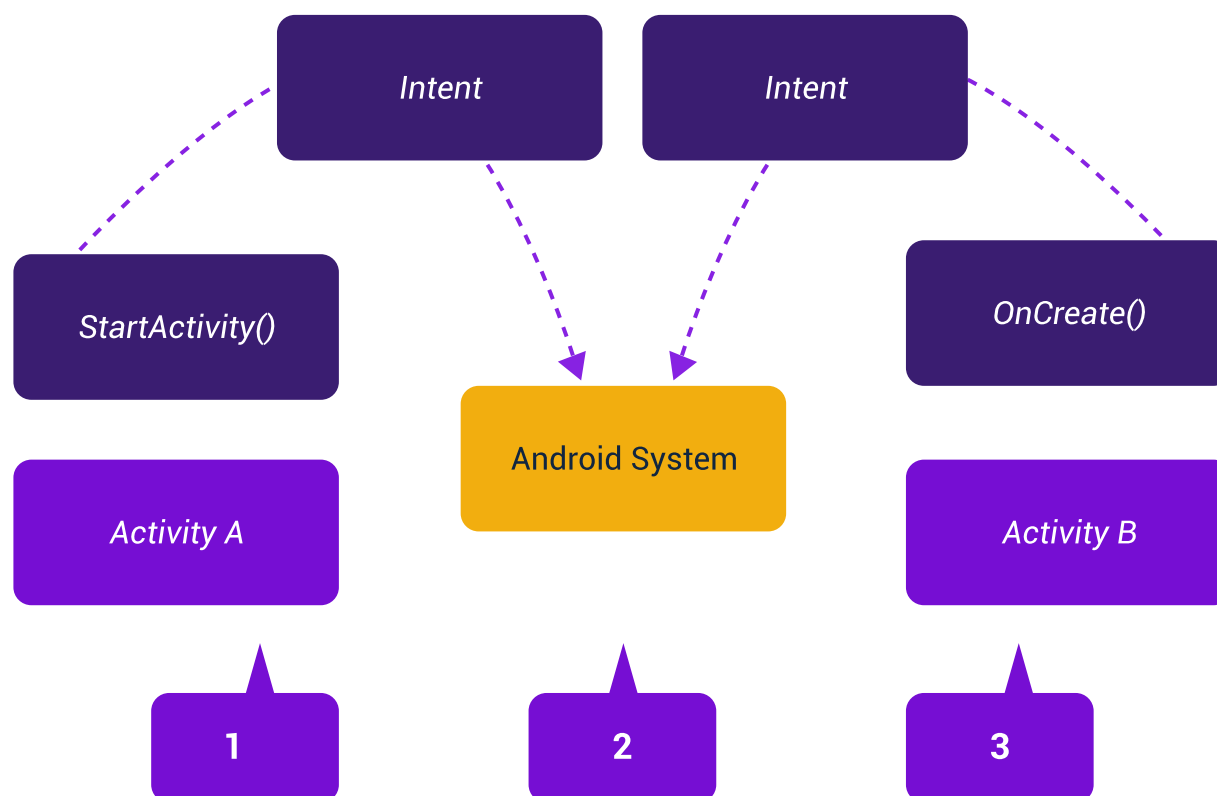
Intent explícito

Se utiliza para llamar a un componente específico. Por ejemplo, si se tiene una aplicación que contiene dos actividades, Actividad A y Actividad B, y se requiere navegar desde la Actividad A hasta la Actividad B, se define una intención explícita.

Para ello, se utiliza la clase Intent donde se especifica dónde está y a dónde se quiere ir:

```
Intent intent = new Intent(ActividadA.this, ActividadB.class);  
  
startActivity(intent);
```

Figura 28. Uso intent explícito



La figura presenta cómo se utiliza un intent para iniciar una actividad. Cuando el objeto Intent nombra un componente de actividad específico de forma explícita, el sistema inicia ese componente al instante.

Intent implícito

A diferencia de los explícitos, los intent implícitos no nombran el componente específico. Se usan para iniciar un componente específico de la aplicación. Por ejemplo, si se desea abrir la cámara, una ubicación en un mapa o una página web, se utilizan los intent implícitos para solicitar que se activen estos servicios.

Ejemplo:

```
Intent intent = new Intent(Intent.ACTION_VIEW,  
Uri.parse("http://www.example.com"));  
  
startActivity(intent);
```

5. Tareas asincrónicas

Todos los componentes de una aplicación Android, como las actividades, los servicios o los broadcast receivers, se ejecutan en el mismo hilo de ejecución, llamado hilo principal, main thread o GUI thread. Como sugiere este último nombre, también es el hilo donde se realizan todas las operaciones que gestionan la interfaz de usuario de la aplicación. Por ello, cualquier operación larga o costosa que se realice en este hilo bloqueará la ejecución del resto de componentes de la aplicación y, por supuesto, también la interfaz, produciendo un efecto evidente de lentitud, bloqueo o mal funcionamiento en general para el usuario.

Android utiliza la clase AsyncTask para ejecutar tareas en segundo plano.

La forma básica de utilizar la clase AsyncTask consiste en crear una nueva clase que extienda de ella y sobrescribir varios de sus métodos, distribuyendo la funcionalidad de la tarea entre ellos. Estos métodos son los siguientes:

- **onPreExecute()**

Se ejecutará antes del código principal de nuestra tarea. Se suele utilizar para preparar la ejecución de la tarea, inicializar la interfaz, etc.

- **doInBackground()**

Contendrá el código principal de nuestra tarea.

- **onProgressUpdate()**

Se ejecutará cada vez que llamemos al método publishProgress() desde el método doInBackground().

- **onPostExecute()**

Se ejecutará cuando finalice nuestra tarea, o, dicho de otra forma, tras la finalización del método `doInBackground()`.

- **onCancelled()**

Se ejecutará cuando se cancele la ejecución de la tarea antes de su finalización normal.

Estos métodos tienen una particularidad esencial para nuestros intereses. El método `doInBackground()` se ejecuta en un hilo secundario, por lo tanto, no se puede interactuar con la interfaz en su interior. Sin embargo, todos los demás métodos se ejecutan en el hilo principal, lo que significa que dentro de ellos se puede hacer referencia directa a los controles de usuario para actualizar la interfaz.

6. Persistencia de datos

En el desarrollo móvil, es indispensable que las aplicaciones puedan guardar datos, ya sea para no perder información en una nueva ejecución o para manejar información en bases de datos. La persistencia de datos es una habilidad esencial para que un desarrollador cree aplicaciones de alta calidad. Uno de los métodos más comunes de almacenamiento en Android es el de las Bases de Datos en SQLite.

6.1. Bases de datos local SQLite

SQLite es un motor de bases de datos relacional de código abierto y actualmente muy popular, ya que ofrece particularidades muy útiles como su pequeño tamaño, ligereza, no necesitar servidor, requerir poca configuración y guardar toda la base de datos en un solo fichero. Su uso es muy habitual en aplicaciones de tamaño pequeño.

- Android incluye soporte para SQLite y utiliza el paquete `android.database.sqlite` para trabajar con bases de datos.
- Permite ejecutar la aplicación en tiempo real en el dispositivo móvil.
- Para crear un esquema de base de datos o actualizar el mismo, se debe crear una clase que herede de `SQLiteOpenHelper`, donde se implementa el constructor y dos métodos abstractos (obligatorios de implementar): `onCreate()` y `onUpgrade()`.
- A través de los métodos `getReadableDatabase()` y `getWritableDatabase()`, se abre y se obtiene una instancia de la base de datos de solo lectura o de lectura y escritura, respectivamente.
- Para la creación de las tablas, se ejecutan las sentencias SQL dentro del método `onCreate()` utilizando el método `db.execSQL()`.

Para la administración de una base de datos SQLite se utiliza la clase SQLiteDatabase:

SQLiteDatabase: tiene métodos para crear, eliminar, ejecutar comandos SQL y realizar otras tareas comunes de administración de bases de datos.

Los métodos más utilizados son:

Tabla 7. Métodos de administración de bases de datos de la clase SQLiteDatabase

Acción	Definición
Insertar datos	db.insert
Consultar datos	db.rawQuery() y db.query()
Eliminar datos	db.delete
Actualizar datos	db.update
Ejecutar sentencia SQL	db.execute

Para reflejar la utilidad de las bases de datos SQLite, se realizará un ejemplo de una base de datos para registro de usuarios y de entidades. Se tendrá una actividad de solicitud de datos para la creación, actualización y borrado en la base de datos, permitiendo gestionar la información de forma persistente.

6.2. Bases de datos en tiempo real

Una base de datos en tiempo real se refiere a un sistema de base de datos que procesa datos en tiempo real, tomando los datos entrantes a medida que fluyen del servidor. La base de datos en tiempo real de Firebase (Firebase Realtime Database) es sin duda uno de los servicios más populares de la plataforma.

Firebase es una plataforma para el desarrollo de aplicaciones web y móviles, creada por James Tamplin y Andrew Lee en 2011, y adquirida por Google en 2014. Provee servicios muy útiles que se pueden incorporar en aplicaciones Android (además de otros sistemas operativos), tales como:

- **Analytics**

Firebase Analytics proporciona una visión profunda sobre el uso de la aplicación por parte de los usuarios, con datos que sirven para analizar el comportamiento de los usuarios con la aplicación.

- **Auth**

Servicio que puede autenticar a los usuarios utilizando únicamente código del lado del cliente. Incluye la autenticación mediante Facebook, GitHub, Twitter y Google. Además, incluye un sistema de administración del usuario por el cual los desarrolladores pueden habilitar la autenticación de usuarios con email y contraseña.

- **Database**

Proporciona una base de datos en tiempo real y back-end. El servicio proporciona a los desarrolladores de aplicaciones una API que permite que la información de las aplicaciones sea sincronizada y almacenada en la nube de Firebase.

- **Storage**

Proporciona cargas y descargas seguras de archivos para aplicaciones Firebase, sin importar la calidad de la red. El desarrollador lo puede utilizar para almacenar imágenes, audio, vídeo, u cualquier otro contenido generado por el usuario.

- **Notifications**

Con Firebase se puede usar la API FirebaseMessaging que permite el envío de mensajes desde Firebase hacia un dispositivo concreto o múltiples dispositivos, pudiendo gestionar las notificaciones desde la consola de Firebase.

Principales características de Firebase Realtime Database:

- No requiere la instalación de un servidor ni de software adicional, ya que la base de datos está alojada en la nube.
- El formato de almacenamiento de los datos es JSON, utilizando una base de datos no relacional.
- La actualización de los datos se realiza en milisegundos debido a la sincronización en tiempo real.
- Permite el desarrollo de aplicaciones multiplataforma para iOS, Android y Web (JavaScript).
- Incluso cuando el dispositivo se encuentra sin conexión a Internet, la aplicación sigue funcionando gracias a la caché.

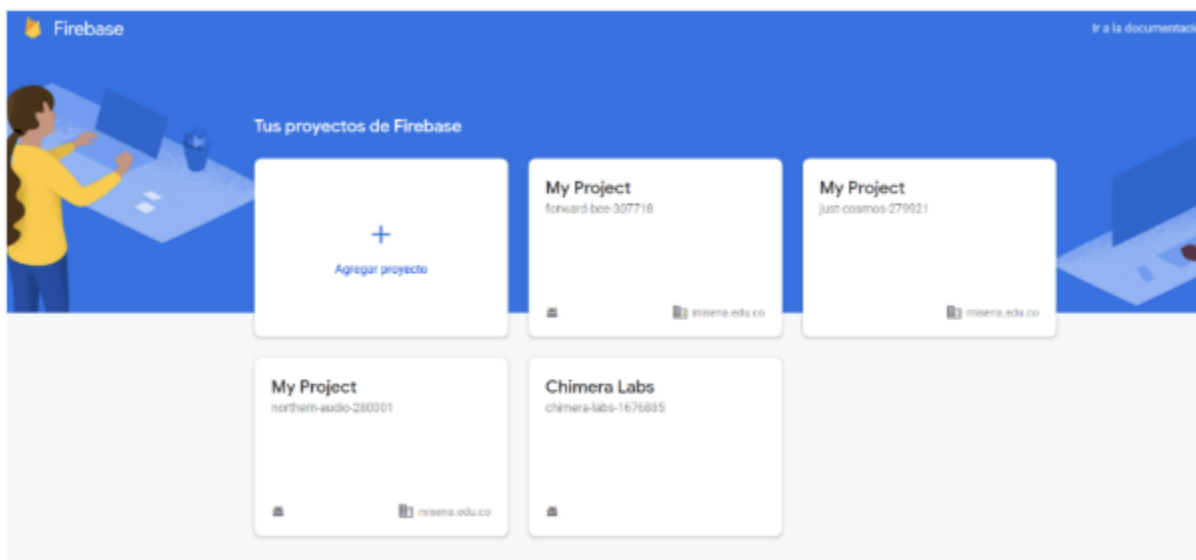
Para crear un proyecto con Firebase se deberán tener lo siguiente:

- Una cuenta de Gmail para acceder a la consola de Firebase
<https://console.firebase.google.com>
- Un proyecto de Android Studio para poder hacer la conexión con la base de datos de Firebase.

Para la configuración de Firebase en el proyecto de Android Studio se enumera los pasos siguientes:

a) Crear un Proyecto en la consola de Firebase.

Figura 29. Pantalla principal de Firebase



b) Agregar el proyecto.

Figura 30. Proceso de creación de un proyecto en Firebase



c) Seleccionar la plataforma para agregar a la APP.

Figura 31. Proyecto recién creado en Firebase



d) Ingresar el nombre del paquete de la APP en el campo Nombre del paquete.

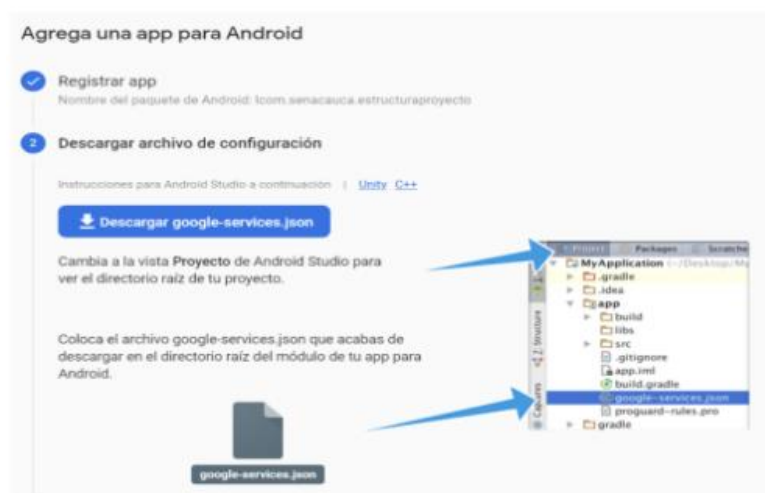
Lo más importante es copiar el paquete de su proyecto de Android Studio que se encuentra en el manifest `com.dominio.proyecto`.

Figura 32. Formulario para registrar una aplicación Android en Firebase



- e) Descargar archivo Json google-services.json y copiarlo en la raíz del proyecto de Android Studio.

Figura 33. Proceso de agregar una app para Android en Firebase



- f) Agregar SDK de Firebase.

Copiar las librerías al gradle del proyecto en Android Studio y ya está lista la configuración de Android Studio con Firebase.

Figura 34. Instrucciones para agregar el SDK de Firebase al proyecto



7. Multimedia

En Android Studio, la integración de contenido multimedia en las aplicaciones es muy sencilla gracias a las opciones que presenta la API. Las principales clases que facilitan el acceso a los servicios multimedia son MediaPlayer y VideoView, siendo estas las más utilizadas.

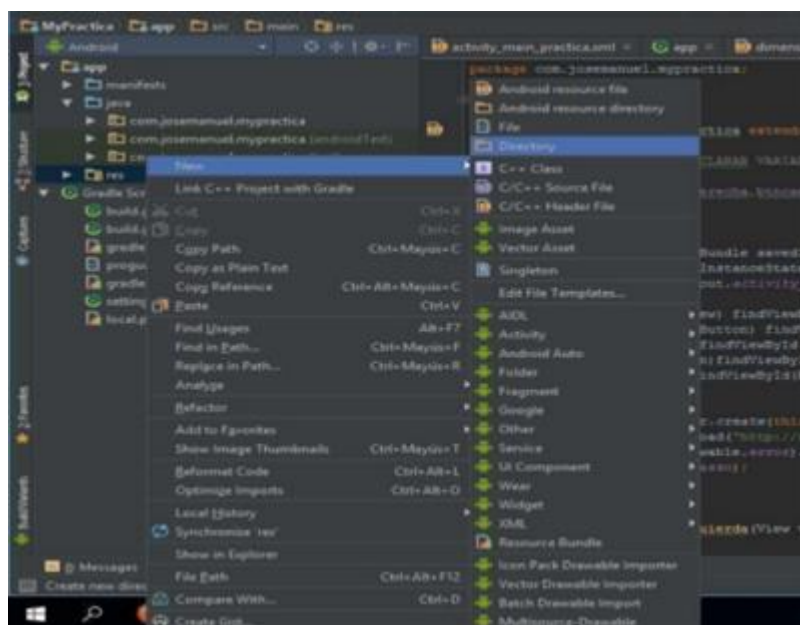
7.1. MediaPlayer

MediaPlayer permite reproducir sonido y video a través de su API. Es uno de los componentes principales del marco de trabajo de medios. Un objeto de esta clase puede recuperar, decodificar y reproducir audio y video con una configuración mínima. Es compatible con varias fuentes de medios diferentes, entre las que se incluyen las siguientes: URI, recursos locales y URL externas.

A continuación, se presenta un ejemplo de cómo reproducir un archivo de audio que está disponible como recurso local (ubicado en el directorio res/raw/ de la app):

- Crear un directorio dentro de la carpeta res llamada raw.

Figura 35. Pantalla de Android Studio que presenta el menú contextual



- Copiar un sonido en la carpeta raw de extensión MP3.

Button Click Sound

How to play audio on button click

Botón 1

Botón 3

Botón 3

- En la clase Java escribe el siguiente código.

```
public class MainActivityPractica extends AppCompatActivity {
    MediaPlayer cancion;
}
```

- Se crea la instancia del sonido.

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
```

```
cancion = MediaPlayer.create(this, R.raw.bruja);  
}
```

- Ejecución del sonido.

```
public void iniciarSonido(View view) {  
    cancion.start();  
}
```

7.2. VideoView

La clase VideoView permite presentar un archivo de video. Puede cargar videos de varias fuentes (como recursos o proveedores de contenido). La propiedad principal de VideoView es setVideoURI, donde se establece la ruta del video.

A continuación, se describe un ejemplo para utilizar VideoView:

- a) Al igual que con el MediaPlayer, se debe crear la carpeta res/raw y guardar en esta carpeta el video con extensión .mp4.
- b) En el layout, se crea el componente VideoView.

```
<VideoView  
    android:id="@+id/videoview"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content" />
```

- c) En la clase Java se realiza todo el código:

```
public class MainActivity extends AppCompatActivity {  
  
    // Paso 1. Se declara el VideoView  
  
    VideoView videoView;
```



```
@Override

protected void onCreate(Bundle savedInstanceState) { // Método principal

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_main);

    // Paso 2. Referenciar el VideoView

    videoView = findViewById(R.id.videoview);

    // Paso 3. Se asigna la ruta al VideoView

    videoView.setVideoPath("android.resource://" + getPackageName() + "/" +
R.raw.video);

    // 4. La propiedad SetMediaController permite visualizar los controles de
    pausar, adelantar, iniciar

    videoView.setMediaController(new MediaController(this));

    // 5. Se inicia la ejecución de video

    videoView.start();

}

}
```

Al ejecutar se presentará de la siguiente manera:

Figura 36. Vista Ejecución VideoView



Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Estructura de un proyecto en Android Studio	Android Developers. (2021, May 25). Aspectos fundamentales de la aplicación.	Página	https://developer.android.com/training/basics/activity-lifecycle
Estructura de un proyecto en Android Studio	Developers. (2021, June 14). Notas de la versión del complemento de Android para Gradle.	Página	https://developer.android.com/studio/releases/gradle-plugin
Interfaz de usuario en Android	Developers. (2021) Cómo crear una IU responsiva con ConstraintLayout.	Página	https://developer.android.com/training/constraint-layout
Interfaz de usuario en Android	Castillo, J. D. L. (2015, May 21). Capítulo 14 - Interacción con Botones, TextView, ImageView. YouTube.	Video	https://www.youtube.com/watch?v=z8zTGTo7vA&ab_channel=Jos%C3%A9DimasLuj%C3%A1nCastillo
Interfaz de usuario en Android	Castillo, J. D. L. (2015, May 21). Capítulo 15 - Android Studio Intent Abrir una Activity. YouTube.	Video	https://www.youtube.com/watch?v=LEVXzsXhO0A&ab_channel=Jos%C3%A9DimasLuj%C3%A1nCastillo

Glosario

Diseño: un diseño define la estructura visual de una interfaz de usuario.

Scrolling: acción de desplazarse internamente.

SDK: kit de desarrollo de software.

String: recurso XML que ofrece una sola string.

View: es la clase base de los widgets.

Wearable: dispositivo conectado que se puede llevar puesto y que se conecta al teléfono móvil.

Referencias bibliográficas

Android Developers. (2021). Documentation | Desarrolladores de Android.

<https://developer.android.com/docs>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Zulema Yidney León Escobar	Experta Temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Andrés Felipe Herrera Roldán	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edward Leonardo Pico Cabra	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Jaime Hernán Tejada Llano	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Tecnologías emergentes y disruptivas

Breve descripción:

La innovación que conlleva las tecnologías emergentes está ofreciendo cada vez más la creación de productos y servicios por parte de las empresas u organizaciones que hacen un gran aporte a la sociedad. Tecnologías como IoT, Blockchain, Machine learning, Deep learning, traen grandes desarrollos.

Julio 2024

Tabla de contenido

Introducción	4
1. Internet de las cosas (IoT)	5
2. Blockchain	10
2.1. Historia	13
2.2. Aplicaciones	18
2.3. Desarrollos	19
3. Machine learning	21
3.1. Conceptos.....	23
Inteligencia artificial	23
Aprendizaje supervisado	25
Aprendizaje no supervisado	27
Aprendizaje por refuerzo.....	27
3.2. Aplicaciones	28
3.3. Deep learning	29
Síntesis	31
Material complementario	32
Glosario	33
Referencias bibliográficas	34



Créditos	36
----------------	----

Introducción

Las tecnologías emergentes o disruptivas facilitan la creación de un marco de innovación que es aprovechado por organizaciones para la creación de nuevos servicios y productos. La palabra disruptiva viene del francés y se refiere a un cambio concluyente o violento. Es así como cada tecnología que genera transformaciones intensas en los conocimientos de productos o servicios, es una tecnología disruptiva.

A medida que avanza el tiempo, avanzan también las innovaciones tecnológicas y surgen nuevos términos utilizados para distinguir la convergencia y emergencia de nuevas tecnologías, con potencial suficiente para manifestarse como tecnologías disruptivas.

Es muy importante reconocer las tendencias que pueden encaminar las innovaciones o condiciones de vida en los siguientes años, en la actualidad, hay muchas que son consideradas tecnologías emergentes y disruptivas, las cuales se estudiarán en el presente componente formativo.

Le deseamos muchos éxitos en este proceso de aprendizaje.

1. Internet de las cosas (IoT)

Gracias a su relevancia y avance significativo, la IoT se considera una de las tecnologías con mayor influencia para el año 2025. Se prevé que una amplia gama de objetos incorporará diversos tipos de sensores conectados a Internet, lo que generará un enorme volumen de datos. Estos datos deberán ser gestionados, almacenados y presentados de manera accesible y comprensible.

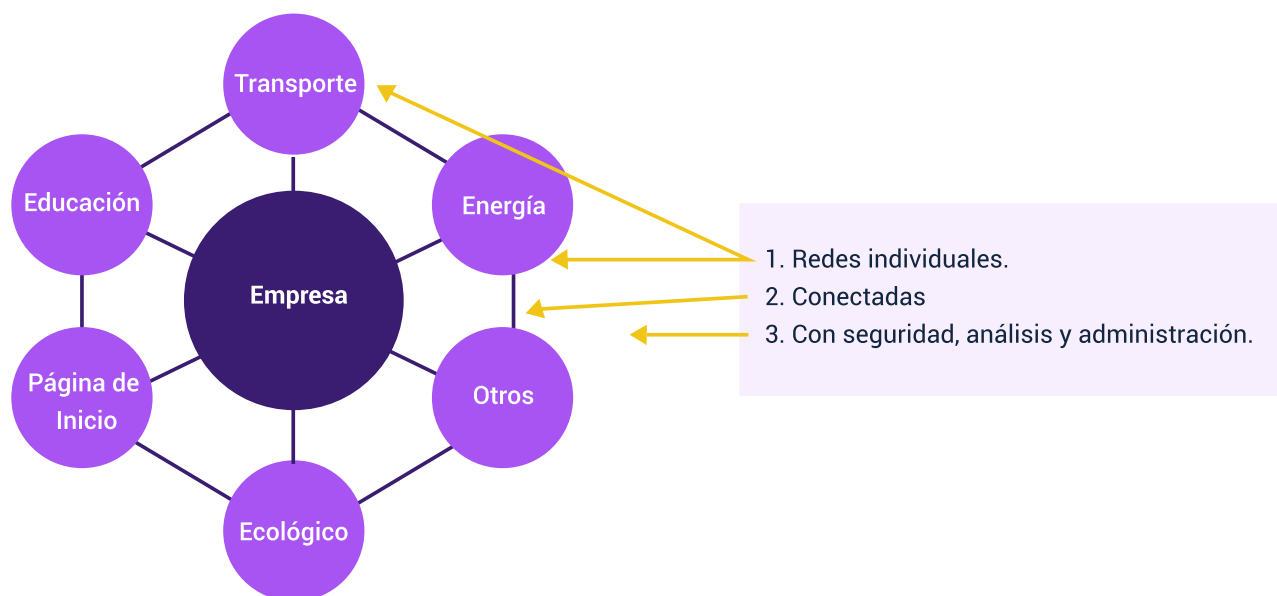
El concepto de IoT (Internet de las cosas), fue presentado por primera vez en Procter y Gamble en el año de 1999 por Kevin Ashton, describiendo que consistía en la generalización de una variedad de diferentes cosas, objetos o elementos de tipo cotidiano, integrados como etiquetas de identificación por medio de radiofrecuencias (RFID), los cuales eran controlados por Internet, generando una gran cantidad de datos que necesitaban ser recolectados y almacenados, razón por la cual IoT se integra con Cloud Computing para que estos datos se alojen en Internet, facilitando que estén en continua disponibilidad, para que las empresas y usuarios puedan acceder desde cualquier lugar.

En resumen, se puede decir que IoT es la agrupación de sensores y equipos electrónicos conectados entre sí, con el objetivo de calcular, recolectar y mandar datos a un servicio de almacenamiento en la nube.

El IoT en la actualidad, está organizado por una recolección extendida de diferentes redes y con propósitos distintos, por ejemplo, los carros modernos cuentan con una cantidad variada de redes para moderar la puesta en marcha del motor, el monitoreo de comunicación, los sistemas de seguridad. De esta manera parecida, los centros comerciales, las oficinas, los apartamentos manejan diferentes sistemas que permiten controlar la calefacción, el aire acondicionado, los sistemas de seguridad, etc.

En tanto que el IoT avanza, todas estas redes quedarán conectadas incorporando medidas de análisis, administración y seguridad, haciendo del IoT una herramienta todavía más poderosa.

Figura 1. Internet de las cosas red de redes



Entre las diferentes áreas de aplicación que tiene el mundo del IoT, se destacan las siguientes:

- **Domótica**

Consiste en la sistematización de una edificación, casa, o inmueble, conectando entre sí diferentes tipos de servicios como energía, agua, ventilación y dispositivos como cerrojos, electrodomésticos, optimizando la seguridad y eficiencia energética, garantizando un mejor bienestar para el usuario.

- **Automatización y control de procesos de producción**

Se emplea la tecnología de IoT para incrementar la eficiencia en la fabricación de los productos, así como su fiabilidad y el continuo seguimiento del producto en la fábrica.

- **Transporte y logística**

Consiste en monitorizar medios de transporte y el proceso de envío de productos, validar estados de conservación y ubicación, evitando robos o pérdidas, logrando realizar un análisis de mejores rutas o administración de tráfico. Unos ejemplos donde son utilizados son en las empresas de envíos o en las plataformas de servicios de carros, gestión de tiempos de traslados, gestión y control de vehículos.

- **Agricultura, ganadería y silvicultura**

Al igual que en las fábricas, IoT es utilizado para la automatización de trabajos primordiales en lo relacionado con el agro, como por ejemplo, sistema de riego, sistema de detector de temperatura para siembra, detección de enfermedades de los animales, entre otros.

- **Videovigilancia y seguridad**

Consiste en la configuración y control de una variedad de equipos de seguridad, tales como sensores de movimiento, alarmas inteligentes, detección de amenazas para proteger la seguridad de edificaciones, casas y así contar con una prevención cuando se presenten situaciones de riesgo.

- **Medicina**

Sus objetivos son detectar en una persona sus signos vitales, así como las variaciones para el suministro de medicinas. Aunque la orientación es más

a los humanos, es posible también monitorear el estado de salud de los animales domésticos. La incorporación de IoT en el sector de la salud transformará el cuidado médico, porque los centros de atención serán mucho más eficientes, brindando al personal de la salud datos muy importantes de los pacientes en tiempos más cortos, para poder agilizar los procesos médicos.

En la actualidad, existen muchos desarrollos que incorporan IoT en sus diferentes áreas de aplicación.

A continuación, se nombran unos de los más novedosos y curiosos.

- **Hidrate Spark**

Se trata de un termo para el agua que tiene colores metálicos muy llamativos, el cual informa el tiempo en que se debe tomar agua y reporta la cantidad de agua tomada según lo que se configure, este funciona por medio de un sensor que recolecta cada trago de agua y lo envía al celular por medio de bluetooth.

- **Ability MyCite**

Es una pastilla inteligente que fue creada por Otsuka Pharmaceutical Co y Proteus Digital, utilizada para tratar a las personas que sufren de trastorno bipolar o esquizofrenia. La pastilla incluye un sensor que envía señales a una aplicación, el sensor es activado cuando la pastilla se disuelve con los ácidos gástricos del estómago, permitiendo así realizar un seguimiento al paciente, verificando la hora de toma del medicamento y si en efecto se lo está tomando.

- **Waymo**

Es el primer carro autónomo creado por Google, que permite a un automóvil conducirse autónomamente por carreteras, detecta señales de tráfico, peatones y otros carros; inicialmente el nombre del proyecto era Google self-driving car project y luego recibió el nombre de Waymo.

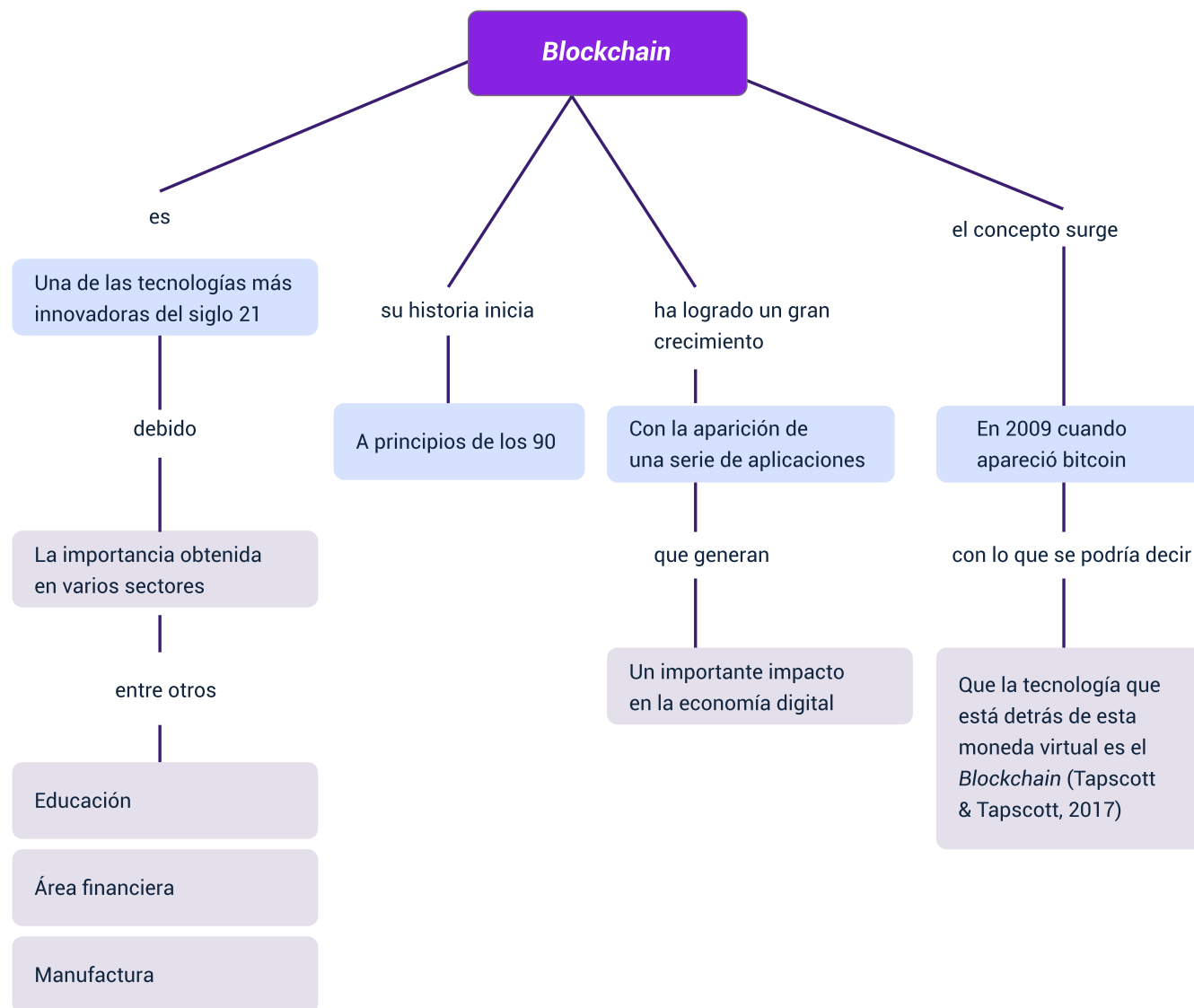
- **Google Glass**

Son unas gafas desarrolladas por Google para realidad aumentada, su función principal es poder manejar aplicaciones sin utilizar las manos, solo con comandos de voz; estas gafas incorporan un miniproyector que muestra una imagen virtual al ojo. Estas gafas son usadas en diferentes áreas como la educación, la industria y la medicina.

2. Blockchain

Para conocer qué es el blockchain, lo invitamos a revisar la siguiente figura.

Figura 2. Blockchain



El problema que quiere solucionar esta tecnología se puede explicar en el siguiente ejemplo:

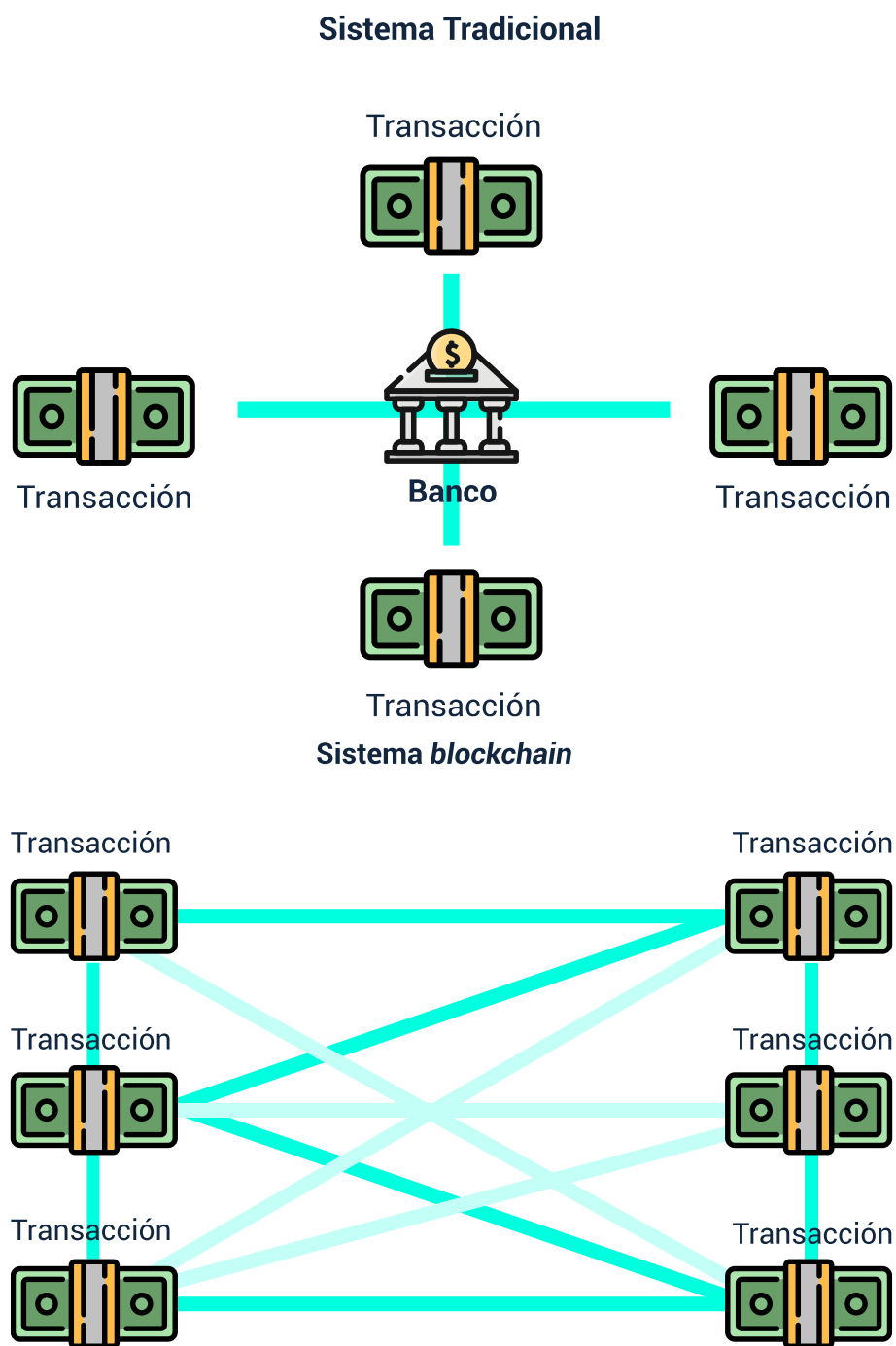
- Cuando una persona llamada Pepito quiere realizar un envío de un millón de pesos a otra persona llamada Mario, normalmente se hace por medio

de un banco, donde el banco actúa como intermediario de esta transacción, haciendo una centralización efectiva para hacer el movimiento de un lado al otro. En este proceso, ni Pepito ni Mario intervienen, son totalmente dependientes del banco y de las condiciones de este como comisiones, por ejemplo, es aquí donde el blockchain o cadena de bloques interviene, porque su principal objetivo es eliminar los intermediarios, haciendo una gestión totalmente descentralizada, donde el control ya no va a ser de los bancos sino de los usuarios, convirtiéndose estos en una conexión de muchos nodos del banco.

En resumen, se puede decir que el blockchain o cadena de bloques, es un gran libro digital de cuentas en donde sus bloques (registros), se encuentran conectados y cifrados, con el objetivo de proteger la privacidad y la seguridad de cualquier tipo de transacciones.

En la siguiente figura se muestra gráficamente la comparación entre un sistema tradicional y un sistema blockchain.

Figura 3. Sistema tradicional vs. Sistema blockchain



2.1. Historia

A continuación, conozcamos un poco sobre la historia del blockchain:

- **Historia del blockchain:**

El blockchain tuvo su inicio en los años de 1991-2008, donde en un primer trabajo, Stuart Haber y Scott realizaron tareas en una cadena de bloques que era protegida criptográficamente, esto quería decir que no se podía realizar ningún tipo de manipulación en las marcas de tiempo de los documentos.

- **Árboles de Merkle**

En el año 1991 se añadieron árboles de Merkle que lograron una mejor eficiencia, permitiendo que en un solo bloque se recopilara más de un documento. Pero es en el año 2008 que el blockchain inicia su crecimiento, cuando Satoshi Nakamoto y un grupo de personas lanzan su trabajo con bitcoin, creando la primera aplicación de registro digital que permitía administrar la red del bitcoin, permitiendo la interacción por medio de foros, entre los interesados.

- **Nacimiento del Bitcoin**

Desde los años 2008 al 2013 emerge el bitcoin y las transacciones, es importante aclarar que no hay que confundir el concepto de blockchain y bitcoin, porque muchas personas piensan que es lo mismo, y no es así, debido a que blockchain es la tecnología en que se fundamentan las aplicaciones y una de ellas es la criptomoneda. El nacimiento del bitcoin fue en 2008 y fue la primera aplicación que incluía la tecnología

blockchain, la definición dada por Satoshi Nakamoto en su artículo fue que se trataba de un sistema eléctrico peer to peer (punto a punto).

- **Se establece el bloque de origen**

Desde el momento en que el bitcoin, que es una aplicación de blockchain, se puso en conocimiento, han surgido una gran variedad de aplicaciones que tienen como objetivo el aprovechamiento de la tecnología digital.

Satoshi estableció el bloque origen de donde salieron otros bloques, los cuales se interconectaban, obteniendo como resultado una gran cadena de bloques que enviaban varios fragmentos de información y transacciones.

- **Surgimiento de los contratos inteligentes**

Para los años 2013-2015, blockchain evolucionó a su versión 2.0 y trajo los llamados contratos inteligentes (smart contracts), desarrollo realizado por Vitalik Buterin y al que llamaron Ethereum, que incorporaba una función que ofrece a los individuos registrar otro tipo de activos como lo son los contratos, ampliando a una plataforma que también desarrolla aplicaciones descentralizadas. Un contrato inteligente es un conjunto de instrucciones que se almacenan en un blockchain, las cuales se ejecutan automáticamente, dependiendo de los parámetros programados, esta ejecución se realiza de forma segura e inmutable.

- **Revolución de las aplicaciones del blockchain**

A partir del 2018 surgen una serie de proyectos, explotando la gran capacidad de la tecnología blockchain, estos proyectos buscaban tratar las fallas de Ethereum y bitcoin, incluyendo también nuevas características. Una de las aplicaciones más representativas fueron IOTA y NEO, enfocadas

en proporcionar más seguridad y escalabilidad que las aplicaciones iniciales de blockchain.

Ahora, revise la siguiente línea de tiempo, la cual resume la historia del blockchain:

- **1991 – 2008**

Stuar Haber y Scott Stornetta trabajan en el primer blockchain.

- **2009**

Satoshi Nakamoto lanza el whitepaper de bitcoin.

- **2010**

La primera compra de bitcoin tiene lugar 10.000 BTC.

- **2013**

El mercado de bitcoin supera los \$1 mil millones. Vitalik Buterín lanza el whitepaper de Ethereum.

- **2014**

Ethereum blockchain es financiado por crowdsale. La tecnología blockchain R3 se forma e inicia un consorcio de más de 40 compañías financieras legadas para implementar la tecnología blockchain.

- **2015**

El bloque Génesis de Ethereum es creado. Linux Foundation presenta hyperledger para mejorar.

- **2016**

Error en el código de Ethereum DAO explotado y atacado.

- **2017**

EOS es presentado por Blockone como un nuevo protocolo de blockchain para el despliegue de aplicaciones descentralizadas.

Es así como podemos concluir que el blockchain se define como un tipo de comercio, donde interactúan criptomonedas y transacciones, aunque la realidad es que es más que un seguro sistema bancario en línea, porque permite, además, el fácil intercambio de objetos e ideas.

Sus características principales son:

- **Inmutabilidad**

Los datos grabados en las cadenas son irreversibles.

- **Transparencia**

Cada uno de los bloques se pueden consultar y son visibles.

- **Sin intermediarios**

Se sostiene con una cantidad de computadores en el mundo, no existe ninguna entidad o empresa intermediaria.

- **Distribuido**

La información se transmite por un conjunto de computadores que comparten información, donde todos están de acuerdo con esta.

- **Veloz**

Solo se necesitan unos pocos minutos para realizar transacciones internacionales.

- **Global**

Solo es necesario Internet para que la información pueda ser vista.

Lo invitamos a consultar el siguiente video, el cual presenta cómo funciona bitcoin:

Video 1. ¿Cómo funciona Bitcoin?



Enlace de reproducción del video

Síntesis del video: ¿Cómo funciona Bitcoin?

Ana quiere enviar bitcoins a Pedro. Ana usa su llave privada y firma un mensaje con la cantidad de bitcoins y la dirección de Pedro. La transacción solicitada por Ana es incluida en un bloque junto con otras transacciones. El primer minero en validar el nuevo bloque para blockchain recibe una recompensa en bitcoins. La red valida la transacción de Ana. El bloque se propaga en toda la red de mineros. La transacción es completada y se añade el nuevo bloque a blockchain. Pedro recibe bitcoins de Ana.

A continuación, se describen unos puntos de funcionamiento del bitcoin:

- Los bloques pueden ser creados por cualquier persona desde su computador.
- Los bloques se crean utilizando un software especial.
- La capacidad de registro de un bloque está entre 2000 y 2400 transacciones.
- La recompensa que gana el primero que realice la validación, es de 50 bitcoins.
- Para agregar un bloque nuevo, la persona que lo genere lo debe enviar al resto de la red.
- No se pueden modificar ni borrar los bloques, estos son permanentes.

2.2. Aplicaciones

Son muchas las aplicaciones que en la actualidad proporciona blockchain, y que son utilizadas por empresas o instituciones. A continuación, se nombran unos ejemplos:

- **Industria alimentaria**

Por medio de la tecnología blockchain, la industria alimentaria puede realizar un seguimiento a cada uno de los productos, partiendo desde donde nacen hasta su destino final.

- **Sector energético**

Por medio del blockchain se puede pensar en una red entre varias casas para vender y comprar energía.

- **Sector bancario**

Por medio de las criptodivisas, los bancos pueden realizar transacciones a un costo menor, con una mayor eficiencia, segura y sin intermediarios.

- **Agricultura**

Al igual que la industria alimentaria, en la agricultura el blockchain se enfoca en optimizar la calidad de los alimentos, con aplicaciones que monitorean en tiempo real el estado de las hortalizas y verduras.

- **Sector inmobiliario**

Gracias a los contratos inteligentes, se pueden formalizar acuerdos con tal solo ingresar las condiciones y almacenarlas en blockchain, una vez se cumplan las condiciones, se ejecuta el contrato de forma automática.

- **Sistemas de votación**

La aplicación de blockchain en los sistemas de votación, facilita la regulación de estas empresas norteamericanas, debido a que han utilizado estas metodologías para garantizar la votación electrónica.

2.3. Desarrollos

En la actualidad, existen muchos desarrollos que incorporan blockchain.

Conozcamos algunos de los más novedosos:

- **IBM Food Trust**

Es una plataforma de intercambio soportada en blockchain, que favorece a los usuarios participantes de una red de alimentos para mayoristas, productores y distribuidores, la cual permite administrar datos de

transacciones, procesamientos y procedencia de los alimentos, entre otros, logrando reducir la pérdida de los alimentos.

- **Pylon Network**

Desarrollado por una empresa española, permite la venta y compra de energía renovable sin intermediarios, este desarrollo lanzó su proyecto de código abierto (open-source), concretamente para las condiciones del sector energético.

- **Steemit**

Consiste en una red social no centralizada que incorpora la tecnología steemit blockchain, su principal función es premiar a todos los usuarios bitcoin que estén activos, esto se consigue realizando publicaciones, comentarios, etc.

- **Koibanx**

Es una plataforma basada en blockchain para el sector financiero, que permite hacer el proceso de pagos, gestión de costos, plazos, fidelización de programas a bancos, cooperativas, fiduciarias, etc.

3. Machine learning

El machine learning es una rama de la AI (Inteligencia Artificial), que consiste en lograr que las máquinas aprendan automáticamente sin necesidad de ser programadas. El inicio de machine learning se da en el año 1943, cuando el neurofisiólogo Warren McCulloch y el matemático Walter Pitts, dan a conocer el concepto de inteligencia artificial y en el trabajo que presentaron, plantearon que se analizaría el cerebro como un ente computacional y la creación de supercomputadoras capaces de funcionar mucho mejor que la red neuronal humana, es aquí donde el mundo comenzaría a inquietarse sobre qué tanta inteligencia llegará a tener una máquina y qué implicaciones traería. (Hinestroza, 2018)

Conozcamos la evolución del machine learning:

- **1950**

El científico Alan Mathison Turing crea el “Test de Turing”, cuyo objetivo principal consistía en determinar la inteligencia de una computadora por medio de una conversación con un humano, la idea era imitar el comportamiento del ser humano sin que se diera cuenta que estaba conversando con una máquina.

- **1952**

El informático y profesor Arthur Samuel lanza el primer programa para computadores preparado para aprender. El software consistía en un juego de damas que almacenaba información y estrategias de juego y que entre más partidas se jugaba, había un mayor mejoramiento en el juego.

- **1956**

Con el apoyo de Minsky, John McCarthy, Claude Shannon y Nathan Rochester, dan nacimiento a la inteligencia artificial.

- **1958**

Rosenblat da a conocer la primera red neuronal artificial.

- **Años 70**

Se crea el robot “Stanford Car”, un robot móvil que tenía la capacidad de moverse de manera autónoma en un cuarto sin chocar con obstáculos y también Neighbor implementa el reconocimiento de patrones en computadores, dando inicio al machine learning, debido a que cuando se le daba la posibilidad a una máquina el aprendizaje de patrones, la solución de respuesta era muy efectiva.

- **Años 80**

La revolución se enfoca en el campo del procesamiento de datos cuando Gerald Dejong incorpora el concepto de “Explanation Based Learning” (EBL), que se trataba de una especie de aprendizaje automático, el computador realiza el análisis de datos entrenados y genera reglas, cuyo objetivo es filtrar los datos que tienen menos importancia.

- **2003**

Se hace la publicación de un estudio referente a un sistema distribuido de ficheros que recibe el nombre de “Google File System” (GFS), y toma más fuerza el año 2004, debido a que Google muestra la creación de un nuevo paradigma llamado “Map & Reduce”, enfocado al procesamiento distribuido.

- **2006**

Sigue el crecimiento y Google apoyando este progreso, crea “Cloud Bigtable”, servicio de bases de datos NoSQL de Big Data, culminando con Hadoop que es una plataforma “Big Data Open Source”.

- **2010 en adelante**

Sigue el crecimiento y apoyando Google este progreso, crea “Cloud Bigtable”, servicio de bases de datos NoSQL de Big Data, culminando con Hadoop que es una plataforma “Big Data Open Source”.

- **Actualidad**

No paran los grandes avances relacionándose con aplicaciones del mundo empresarial que se benefician del machine learning, logrando crear estrategias para la creación de completos mercados, incorporando cada vez más sectores.

3.1. Conceptos

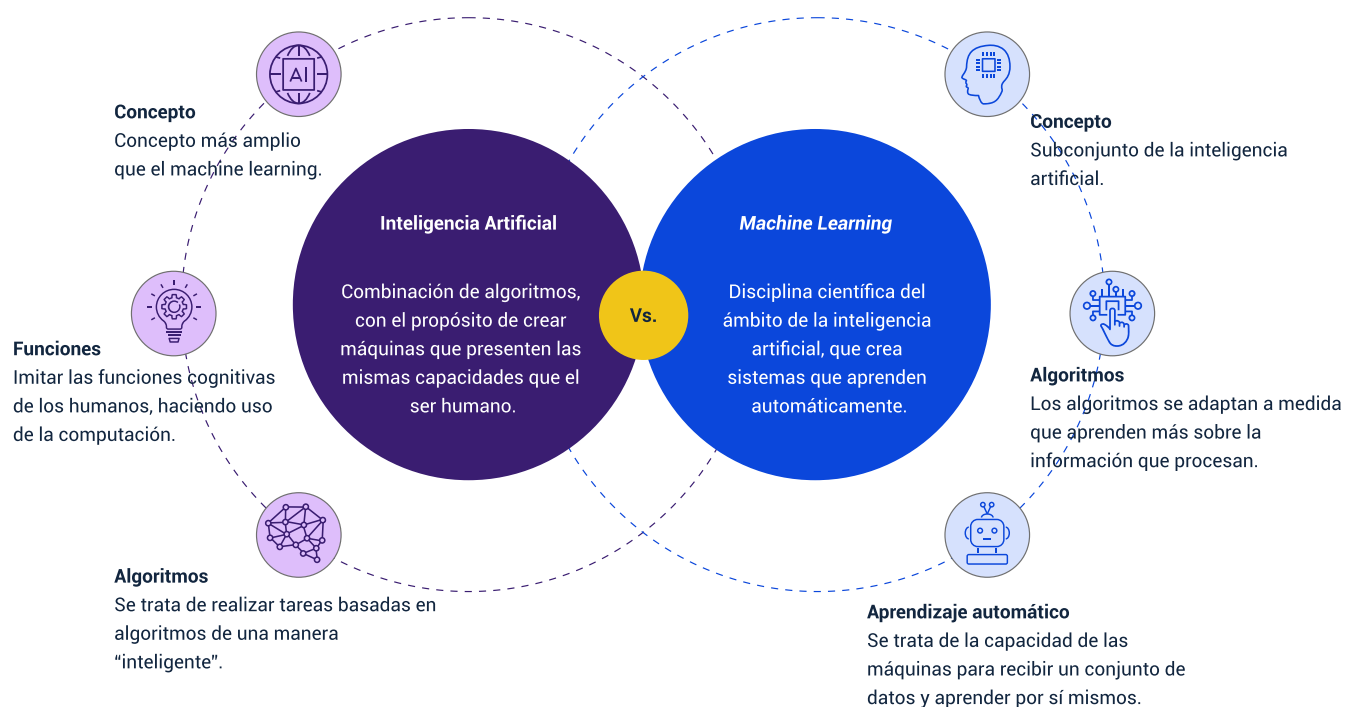
Existen muchos conceptos alrededor del machine learning. A continuación, se describen los más importantes.

Inteligencia artificial

Consiste en aplicar técnicas sistematizadas que proporcionan a las computadoras recibir algunas características que tiene la inteligencia humana, como por ejemplo, reconocer el significado de objetos y poder identificarlos, resolución de problemas autónomamente, aprendizaje de tareas nuevas, procesamiento de lenguaje natural, reconocimiento de imágenes, entre otros.

Se tiende a confundir mucho el concepto de inteligencia artificial con machine learning, pero son diferentes, machine learning es una rama de la inteligencia artificial que se fundamenta en el potencial que tienen las máquinas para recoger un grupo de datos y aprender por sí mismo, apropiando los algoritmos a medida que se va aprendiendo de la información procesada. La siguiente figura muestra otras de las características que diferencian los dos conceptos.

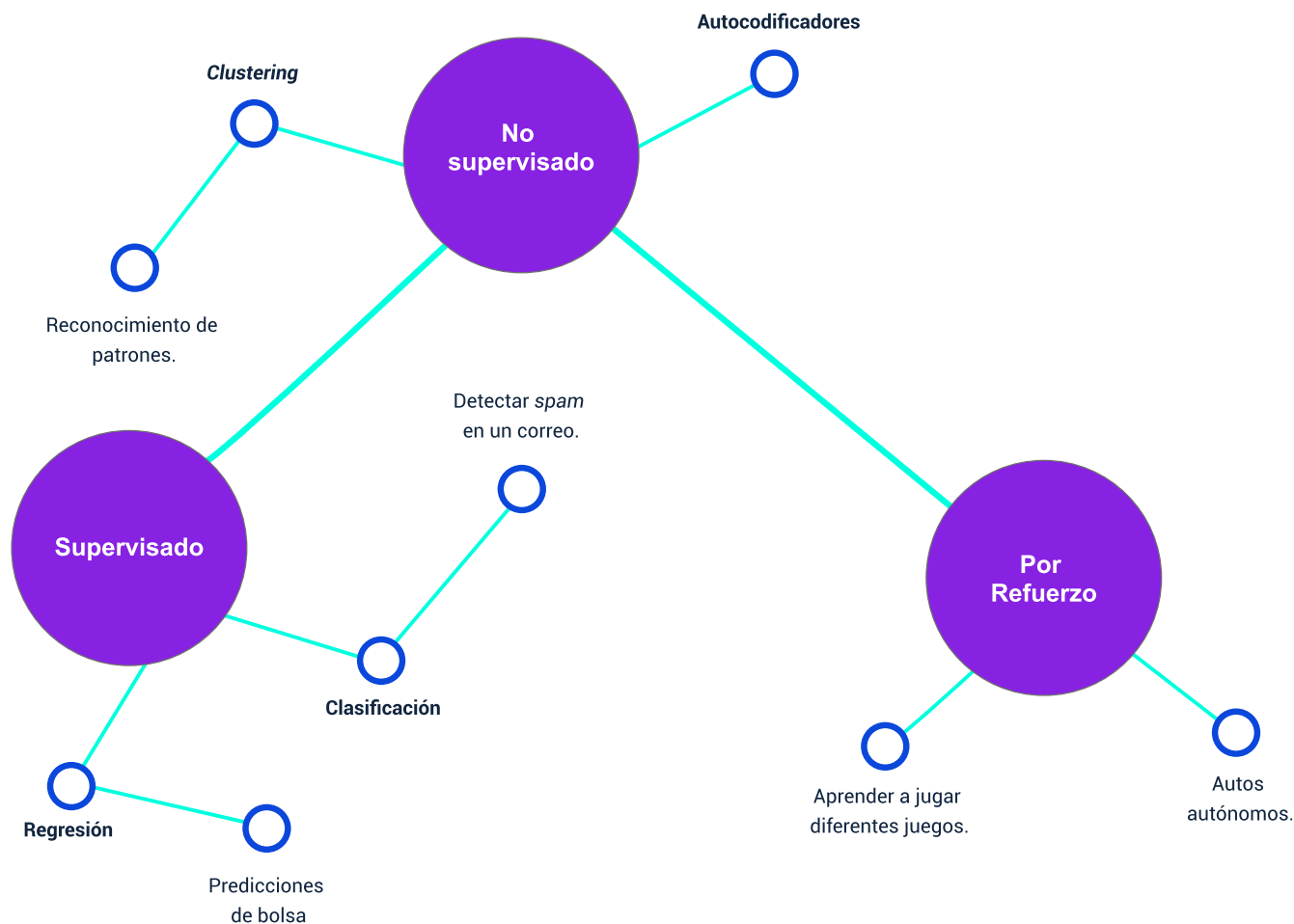
Figura 4. Inteligencia artificial vs. machine learning



Nota. Tomado y adaptado de Nalda, V. (s.f.).

También encontramos varios tipos de machine learning.

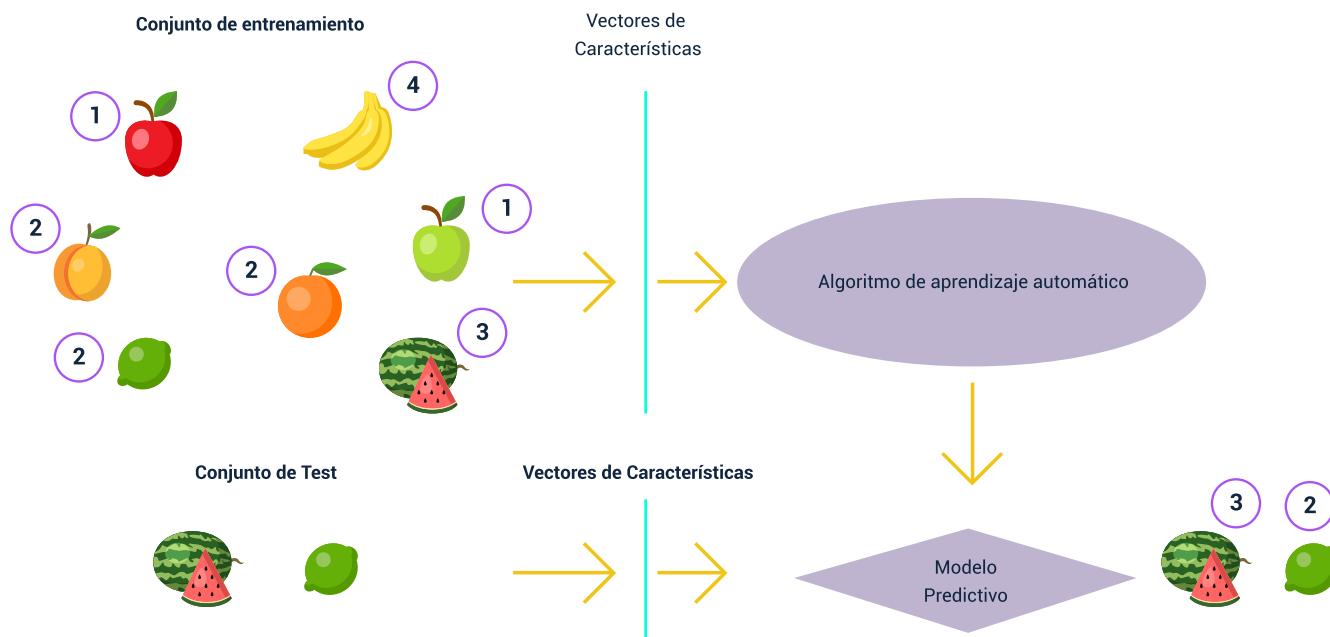
Figura 5. Tipos de machine learning



Aprendizaje supervisado

En este tipo de aprendizaje se utiliza información de entrenamiento llamada dataset, en donde lo que se hace es entrenar con muchas imágenes (labels) un modelo, para que finalmente pueda determinar de cuál label se trata. Básicamente es un grupo de técnicas, cuya función es permitir hacer predicciones fundamentadas en conductas que son analizadas en datos que han sido históricamente etiquetados.

Figura 6. Aprendizaje supervisado



La figura 6 muestra un ejemplo que contiene un grupo de datos que están etiquetados a propósito: 1 manzana, 2 cítricos, 3 sandías y 4 plátanos. Después, por medio de un vector, son extraídas las características de todo el grupo y con esta información se realiza el entrenamiento al algoritmo de aprendizaje automático. Lo que hace el algoritmo es crear un modelo al que se le pueden agregar nuevos grupos de datos para que sean clasificados nuevos elementos nunca vistos, aplicando lo aprendido en el entrenamiento inicial.

El aprendizaje supervisado se clasifica en dos tipos de métodos:

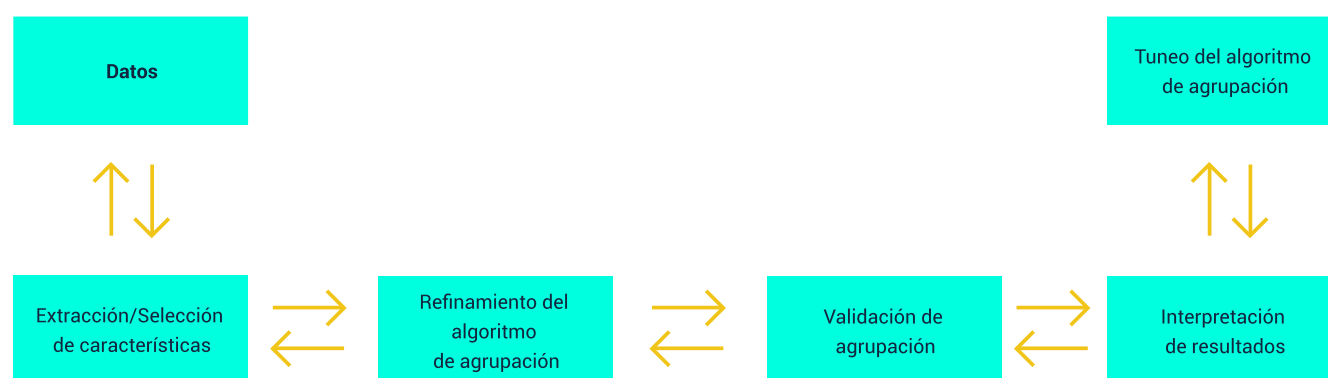
- El método de clasificación consiste, como su nombre lo dice, en hacer una clasificación en un grupo, reconociendo datos que ya han sido etiquetados anteriormente.

- El método de regresión realiza la predicción de datos continuos, partiendo de datos etiquetados históricamente, un ejemplo es predecir lluvias o el cambio del valor de un producto.

Aprendizaje no supervisado

Al contrario del aprendizaje supervisado, el no supervisado no tiene etiquetas, su objetivo es entender y abstraer patrones de información; se clasifica en clustering y en reducción. La siguiente figura muestra el proceso de aprendizaje no supervisado.

Figura 7. Aprendizaje supervisado



Aprendizaje por refuerzo

Se trata de realizar el entrenamiento de un algoritmo para crear un modelo que esté capacitado para obtener una salida con el resultado óptimo. El entrenamiento del modelo se hace programando que cada acción tenga sus premios o penalidades, así por cada prueba y error logrará encontrar el resultado que propague los premios.

Figura 8. Aprendizaje supervisado



Toma una acción y
obtiene una recompensa

Nota. Tomado y adaptado de Jurado, M. (2024).

3.2. Aplicaciones

El machine learning se aplica en muchas cosas de nuestra vida cotidiana, a continuación, se nombrarán algunos de los ejemplos donde se aplica y que hacen un gran aporte:

- **Reconocimiento de imágenes**

Para este proceso existen aplicaciones que comparan las letras, las agrupan y descifran aquellos textos que contienen imágenes. Un ejemplo más claro es el reconocimiento facial que realizan aplicaciones como Nequi o Facebook por medio de redes neuronales complejas.

- **Reconocimientos de voz**

Los modelos entrenados para el reconocimiento de voz detectan el sonido y pueden entender las órdenes dadas desde nuestro celular o computador y un ejemplo de estos son Cortana y Siri.

- **Predicción de datos**

Machine learning facilita predecir efectivamente datos que se obtienen a través del ingreso de estos a un sistema, se utilizan normalmente en estrategias de comercialización.

- **Autos autónomos**

Se aplican los modelos que realizan reconocimiento en tiempo real, los cuales permiten que los carros se desplacen de forma autónoma. Uno de los ejemplos de la actualidad son los carros Tesla.

- **Sistemas de recomendación**

Es uno de los ejemplos que aplica machine learning con el que se interactúa en la cotidianidad, porque aplica algoritmos que siguen patrones de comportamiento según lo que se vea, por ejemplo, las plataformas de películas como Netflix o las aplicaciones de música como YouTube y Spotify.

- **Watson**

Software de inteligencia artificial creado por IBM que puede dar respuesta a preguntas realizadas en un lenguaje natural.

- **DeepFace**

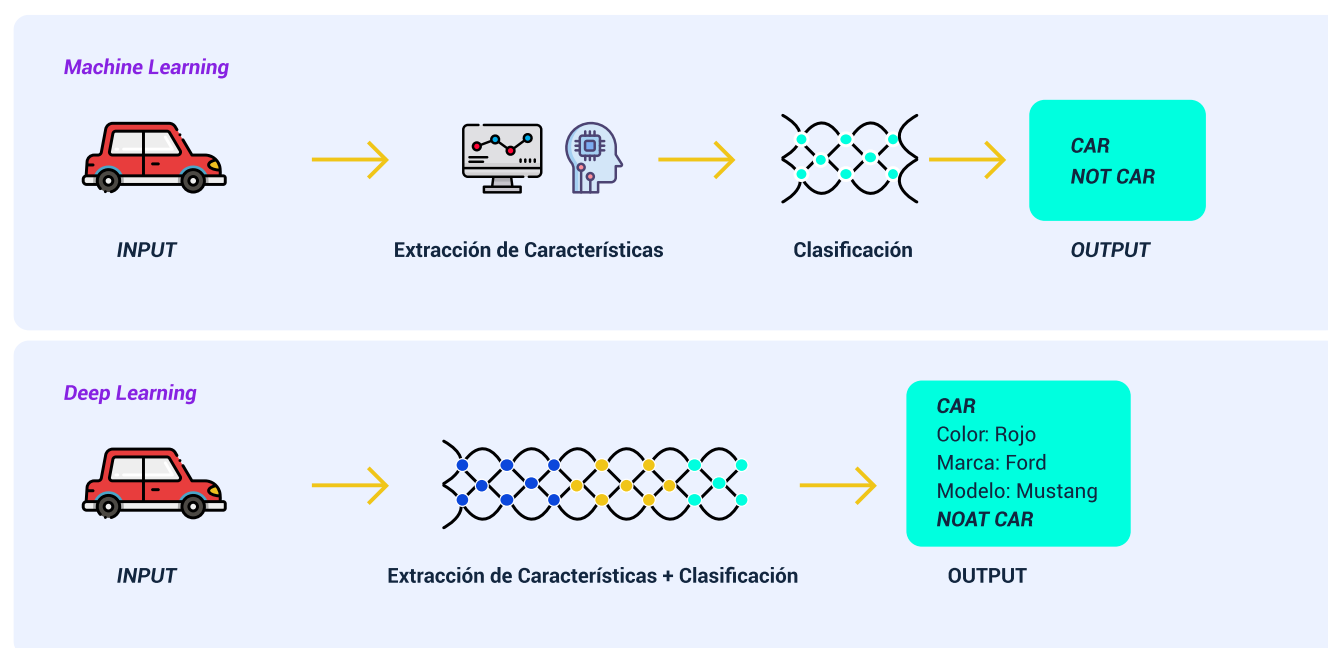
Algoritmo creado por Facebook que reconoce personas en fotos.

3.3. Deep learning

El deep learning se fundamenta también en la inteligencia artificial (AI), es traducido del inglés como aprendizaje profundo y está compuesto por redes neuronales

profundas. Su objetivo es tratar de hacer modelos muy similares al aprendizaje que utilizan los humanos para adquirir conocimiento de algo. Aunque se tiende a pensar que el machine learning y el deep learning son lo mismo, no es así, porque los algoritmos que utiliza el machine learning son tradicionales y lineales, mientras que los del deep learning son fuertemente complejos.

Figura 9. Machine learning y deep learning

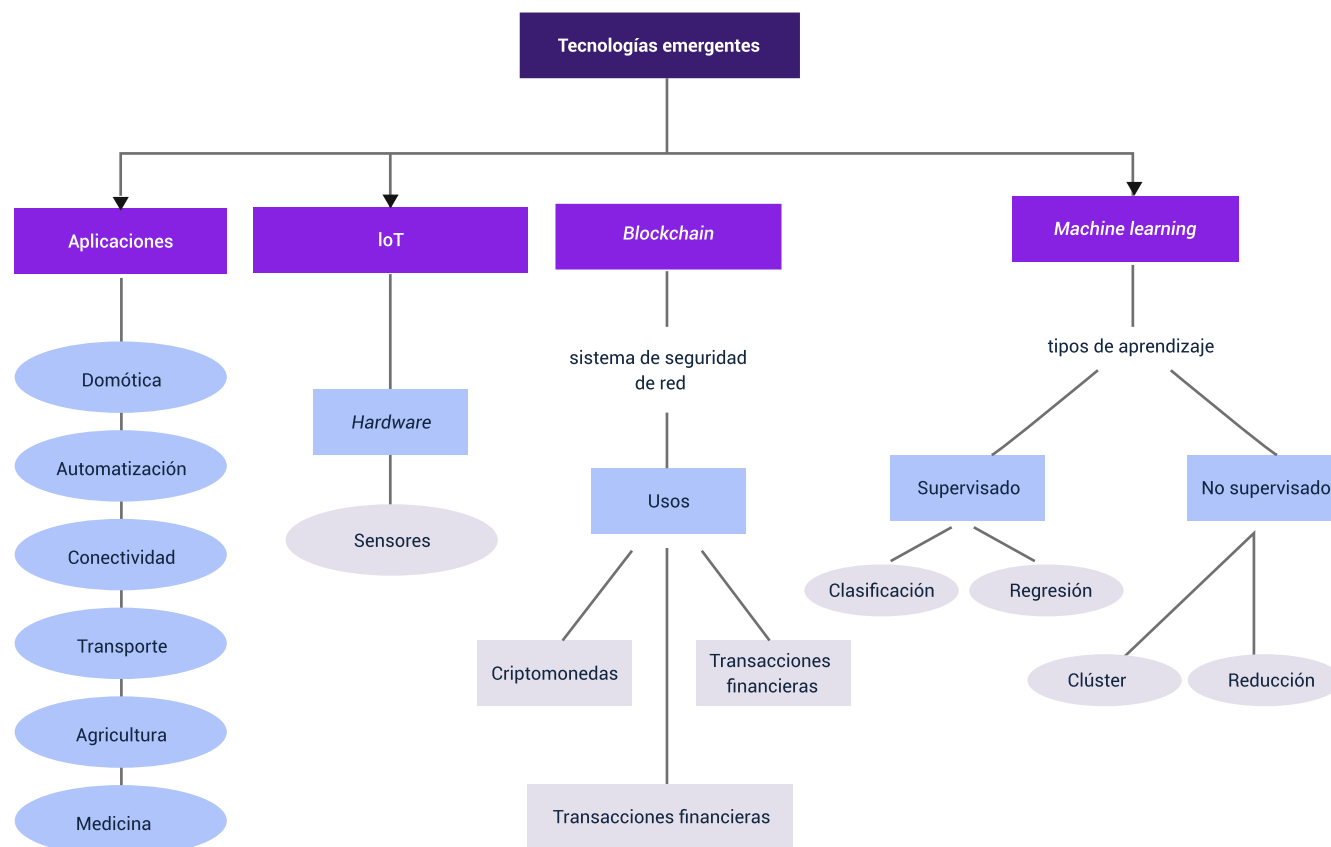


Algunos objetivos del deep learning son:

- Lograr que los computadores se instruyan para poder clasificar sonidos, imágenes y textos, sin que un humano intervenga.
- Perfeccionar los procesos pesados, respondiendo a necesidades de los humanos, resolviendo problemas de la sociedad.
- Exactitud, lograr un comportamiento que supere al del humano.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Blockchain	IBM. (s.f.). What is blockchain?	Página web	https://www.ibm.com/topics/blockchain
Desarrollos	Ayudaley. (s.f.). Google Glasses 2020: Resurgen las gafas inteligentes de Google.	Página web	https://ayudaleyprotecciondatos.es/2020/07/21/google-glasses

Glosario

Criptografía: es el arte de escribir con clave secreta o de un modo enigmático.

Criptomoneda: son monedas digitales que se intercambian online.

Referencias bibliográficas

Hinestroza, D. (2018). El machine learning a través de los tiempos y los aportes a la humanidad. Universidad Libre Seccional Pereira.
<https://repository.unilibre.edu.co/bitstream/handle/10901/17289/EL%20MACHINE%20LEARNING.pdf?sequence=1&isAllowed=y>

Jurado, M. (2024). ¿Qué es el Aprendizaje por Refuerzo y Aprendizaje por Imitación, y su Importancia con la Nueva IA?
<https://medium.com/@martin.jurado.p/aprendizaje-por-refuerzo-y-aprendizaje-por-imitaci%C3%B3n-ceba692a561a>

Lovelle, J., Molano, J. & Marin, C. (2015). Introducción al Internet de las cosas. Redes de Ingeniería, 6.
<https://revistas.udistrital.edu.co/index.php/REDES/article/view/8505/10464>

Nakamoto, S. (2008). Bitcoin: un sistema de efectivo electrónico peer to peer.
https://bitcoin.org/files/bitcoin-paper/bitcoin_es.pdf

Nalda, V. (s.f.). Machine Learning: Los orígenes y la evolución.
<https://www.futurespace.es/machine-learning-los-origenes-y-la-evolucion/>

Tapscott, D. & Tapscott, A. (2017). La revolución blockchain. Descubre cómo esta nueva tecnología transformará la economía global. Ediciones Deusco.
https://static0planetadelibroscommx.cdnstatics.com/libros_contenido_extra/35/34781_La_revolucion_blockchain.pdf

Vidal, M., Carnota, O. & Rodríguez, A. (2019). Tecnologías e innovaciones disruptivas. Educación Médica Superior, 33(1).



Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Zulema Yidney León Escobar	Experta Temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Jonathan Guerrero Astaiza	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Ana Catalina Córdoba Sus	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Carlos Julián Ramírez Benítez	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edward Leonardo Pico Cabra	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Jaime Hernán Tejada Llano	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Las pruebas de software

Breve descripción:

Las pruebas de software evalúan la calidad del código a través de diversos métodos. Las pruebas funcionales, unitarias y de integración se enfocan en la funcionalidad y la interacción de componentes. Por otro lado, las pruebas no funcionales y de rendimiento miden aspectos como la escalabilidad y la eficiencia. Cada tipo tiene ventajas, como la precisión, y desventajas, como el tiempo requerido.

Junio 2024

Tabla de contenido

Introducción	3
1. Generalidades de las pruebas de software	4
2. Tipos de pruebas y sus características	6
2.1. Pruebas funcionales	7
2.2. Pruebas unitarias.....	8
2.3. Pruebas de integración.....	9
2.4. Pruebas no funcionales	11
2.5. Pruebas de rendimiento.....	12
2.6. Ventajas y desventajas de los tipos de pruebas	13
3. Los casos de pruebas.....	15
4. Desarrollo guiado por pruebas.....	17
Ciclo del desarrollo guiado por pruebas.....	20
Síntesis	21
Material complementario	22
Glosario	23
Referencias bibliográficas	24
Créditos	25

Introducción

El intercambio y manejo de información, es esencial en todos los aspectos, porque involucra los niveles operativos, económicos, culturales e intelectuales de la humanidad.

Como parte fundamental, son los sistemas de información, que se desarrollan para optimizar el cambio y almacenamiento de esta información.

Sanz (2005), afirma que las pruebas de software son parte fundamental en el desarrollo de los sistemas de información, debido a que son el conjunto de procesos encaminados a encontrar posibles fallos de funcionamiento, usabilidad o configuración de un programa o aplicación.

Las pruebas de software, también conocidas como software testing, son la actividad de control de calidad más realizada en los proyectos de desarrollo o mantenimiento de software. En ellas se encuentran errores como los bugs, que a simple vista son insignificantes, pero pueden ser trágicos en los procesos que el programa desarrolla.

Bienvenido a este proceso de aprendizaje.

1. Generalidades de las pruebas de software

Las pruebas software son parte fundamental del proceso de calidad y adopción de buenas prácticas, estas deben realizarse en todo el proceso de desarrollo de software.

“Este proceso debe incluir revisión de los requerimientos, realización de análisis documentales, identificación de defectos, pruebas funcionales y no funcionales, pruebas dinámicas y estáticas, pruebas de integración, informes de confianza en el nivel de calidad, información para la toma de decisiones, planes de mejora continua”. Mera (2016)

El mismo autor, afirma que dentro de las actividades de un proceso de prueba, se deben incluir las siguientes fases:

¿Qué debemos tener en cuenta al momento de elaborar una lista de chequeo?

- El control.
- Comprobación de resultados.
- Elaboración de informes de proceso.
- Aplicación objeto de las pruebas.
- Bitácoras de experiencias.

Es de suma importancia realizar una bitácora y una matriz, con el fin de ejecutar los casos de prueba y así concluir si el caso funciona como se esperaba, para agregarlo como una conformidad, o en su defecto, pasaría a ser una no conformidad.

Según la organización ISO (2020), estas pruebas se regulan por la norma ISO/IEC 25000 Sistemas y Requisitos de Calidad y Evaluación de software (SQuaRE).

Las pruebas de software son aplicables a todos aquellos que deseen crear especificaciones de prueba, marcos correspondientes, o crear automatización de pruebas.

2. Tipos de pruebas y sus características

En los últimos años, se han propuesto algunos principios, con el fin de establecer pautas universales para que los programadores de software las adapten a sus procesos de pruebas. Mera (2016), lista los siguientes principios:

- **Las pruebas muestran la presencia de defectos**

Las pruebas son herramientas que facilitan detectar defectos, no obstante, no garantizan que no haya defectos ocultos y no es una evidencia de que el software esté óptimo.

- **Pruebas tempranas**

Encontrar los errores en etapas tempranas, permite un importante ahorro de recursos.

- **No es posible realizar pruebas exhaustivas**

Hay que realizar un análisis de riesgos, para identificar prioridades y tener una óptima toma de decisiones, centralizando esfuerzos y utilizando el talento humano y recursos de manera correcta.

- **Las pruebas dependen del contexto**

Estas dependen del contexto en el cual se ejecutan, por lo cual, se debe dar más énfasis en las que sean para un sistema crítico, como un software financiero, para el cual se requiere realizar más pruebas, en comparación con otras aplicaciones con niveles críticos bajos.

- **Agrupación de defectos**

Las pruebas deben agruparse, la mayoría de fallos operativos se enfocan en un número reducido de módulos.

- **La paradoja de los pesticidas**

No hay que repetir la misma prueba varias veces, por lo cual es importante que los casos de prueba se revisen constantemente y escribir nuevos, con el fin de encontrar más defectos.

- **Ausencia de error**

De nada sirve corregir en el sistema si este no cumple con los requerimientos o necesidades del usuario.

Conozcamos cuáles son los tipos de pruebas que se pueden realizar.

2.1. Pruebas funcionales

IBM (2021), plantea que estas pruebas se basan en la ejecución y revisión de las funciones y en su interoperabilidad con sistemas específicos, se llevan a cabo en todos los niveles de prueba. Buscan evaluar cada una de las opciones con las que cuenta el paquete informático, principalmente en el comportamiento externo del producto o aplicativo, también se las identifica como de tipo caja negra.

Figura 1. Pruebas funcionales



2.2. Pruebas unitarias

Las pruebas unitarias, o como también se las conocen, Unit testing, son pequeños test, en los cuales se revisa que el comportamiento de un objeto y su lógica funcionen adecuadamente. Por lo general, se realiza durante la fase de desarrollo de las aplicaciones o software, y es ejecutada por los programadores, aunque también pueden realizarlas los aseguradores de calidad. Las razones por las cuales se desarrollan este tipo de pruebas, son:

- **Prueba uno**

Son rápidas de realizar, por lo que se pueden realizar en gran cantidad.

- **Prueba dos**

Demuestran que la lógica del código funciona en todos los casos.

- **Prueba tres**

Permite a los programadores entender bien el código base, lo que facilitará realizar cambios oportunos.

- **Prueba cuatro**

Se obtendrá un código de calidad.

En algunos casos, las pruebas pueden hacerse de manera manual, pero lo mejor es usar herramientas que permitan ejecutar el servicio de la mejor manera posible. Existen muchas herramientas en el mercado y estas varían en función del lenguaje de programación que se esté utilizando; a continuación, se mencionarán algunas de las más conocidas:

- **xUnit**

Es una herramienta de pruebas unitarias para el framework.NET.

- **Junit**

Es una herramienta de pruebas de aplicaciones Java.

- **Nunit**

Es una herramienta de pruebas de plataformas .NET.

- **PHPUnit**

Es una herramienta de pruebas de programación PHP.

Para realizar las pruebas unitarias se deben tener en cuenta los siguientes criterios:

- a) Probar un solo código a la vez.
- b) Hay que realizar pruebas frecuentemente mientras se programa.
- c) Corregir los bugs identificados en las pruebas antes de continuar.
- d) Cualquier cambio que se realice también debe pasar la prueba.

2.3. Pruebas de integración

De acuerdo con Mera (2016), estas pruebas son las encargadas de verificar el óptimo acople entre los componentes y probar las interfaces entre los módulos, como el sistema de archivos en integración con el hardware y el componente de validación de usuario con el sistema operativo.

Se deben tener en cuenta los objetos típicos de prueba, los cuales son:

- a) Bases de datos de subsistemas.
- b) Datos de configuración.
- c) Configuración del sistema.
- d) Infraestructura.
- e) Interfaces.

Ahora, las principales funcionalidades de las pruebas de integración, son: verificar que un gran conjunto de partes de la aplicación funcionen juntas; verificar el correcto ensamble entre componentes, para que funcionen adecuadamente; no necesita resguardo, lo cual es un gran beneficio; se realiza en cada uno de los módulos del sistema y se inicia desde los módulos de abajo hacia arriba.

El mismo autor argumenta que en las pruebas de integración se revisan las interfaces entre componentes o subsistemas, para cerciorarse de que son llamados cuando se necesite y que los datos o mensajes que se transmiten son los adecuados. Existen los siguientes tipos de pruebas de integración:

- **Integración incremental**

Se prueba el nuevo componente en conjunto con los que ya se probaron y se incrementa progresivamente con los nuevos componentes que se probarán.

- **Descendente**

También conocidas como top down, se realizan siguiendo el flujo de control, ejemplo, desde el GUI o menú principal y los componentes o sistemas se sustituyen por stubs.

- **Ascendente**

También conocidas como bottom-up, las pruebas se ejecutan desde la parte inferior del flujo hacia arriba y los componentes o sistemas se sustituyen por controladores.

- **Integración no incremental**

También conocida como Big-bang, se prueba cada componente por separado y al final, se integran todos de una vez, realizando las pruebas pertinentes.

- **Combinadas**

Se desarrolla combinando el enfoque ascendente y descendente, pero hay que realizar una planificación, para que los componentes individuales se encuentren en el medio.

2.4. Pruebas no funcionales

En este tipo de pruebas se comprueban los requisitos basados en la operación de un software, no en su funcionalidad. Estas pruebas pueden ayudar a establecer la carga que resiste el producto, si su rendimiento es el adecuado o si es estable a nivel de contacto con el servidor, en otras palabras, nos dice si tienen un bajo desempeño o rendimiento en la producción. Aquí se encuentran:

- **Pruebas de carga**

Simulan la demanda esperada que puede tener una aplicación y mide el resultado.

- **Pruebas de rendimiento**

Se calcula la respuesta del programa con diferentes medidas de peticiones del usuario.

- **Pruebas de volumen**

Se prueba el funcionamiento del programa con ciertos volúmenes de datos.

- **Pruebas de estrés**

Cuántos usuarios o peticiones puede soportar el programa, con esta prueba se determina si el sistema es estable.

- **Pruebas de configuración**

Aquí se valida qué efectos en el desempeño tienen los cambios de configuración que se pueden aplicar, como en diferentes hardware, software, versiones y navegadores.

- **Prueba de resistencia**

Aquí se prueba cómo se comporta una aplicación luego de un uso prolongado.

- **Pruebas de evaluación**

Se evalúa si la aplicación cumple con los objetivos para la cual fue diseñada.

- **Pruebas de usabilidad**

Se centran en validar qué tan fácil se utiliza el programa en cuanto a facilidad de aprendizaje, eficiencia, memorización, errores y satisfacción.

2.5. Pruebas de rendimiento

Según IBM (2021), estas pruebas sirven para evaluar lo rápido que el programa realiza una tarea en condiciones específicas de trabajo, también evalúa la escalabilidad, fiabilidad y uso de los recursos. Los objetivos de estas pruebas son:

- **Identificar y localizar problemas de rendimiento**

Es permitir encontrar un fallo en el rendimiento, y también ayudar a localizar en qué parte está ese problema de rendimiento, para poder solucionarlo.

- **Verificar el cumplimiento de los SLA**

SLA - Acuerdos de Nivel de Servicio. Se utiliza para medir los tiempos de respuesta del software en condiciones y tiempos específicos.

- **Localizar cuellos de botella**

Ayudar a detallar en qué lugar se generan estos cuellos de botella, si se debe a problemas de hardware como CPU, memoria, o es el ancho de banda.

2.6. Ventajas y desventajas de los tipos de pruebas

Todo proceso tiene sus ventajas y desventajas, a continuación, se presentan cada una de ellas.

a) Pruebas funcionales

- Optimiza las funcionalidades y la calidad del producto.
- Verifica que la seguridad y la arquitectura sean las adecuadas.
- Asegura que el comportamiento del software y de todas sus funciones sea el esperado.
- Se asegura que el software o aplicación esté libre de errores.

b) Pruebas unitarias

- Simplifica la integración y permite una buena documentación.
- Funciona con el proceso de desarrollo ágil.
- Detección temprana de errores en sus funcionalidades, lo que minimiza tiempo y costos.
- Optimiza la efectividad del código.

c) Pruebas de integración

- Detalla los errores y conflictos en cada módulo y en sus conexiones.

- Mejora la calidad del código.
- Garantiza que las funciones del software serán confiables y estables entre sus módulos.
- Garantiza que todos los módulos del software estén correctamente integrados y funcionen en conjunto como se necesita.

d) Pruebas de rendimiento

- Mejora las características del software o aplicación, perfeccionando el sistema.
- Identifica errores que no se detectan en las pruebas funcionales.
- Localiza los cuellos de botella para solucionarlos y optimizar su rendimiento.
- Valora la escalabilidad y velocidad de la aplicación o website.

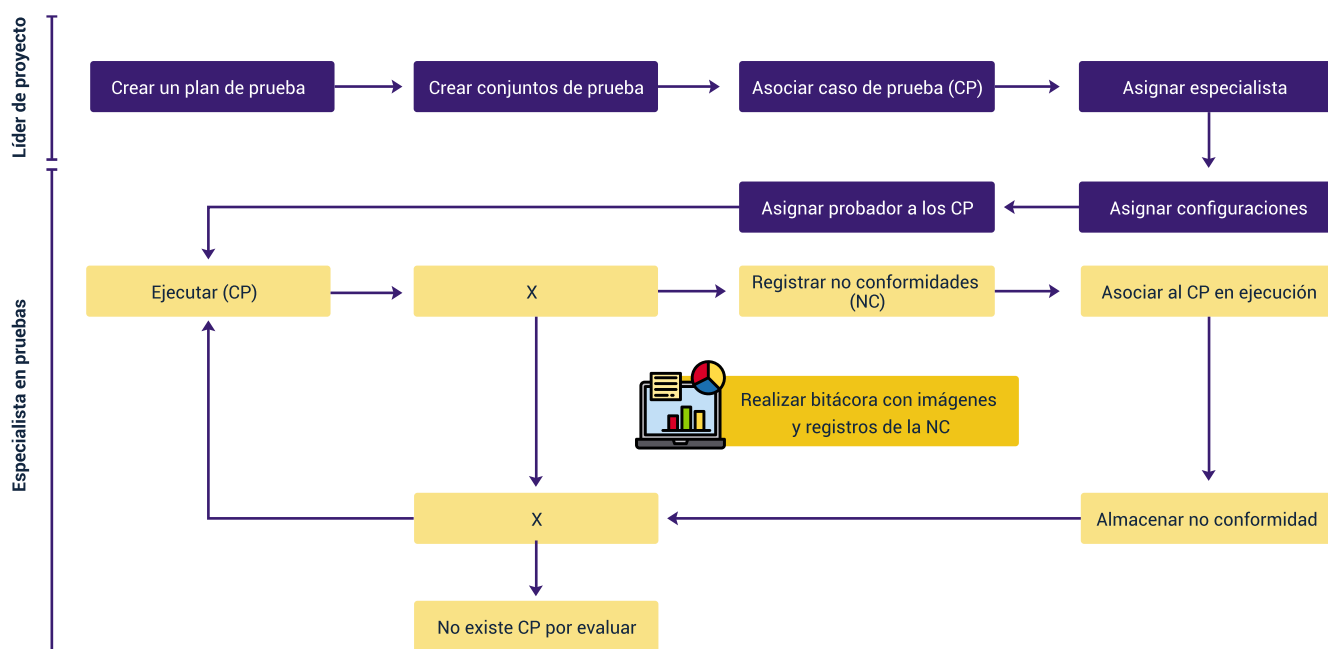
Conclusión de los factores que afectan o determinan la calidad del software

Por lo general, todas las pruebas de software traen beneficios al desarrollador, dependiendo del tipo de la prueba los beneficios pueden ser más altos y enfocados en un fin o propósito específico, dando como resultado una mejor calidad de producto, mejorando su funcionalidad, reduciendo costos, tiempos, recursos humanos y mejorando la experiencia del usuario. Se podría considerar como una desventaja, si se comparan los tipos de prueba, porque puede variar en tiempo, costo, esfuerzo, recursos humanos y exactitud de las pruebas. Sin embargo, la única desventaja de las pruebas de software es no hacerlas, porque el omitir este paso, desencadena una nefasta serie de errores como un producto no funcional y colocarían en tela de juicio la profesionalidad del programador y su equipo de trabajo.

3. Los casos de pruebas

Son los documentos de los productos que se generan al momento de realizar una prueba de software, en los cuales se plasman las precondiciones, entradas y resultados esperados, incluyendo la implementación correcta, la identificación de errores, el chequeo de la calidad y otras informaciones consideradas importantes. En general, no hay una plantilla específica, solo la bitácora, listas de chequeo e imágenes en las que se registran las no conformidades que se encuentran en las pruebas. A continuación se presenta un esquema de casos de prueba.

Figura 2. Esquema de casos de prueba



En el diseño de las pruebas de software, se deben identificar y describir casos de prueba, por lo tanto, para documentar un caso de prueba se debe comprender que este contiene un conjunto de variables o condiciones, en el cual un tester determina si un sistema funciona según lo esperado; así mismo, cabe resaltar que la documentación de un caso de prueba se convierte en una actividad sin dificultad, si se tiene la

información necesaria para su proceso de elaboración, teniendo en cuenta que al momento de probar un software, es de mucho aporte, puesto que se transforma en una herramienta esencial en el proceso de registro, seguimiento y control. A continuación, se nombran los elementos principales que debe contener un caso de prueba.

Tabla 1. Elementos de un caso de prueba

Elemento	Descripción
Identificador	Puede ser alfanumérico o numérico.
Nombre	Nombre del caso de prueba de manera concisa.
Descripción	Objetivo del caso de prueba, también describe qué probará, en ciertas ocasiones se incluye el ambiente de pruebas.
Número de orden de ejecución	Orden en la cual se ejecuta el caso de prueba, en la situación donde se tengan múltiples casos de prueba.
Requerimiento asociado	Si se plantea un caso de prueba, se debe saber a qué requerimiento va asociado, para mantener la trazabilidad.
Precondición	Estado en el cual se debe encontrar el sistema antes de comenzar la prueba.
Postcondición	El estado en que debe encontrarse el sistema luego de ejecutar la prueba.
Resultado esperado	Objetivo que debe ser alcanzado, posterior a la ejecución de la prueba.

4. Desarrollo guiado por pruebas

El desarrollo de software dirigido por pruebas, es una técnica de programación en la cual las pruebas, que normalmente son las unitarias, se escriben primero y luego se escribe el código que pase las pruebas. Con esta práctica se consigue, entre otras cosas, un código más robusto, más seguro, más mantenible y una mayor rapidez en el desarrollo.

Según Herranz (2021), cuando el TDD (Desarrollo Dirigido por Pruebas), se combina con el desarrollo ágil, tiene como consecuencia un espacio más amplio, el cual cubre el desarrollo de software, hasta el diseño de software, el cual propone el proceso denominado:

- **Rojo.** Hacer que las pruebas fallen.
- **Verde.** Hacer que las pruebas pasen.
- **Refactory.** Corregir errores lógicos.

“Para usar TDD, el desarrollador de software tiene que cambiar su método de trabajo. Ahora, en vez de crear los casos de uso y después implementarlos, va a intentar convertir un caso de uso en un ejemplo. Si se realiza esto con todos los casos de uso, llegará un momento en el que los ejemplos describirán por sí solos la próxima tarea a implementar”. Sánchez (2017)

El proceso de diseño de software, combinando TDD con metodologías ágiles, sería el siguiente:

- **Historia del usuario**

El cliente escribe su historia de usuario.

- **Aceptación de la historia de usuario**

Se definen, en conjunto con el cliente, los criterios de aceptación de la historia de usuario, descomponiéndola todo lo que se pueda, para simplificar lo más posible.

- **Punto de aceptación**

Se selecciona el punto de aceptación más sencillo y se transforma en la prueba unitaria.

- **Verifica**

Se verifica que esta prueba falla.

- **Prueba correcta**

Se escriben las líneas de código para hacer que la prueba sea correcta.

- **Automático**

Se llevan a cabo todas las pruebas de carácter automático.

- **Vuelve**

Se vuelve a escribir el código para que quede limpio.

- **Volvamos al punto 3**

Se debe regresar al paso número 3 en cuanto se detecten criterios de aceptación que hagan falta, y se debe repetir el ciclo una y otra vez hasta lograr la aplicación completa.

Es de comentar que este es un proceso de desarrollo relativamente nuevo, el cual está comenzando a ganar terreno y ser conocido en el ámbito del desarrollo de software en las organizaciones a nivel mundial, el cual se basa en evidencias anecdóticas y una serie de evaluaciones empíricas. Hablando del desarrollo tradicional,

las pruebas, se usan para validar y verificar la calidad del software y su creación se realiza después de que existe la funcionalidad.

Nota:

En el TDD, las pruebas adicionales al tema de validación y verificación se usan con fines de especificar, donde primero se desarrolla la prueba y posteriormente la funcionalidad. Esto permite que la calidad del software aumente y la productividad del desarrollador mejore, dado que se lleva un mejor esfuerzo en depurar y se avanza rápidamente con el progreso del producto software.

A continuación, se pueden observar los beneficios del desarrollo guiado por pruebas:

- **TDD**

Permite una verificación de las funcionalidades desarrolladas en ciclos más pequeños, lo que permite la detección de errores y así mismo, su solución temprana.

- **Implementación**

La implementación de esta forma de trabajo permitirá elevar la fiabilidad del producto software, no solo desde el punto de vista de errores funcionales, sino avanzar con las necesidades requeridas por el cliente.

- **Optimización**

Otro de los beneficios es la optimización y mejora en el rendimiento del código, dado que se hace énfasis en escribir solamente el código necesario para la resolución de la necesidad.

- **Proceso**

El proceso de realizar pruebas por cada nueva funcionalidad o requisito, facilita el proceso de desarrollo de forma modular, lo que permite la escalabilidad y el futuro mantenimiento del producto software.

Ciclo del desarrollo guiado por pruebas

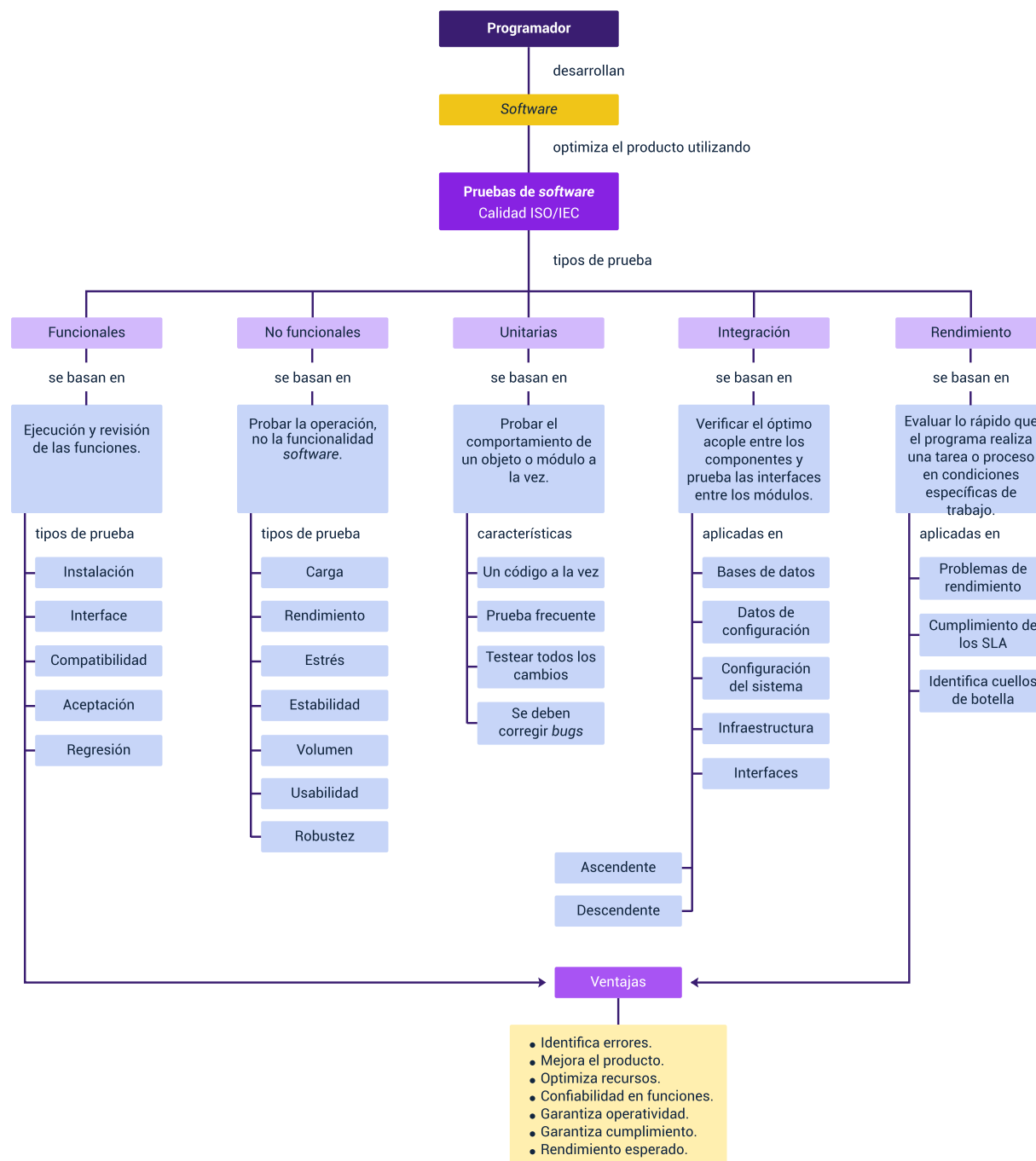
Ahora que se conocen los beneficios que implica aplicar TDD, es posible ver el ciclo completo del proceso, revisando cada uno de sus pasos. En la siguiente figura se presenta el flujo de manera gráfica.

Figura 3. Ciclo TDD



Síntesis

A continuación, se presenta a manera de síntesis, un esquema que articula los elementos principales abordados en el desarrollo del componente formativo, con relación a las pruebas de software.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Tipos de pruebas y sus características	IBM. (2021). Pruebas funcionales.	Página web	https://www.ibm.com/docs/es/rtw/9.1.0?topic=SSBLQQ_9.1.0/com.ibm.rational.test.ft.doc/topics/Getting_Started_With_Ivory.html
Tipos de pruebas y sus características	IBM. (2021). Pruebas de rendimiento.	Página web	https://www.ibm.com/docs/es/rtw/9.0.0?topic=phases-performance-testing

Glosario

Bottom-up: pruebas ascendentes.

QA (Quality Assurance): especialistas en pruebas de software, verifican que un software no contenga fallos.

Refactory: acción de limpiar o reconstruir el código de una aplicación.

TDD: desarrollo dirigido por pruebas.

Top-down: pruebas descendentes.

Referencias bibliográficas

Herranz, J.I. (2021). TDD como metodología de diseño de software.

<https://www.paradigmadigital.com/dev/tdd-como-metodologia-de-diseno-de-software/>

IBM (2021). Pruebas de rendimiento.

<https://www.ibm.com/docs/es/rtw/9.0.0?topic=phases-performance-testing>

IBM (2021). Pruebas funcionales.

https://www.ibm.com/docs/es/rtw/9.1.0?topic=SSBLQQ_9.1.0/com.ibm.rational.test.ft.doc/topics/Getting_Started_With_Ivory.html

Mera Paz, J. A. (2016). Análisis del proceso de pruebas de calidad de software.

Ingeniería solidaria, 12 (20).

Organización ISO. (2020). Ingeniería de sistemas y software - Requisitos de calidad y evaluación de sistemas y software (SQuaRE) - Guía de SQuaRE.

<https://www.iso.org/standard/64764.html>

Sánchez, A.F (2017). Agile Testing. Estado del arte. Su aplicación en empresas TIC de Extremadura. Universidad De Extremadura.

Sanz, L. F. (2005). Un sondeo sobre la práctica actual de pruebas de software en España. REICIS. Revista Española de Innovación, Calidad e Ingeniería del Software,1(2), pp. 43-54.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Mario Fernando Meneses Calvache	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Carlos Hernán Muñoz	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Ana Catalina Córdoba Sus	Evaluable Instruccionale	Centro de Servicios de Salud - Regional Antioquia
Yerson Fabián Zárate Saavedra	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edward Leonardo Pico Cabra	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Jaime Hernán Tejada Llano	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia